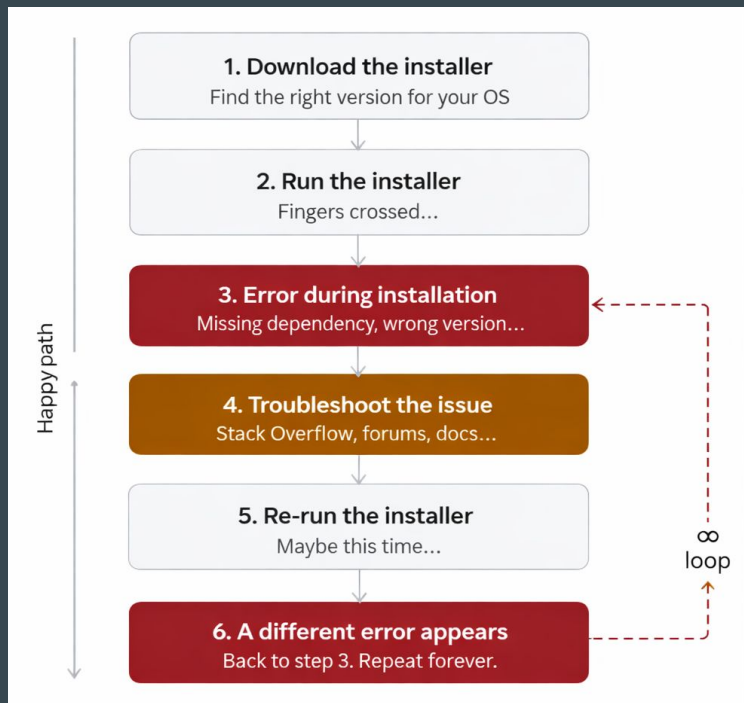


Docker Certified Associate

Introduction to Docker

Traditional software install experience

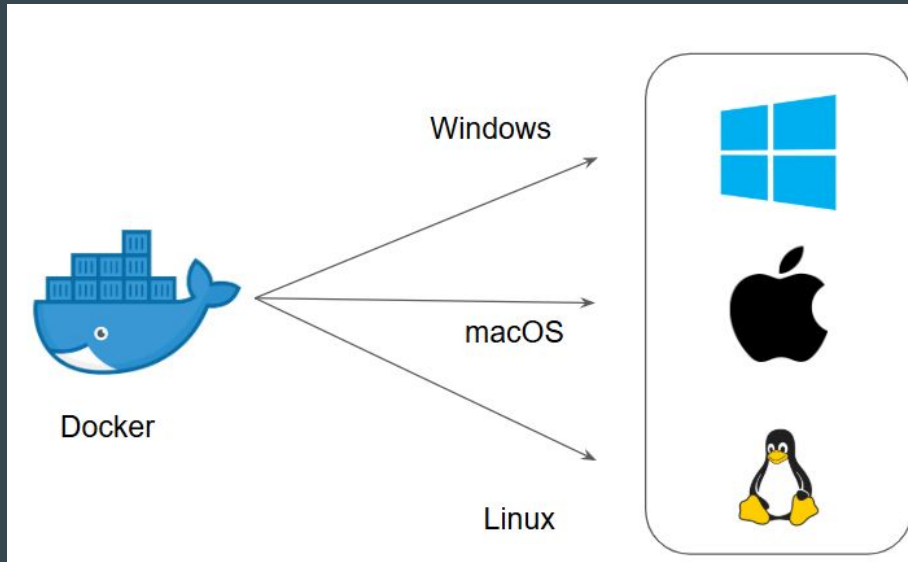
Installing software traditionally is a painful cycle — download, run, hit errors, troubleshoot, and repeat.



Installation Methods - Docker

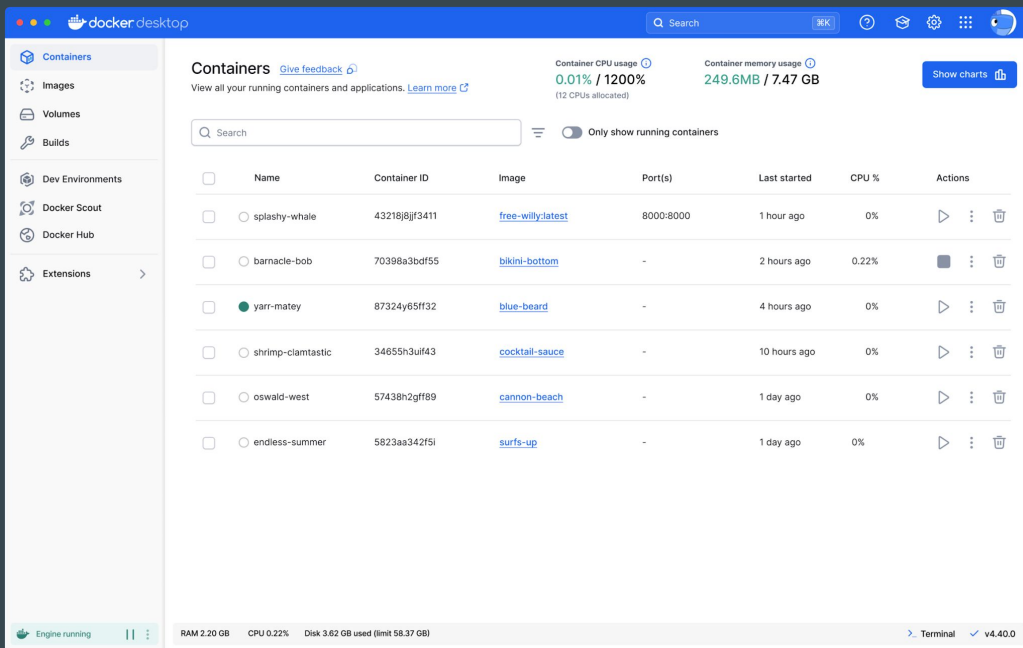
Setting the Base

Docker is supported on Linux, macOS, and Windows, across a wide range of distributions and versions.



Preferred Installation Method

For local development, **Docker Desktop** is the recommended way to get started on your laptop (macOS or Windows).



Production Option (Linux Recommended)

Most organization uses Linux servers extensively.

You will use Docker CLI heavily in those environments.

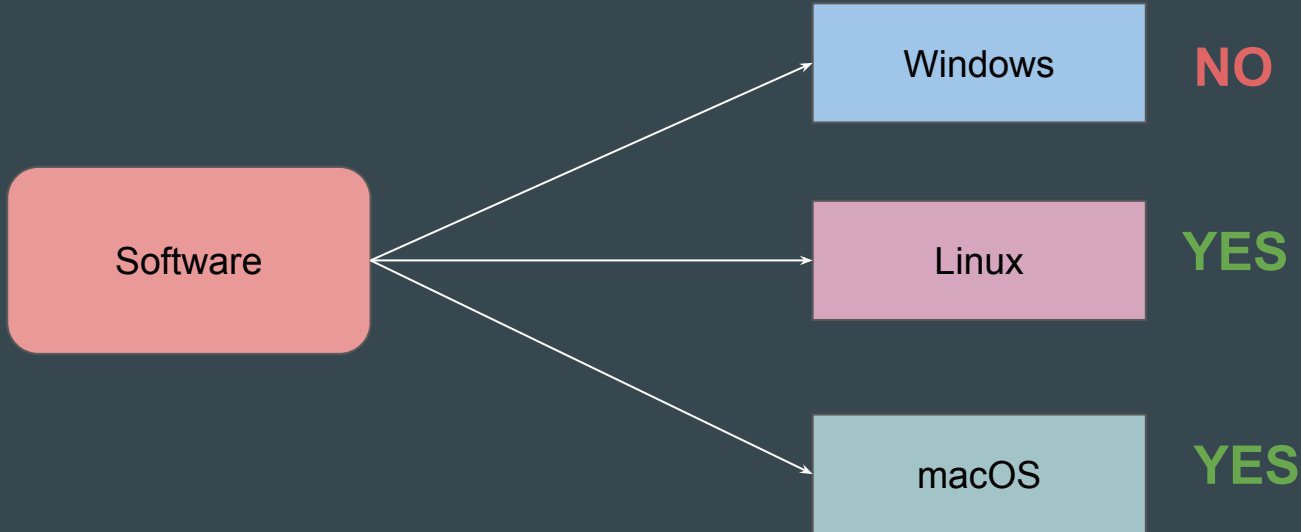


Points to Note

Certain Linux-specific features — such as host networking and privileged mode — behave differently or have limitations on Docker Desktop (macOS/Windows), due to the underlying Linux VM layer.

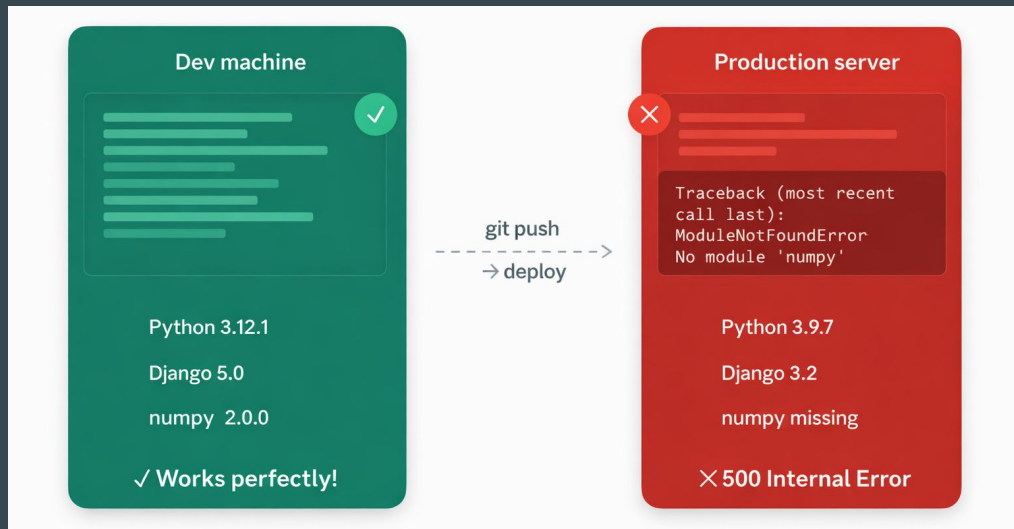
OS Compatibility Issues

Some application will only work on a specific set of operating systems.



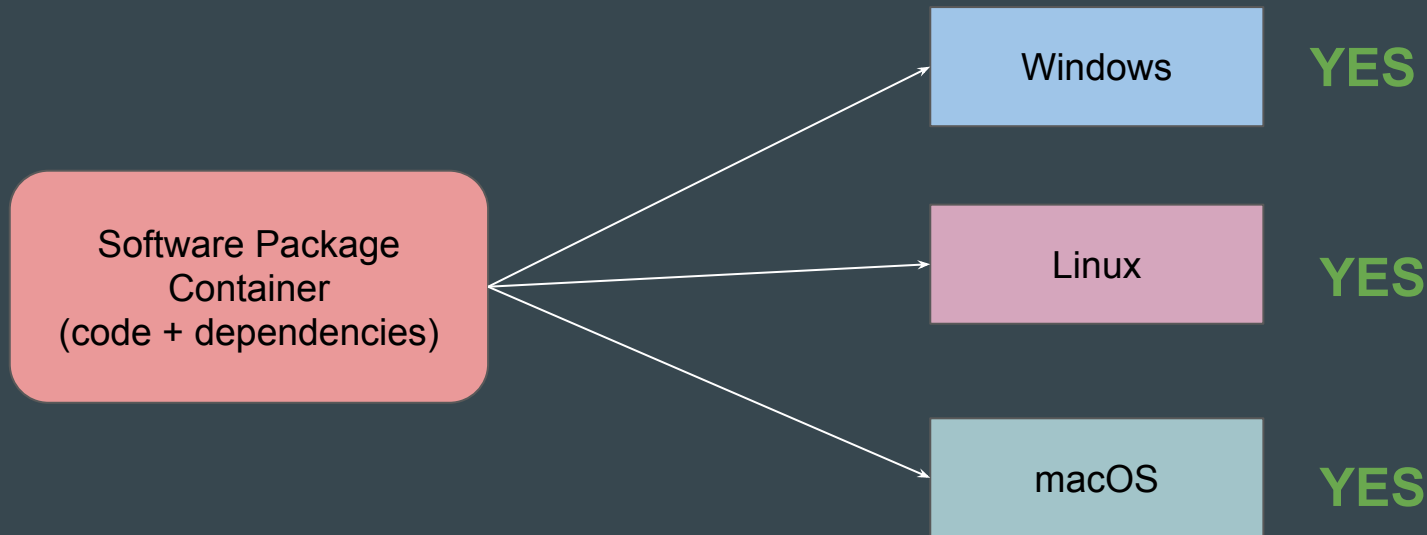
Issue with Consistency

The most famous problem in software is when code works for a developer but breaks in production because the server has a different version of Python, etc.



What Docker is Trying to Achieve

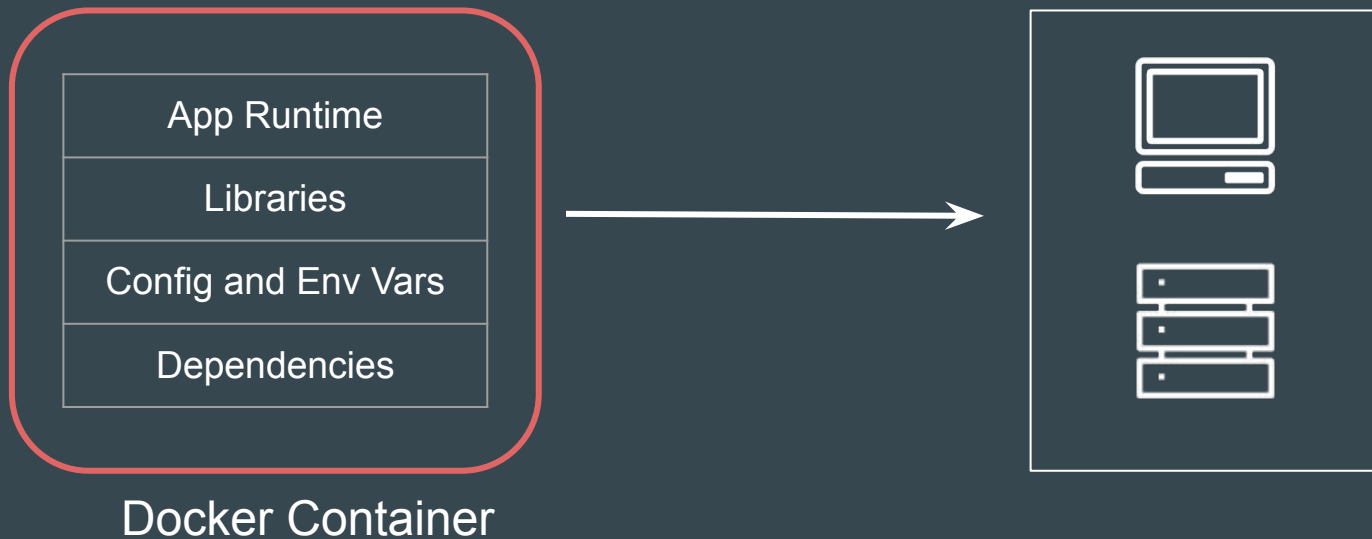
Docker is based on principle of **Build once, run anywhere**



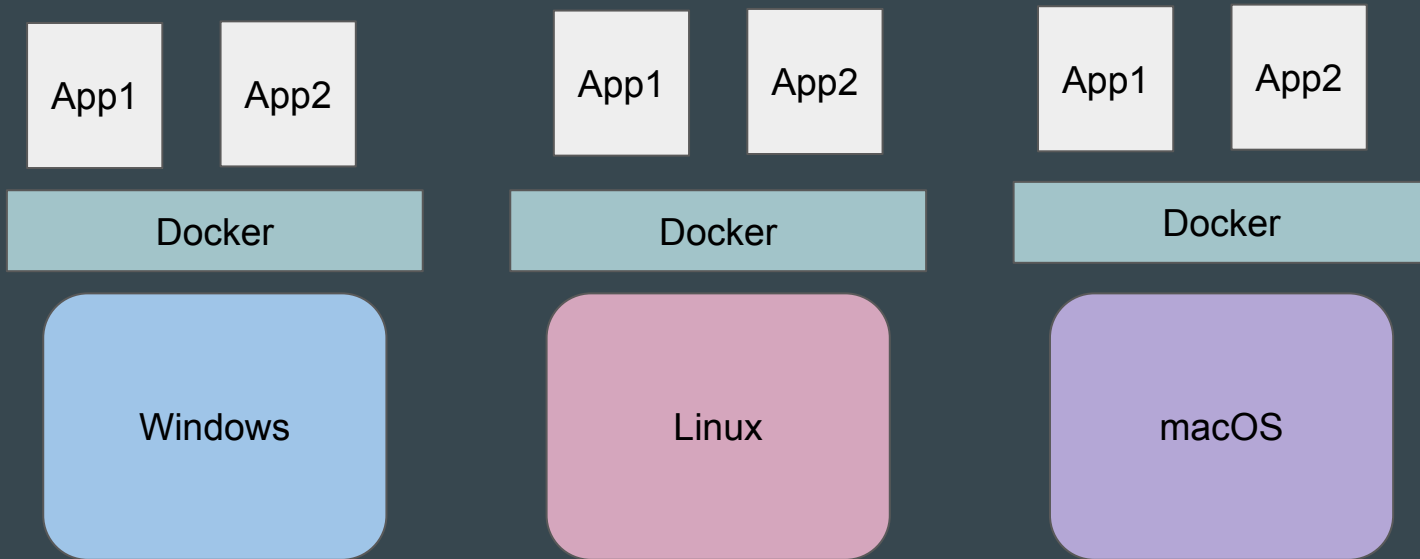
Introduction to Docker

Docker allows users to package applications into containers.

Those containers can then be pushed onto servers, laptops, etc.



High-Level Architecture (Over Simplified)



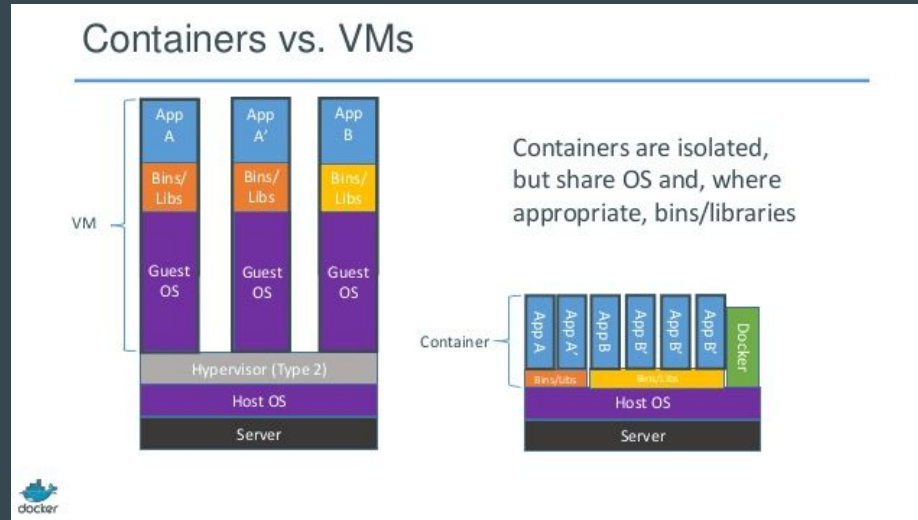
Point to Note

On Windows and macOS, Docker Desktop runs a lightweight Linux virtual machine (via **WSL2 on Windows**, and **Apple's Hypervisor framework on macOS**) to host Linux containers.

Containers vs Virtual Machines

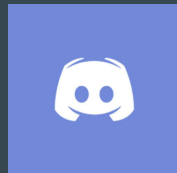
A Virtual Machine contains an entire Operating System

A Container uses the resources of the host operating system.



Join us in our Adventure

Be Awesome



kplabs.in/chat

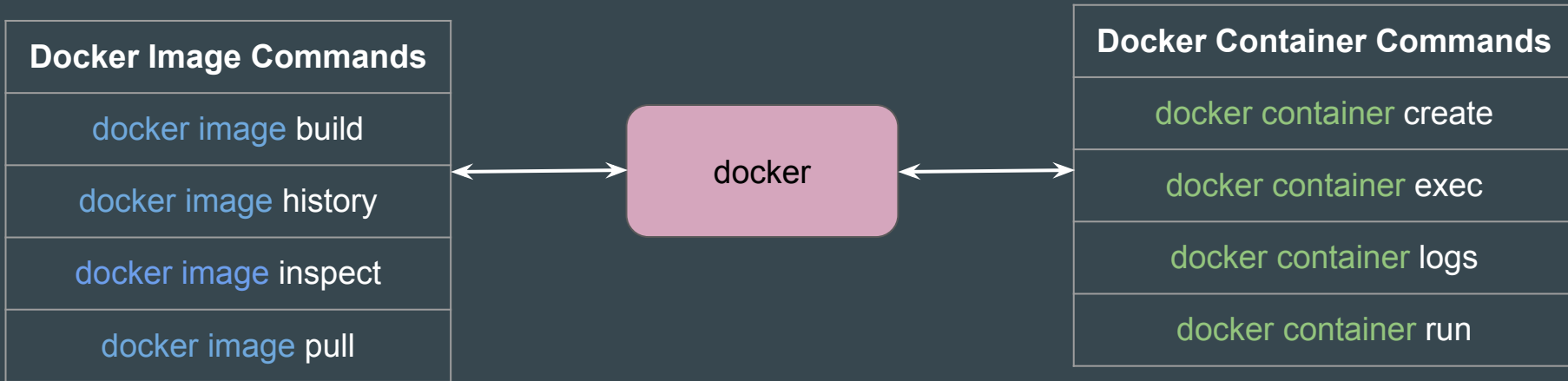


kplabs.in/linkedin

docker - Base Command for Docker CLI

Setting the Base

docker is the base command for Docker CLI



Important Point to Note

Previously, the generic syntax for commands was `docker <command>` (e.g., `docker run`, `docker ps`)

From Docker 1.13 onwards, they **regrouped commands** to sit under the logical object they interact with, such as `docker container run` or `docker container ls`.

Action	Old Classic Syntax	Newer Approach
Run a Container	<code>docker run <image></code>	<code>docker container run <image></code>
List containers	<code>docker ps</code>	<code>docker container ls</code>
Fetch Logs	<code>docker logs <id></code>	<code>docker container logs <id></code>

Point to Note

Both the new syntax (e.g., docker container ls) and the Classic' syntax (e.g., docker ps) are fully supported.

The older commands act as shortcuts to the newer, structured ones.

Docker - Images and Containers

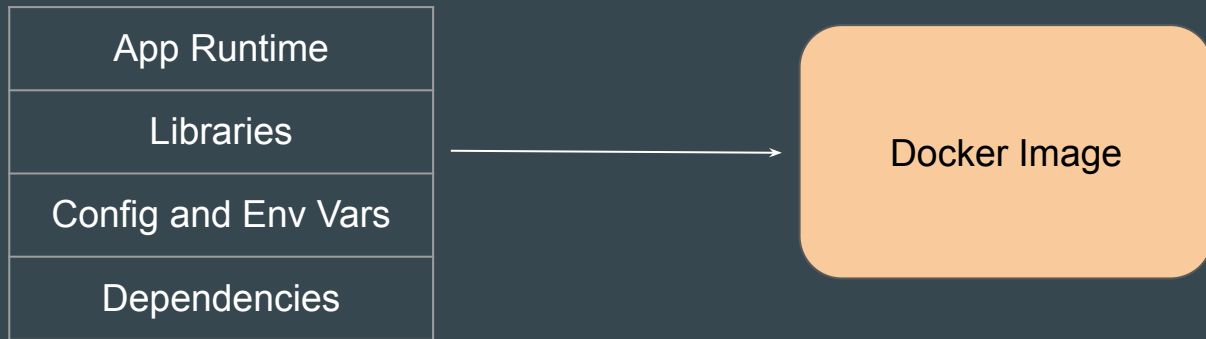
Use-Case: Install an Operating System

Whenever you want to install an Operating System, you need a Image (ISO) of that specific OS.



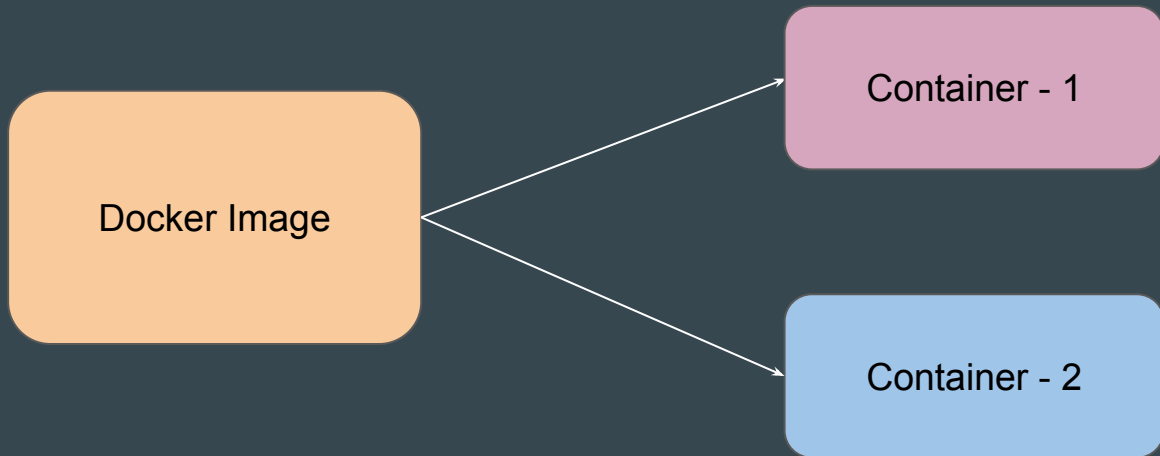
Docker Image

A Docker image is standalone, read-only package that **includes everything needed to run a software application**: the code, runtime, system tools, libraries, and settings.



Docker Container

Docker Containers is a **running instance of an image**.



Points to Note

1. You can launch multiple Containers from Single Docker Image.
2. You can launch multiple containers from different Docker Images.

Practical - Creating our First Container

Setting the Base

To create a new Docker container, you can use the **docker container run** command.

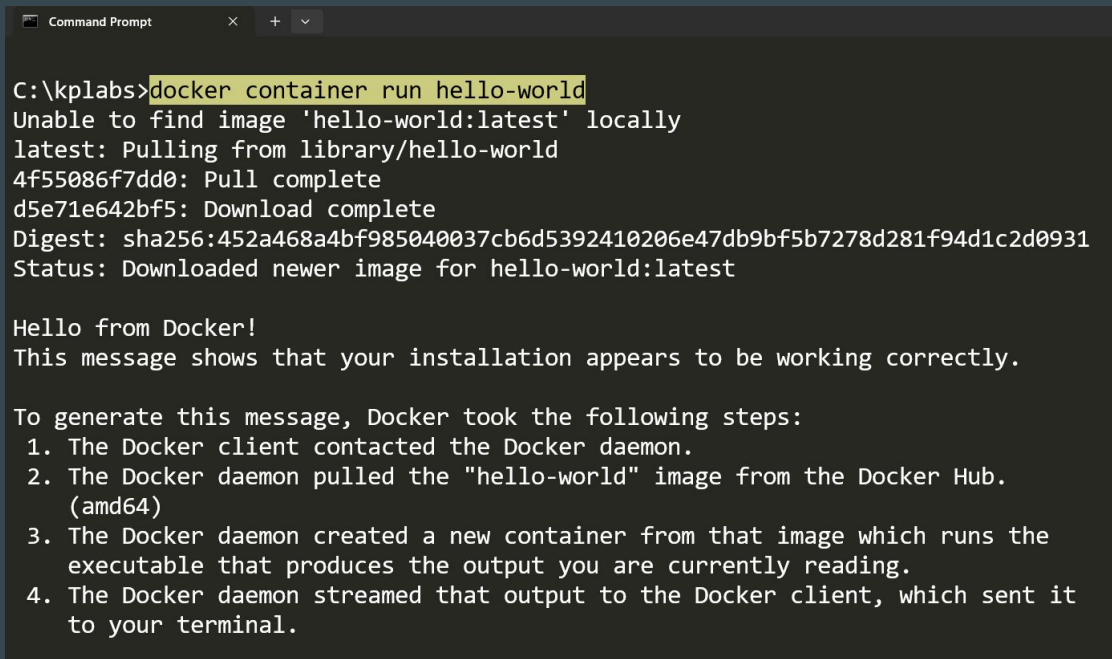


```
C:\kplabs>docker container run nginx
```

```
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/04/05 16:55:14 [notice] 1#1: using the "epoll" event method
2026/04/05 16:55:14 [notice] 1#1: nginx/1.29.5
2026/04/05 16:55:14 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/04/05 16:55:14 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/04/05 16:55:14 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/04/05 16:55:14 [notice] 1#1: start worker processes
2026/04/05 16:55:14 [notice] 1#1: start worker process 29
2026/04/05 16:55:14 [notice] 1#1: start worker process 30
2026/04/05 16:55:14 [notice] 1#1: start worker process 31
2026/04/05 16:55:14 [notice] 1#1: start worker process 32
2026/04/05 16:55:14 [notice] 1#1: start worker process 33
2026/04/05 16:55:14 [notice] 1#1: start worker process 34
2026/04/05 16:55:14 [notice] 1#1: start worker process 35
```


Important Point to Note

If the necessary Docker image is not available locally, Docker will automatically pull the latest version of the image from Docker Hub.

A screenshot of a Windows Command Prompt window. The title bar says 'Command Prompt'. The command prompt shows the command 'C:\kplabs>docker container run hello-world' being entered. The output of the command is displayed below the command line. It shows that Docker was unable to find the 'hello-world:latest' image locally, so it pulled it from the Docker Hub library. The pull is complete, and the image is downloaded. The output ends with 'Hello from Docker!' and a message stating that the installation appears to be working correctly. Below this, a list of steps is provided, explaining the process from contacting the Docker daemon to streaming the output to the terminal.

```
Command Prompt
C:\kplabs>docker container run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:452a468a4bf985040037cb6d5392410206e47db9bf5b7278d281f94d1c2d0931
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

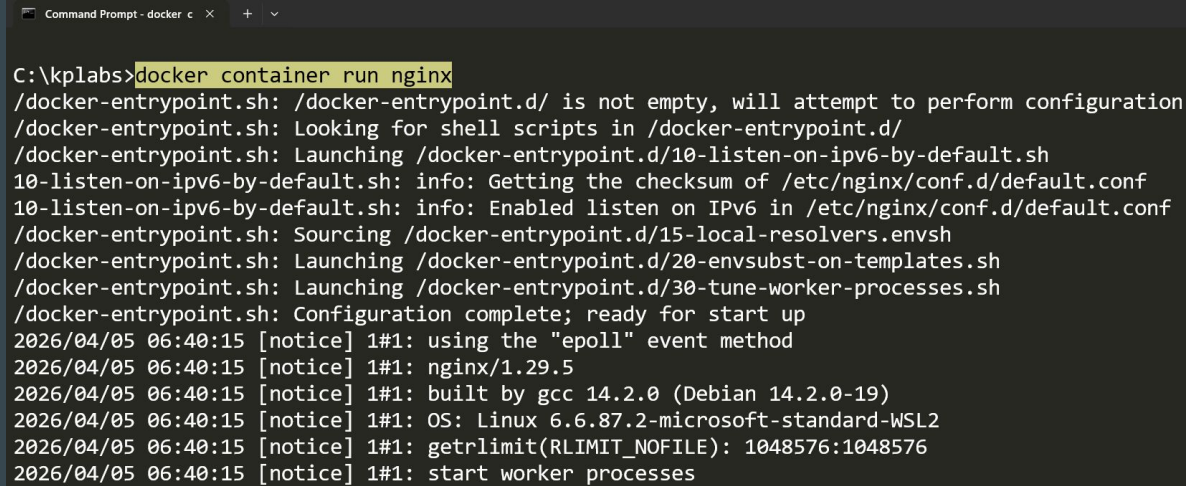
To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.
```

Containers - Attached vs Detached Modes

Attached Mode (Default)

When you use the docker container run <image> without any extra flags, Docker starts the container in Attached Mode.

In this mode, your terminal's local standard input, output, and error streams are "attached" to the container.



```
Command Prompt - docker c  X + v
C:\kplabs>docker container run nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/04/05 06:40:15 [notice] 1#1: using the "epoll" event method
2026/04/05 06:40:15 [notice] 1#1: nginx/1.29.5
2026/04/05 06:40:15 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/04/05 06:40:15 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/04/05 06:40:15 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/04/05 06:40:15 [notice] 1#1: start worker processes
```

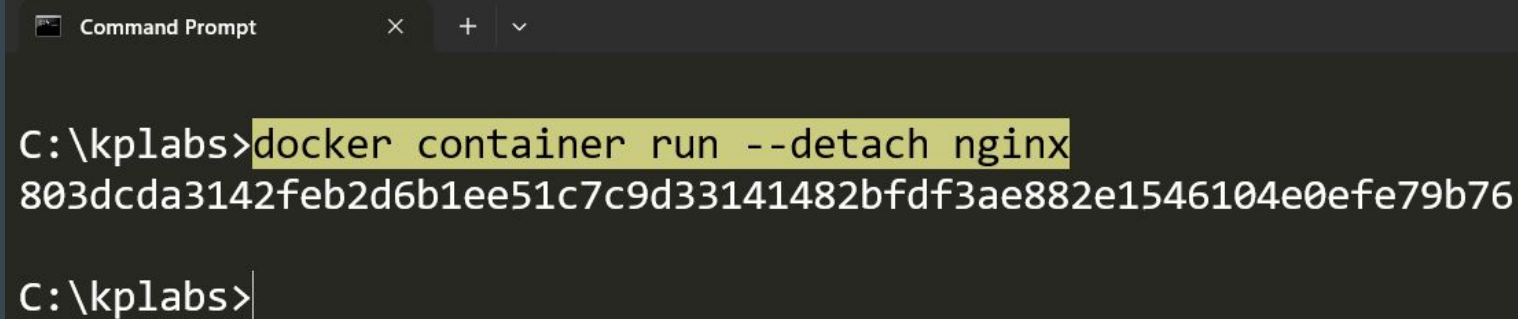
Point to Note - Attached Mode

If you press Ctrl + C in the terminal, you send a SIGINT signal to the container's primary process, which usually stops the container entirely.

Detached Mode

To run a container in the background, you use the `-d` or `--detach` flag.

This is the standard for any long-running service.



```
Command Prompt
C:\kplabs>docker container run --detach nginx
803dcda3142feb2d6b1ee51c7c9d33141482bdfd3ae882e1546104e0efe79b76
C:\kplabs>
```

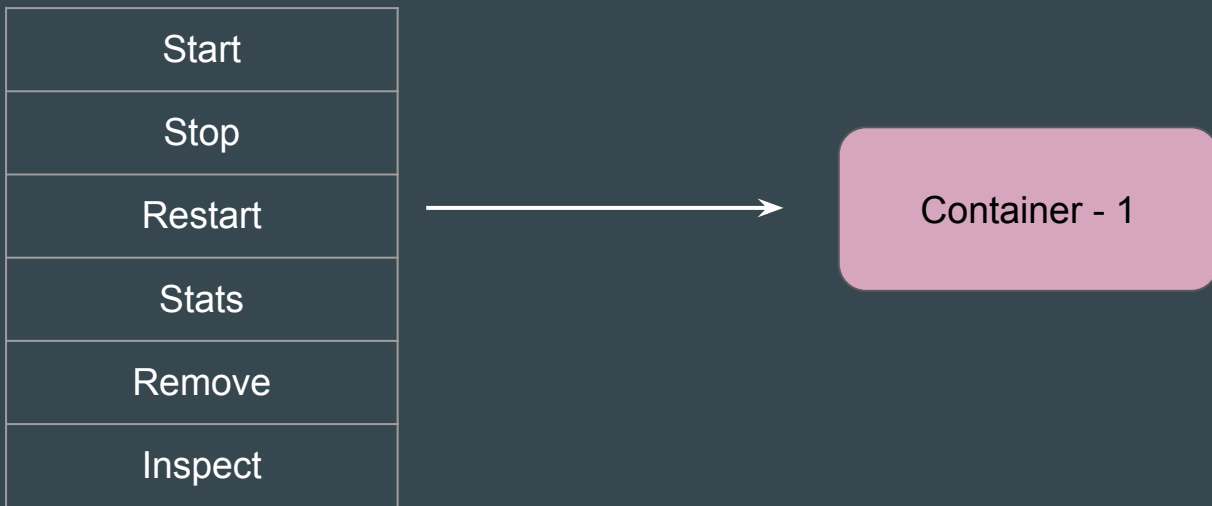
Comparison Table

Feature	Attached Mode	Detached Mode
Flag	None (Default)	-d or --detach
Terminal Control	Occupied by Container	Immediate return to shell
Logs	Streamed live to terminal	Hidden (accessible via docker logs)
Ctrl + C	Usually stops/kills the container	N/A (Doesn't affect the container)
Best For	Debugging, interactive CLI tools	Production services, DBs, Web servers

Basic Container Management Commands

Setting the Base

You can perform multiple operations on a container. Let's look at some common ones.



Managing State (Stop, Start, Restart)



Monitoring Performance

```
docker container stats <name>
```



Container - 1

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
4c844f4d967b	nginx-01	0.00%	24.48MiB / 45.87GiB	0.05%	1.68kB / 126B	0B / 12.3kB	33

Inspect (Detailed Information of Container)

`docker container inspect <name>`



Container - 1

```
Command Prompt
C:\kplabs>docker container inspect nginx-01
[
  {
    "Id": "4c844f4d967bc6352c587199538a52bf472c2e2b1c71b0ad4c3d2ded96169586",
    "Created": "2026-04-05T16:47:46.529729461Z",
    "Path": "/docker-entrypoint.sh",
    "Args": [
      "nginx",
      "-g",
      "daemon off;"
    ],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 11329,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2026-04-05T16:47:46.568218516Z",
      "FinishedAt": "0001-01-01T00:00:00Z"
    }
  },
]
```

Remove a Container

`docker container rm <name>`



~~Container - 1~~

```
Command Prompt
C:\kplabs>docker container rm objective_perlman
objective_perlman
```

Revision Slide

Commands	Description
docker container stop	Gracefully stop a running container
docker container start	Start a stopped container
docker container restart	Stop + start in one step
docker container inspect	View full container metadata
docker container stats	Monitor CPU & memory usage
docker container rm	Remove a Container

docker container exec

Setting the Base

`docker exec` lets you run a command inside an already running container.



`docker container exec nginx ls`



Nginx Container

Command Prompt

```
C:\kplabs>docker container exec nginx ls -l
```

```
total 64
```

lrwxrwxrwx	1	root	root	7	Mar	2	21:50	bin -> usr/bin
drwxr-xr-x	2	root	root	4096	Mar	2	21:50	boot
drwxr-xr-x	5	root	root	340	Apr	6	04:04	dev
drwxr-xr-x	1	root	root	4096	Mar	24	22:12	docker-entrypoint.d
-rwxr-xr-x	1	root	root	1620	Mar	24	22:11	docker-entrypoint.sh
drwxr-xr-x	1	root	root	4096	Apr	6	04:04	etc
drwxr-xr-x	2	root	root	4096	Mar	2	21:50	home
lrwxrwxrwx	1	root	root	7	Mar	2	21:50	lib -> usr/lib
lrwxrwxrwx	1	root	root	9	Mar	2	21:50	lib64 -> usr/lib64
drwxr-xr-x	2	root	root	4096	Mar	16	00:00	media
drwxr-xr-x	2	root	root	4096	Mar	16	00:00	mnt
drwxr-xr-x	2	root	root	4096	Mar	16	00:00	opt
dr-xr-xr-x	438	root	root	0	Apr	6	04:04	proc
drwx-----	2	root	root	4096	Mar	16	00:00	root

docker container exec - Importance of -i and -t flags

Setting the Base

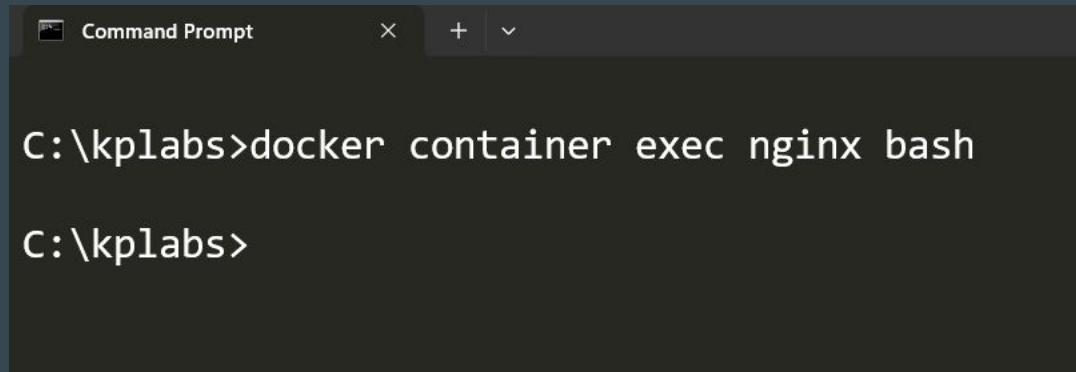
For simple commands that just print output, you don't need any flags. The command runs, prints its result to your terminal, and exits. No interaction needed.

```
Command Prompt
C:\kplabs>docker container exec nginx ls -l
total 64
lrwxrwxrwx    1 root root      7 Mar  2 21:50 bin -> usr/bin
drwxr-xr-x    2 root root 4096 Mar  2 21:50 boot
drwxr-xr-x    5 root root  340 Apr  6 04:04 dev
drwxr-xr-x    1 root root 4096 Mar 24 22:12 docker-entrypoint.d
-rwxr-xr-x    1 root root 1620 Mar 24 22:11 docker-entrypoint.sh
drwxr-xr-x    1 root root 4096 Apr  6 04:04 etc
drwxr-xr-x    2 root root 4096 Mar  2 21:50 home
lrwxrwxrwx    1 root root      7 Mar  2 21:50 lib -> usr/lib
lrwxrwxrwx    1 root root      9 Mar  2 21:50 lib64 -> usr/lib64
drwxr-xr-x    2 root root 4096 Mar 16 00:00 media
drwxr-xr-x    2 root root 4096 Mar 16 00:00 mnt
drwxr-xr-x    2 root root 4096 Mar 16 00:00 opt
dr-xr-xr-x 438 root root      0 Apr  6 04:04 proc
drwx-----  2 root root 4096 Mar 16 00:00 root
```

Understanding the Challenge

If you want to connect inside a container using shell like bash, sh (similar to SSH), the default approach will not work. Bash will close immediately

Bash closes because it is an interactive shell that detects there is no input stream (STDIN) to read from. Without an active input stream or a terminal, it assumes its job is done and exits immediately.

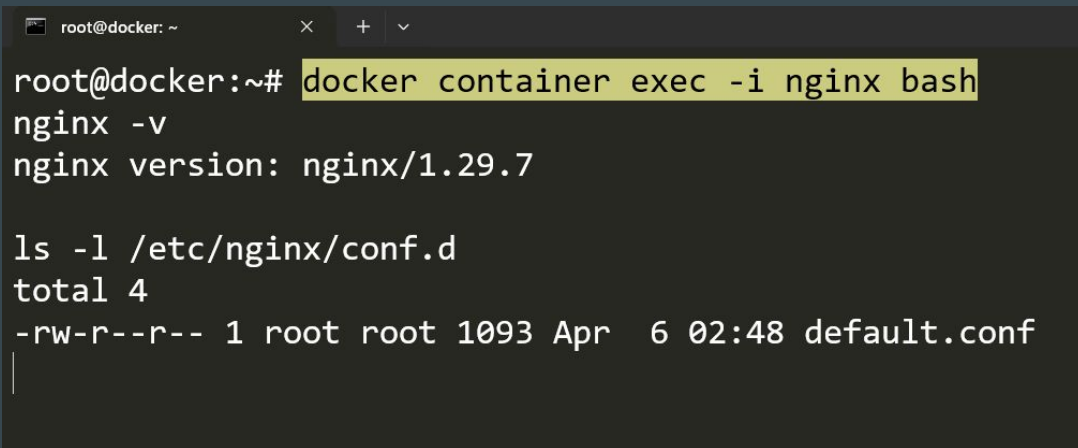
A screenshot of a Windows Command Prompt window. The title bar reads 'Command Prompt' with standard window controls. The command prompt shows the user at 'C:\kplabs' typing 'docker container exec nginx bash'. The prompt returns to 'C:\kplabs>' without any output, indicating the command failed or the process exited immediately.

```
C:\kplabs>docker container exec nginx bash  
C:\kplabs>
```

The -i Flag (Interactive)

By default, Docker runs commands in non-interactive mode and does not keep the input stream (STDIN) open.

-i keeps STDIN attached to the container process, which is necessary for any interactive process.

A terminal window with a dark background and light text. The window title is 'root@docker: ~'. The prompt is 'root@docker:~#'. The command 'docker container exec -i nginx bash' is entered and highlighted in yellow. The output shows 'nginx -v' followed by 'nginx version: nginx/1.29.7'. Then 'ls -l /etc/nginx/conf.d' is entered, followed by 'total 4' and a file listing: '-rw-r--r-- 1 root root 1093 Apr 6 02:48 default.conf'.

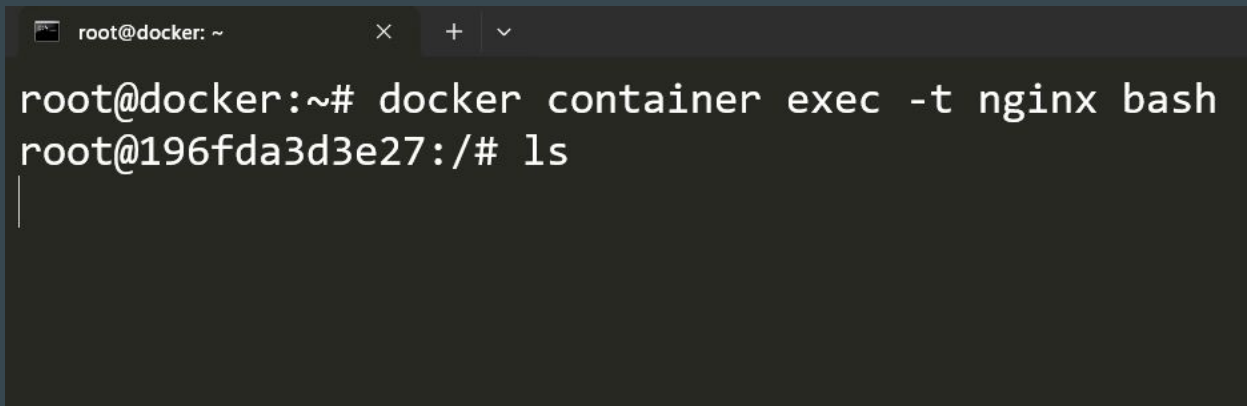
```
root@docker: ~
root@docker:~# docker container exec -i nginx bash
nginx -v
nginx version: nginx/1.29.7

ls -l /etc/nginx/conf.d
total 4
-rw-r--r-- 1 root root 1093 Apr 6 02:48 default.conf
```

The -t Flag (TTY)

In Docker, -t allocates a pseudo-terminal.

If you just use -t flag, the STDIN is closed, so the keystrokes have no path to reach the process inside the container

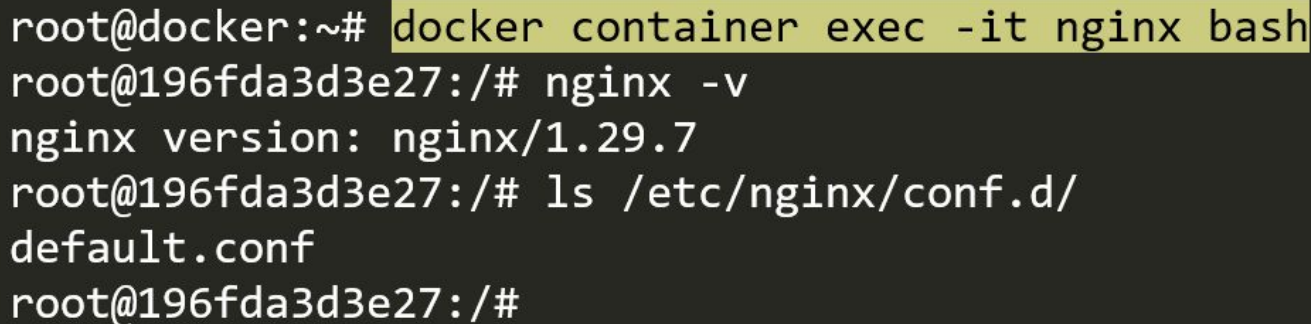
A terminal window with a dark background. The title bar shows 'root@docker: ~' and window control buttons. The terminal text shows a successful execution of 'docker container exec -t nginx bash', resulting in a shell prompt 'root@196fda3d3e27:/#'. The 'ls' command has been entered, and a vertical cursor is visible on the next line.

```
root@docker: ~  
root@docker:~# docker container exec -t nginx bash  
root@196fda3d3e27:/# ls  
|
```

Combination is good (-it)

Using the -it option, users can get a full interactive shell

You can type, navigate, and explore.

A terminal window with a dark background. The title bar shows 'root@docker: ~' and window controls. The terminal text shows a sequence of commands and outputs: 'docker container exec -it nginx bash' is highlighted in yellow; the prompt changes to 'root@196fda3d3e27:/#'; 'nginx -v' is entered and outputs 'nginx version: nginx/1.29.7'; 'ls /etc/nginx/conf.d/' is entered and outputs 'default.conf'; the prompt returns to 'root@196fda3d3e27:/#'.

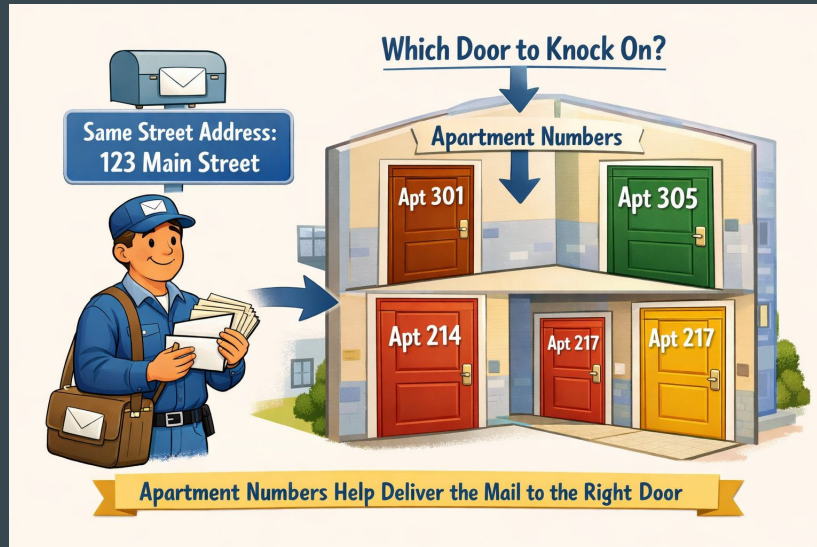
```
root@docker:~# docker container exec -it nginx bash
root@196fda3d3e27:/# nginx -v
nginx version: nginx/1.29.7
root@196fda3d3e27:/# ls /etc/nginx/conf.d/
default.conf
root@196fda3d3e27:/#
```

Basics of Ports

Simple Use-Case

Imagine you live in a massive apartment complex.

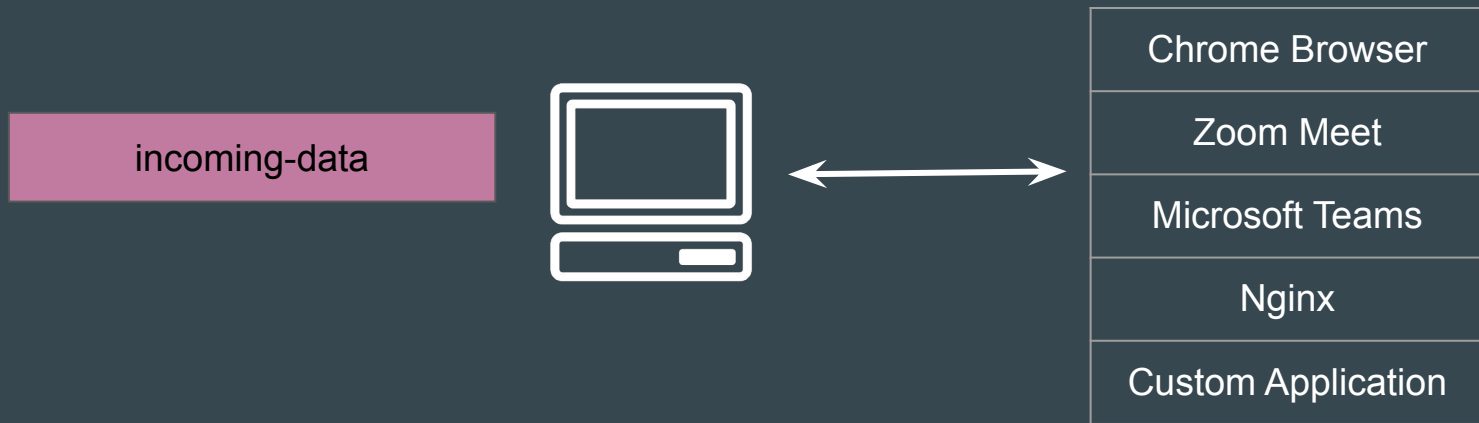
Every resident has the same street address, but how does the mail carrier know which door to knock on? They use apartment numbers.



Setting the Basics

When data arrives at your computer or a server, the operating system needs to know which program should receive it.

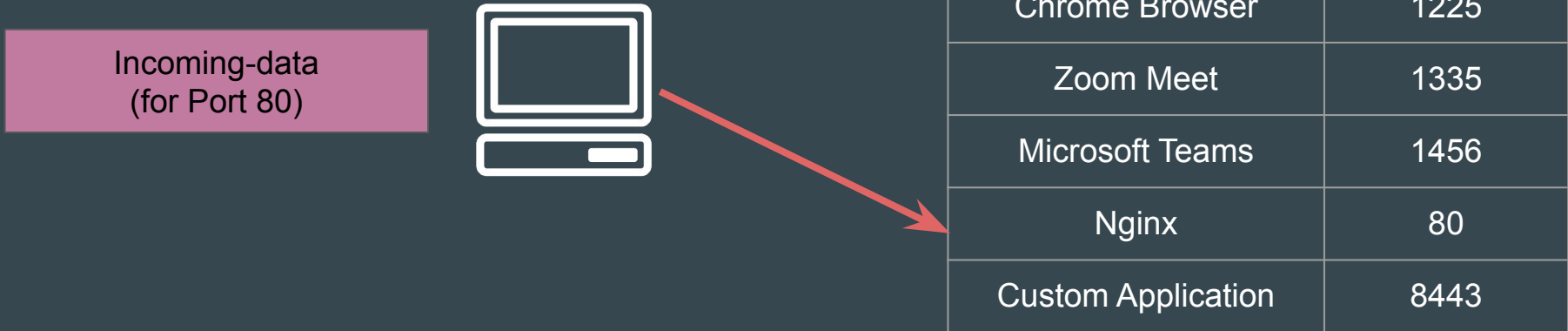
Is it for Chrome? Zoom? A Custom Application?



Basics of Ports

Ports help the computer figure out which application should receive incoming data.

Application can listen on a specific Port Number (**Port Binding**)



Points to Note

Ports are numbers from 0 to 65,535

Only one process can bind to a specific port on the same IP address at a time

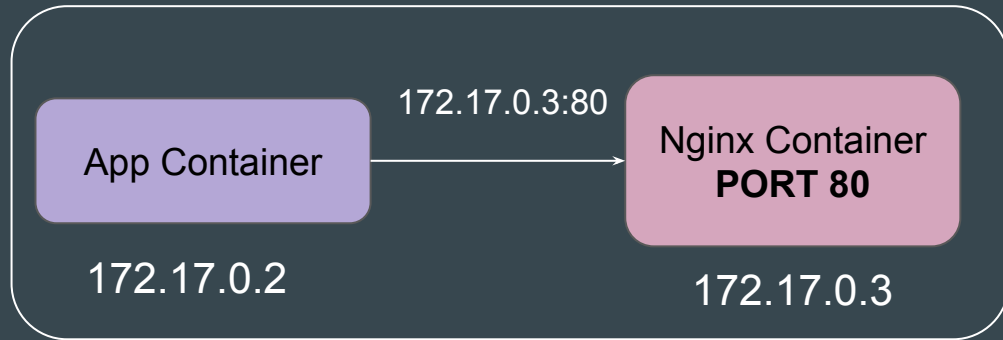


Publishing Ports

Setting the Base

When a container is started, it is assigned a unique IP address within its configured Docker network. The application running inside the container may listen on a specific port.

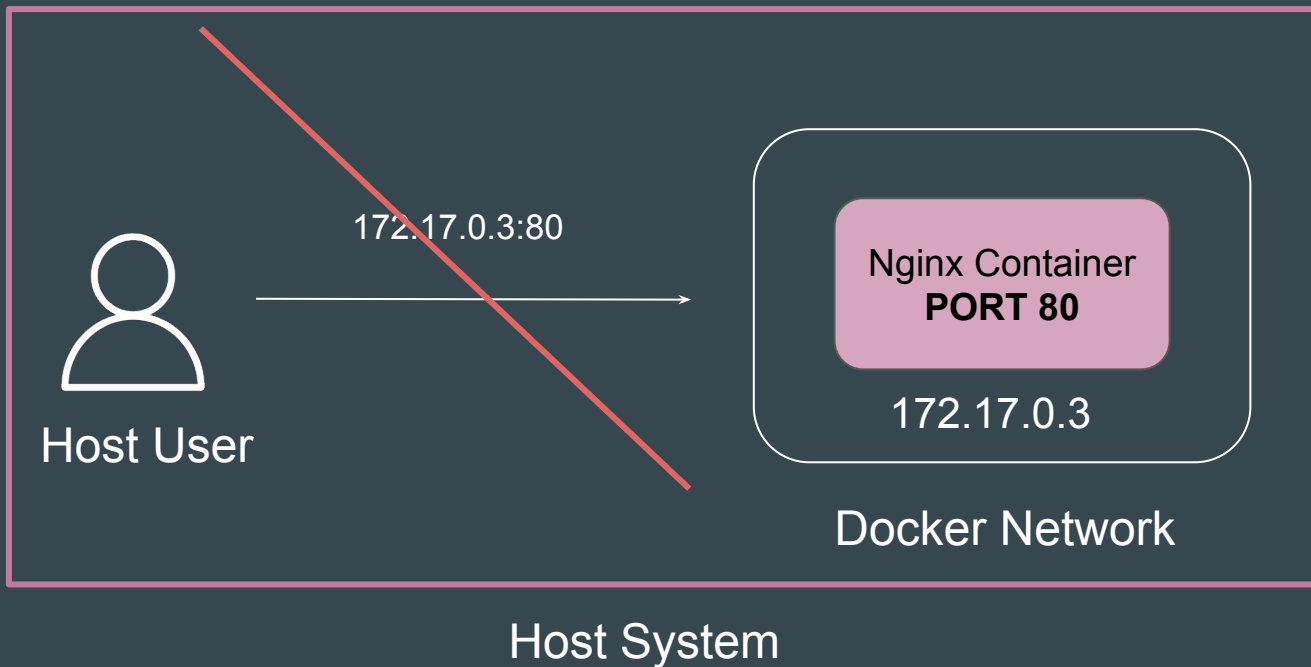
Containers connected to the same network can communicate with each other using the container's IP address and the application's listening port



Docker Network

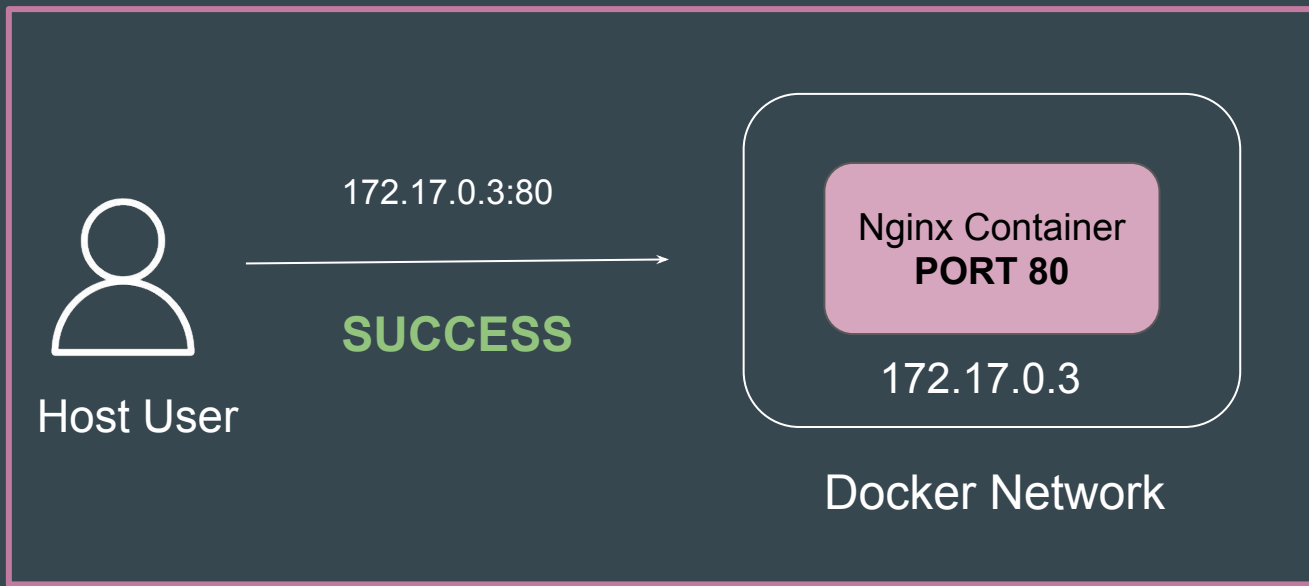
Point to Note (Docker Desktop)

The internal Docker network IP addresses are not directly accessible from the host system (especially on Windows or macOS).



Point to Note - Linux Setup

On Linux hosts, the Docker bridge network is directly reachable from the host via container IP.



Host System

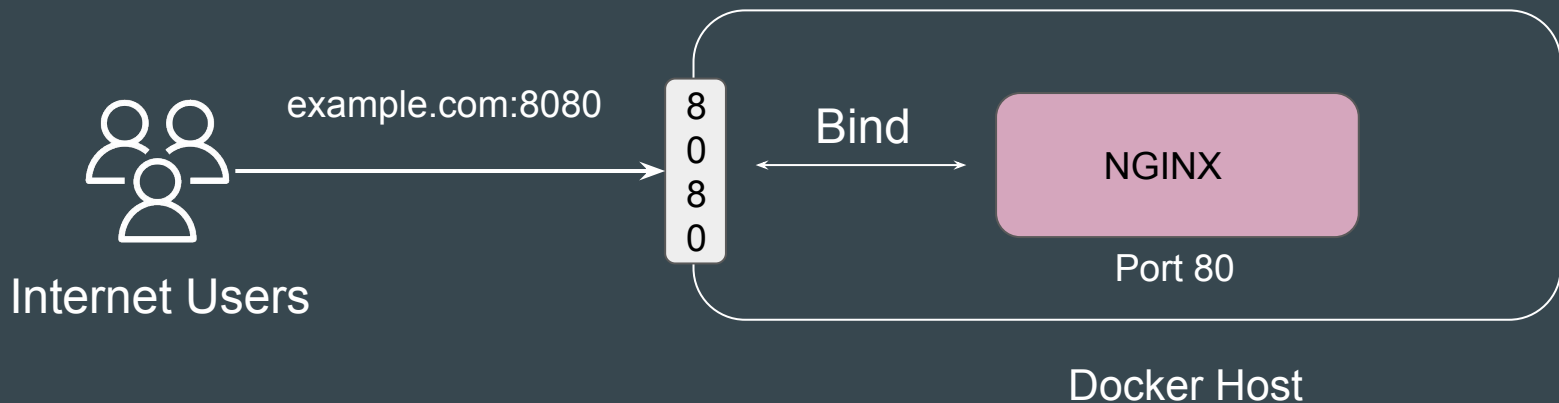
Ports Concepts

When we talk about ports in Docker, we are always talking about two different locations.

Host Port (Outside)	The port you type into your browser (e.g., localhost:8080).
Container Port (Inside)	The port the actual application is configured to use (e.g., 80).

Connecting to Container Application

To allow a container to accept incoming connections from the host or the outside world, you must bind (map) it to a host port.



Reference Point

In a containerized environment, the container has its own internal network.

When you "map" or "bind" a port, you are essentially telling the host computer:

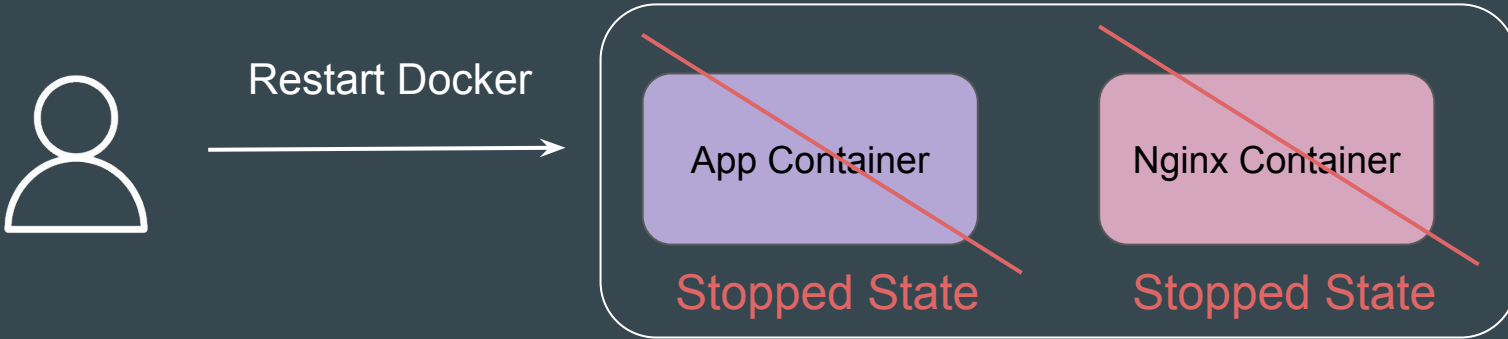
"Any traffic that arrives at Host Port X should be forwarded to Container Port Y."

Docker Restart Policies

Setting the Base

By default, Docker containers will not start when they exit or when docker daemon is restarted.

Docker provides restart policies to control whether your containers start automatically when they exit, or when Docker restarts.



Restart Policies

You can specify the restart policy by using the `--restart` flag with docker run command.

Restart Policy	Description
no	The default. Docker will not restart the container under any circumstance.
on-failure	Restarts only if the container exits with a non-zero exit code.
unless-stopped	Always restarts except when you manually stop the container. If manually stopped, it won't restart even after a daemon restart.
always	Always restarts regardless of exit code, including after a daemon restart — even if the container was manually stopped before the daemon went down.

Choosing the Right Policy

Policy	Restarts on crash?	Restarts on success exit?	Restarts after daemon restart?	Best for
no	No	No	No	Development, testing
always	Yes	Yes	Yes	Critical services, web servers
on-failure	Yes	No	Only if it was running	Batch jobs, scripts
unless-stopped	Yes	Yes	Only if not manually stopped	Databases, background services

Working with Docker Images

Build once, use anywhere

Getting Started with Images

Every Docker container is based on an image.

Till now we have been using images which were created by others and available in Docker Hub.

Docker can build images automatically by reading the instructions from a **Dockerfile**

Understanding Dockerfile

A **Dockerfile** is a text document that contains all the commands a user could call on the command line to assemble an image.



Dockerfile

Building Docker Images

Understanding Dockerfile

A **Dockerfile** is a text document that contains all the commands a user could call on the command line to assemble an image.



Format of Dockerfile

The format of Dockerfile is similar to the below syntax:

```
# Comment
```

```
INSTRUCTION arguments
```

A Dockerfile must start with a **FROM** instruction.

The FROM instruction specifies the Base Image from which you are building.

Dockerfile Commands

Thee format of Dockerfile is similar to the below syntax:

- FROM
- RUN
- CMD
- LABEL
- EXPOSE
- ENV
- ADD
- COPY
- ENTRYPOINT
- VOLUME
- USER
- Many More ...

Use-Case - Create Custom Nginx Image

Simple Use-Case

We want to create a custom Nginx Image from which all containers within organization will be launched.

The base container image should have custom index.html file which states:

“Welcome to Base Nginx Container from KPLABS”

..

COPY vs ADD

Build once, use anywhere

Overview of COPY and ADD

COPY and ADD are both Dockerfile instructions that serve similar purposes.

They let you copy files from a specific location into a Docker image.

Difference between COPY and ADD

COPY takes in a **src** and **destination**. It only lets you copy in a local file or directory from your host

ADD lets you do that too, but it also supports 2 other sources.

- First, you can use a URL instead of a local file / directory.
- Secondly, you can extract a tar file from the source directly into the destination.

Use CURL and WGET whenever possible

Using ADD to fetch packages from remote URLs is strongly discouraged; you should use curl or wget instead.

```
ADD http://example.com/big.tar.xz /usr/src/things/
```

```
RUN tar -xJf /usr/src/things/big.tar.xz -C /usr/src/things
```

```
RUN make -C /usr/src/things all
```

```
RUN mkdir -p /usr/src/things \
```

```
&& curl -SL http://example.com/big.tar.xz \
```

```
| tar -xJC /usr/src/things \
```

```
&& make -C /usr/src/things all
```

EXPOSE

Build once, use anywhere

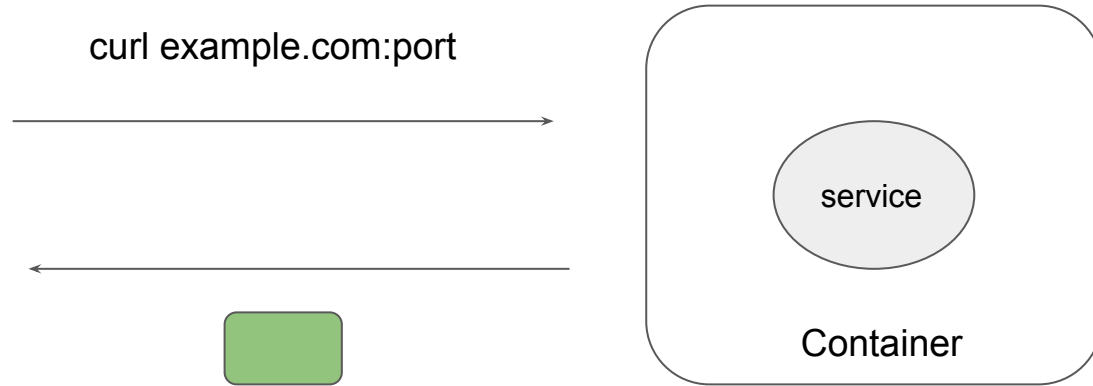
Overview of EXPOSE instruction

The EXPOSE instruction informs Docker that the container listens on the specified network ports at runtime.

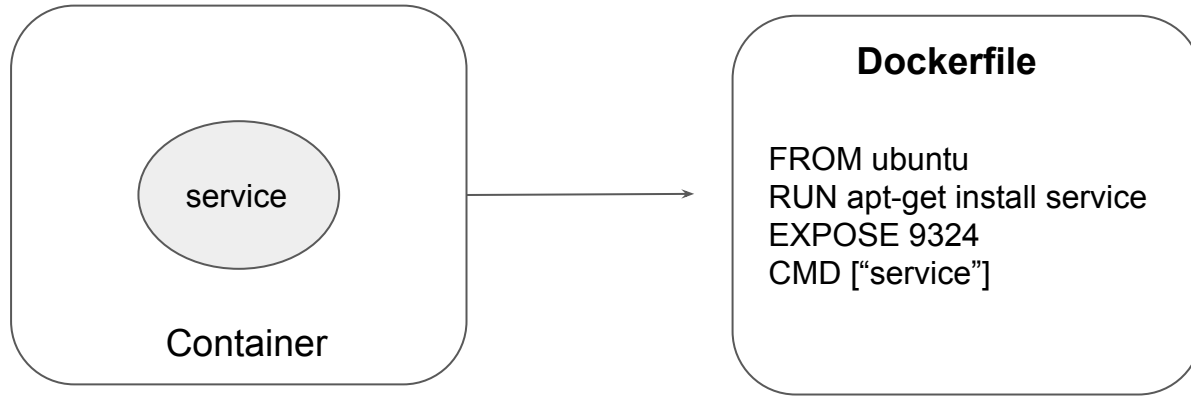
The EXPOSE instruction does not actually publish the port.

It functions as a type of documentation between the person who builds the image and the person who runs the container, about which ports are intended to be published.

Understanding the Use-Case



Understanding the Use-Case



HEALTHCHECK Options

Docker HealthCheck

Overview of Instruction Options

HEALTHCHECK Instruction is combined with various options to get the desired results.

Options	Default Values
--interval=DURATION	30 seconds
--timeout=DURATION	30 seconds
--start-period=DURATION	0 seconds
--retries=N	3

Understanding the Concept

The health check will first run interval seconds after the container is started, and then again interval seconds after each previous check completes.

So if CMD is `curl -I google.com`

With default values, the health check will run after every 30 seconds.

Exit Status

The command's exit status indicates the health status of the container. The possible values are:

Possible Values	Description
0: Success	the container is healthy and ready for use
1: Failure	the container is not working correctly
2: Reserved	do not use this exit code

Sample Use-Case

Check every five minutes or so that a web-server is able to serve the site's main page within three seconds:

```
HEALTHCHECK --interval=5m --timeout=3s \
```

```
CMD curl -f http://localhost/ || exit 1
```

Overview of CURL

CURL is used with -f option to fail silently.

This is mostly done to better enable scripts etc to better deal with failed attempts.

```
bash-4.2# curl dexter.kplabs.in/test
<html>
<head><title>404 Not Found</title></head>
<body bgcolor="white">
<center><h1>404 Not Found</h1></center>
<hr><center>nginx/1.10.1</center>
</body>
</html>
```

Without -f option

```
bash-4.2# curl -f dexter.kplabs.in/test
curl: (22) The requested URL returned error: 404 Not Found
```

With -f option

Dockerfile - ENTRYPOINT

Build once, use anywhere

Overview of ENTRYPOINT

The best use for ENTRYPOINT is to set the image's main command

ENTRYPOINT doesn't allow you to override the command.

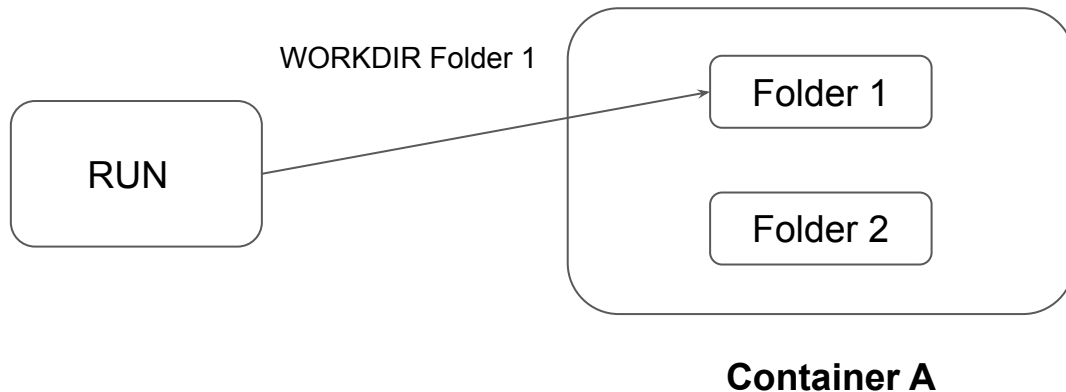
It is important to understand distinction between CMD and ENTRYPOINT.

Dockerfile - WORKDIR

Build once, use anywhere

Overview of the Instruction

The **WORKDIR** instruction sets the working directory for any RUN, CMD, ENTRYPOINT, COPY and ADD instructions that follow it in the Dockerfile



Important Pointer

The WORKDIR instruction can be used multiple times in a Dockerfile

Sample Snippet:

```
WORKDIR /a
```

```
WORKDIR b
```

```
WORKDIR c
```

```
RUN pwd
```

Output = /a/b/c

Dockerfile - ENV

Build once, use anywhere

Overview of the Instruction

The ENV instruction sets the environment variable <key> to the value <value>.

Example Snippet:

```
ENV NGINX 1.2
```

```
RUN curl -SL http://example.com/web-\$NGINX.tar.xz
```

```
RUN tar -xvzf web-\$NGINX.tar.xz
```

Setting Environment Variables from CLI

You can use the `-e`, `--env`, and `--env-file` flags to set simple environment variables in the container you're running, or overwrite variables that are defined in the Dockerfile of the image you're running.

Example Snippet:

```
docker run --env VAR1=value1 --env VAR2=value2 ubuntu env | grep VAR
```

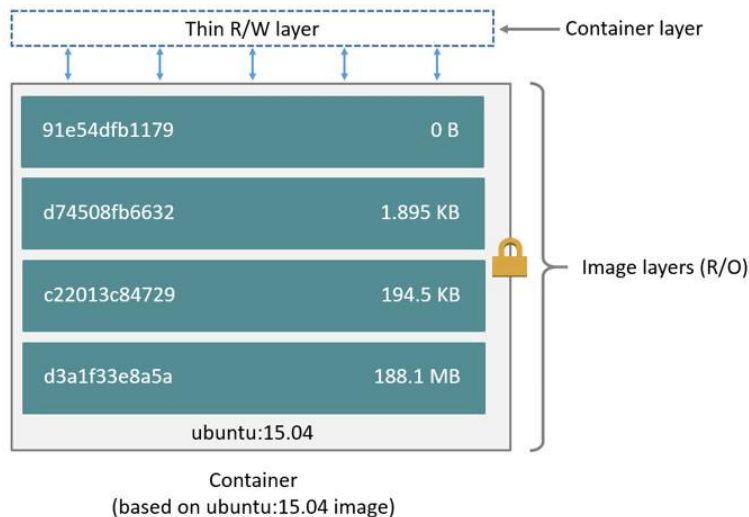
Layers of Docker Image

Building Docker Images

Understanding Layers

- A Docker image is built up from a series of layers.
- Each layer represents an instruction in the image's Dockerfile.

```
FROM ubuntu:15.04  
COPY . /app  
RUN make /app  
CMD python /app/app.py
```



Containers and Layers

The major difference between a container and an image is the top writable layer.

All writes to the container that add new or modify existing data are stored in this writable layer.

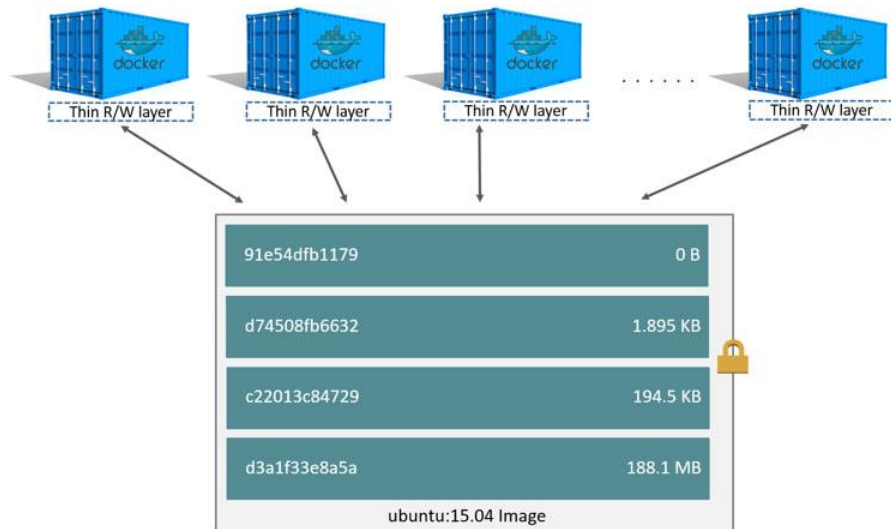
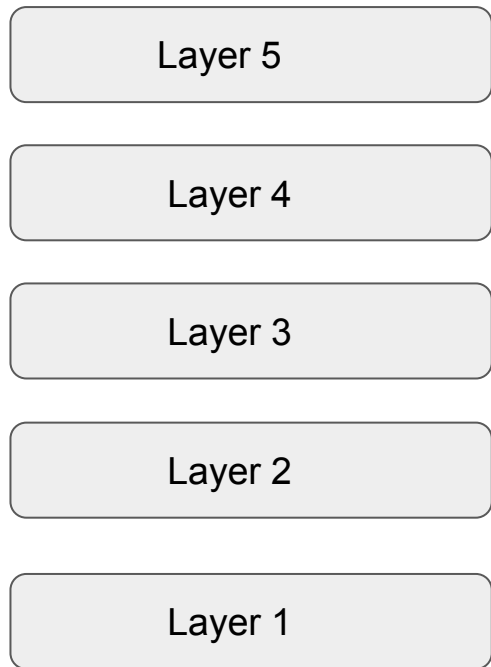


Image and Layer



```
FROM ubuntu
```

```
RUN dd if=/dev/zero of=/root/file1.txt bs=1M count=100
```

```
RUN dd if=/dev/zero of=/root/file2.txt bs=1M count=100
```

```
RUN rm -f /root/file1.txt
```

```
RUN rm -f /root/file2.txt
```

First Three Instructions

```
FROM ubuntu
```

```
RUN dd if=/dev/zero of=/root/file1.txt bs=1M count=100
```

```
RUN dd if=/dev/zero of=/root/file2.txt bs=1M count=100
```

Layer 3



~100 MB

Layer 2



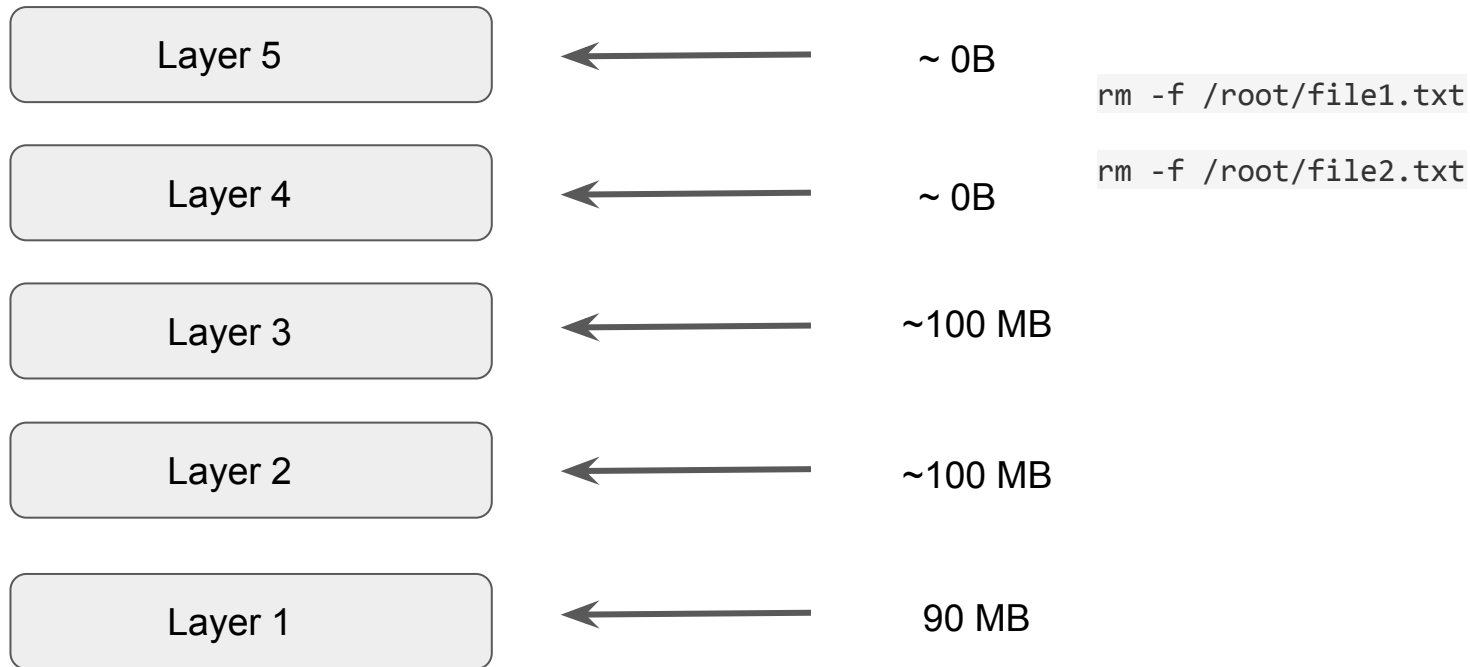
~100 MB

Layer 1



90 MB

ALL Instruction



Managing Images with CLI

Build once, use anywhere

Getting Started

Docker CLI can be used to manage various aspects related to Docker Images which includes building, removing, saving, tagging and others.

We should be familiar with the `docker image child-commands`

Child Commands for Docker Images

As of this date of recording, following child commands are available:

- `docker image build`
- `docker image history`
- `docker image import`
- `docker image inspect`
- `docker image load`
- `docker image ls`
- `docker image prune`
- `docker image pull`
- `docker image push`
- `docker image rm`
- `docker image save`
- `docker image tag`

Inspecting Docker Images

Build once, use anywhere

Getting Started

A Docker Image contains lots of information, some of these include:

- Creation Date
- Command
- Environment Variables
- Architecture
- OS
- Size

`docker image inspect` command allows us to see all the information associated with a docker image.

Docker Image Prune

Build once, use anywhere

Image Pruning Steps

`docker image prune` command allows us to clean up unused images.

By default, the above command will only clean up dangling images.

Dangling Images = Image without Tags and Image not referenced by any container

Modify Image to Single Layer

Build once, use anywhere

Getting Started

In a generic scenerio, the more the layers an image has, the more the size of the image.

Some of the image size goes from 5GB to 10GB.

Flattening an image to single layer can help reduce the overall size of the image.

Docker Registry

Building Docker Registry

Understanding Docker Registry

A [Registry](#) is a stateless, highly scalable server side application that stores and lets you distribute Docker images.

Docker Hub is the simplest example that all of us must have used.

There are various types of registry available, which includes:

- Docker Registry
- Docker Trusted Registry
- Private Repository (AWS ECR)
- Docker Hub

Moving Images Across Hosts

Build once, use anywhere

Example Use-Case

James has created an application based on Docker. He has the image file in his laptop.

He wants to send the image to Matthew over email.



Overview of Docker Save

The `docker save` command will save one or more images to a tar archive

Example Snippet:

```
docker save busybox > busybox.tar
```

Overview of Docker Load

The `docker load` command will load an image from a tar archive

Example Snippet:

```
docker load < busybox.tar
```

Build Cache

Build once, use anywhere

Overview Slide

Docker creates container images using layers.

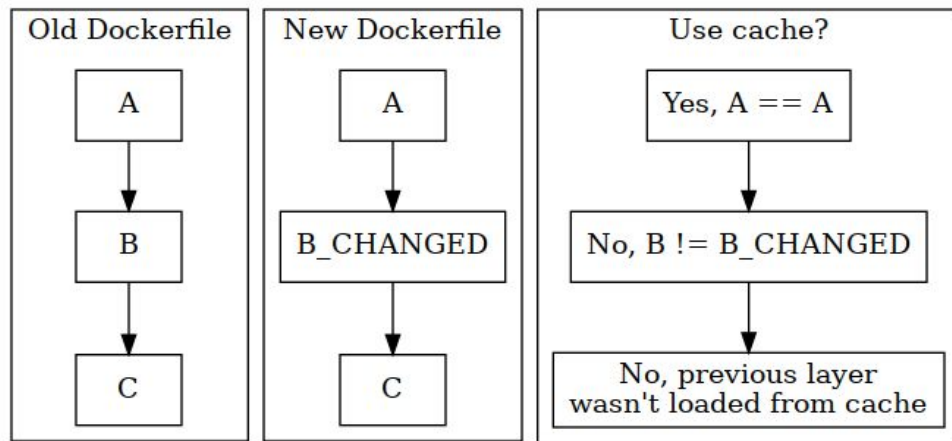
Each command that is found in a Dockerfile creates a new layer.

Docker uses a layer cache to optimize the process of building Docker images and make it faster.

```
[root@swarm02 build-cache]# docker build -t demo2
Sending build context to Docker daemon 3.072kB
Step 1/4 : FROM python:3.7-slim-buster
--> 87b1022604d5
Step 2/4 : COPY . .
--> Using cache
--> fe355c27a8ff
```

Important Pointer

If the cache can't be used for a particular layer, all subsequent layers won't be loaded from the cache.



Overview of Docker Networking

Build once, use anywhere

Getting Started

Docker takes care of the networking aspects so that container can communicate with other containers and also with the Docker Host.

```
[root@docker-demo ~]# ifconfig
docker0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 0.0.0.0
    inet6 fe80::42:b6ff:fea6:f62b prefixlen 64 scopeid 0x20<link>
    ether 02:42:b6:a6:f6:2b txqueuelen 0 (Ethernet)
    RX packets 147774 bytes 42989090 (40.9 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 153893 bytes 273579092 (260.9 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 139.59.82.72 netmask 255.255.240.0 broadcast 139.59.95.255
    inet6 fe80::8838:79ff:feed:1335 prefixlen 64 scopeid 0x20<link>
    ether 8a:38:79:ed:13:35 txqueuelen 1000 (Ethernet)
    RX packets 822197 bytes 2191970977 (2.0 GiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 634774 bytes 44815091 (42.7 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

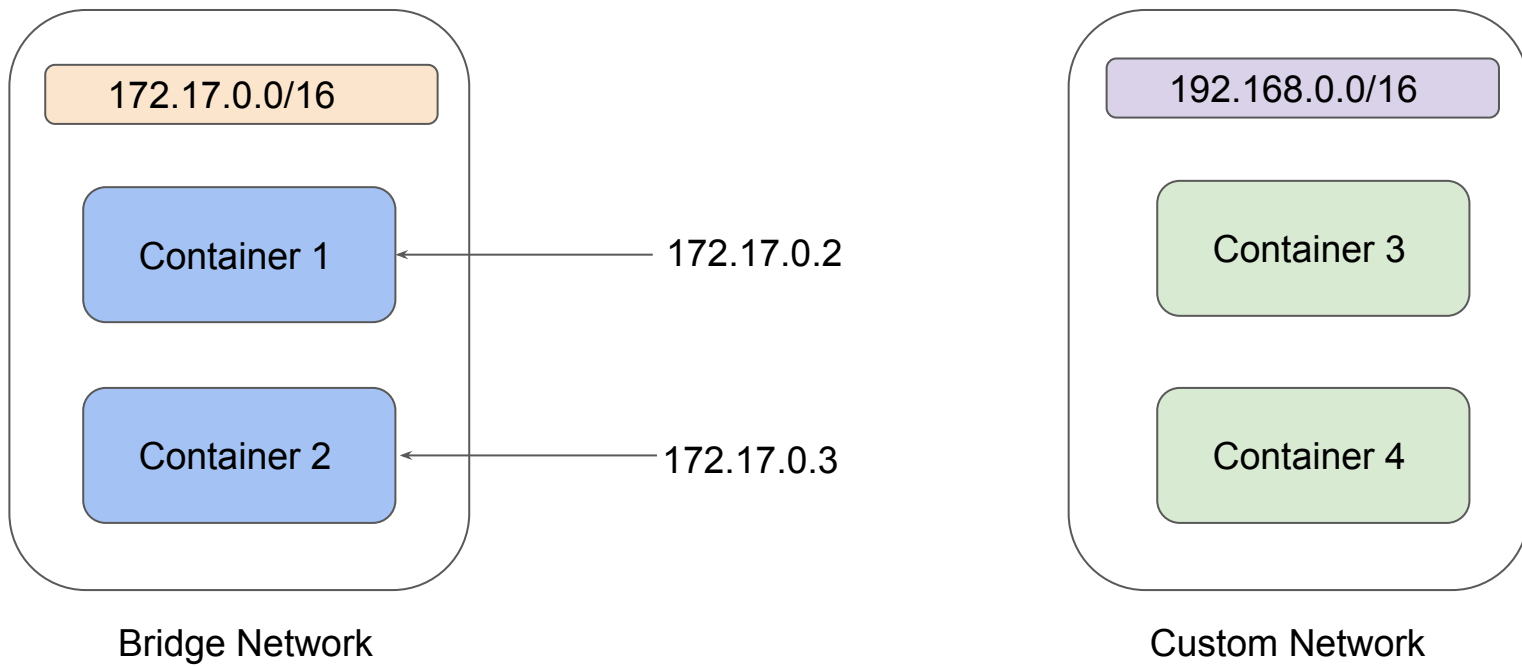
Overview of Network Drivers

Docker networking subsystem is pluggable, using drivers.

There are several drivers available by default, and provides core networking functionality.

- bridge
- host
- overlay
- macvlan
- none

Diagrammatic Representation

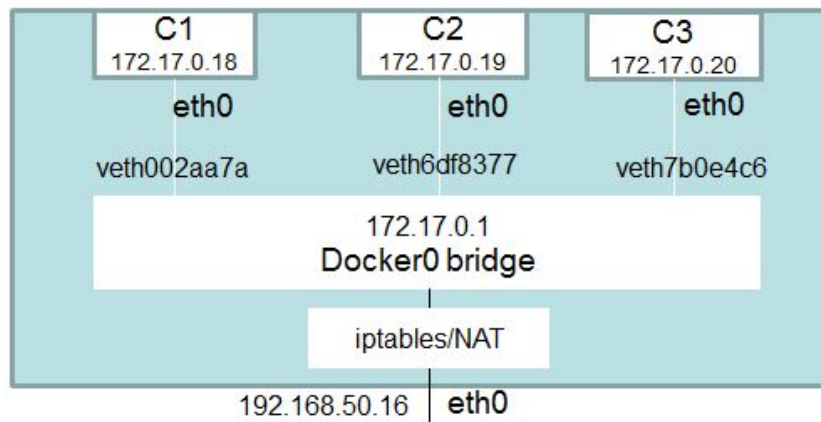


Bridge Network Driver

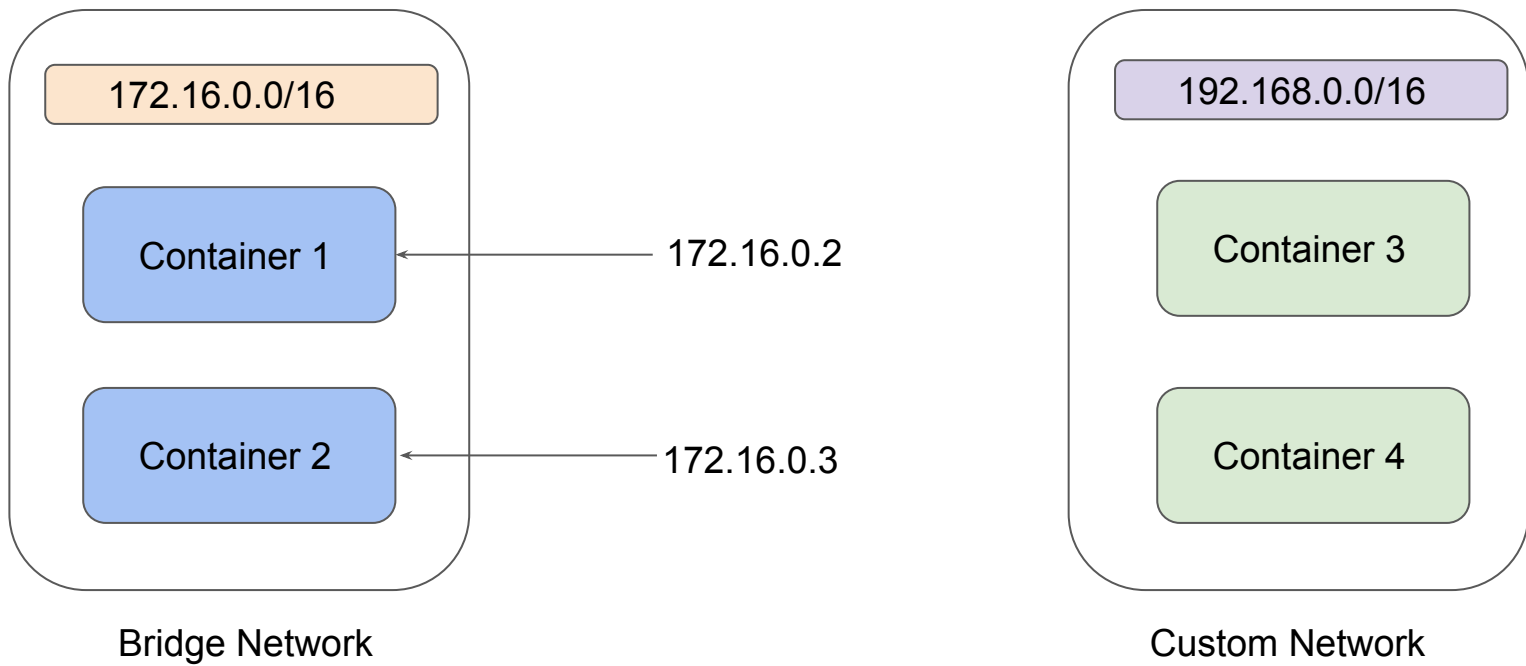
Build once, use anywhere

Overview of Bridge Network

A **bridge network** uses a software bridge which allows containers connected to the same bridge network to communicate, while providing isolation from containers which are not connected to that bridge network.



Diagrammatic Representation



Overview of Network Drivers

Bridge is the default network driver for Docker.

If we do not specify a driver, this is the type of network you are creating.

When you start Docker, a default bridge network (also called bridge) is created automatically, and newly-started containers connect to it unless otherwise specified.

We also can create User-Defined Bridge Network which are superior to the default bridge.

Host Networks

Build once, use anywhere

Overview of Host Network

This driver removes the network isolation between the docker host and the docker containers to use the host's networking directly.

For instance, if you run a container which binds to port 80 and you use host networking, the container's application will be available on port 80 on the host's IP address.

None Network

Build once, use anywhere

Overview of None Network

If you want to completely disable the networking stack on a container, you can use the none network.

This mode will not configure any IP for the container and doesn't have any access to the external network as well as for other containers.

Publishing Exposed Ports of Container

Build once, use anywhere

Overview of Port Publishing

We were discussing about an approach to publishing container port to host.

```
docker container run -dt --name webserver -p 80:80 nginx
```

This is also referred as publish list as it publishes only list of port specified.

Overview of Port Publishing

There is also a second approach to publish all the exposed ports of the container.

```
docker container run -dt --name webserver -P nginx
```

This is also referred as a publish all .

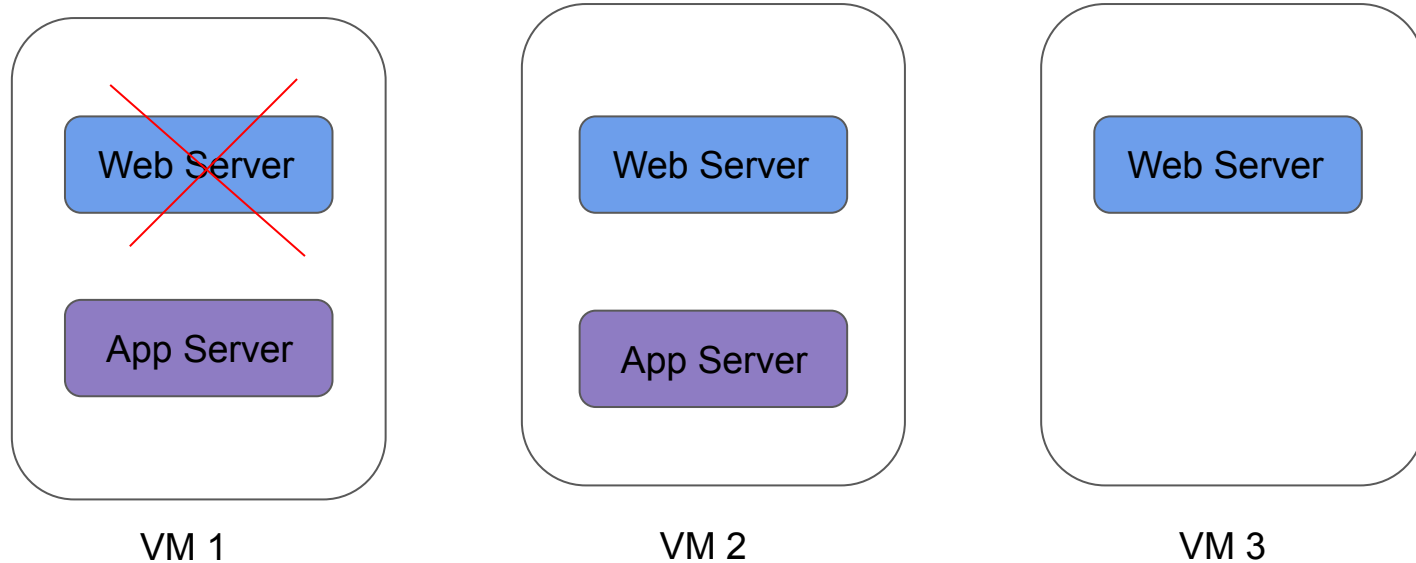
In this approach, all exposed ports are published to random ports of the host.

Container Orchestration

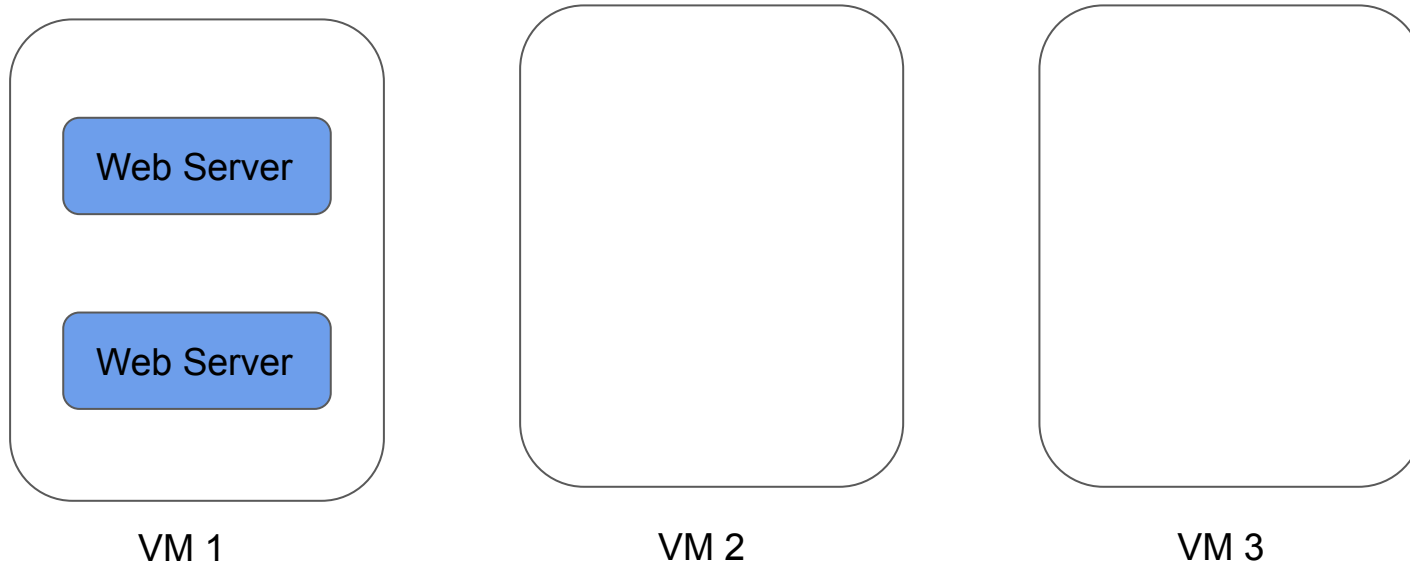
Build once, use anywhere

Getting Started

Container orchestration is all about managing the life cycles of containers, especially in large, dynamic environments.



Requirement: Minimum of 2 web-server should be running all the time.



Importance of Container Orchestration

Container Orchestration can be used to perform lot of tasks, some of them includes:

- Provisioning and deployment of containers
- Scaling up or removing containers to spread application load evenly
- Movement of containers from one host to another if there is a shortage of resources
- Load balancing of service discovery between containers
- Health monitoring of containers and hosts

Container Orchestration Solutions

There are many container orchestration solutions which are available, some of the popular ones include:

- Docker Swarm
- Kubernetes
- Apache Mesos
- Elastic Container Service (AWS ECS)

There are also various container orchestration platforms available like EKS.

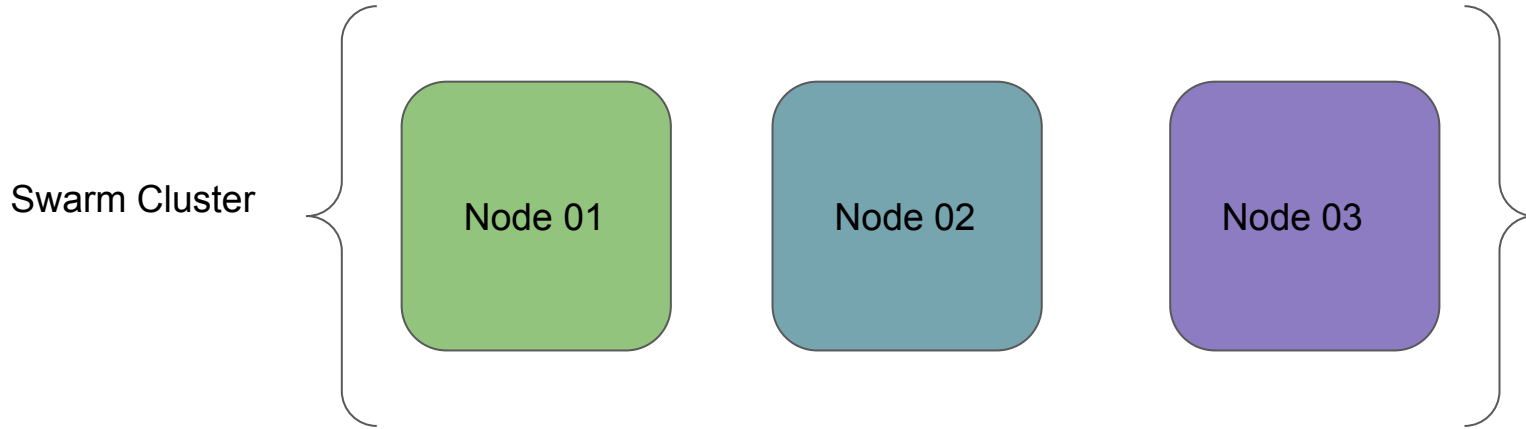
Overview of Docker Swarm

Build once, use anywhere

Getting Started

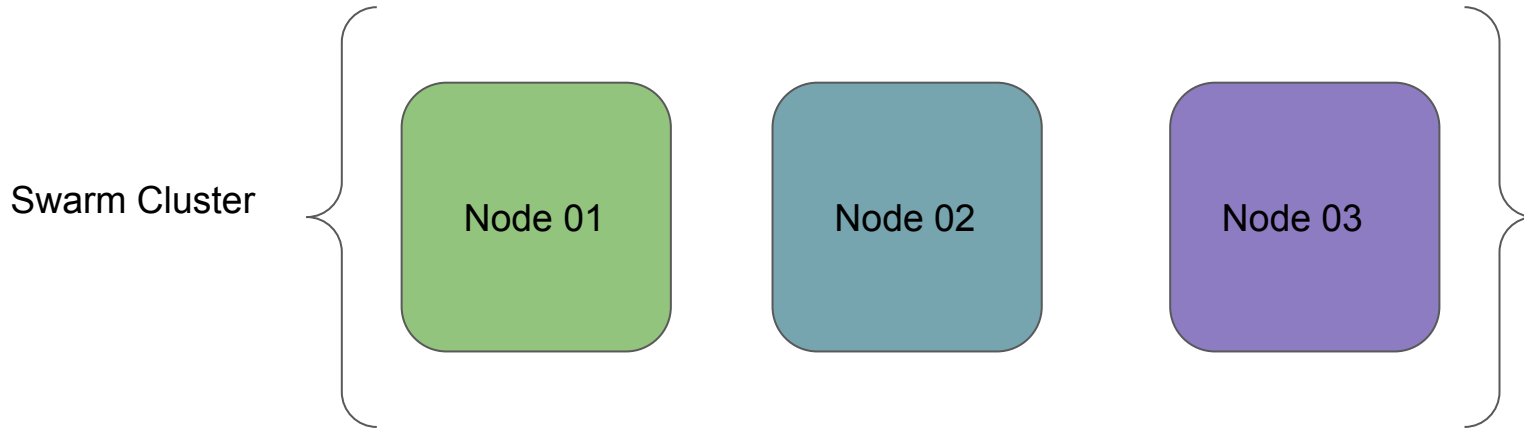
Docker Swarm is a container orchestration tool which is natively supported by Docker.

Our Lab Setup:



Initializing Docker Swarm

- A **node** is an instance of the Docker engine participating in the swarm.
- To deploy your application to a swarm, you submit a service definition to a **manager node**.
- The manager node dispatches units of work called tasks to **worker nodes**.

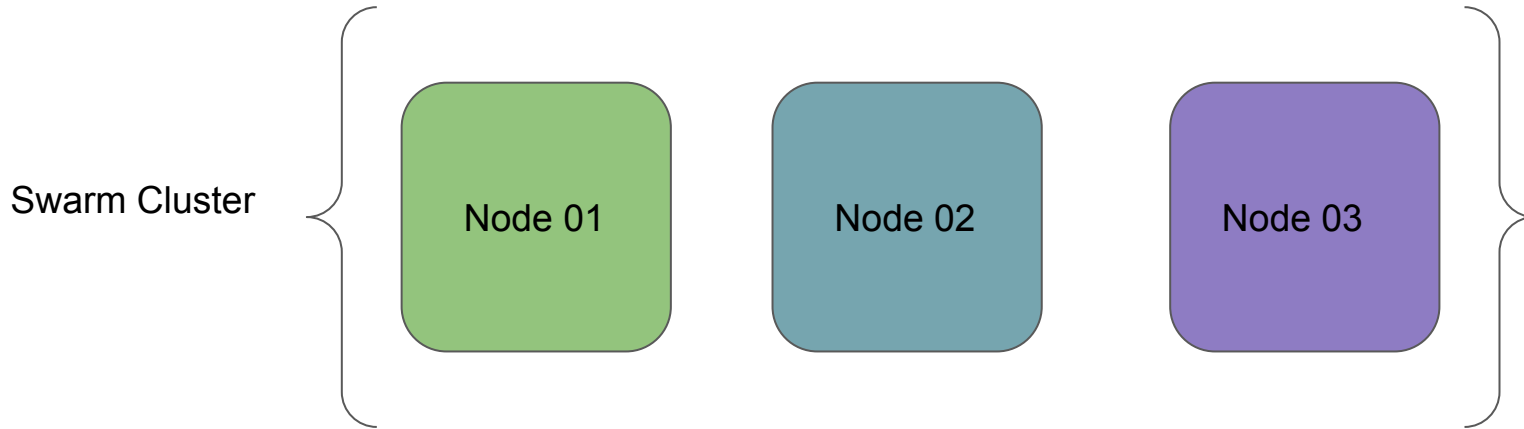


Initializing Docker Swarm

Build once, use anywhere

Initializing Docker Swarm

- A **node** is an instance of the Docker engine participating in the swarm.
- To deploy your application to a swarm, you submit a service definition to a **manager node**.
- The manager node dispatches units of work called tasks to **worker nodes**.



Initializing Docker Swarm

1. Manager Node Command:

```
docker swarm init --advertise-addr <MANAGER-IP>
```

2. Worker Nodes Command:

```
docker swarm join-token worker
```

Services, Tasks, Containers

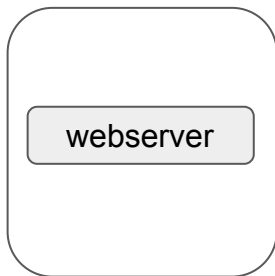
Build once, use anywhere

Getting Started

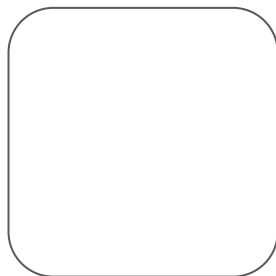
A service is the definition of the tasks to execute on the manager or worker nodes.

```
docker service create --name webserver --replicas 1 nginx
```

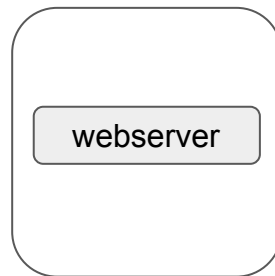
Swarm Cluster



Node 01

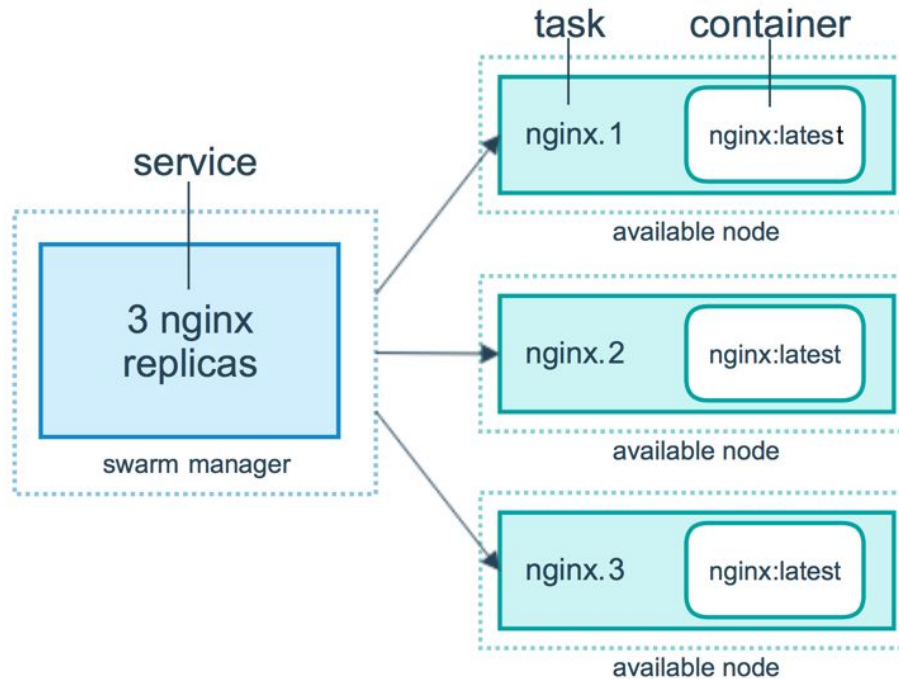


Node 02



Node 03

Services, Tasks and Containers



Scaling Service in Swarm

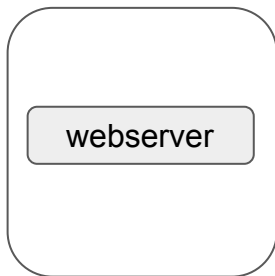
Build once, use anywhere

Getting Started

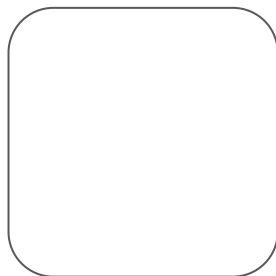
Once you have deployed a service to a swarm, you are ready to use the Docker CLI to scale the number of containers in the service.

Containers running in a service are called “tasks.”

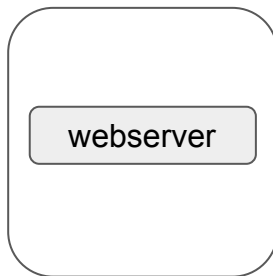
Swarm Cluster



Node 01

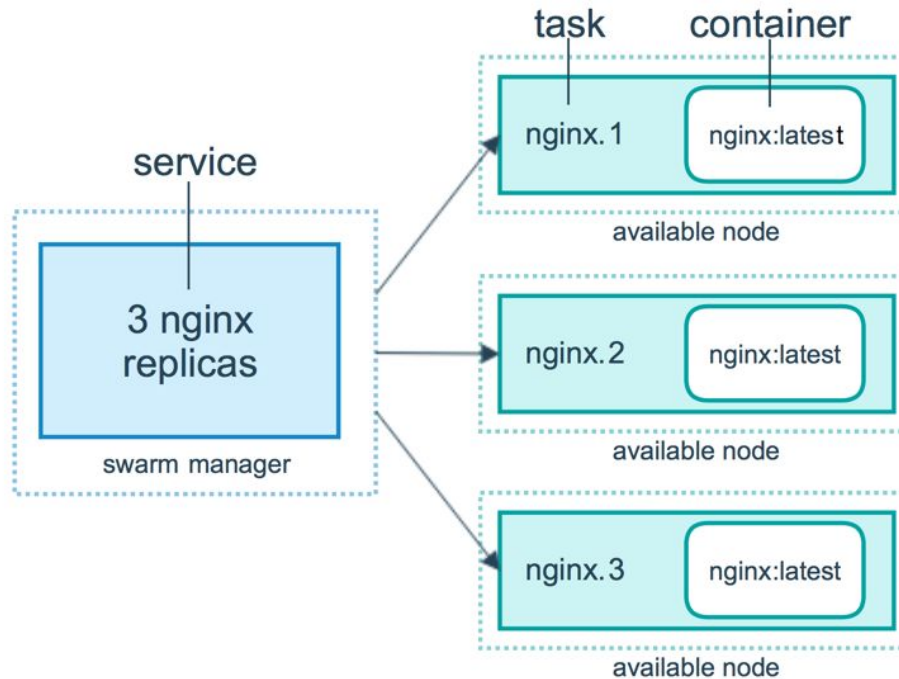


Node 02



Node 03

Services, Tasks and Containers

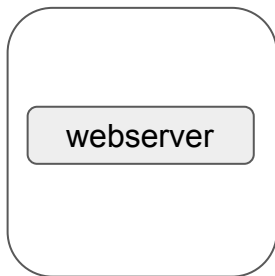


Two Commands to Scale Service

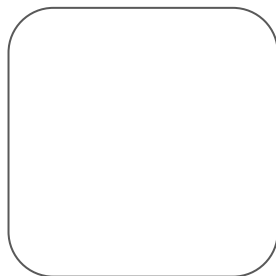
There are two ways in which you can scale service in swarm:

- `docker service scale webserver=5`
- `docker service update --replicas 5 mywebserver`

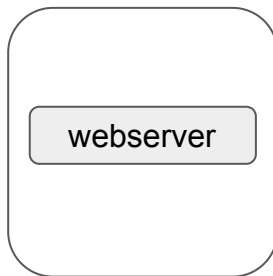
Swarm Cluster



Node 01



Node 02



Node 03

Replicated vs Global Service

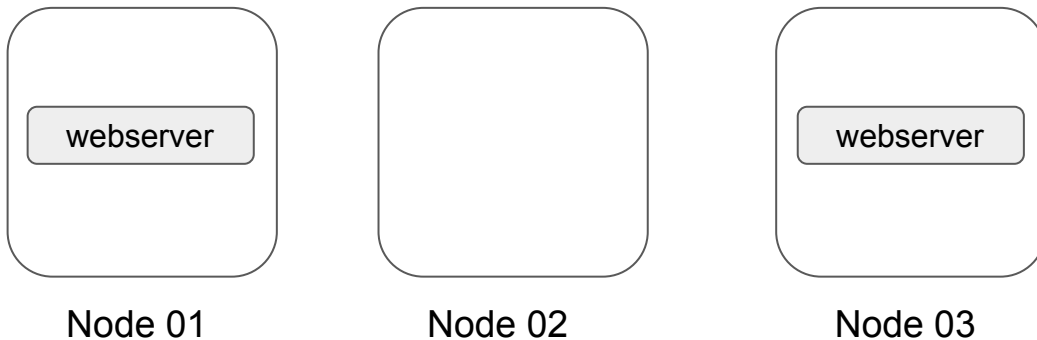
Build once, use anywhere

Getting Started

There are two types of services deployments, **replicated** and **global**

For a replicated service, you specify the number of identical tasks you want to run. For example, you decide to deploy an NGINX service with two replicas, each serving the same content.

Swarm Cluster

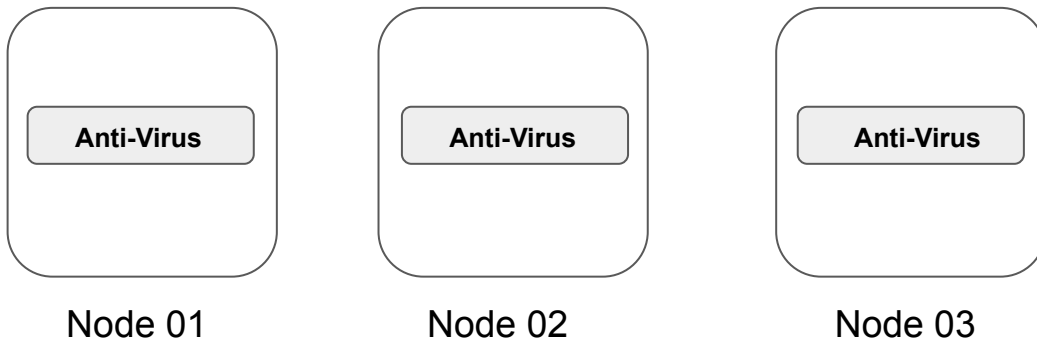


Global Service

A global service is a service that runs one task on every node.

Each time you add a node to the swarm, the orchestrator creates a task and the scheduler assigns the task to the new node.

Swarm Cluster



Inspecting - Swarm Services & Nodes

Build once, use anywhere

Overview of Docker Inspect

Docker Inspect provides detailed information of the Docker Objects.

Some of these Docker Object includes:

- Docker Containers
- Docker Network
- Docker Volumes
- Docker Swarm Services
- Docker Swarm Nodes

Overview of Docker Compose

Build once, use anywhere

Getting Started

Compose is a tool for defining and running multi-container Docker applications.

With Compose, you use a [YAML](#) file to configure your application's services.

We can start all the services with single command - `docker compose up`

We can stop all the services with single command - `docker compose down`

Deploying Multi-Service Apps in Swarm

Build once, use anywhere

Getting Started

A specific web-application might have multiple containers that are required as part of the build process.

Whenever we make use of docker service, it is typically for a single container image.

The [docker stack](#) can be used to manage a multi-service application.

Overview of Docker Stack

A stack is a group of interrelated services that share dependencies, and can be orchestrated and scaled together.

A stack can compose **YAML** file like the one that we define during Docker Compose.

We can define everything within the YAML file that we might define while creating a Docker Service.

Locking Swarm Cluster

Build once, use anywhere

Getting Started

Swarm Cluster **contains lot of sensitive information**, which includes:

- TLS key used to encrypt communication among swarm node
- Keys used to encrypt and decrypt the Raft logs on disk

If your Swarm is compromised and if data is stored in plain-text, an attack can get all the sensitive information.

Docker Lock allows us to have control over the keys.

Troubleshoot Service Deployments

Build once, use anywhere

Getting Started

A service may be configured in such a way that no node currently in the swarm can run its tasks.

In this case, the service remains in state **pending**

There are multiple reasons why service might go into pending state

```
[root@swarm01 ~]# docker service ps demotroubleshoot
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR
ulggnlj0ber	demotroubleshoot.1	nginx:latest		Running	Pending 17 seconds ago	"no suitable node (scheduling ..."
g819kx5dqp9b	demotroubleshoot.2	nginx:latest		Running	Pending 17 seconds ago	"no suitable node (scheduling ..."
k83f9wo0u6xc	demotroubleshoot.3	nginx:latest		Running	Pending 17 seconds ago	"no suitable node (scheduling ..."

Examples when services go in Pending State

If all nodes are drained, and you create a service, it is pending until a node becomes available.

You can reserve a specific amount of memory for a service. If no node in the swarm has the required amount of memory, the service remains in a pending state until a node is available which can run its tasks.

You have imposed some kind of placement constraints

Control Service Placement

Build once, use anywhere

Overview of Placement Constraints

Swarm services provide a few different ways for you to control scale and placement of services on different nodes.

- Replicated and Global Services
- Resource Constraints [requirement of CPU and Memory]
- Placement Constraints [only run on nodes with label `pci_compliance = true`]
- Placement Preferences

Overlay Network

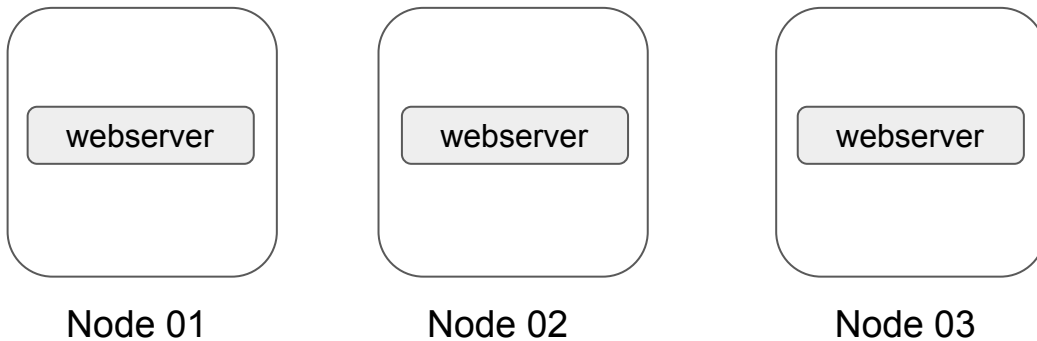
Build once, use anywhere

Overview of Overlay Networks

The overlay network driver creates a distributed network among multiple Docker daemon hosts

Overlay network allows containers connected to it to communicate securely.

Swarm Cluster



Secure Overlay Networks

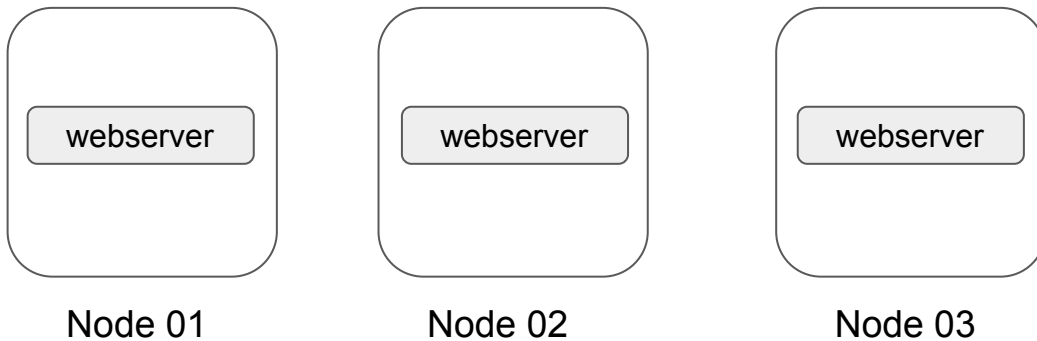
Build once, use anywhere

Overview of Overlay Networks

The overlay network driver creates a distributed network among multiple Docker daemon hosts

Overlay network allows containers connected to it to communicate securely.

Swarm Cluster



Overview of Secure Overlay Network

For the overlay networks, the containers can be spreaded across multiple servers.

If the containers are communicating with each other, it is recommended to secure the communication.

To enable encryption, when you create an overlay network pass the `--opt encrypted` flag:

```
docker network create --opt encrypted --driver overlay my-overlay-secure-network
```

Important Pointers

When you enable overlay encryption, Docker creates IPSEC tunnels between all the nodes where tasks are scheduled for services attached to the overlay network.

These tunnels also use the AES algorithm in GCM mode and manager nodes automatically rotate the keys every 12 hours.

Overlay network encryption is not supported on Windows. If a Windows node attempts to connect to an encrypted overlay network, no error is detected but the node will not be able to communicate.

Creating Services using Templates

Build once, use anywhere

Getting Started

We can make use of **templates** while running the **service create** command in swarm.

Let us understand this with an example:

```
docker service create --name demoservice  
--hostname="{{.Node.Hostname}}-{{.Service.Name}}" nginx
```

Valid Placeholders

Placeholder	Description
Service.ID	Service ID
.Service.Name	Service name
.Service.Labels	Service labels
.Node.ID	Node ID
.Node.Hostname	Node Hostname
.Task.ID	Task ID
.Task.Name	Task name
.Task.Slot	Task slot

Supported Flags

There are three supported flags for the placeholder templates:

`--hostname`

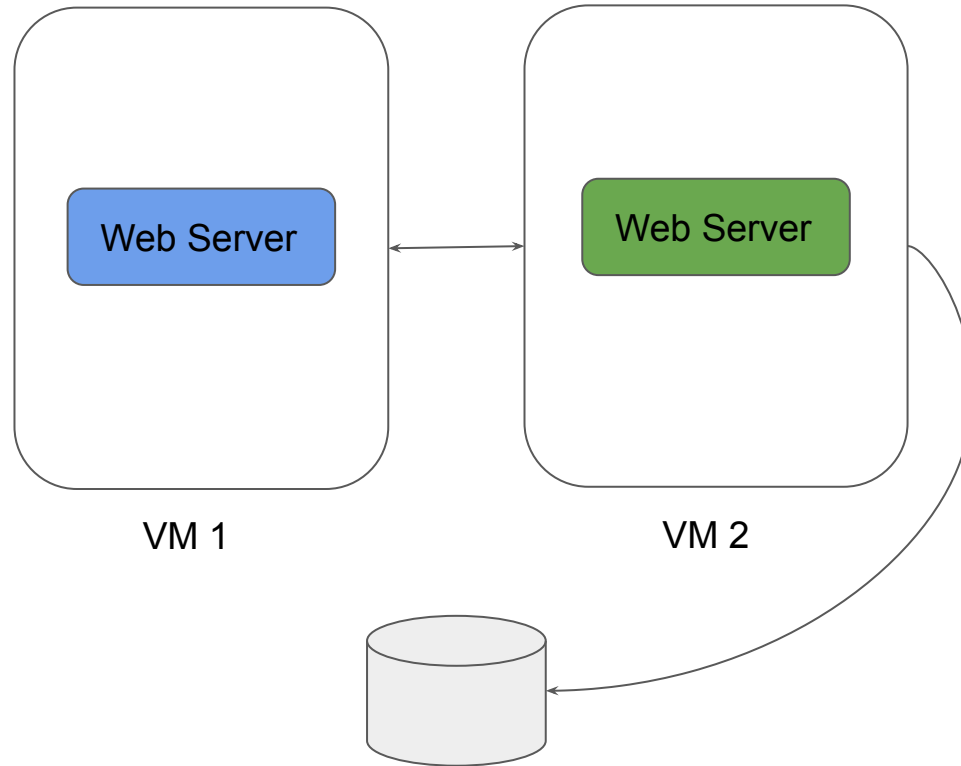
`--mount`

`--env`

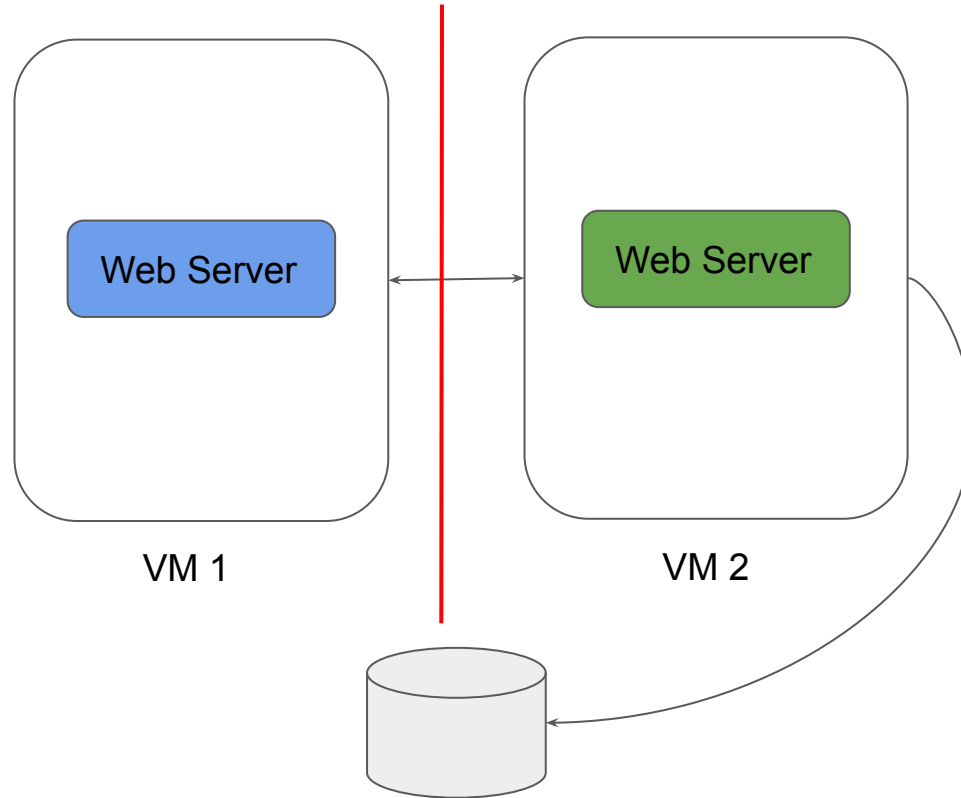
Split-Brain & Quorum

Build once, use anywhere

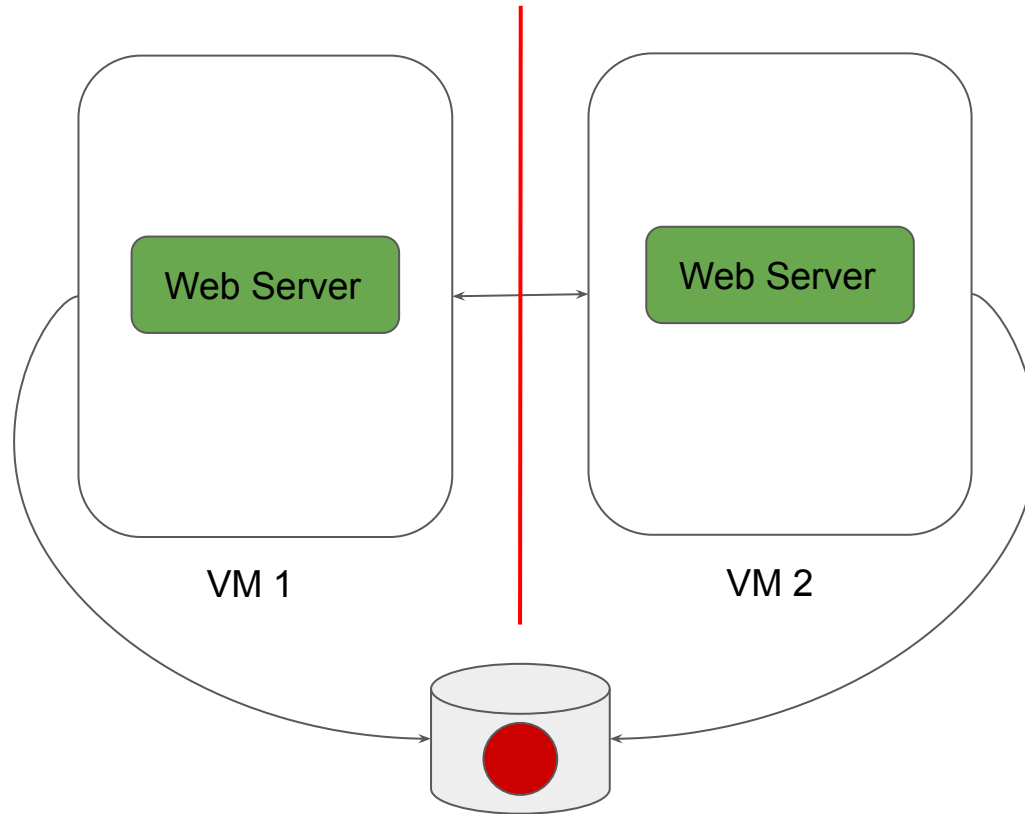
Split Brain Problem



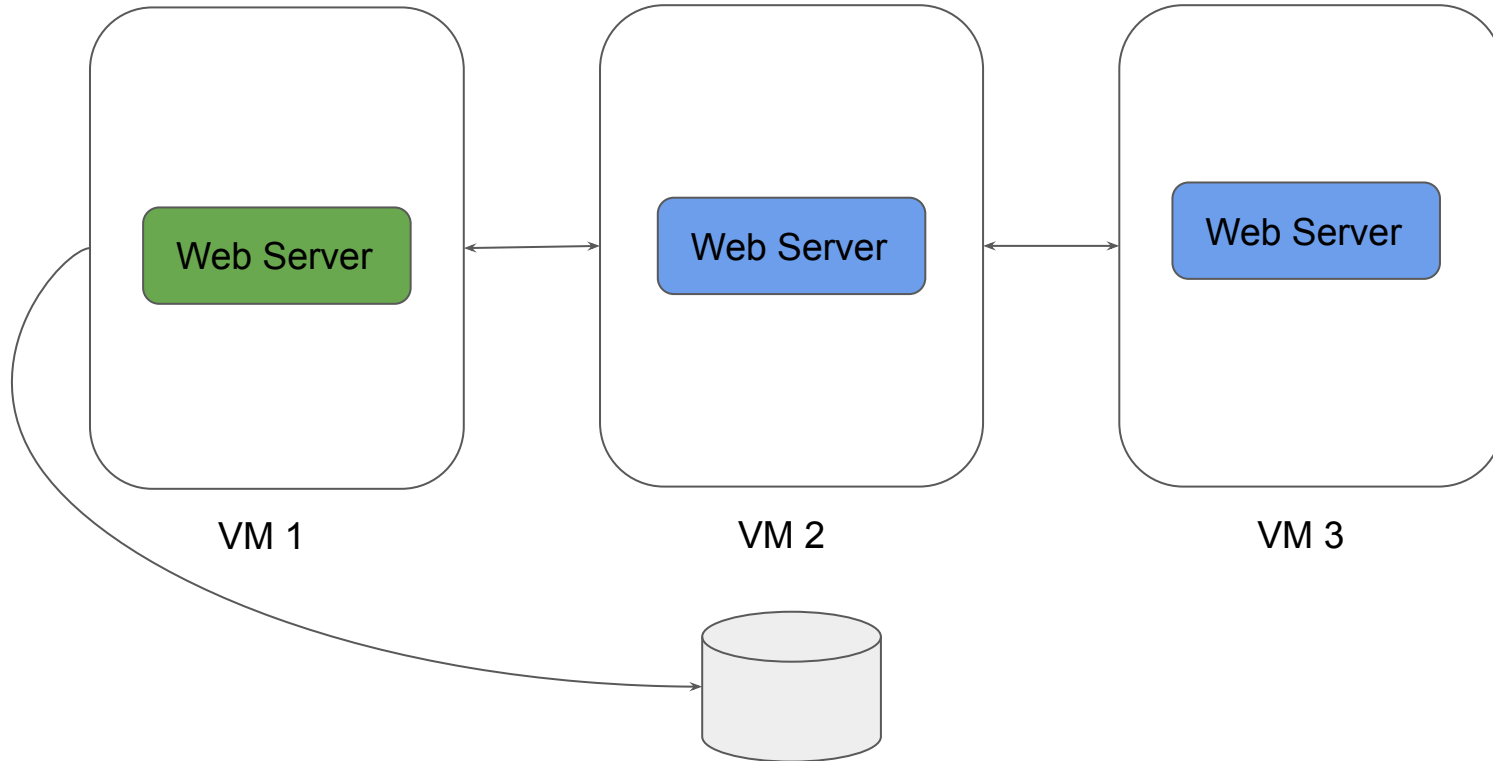
Split Brain Problem



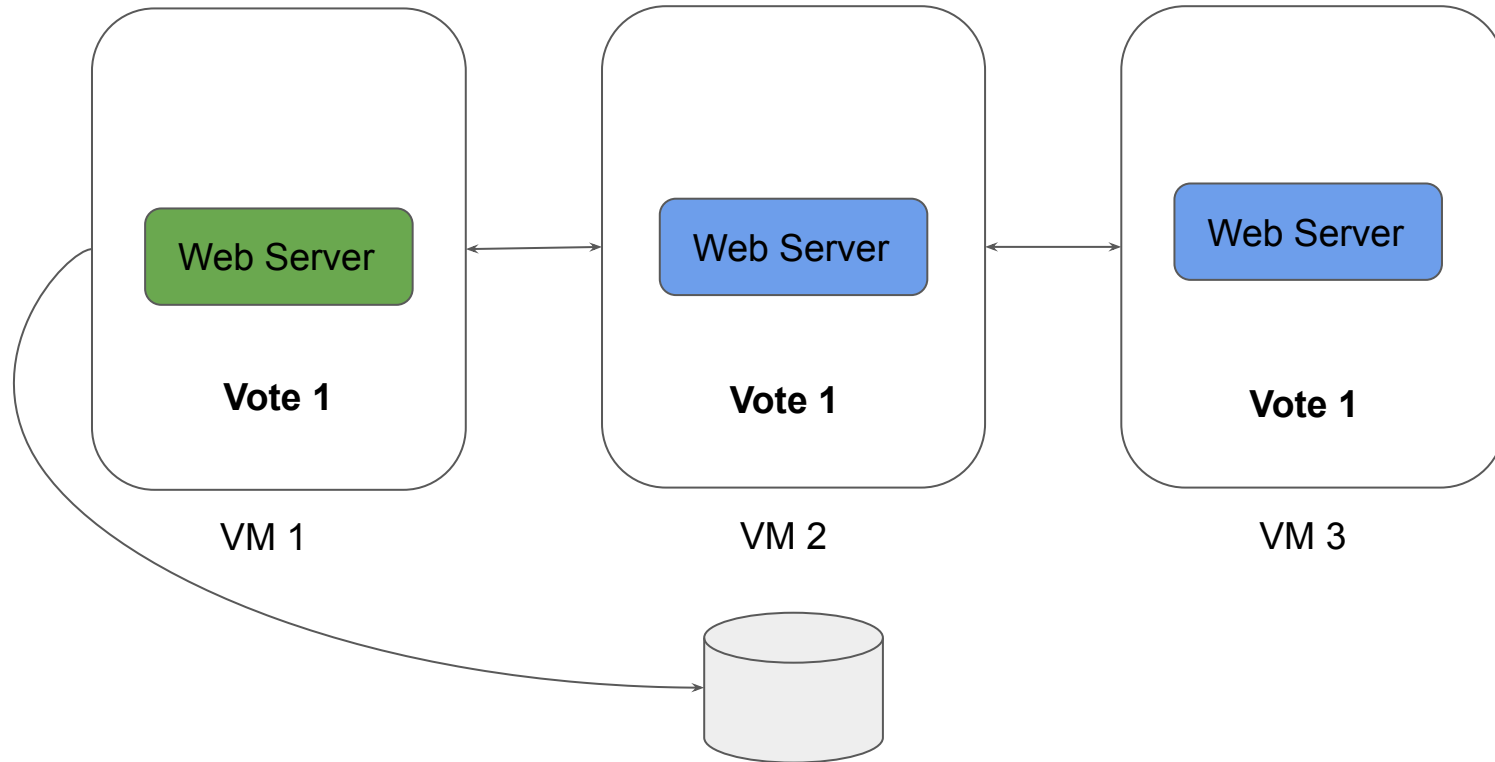
Split Brain Problem



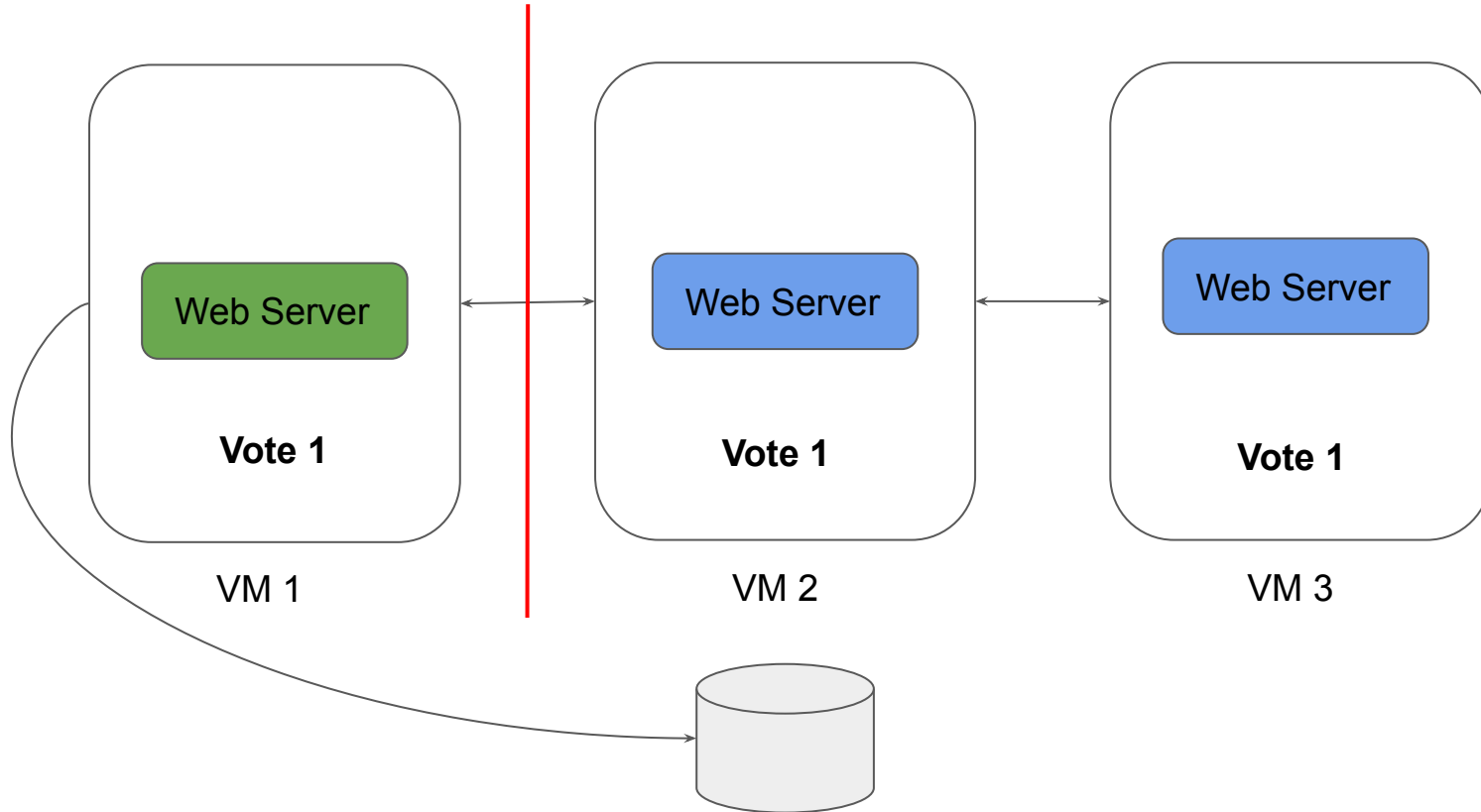
Right Quorum to the Rescue



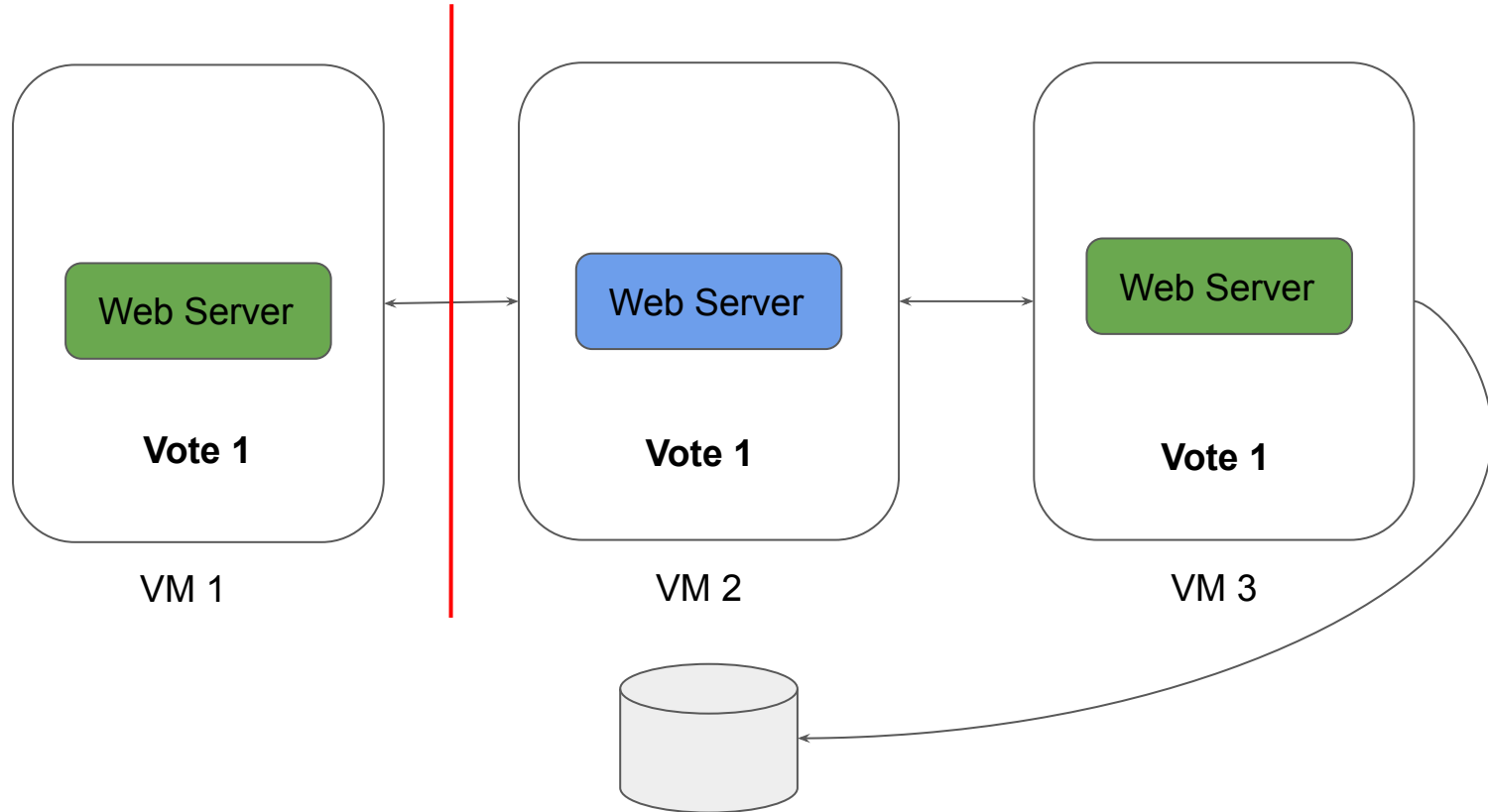
Right Quorum to the Rescue



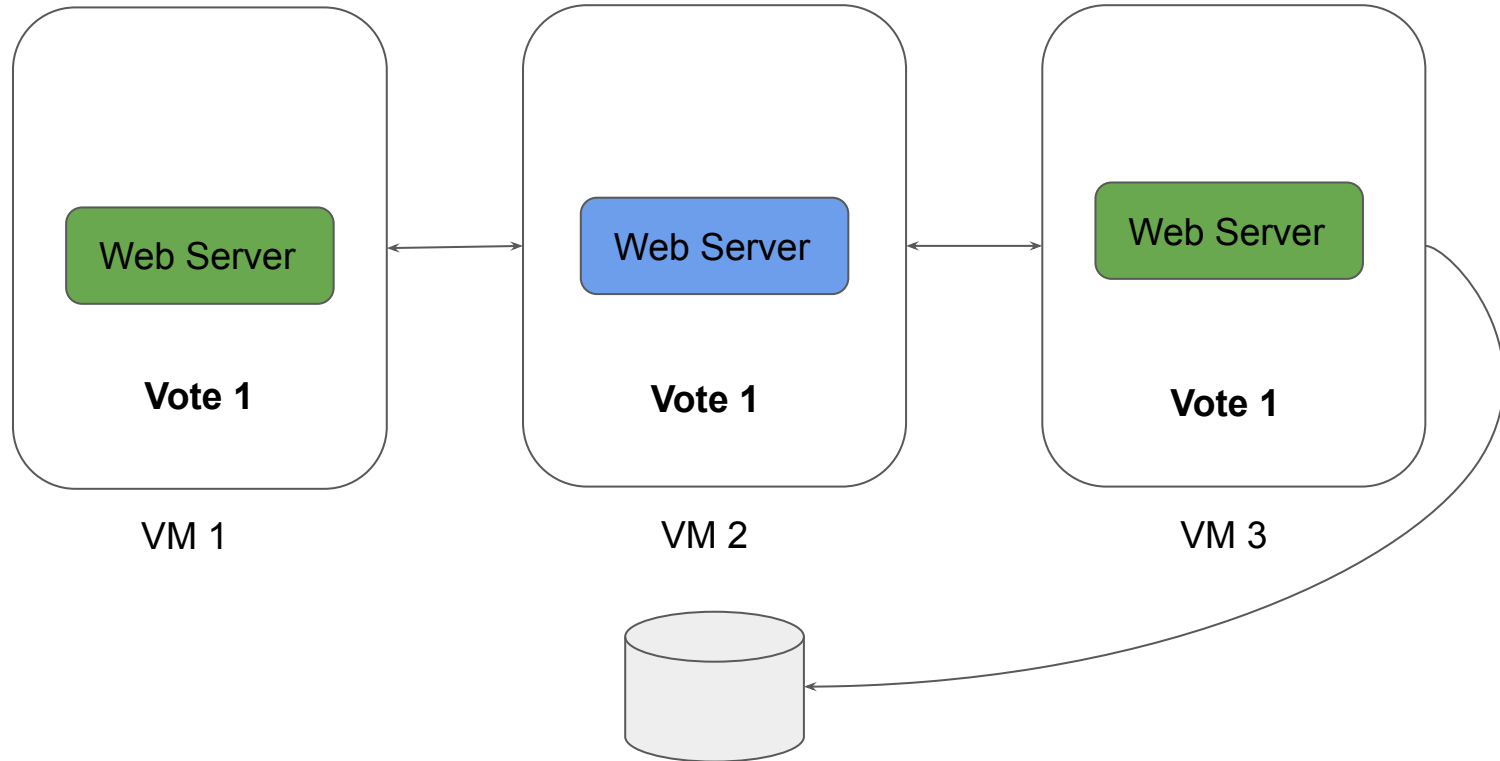
Right Quorum to the Rescue



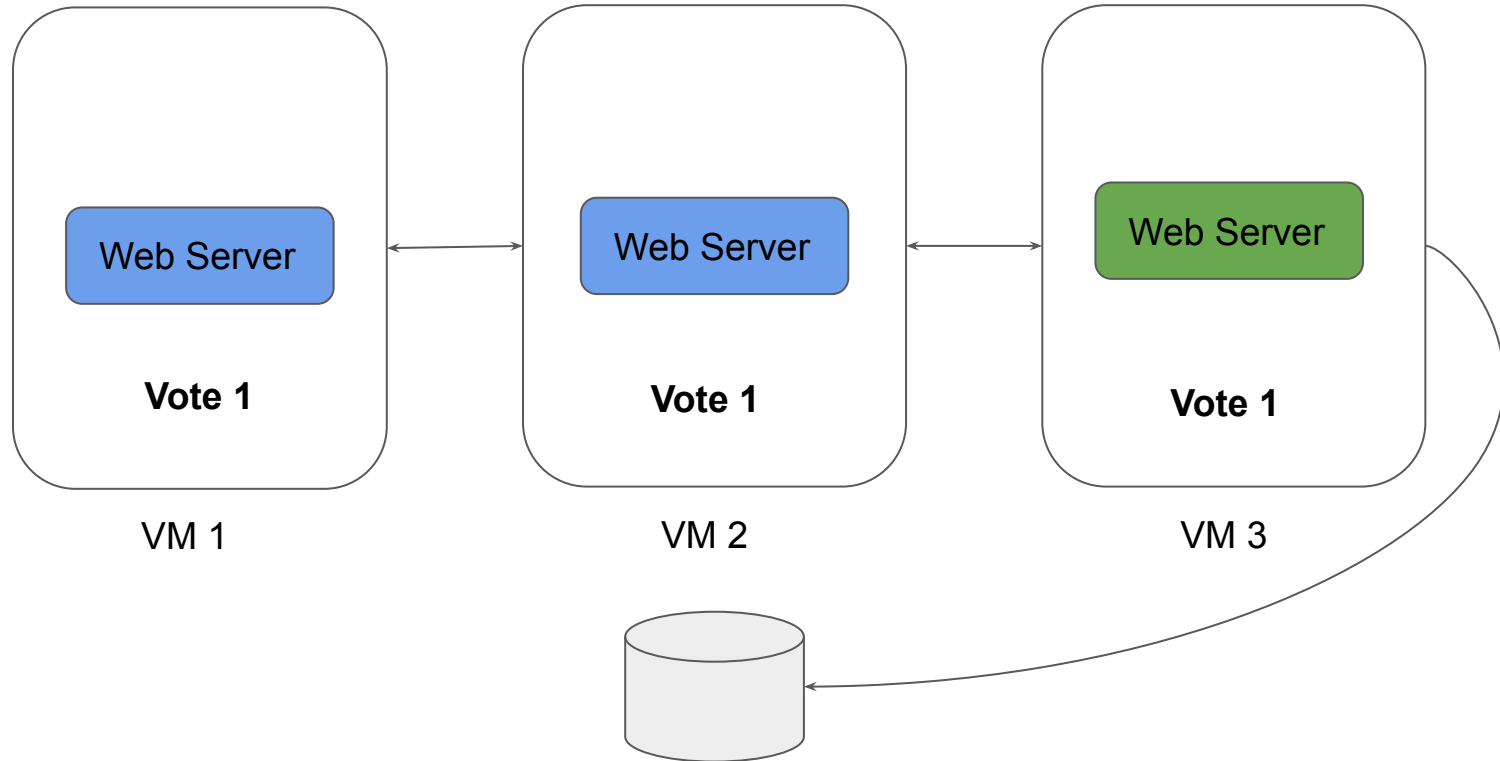
Right Quorum to the Rescue



Right Quorum to the Rescue



Right Quorum to the Rescue



Important Pointers for Quorum

We should maintain an odd number of nodes within the cluster.

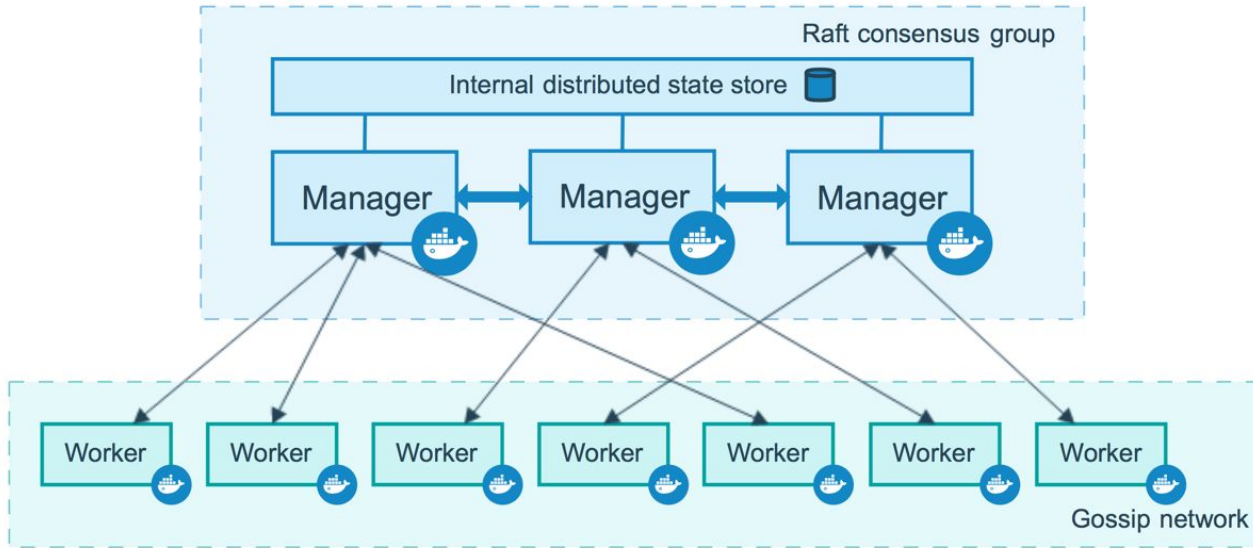
Cluster Size	Majority	Fault Tolerance
1	0	0
2	2	0
3	2	1
4	3	1
5	3	2
6	4	2
7	4	3

Swarm Manager Node HA

Build once, use anywhere

Getting Started

Manager Nodes are responsible for handling the cluster management tasks.



Importance of Container Orchestration

Manager Node has many responsibility within swarm, these include:

- maintaining cluster state
- scheduling services
- serving swarm mode HTTP API endpoints

Using a Raft implementation, the managers maintain a consistent internal state of the entire swarm and all the services running on it

High Availability in Swarm

Swarm comes with its own fault tolerance features.

Docker recommends you implement an odd number of nodes according to your organization's high-availability requirements.

Using a Raft implementation, the managers maintain a consistent internal state of the entire swarm and all the services running on it

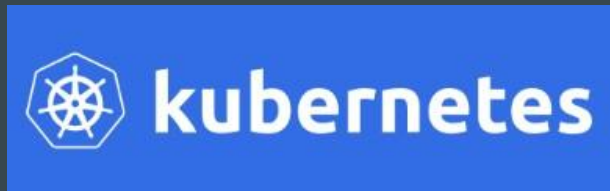
An N manager cluster tolerates the loss of at most $(N-1)/2$ managers.

Introduction to Kubernetes

Setting the Base

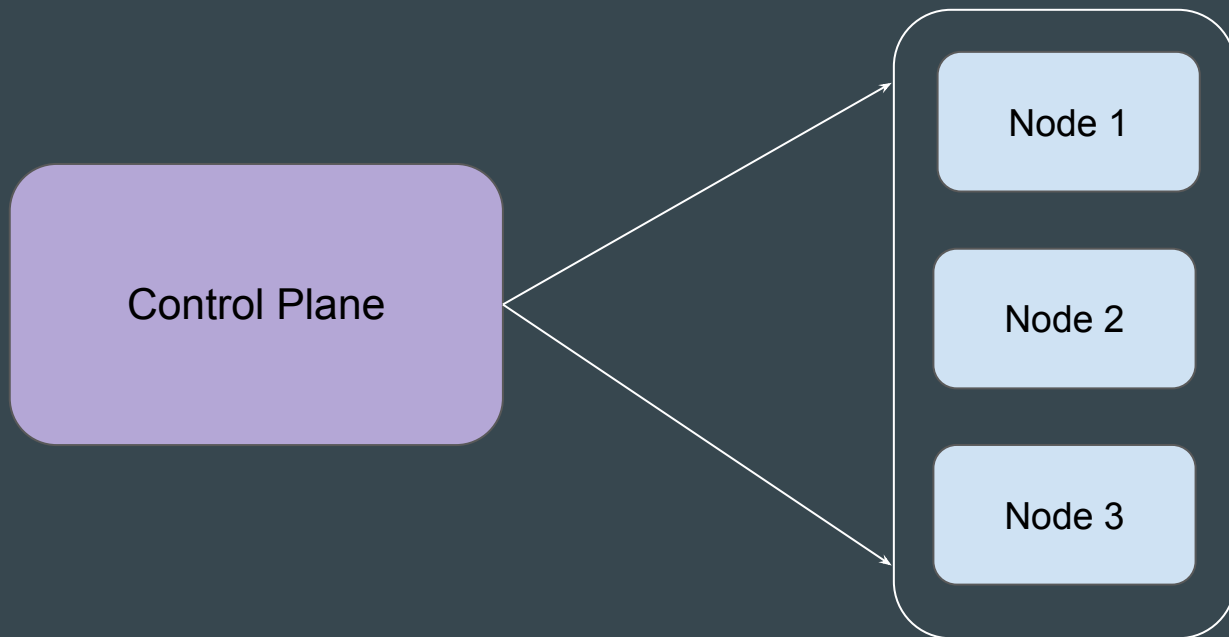
Kubernetes (K8s) is an open-source **container orchestration engine** developed by Google.

It was originally designed by Google, and is now maintained by the Cloud Native Computing Foundation.



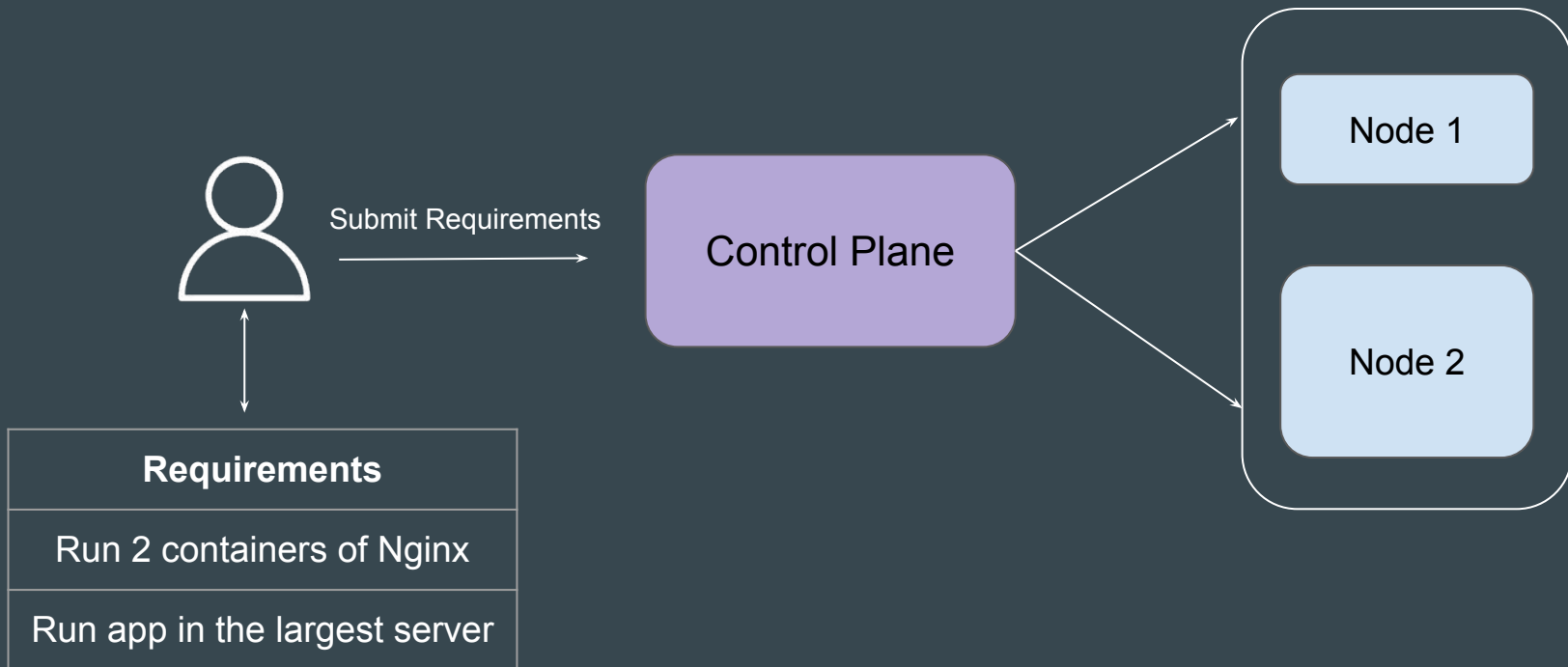
Architecture of Kubernetes

A Kubernetes cluster consists of a **control plane** + a set of worker machines, called nodes, that run containerized applications



Basic Workflow

User communicates to Control Plane and provides necessary instructions to run containerized applications.



Example - Run 1 Container of Apache

If you have instructed Kubernetes to **run 1 container** of Apache, Kubernetes will launch it in one of the worker nodes and will regularly monitor the state of that container to ensure it always runs.



Submit Requirement

Control Plane

Node 1

Node 2

Requirements

Run 1 container of Apache

Kubernetes is Awesome

Kubernetes provides amazing set of features and is designed on the same principles that allow Google to **run billions of containers** a week.

Some of the popular features include:

- Pod Auto-Scaling
- Service discovery and load balancing
- Self-Healing of Containers
- Secret management
- Automated rollouts and rollbacks



Installation Options for Kubernetes

Analogy - Eating your Favorite Food Dish

You want to eat your favourite food dish.

What are the options:

1. Go to store, get ingredients and prepare from scratch.
2. Get ready-made mix, make it hot.
3. Order it from the restaurant.



Kubernetes Reference

You want to launch a Kubernetes cluster.

What are the options:

1. Go to K8s website, download individual components and integrate them one by one.
2. Use tools like Minikube, Kubeadm to quickly setup K8s cluster for you.
3. Use Managed Kubernetes Service.

Option 1 - Managed Kubernetes Service

Various providers like AWS, IBM, GCP and others provides **managed** Kubernetes clusters.

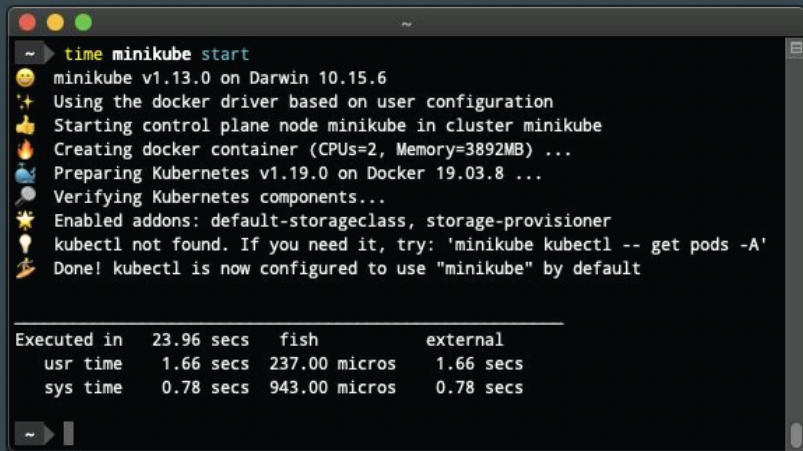
Analogy: Order Ready-Made Food from Store.



Option 2 - K8s Development Tools

Various tools like Minikube, K3d allows you to quickly setup the Kubernetes for local testing.

Analogy: Get ready-made mix, make it hot.



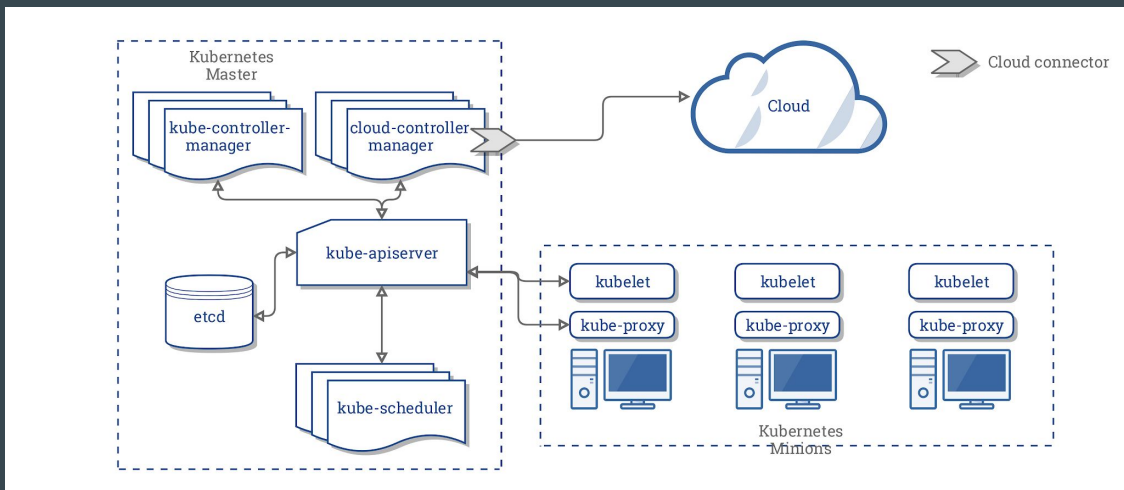
```
~ ➤ time minikube start
🐸 minikube v1.13.0 on Darwin 10.15.6
🌟 Using the docker driver based on user configuration
👉 Starting control plane node minikube in cluster minikube
🔥 Creating docker container (CPUs=2, Memory=3892MB) ...
🐳 Preparing Kubernetes v1.19.0 on Docker 19.03.8 ...
🔍 Verifying Kubernetes components...
🌟 Enabled addons: default-storageclass, storage-provisioner
💡 kubectl not found. If you need it, try: 'minikube kubectl -- get pods -A'
👉 Done! kubectl is now configured to use "minikube" by default
```

Executed in	23.96 secs	fish	external
usr time	1.66 secs	237.00 micros	1.66 secs
sys time	0.78 secs	943.00 micros	0.78 secs

```
~ ➤
```

Option 3 - Setup K8s from Scratch

In this approach, you install and configure each Kubernetes component individually.



Choosing Right Cloud Provider for Practicals

Installation Options for Kubernetes

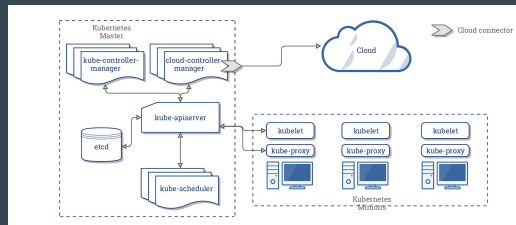
There are **three primary ways** to configure Kubernetes.

1. Using **Managed Kubernetes Service** from a Provider
2. Use tools like **Minikube**, **Kubeadm** to quickly setup K8s cluster for you.
3. Setup **Kubernetes Cluster from Scratch**.



```
~$ time minikube start
minikube v1.13.0 on Darwin 10.15.6
Using the docker driver based on user configuration
Starting control plane node minikube in cluster minikube
Creating docker container (CPUs=2, Memory=3852MB) ...
Preparing Kubernetes v1.19.0 on Docker 19.03.8 ...
Verifying Kubernetes components...
Enabled addons: default-storageclass, storage-provisioner
kubectl not found. If you need it, try: 'minikube kubectl -- get pods -A'
Done! Kubectl is now configured to use "minikube" by default

Executed in 23.96 secs  fish           external
   usr time   1.66 secs  237.00 micros   1.66 secs
   sys time   0.78 secs   943.00 micros   0.78 secs
```



Constraints of Choosing Cloud Provider

1. Easy to Use.
2. Cost Effective.
3. Free Credits to Get Started.

Option 1 - Amazon EKS

Managed Kubernetes service provided by AWS.

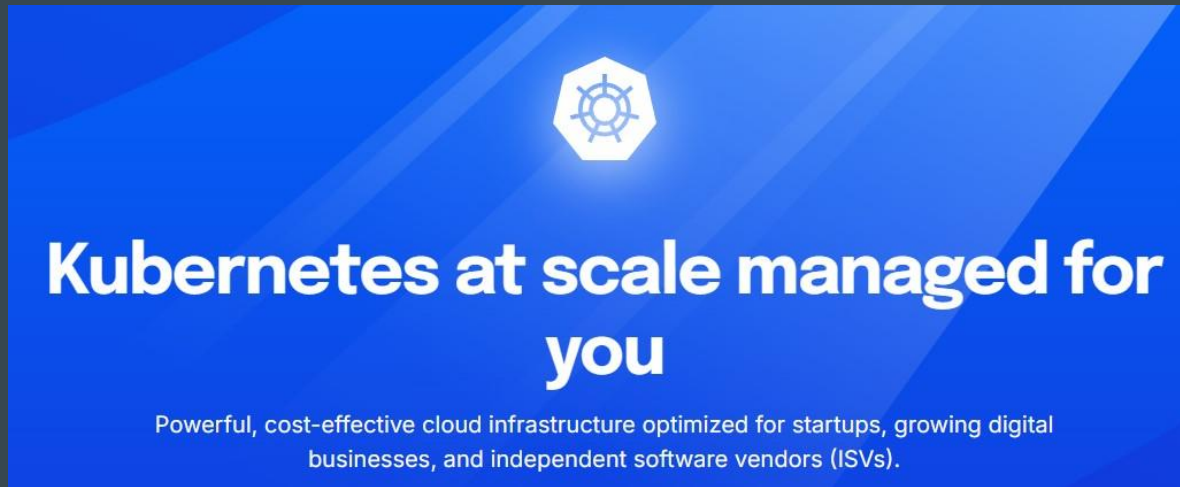
Amazon Elastic Kubernetes Service

Start, run, and scale Kubernetes without thinking about cluster management

[Get started with Amazon EKS](#)

Option 2 - Digital Ocean

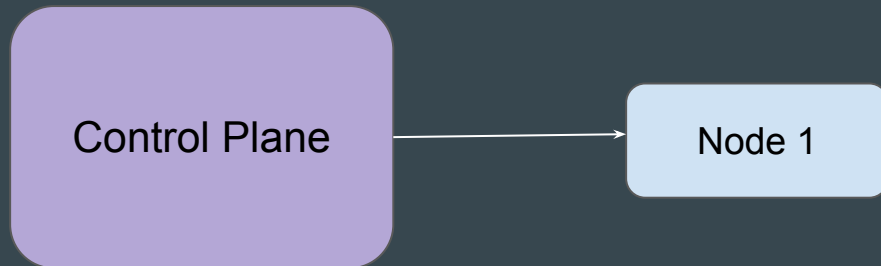
Managed Kubernetes service provided by Digital Ocean.



Why Digital Ocean?

Kubernetes **Control Plane** is completely free.

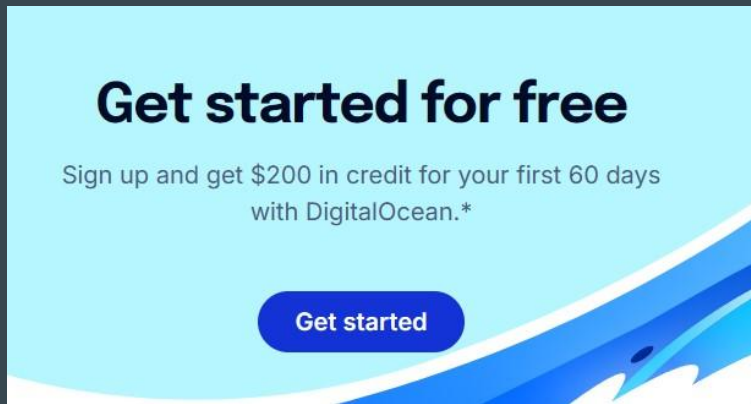
You will be charged only for the Worker Nodes that you run..



Initial Free Credits

Digital Ocean provides decent amount of free credits for new users.

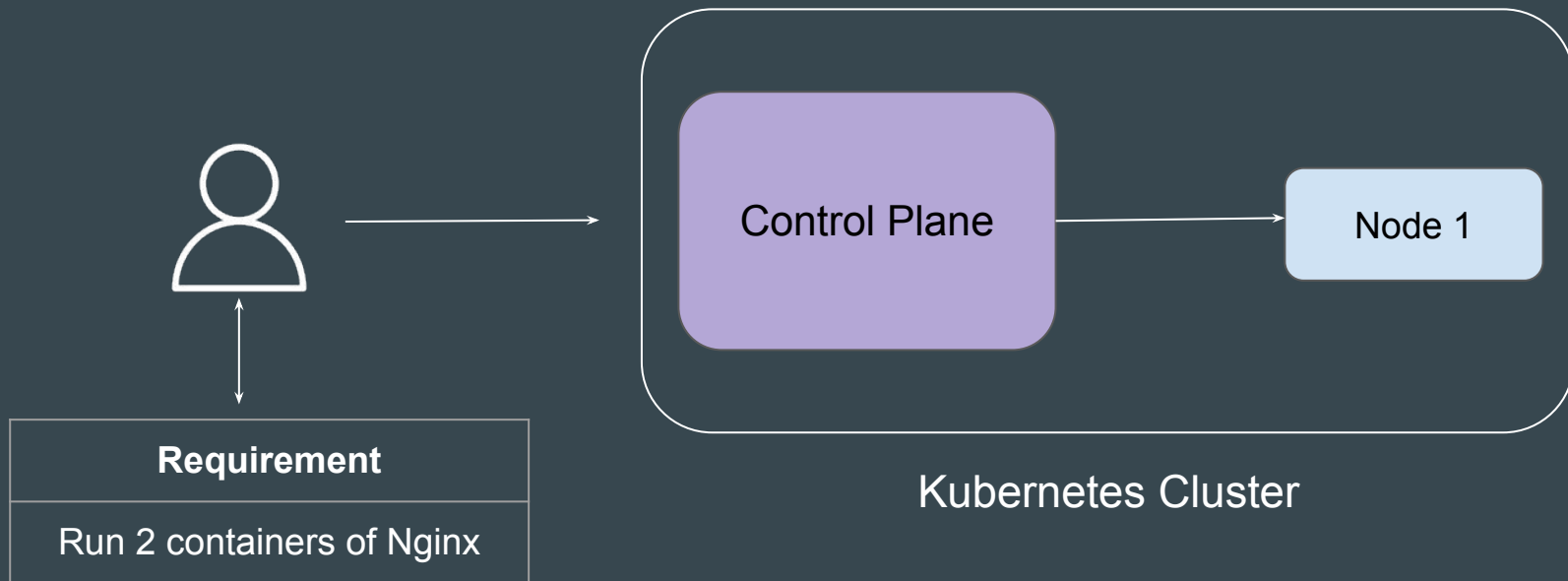
You can literally complete this entire course and ALL practicals for free.



Connectivity Options for Kubernetes

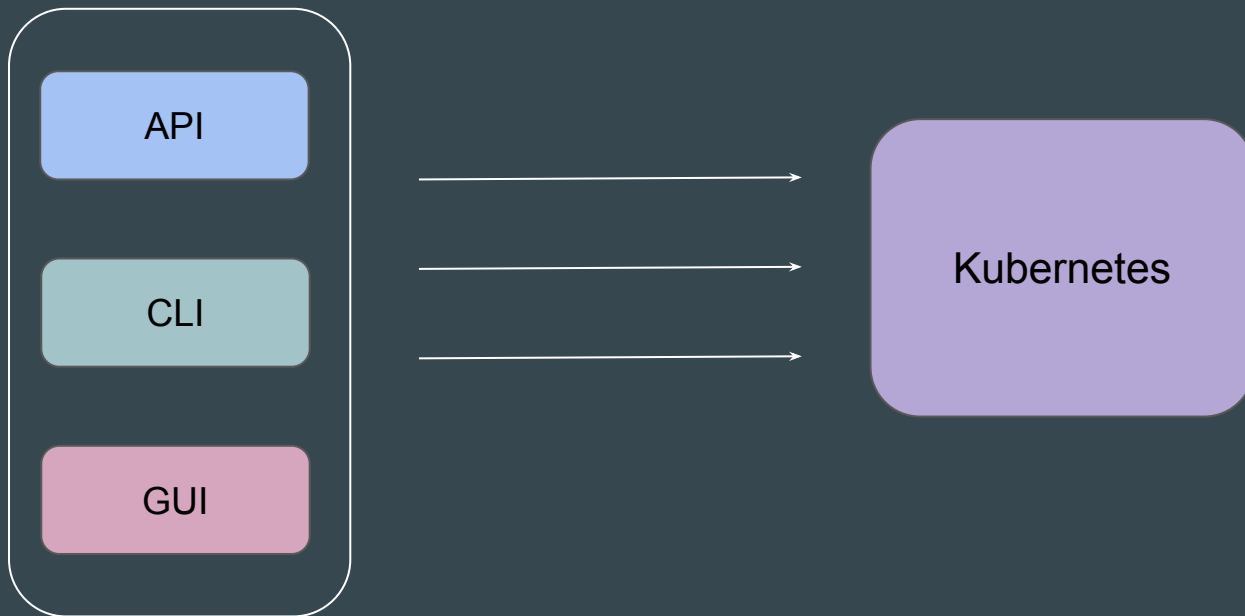
First Important Step - Connect to Cluster

The **first important step** after launching Kubernetes cluster is to **connect to the Control Plane**.



Options for Connectivity

There are **three** primary ways to connect to your Kubernetes cluster.



Option 1 - API

You can use utilities like curl to directly send request to to the API.

```
root@kubeadm-master:~# curl http://localhost:8080/api/v1/namespaces/default/pods
{
  "kind": "PodList",
  "apiVersion": "v1",
  "metadata": {
    "resourceVersion": "1346"
  },
  "items": [
    {
      "metadata": {
        "name": "nginx",
        "namespace": "default",
        "uid": "67850778-3e32-4364-9a91-04c2cb9644d8",
        "resourceVersion": "1338",
        "creationTimestamp": "2024-12-26T05:49:11Z",
        "labels": {
          "run": "nginx"
        }
      },
```

Option 2 - CLI

To connect to Kubernetes using CLI, you need an important tool named **kubectl**



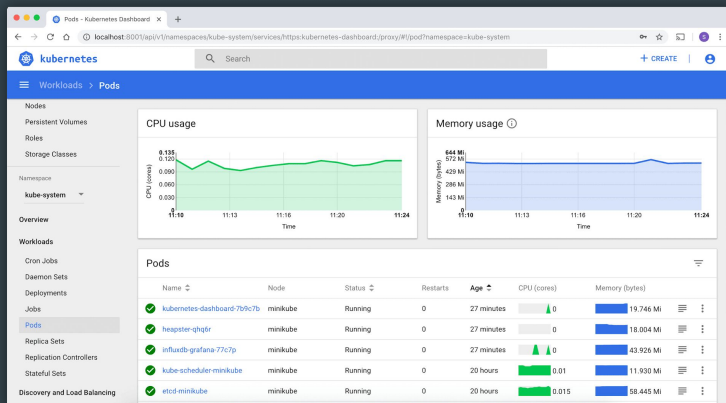
```
root@kubeadm-master:~# kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx	1/1	Running	0	82s

Option 3 - GUI

Dashboard is a web-based Kubernetes user interface.

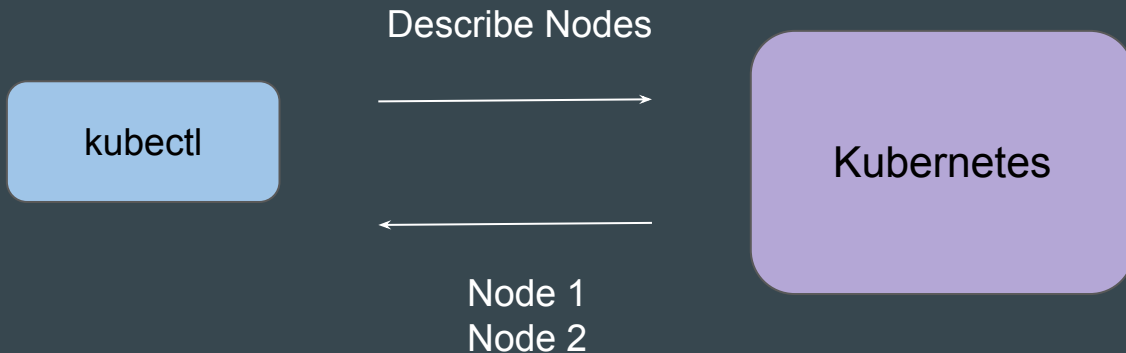
You can use Dashboard to deploy, troubleshoot containerized applications and manage the cluster resources



Overview of kubectl

Basics of Kubectl

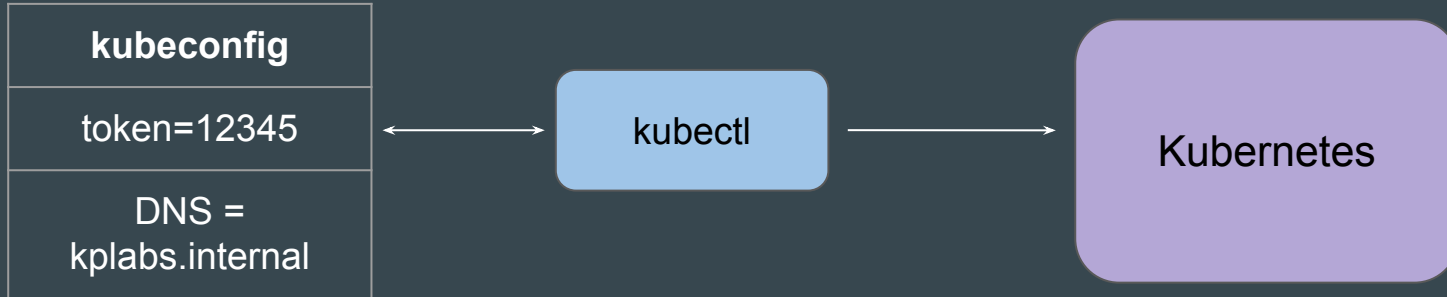
The Kubernetes command-line tool, **kubectl**, allows you to run commands against Kubernetes clusters.



Important Part - Authentication

Kubectl needs **two important details** while connecting to Kubernetes Cluster

1. DNS / IP Address of Kubernetes Control Plane
2. Authentication Credentials.



Default Path of Kubeconfig File

Kubectl by default will look for the kubeconfig file in a file named **config** inside **.kube** folder in your **home directory**.

~/.kube/config

```
root@kubeadm-master:~# ls -l ~/.kube/config
-rw----- 1 root root 5658 Dec 26 05:46 /root/.kube/config
```

Referencing to Custom Config File

You can also refer to custom config file using the `--kubeconfig` flag

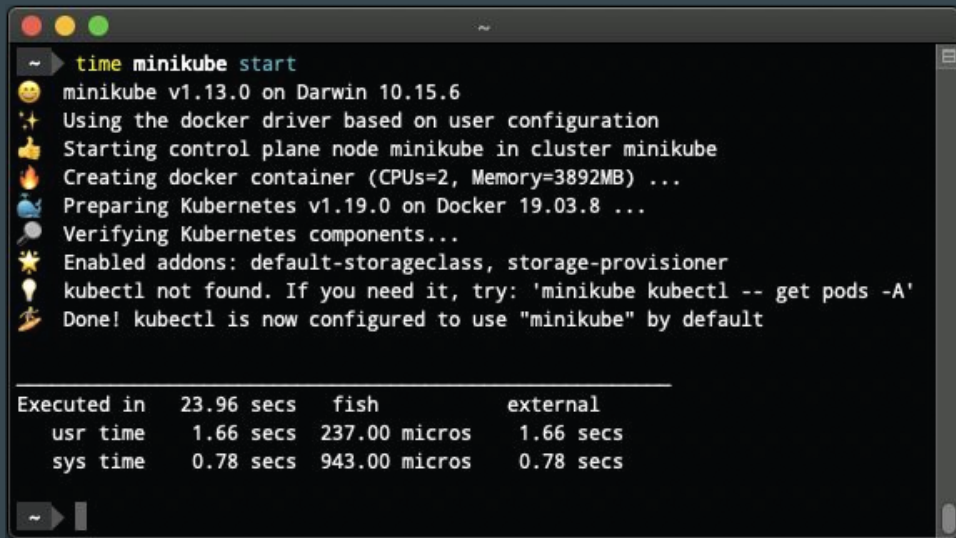
```
C:\kplabs-k8s>kubectl get nodes --kubeconfig "custom-kubeconfig"
```

NAME	STATUS	ROLES	AGE	VERSION
pool-i625o5obd-eob8a	Ready	<none>	3h6m	v1.31.1

Configuring Kubernetes using Minikube

Basics of Minikube

minikube quickly **sets up a local Kubernetes cluster** on macOS, Linux, and Windows.



```
~ ➤ time minikube start
🐼 minikube v1.13.0 on Darwin 10.15.6
🌟 Using the docker driver based on user configuration
👉 Starting control plane node minikube in cluster minikube
🔥 Creating docker container (CPUs=2, Memory=3892MB) ...
🚀 Preparing Kubernetes v1.19.0 on Docker 19.03.8 ...
🔍 Verifying Kubernetes components...
🌟 Enabled addons: default-storageclass, storage-provisioner
💡 kubectl not found. If you need it, try: 'minikube kubectl -- get pods -A'
🏠 Done! kubectl is now configured to use "minikube" by default

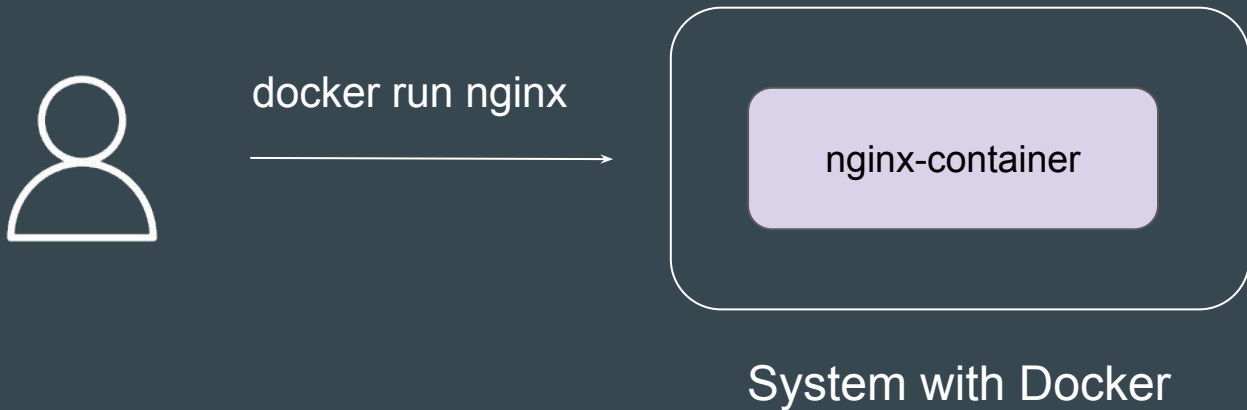
Executed in   23.96 secs   fish           external
   usr time    1.66 secs  237.00 micros    1.66 secs
   sys time    0.78 secs   943.00 micros    0.78 secs
```


Basics of Kubernetes Pods

Setting the Base - Docker Analogy

You have recently **installed Docker** in your system.

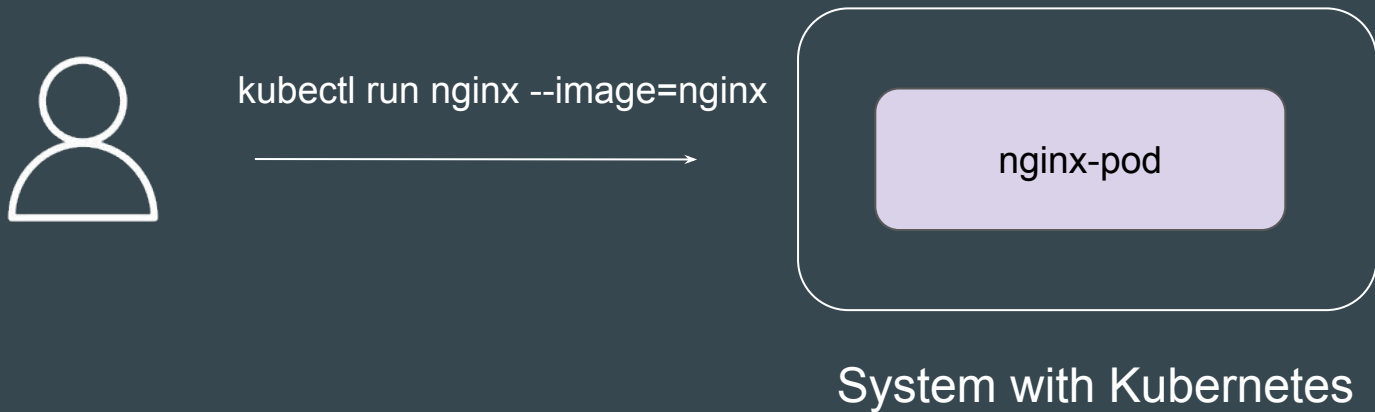
What is the first thing that you might do?



Setting the Base - Kubernetes Analogy

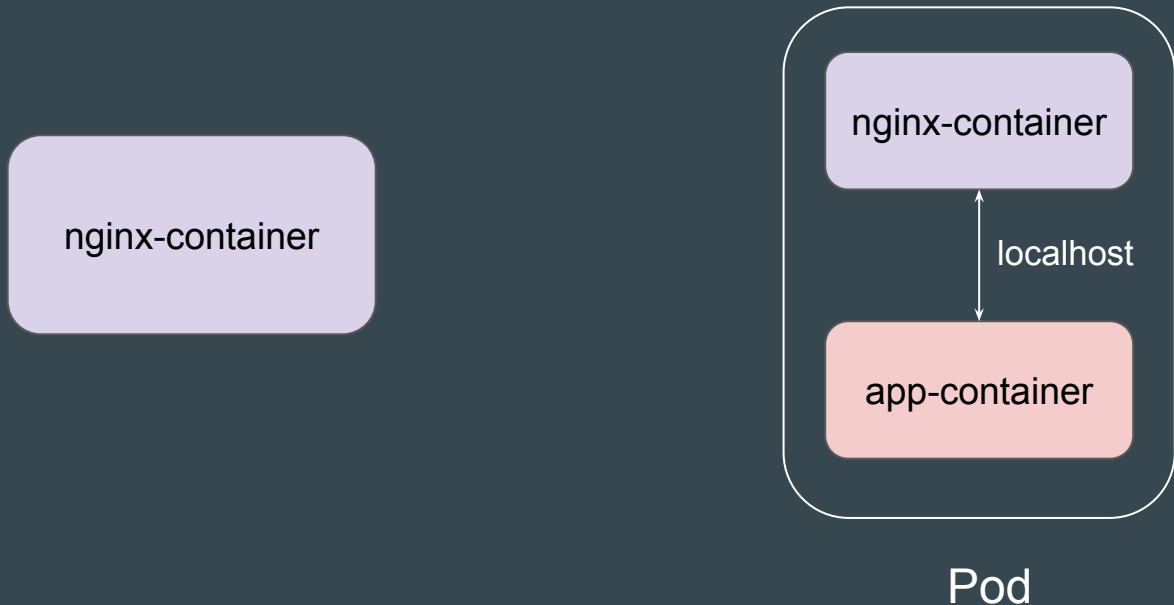
You have recently **configured Kubernetes** cluster.

What is the first thing that you might do?



Difference Between Container and Pods

A Pod can contain **one or more Docker containers** that share the same network namespace and storage volumes



Comparison Table - Docker vs Kubernetes Commands

Docker Command	Kubernetes Command	Purpose
docker run nginx	kubectl run nginx --image=nginx	Create and run a container/pod
docker ps	kubectl get pods	List running containers/pods
docker logs <container>	kubectl logs <pod>	View logs
docker inspect <container>	kubectl describe pod <pod-name>	Get detailed info
docker exec -it <container> bash	kubectl exec -it <pod-name> -- bash	Execute interactive shell
docker rm <container>	kubectl delete pod <pod-name>	Remove container/pod

Points to Note - Kubernetes Pod

A Pod always runs on a Node.

A Node is a worker machine in Kubernetes.

Each Node is managed by the Kubernetes Control Plane.

A Node can have multiple pods.

Multiple Ways to Create Kubernetes Objects

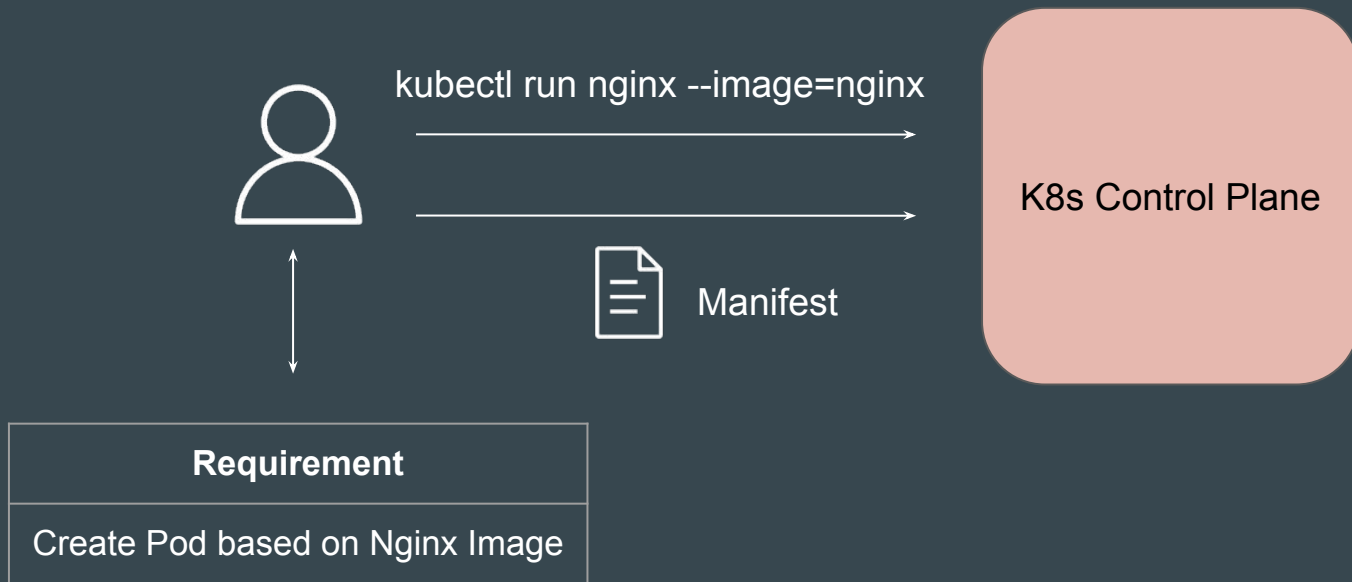
Setting the Base

We have been using simple approach of using `kubectl` command to create object like Pods in Kubernetes.

```
root@minikube:~# kubectl run nginx --image=nginx  
pod/nginx created
```


2 Primary Ways to Create Object

Use **kubectl run** command or supply kubectl with **Manifest file** that contains information of resource to be created.



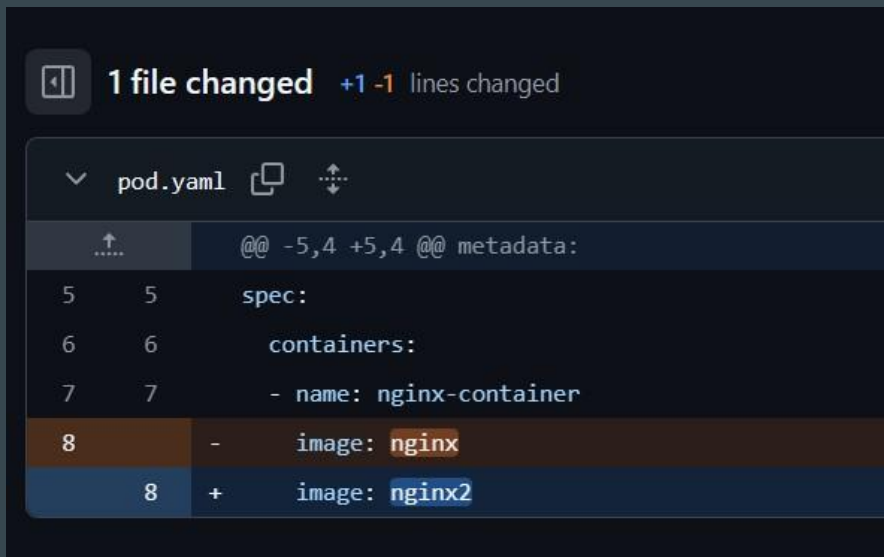
Manifest File Example

A Kubernetes manifest file is a **YAML or JSON file** that defines the desired state of a Kubernetes object.

```
! pod.yaml
  apiVersion: v1
  kind: Pod
  metadata:
    name: nginx
  spec:
    containers:
    - name: nginx-container
      image: nginx
```

Benefits of Manifest File - Version Control

Manifest files can be **stored in version control systems like Git**, allowing you to track changes to your infrastructure over time and easily roll back to previous versions.



The screenshot shows a diff view for a file named `pod.yaml`. At the top, a summary bar indicates "1 file changed" with a net change of "+1 -1 lines changed". Below this, the file name `pod.yaml` is shown with icons for a dropdown, copy, and diff. The diff content shows a change in the `image` field of a container specification. Line 8 of the previous version (marked with a minus sign) had `image: nginx`, while the current version (marked with a plus sign) has `image: nginx2`. The surrounding context includes a `metadata:` section and a `spec:` section with a `containers:` list.

```
@@ -5,4 +5,4 @@ metadata:
5      5      spec:
6      6      containers:
7      7      - name: nginx-container
8      -      image: nginx
8      +      image: nginx2
```

Benefits of Manifest File - Multiple Resources

You can define multiple rest of dependent resource objects that you want to create in a single manifest file.

```
! demo.yaml ●
! demo.yaml
  apiVersion: v1
  kind: Pod
  metadata:
    name: nginx-pod
    labels:
      app: nginx
  spec:
    containers:
      - name: nginx-container
        image: nginx:latest
        ports:
          - containerPort: 80

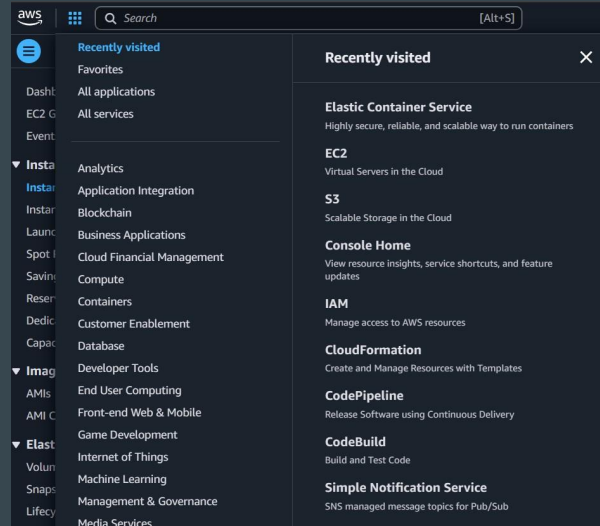
---
  apiVersion: v1
  kind: Service
  metadata:
    name: nginx-service
  spec:
    type: NodePort
    selector:
      app: nginx
    ports:
```

Basics of Kubernetes Resource Types

Sample Analogy - Hosting Provider

During the early times, one of the primary offerings of hosting providers was servers / virtual machines.

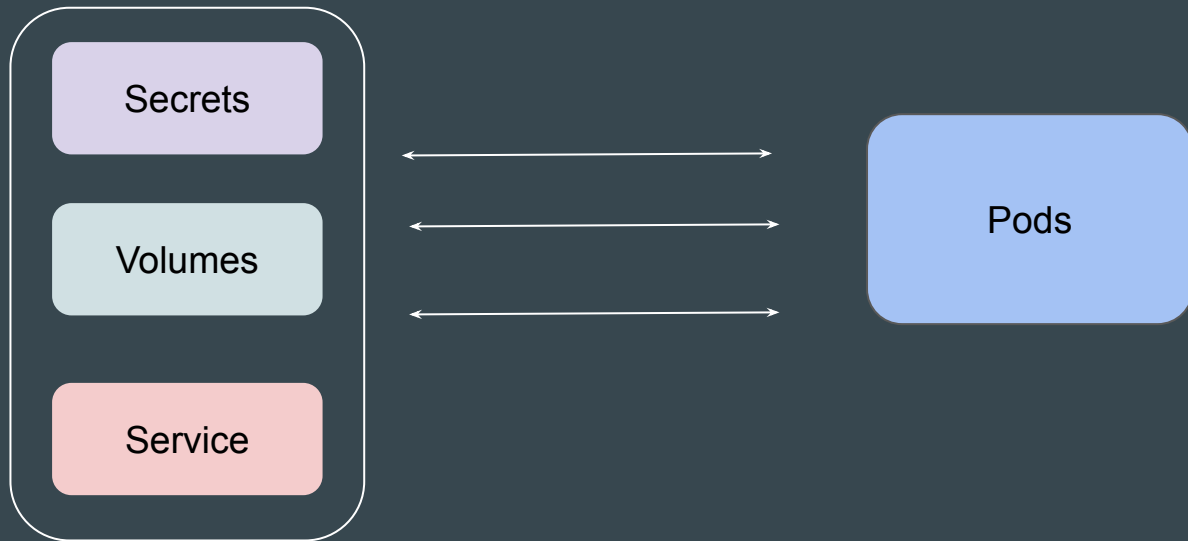
Nowadays, there are 100s of services offered by Cloud providers that aid in the entire lifecycle management of applications and custom use-cases of organizations.



Understanding from Kubernetes Perspective

Although Pods allow users to host custom applications in containers, it might not always be enough.

Based on use-case, Pod might also require access to other Kubernetes Objects.



Kubernetes Resource Types

Kubernetes has a broad set of **resource types** that organizations can use based on their requirements.

Pod The smallest deployable unit in Kubernetes, representing a single instance of a running process.	Deployment Manages multiple replicas of Pods and supports rolling updates.	Service Exposes Pods to the network and provides stable load balancing.
ConfigMap Stores non-sensitive configuration data for applications.	Secret Stores sensitive data like passwords and keys securely.	PersistentVolume Represents storage resources for persistent data in the cluster.
PersistentVolumeClaim Requests a specific amount of storage from a PersistentVolume.	Ingress Manages external HTTP and HTTPS traffic to Services in the cluster.	Namespace Provides a way to group and isolate resources within the cluster.

Reference Screenshot

```
C:\Users\zealv>kubectl api-resources
```

NAME	SHORTNAMES
bindings	
componentstatuses	cs
configmaps	cm
endpoints	ep
events	ev
limitranges	limits
namespaces	ns
nodes	no
persistentvolumeclaims	pvc
persistentvolumes	pv
Pods	po
podtemplates	
replicationcontrollers	rc
resourcequotas	quota
secrets	
serviceaccounts	sa
services	svc

Basic Structure of a Manifest File

Sample - Pod Manifest File

```
! pod.yaml
  apiVersion: v1
  kind: Pod
  metadata:
    name: nginx
  spec:
    containers:
      - name: nginx-container
        image: nginx
```

Comparing Manifest Structure with Pod Manifest

```
! manifest.yaml
  apiVersion: [API version]
  kind: [Resource type]
  metadata:
    name: [Resource name]
  spec:
    [Resource-specific configuration]
```



```
! pod.yaml
  apiVersion: v1
  kind: Pod
  metadata:
    name: nginx
  spec:
    containers:
      - name: nginx-container
        image: nginx
```

Basic Structure of Manifest File

```
! manifest.yaml
  apiVersion: [API version]
  kind: [Resource type]
  metadata:
    name: [Resource name]
  spec:
    [Resource-specific configuration]
```



Component	Description
apiVersion	Specifies which version of the Kubernetes API to use
kind	Type of resource you are creating.
metadata	Information about resource
spec	Details about resource to be created

Generating Manifest File through CLI Command

Setting the Base

CLI commands are easy to run, and Manifest file provides a lot of benefits for organizations and team collaboration.

Finding an easier way to generate a manifest file is needed.

Kubectrl Command

Generate

```
! pod.yaml
  apiVersion: v1
  kind: Pod
  metadata:
    name: nginx
  spec:
    containers:
      - name: nginx-container
        image: nginx
```

Basics of Dry Runs

The `--dry-run=client` option allows you to validate a Kubernetes resource definition without actually applying it to the cluster.

```
C:\kplabs-k8s>kubectl run nginx --image=nginx --dry-run=client  
pod/nginx created (dry run)
```


Exploring Output of Kubectl command

By default, when you run the basic **kubectl** command to create an object like Pod, in the output, it will just print the confirmation message.

```
C:\>kubectl run nginx --image=nginx  
pod/nginx created
```

Kubectl with Output of YAML

When kubectl command is used with output of yaml, the command doesn't just create the resource in the cluster; it also **prints the full YAML configuration of the created resource** to your terminal.

```
C:\kplabs-k8s>kubectl run nginx --image=nginx -o yaml
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: "2025-01-01T04:11:42Z"
  labels:
    run: nginx
  name: nginx
  namespace: default
  resourceVersion: "1599760"
  uid: 2f321eaf-15b5-457d-8d58-d644ede8a6cc
spec:
  containers:
  - image: nginx
    imagePullPolicy: Always
    name: nginx
```

Combining Dry Run Output of YAML

When `--dry-run=client` is combined with `-o yaml`, it will generate manifest file for you associated with the command without actually deploying the object in Kubernetes.

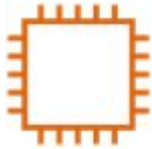
```
C:\>kubectl run nginx --image=nginx --dry-run=client -o yaml
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: nginx
  name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
```

Labels and Selectors

Let's get started

Overview of Labels

Labels are key/value pairs that are attached to objects, such as pods.



Server



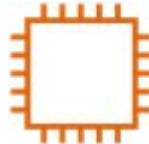
Database



Load Balancer



Load Balancer

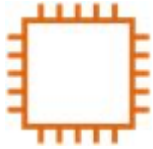


Server



Database

Adding Labels to Resource



name: kplabs-gateway
env: production



name: kplabs-db
env: production



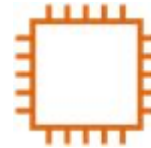
name: kplabs-elb
env: production



name: kp-elb
env: dev



name: kp-db
env: dev



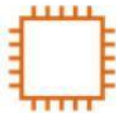
name: kp-gateway
env: dev

Overview of Selectors

Selectors allows us to filter objects based on labels.

Example:

1. Show me all the objects which has label where **env: prod**



name: kplabs-gateway
env: production



name: kplabs-db
env: production



name: kplabs-elb
env: production

Overview of Selectors

Selectors allows us to filter objects based on labels.

Example:

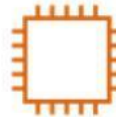
2. Show me all the objects which has label where **env: dev**



name: kp-elb
env: dev



name: kp-db
env: dev



name: kp-gateway
env: dev

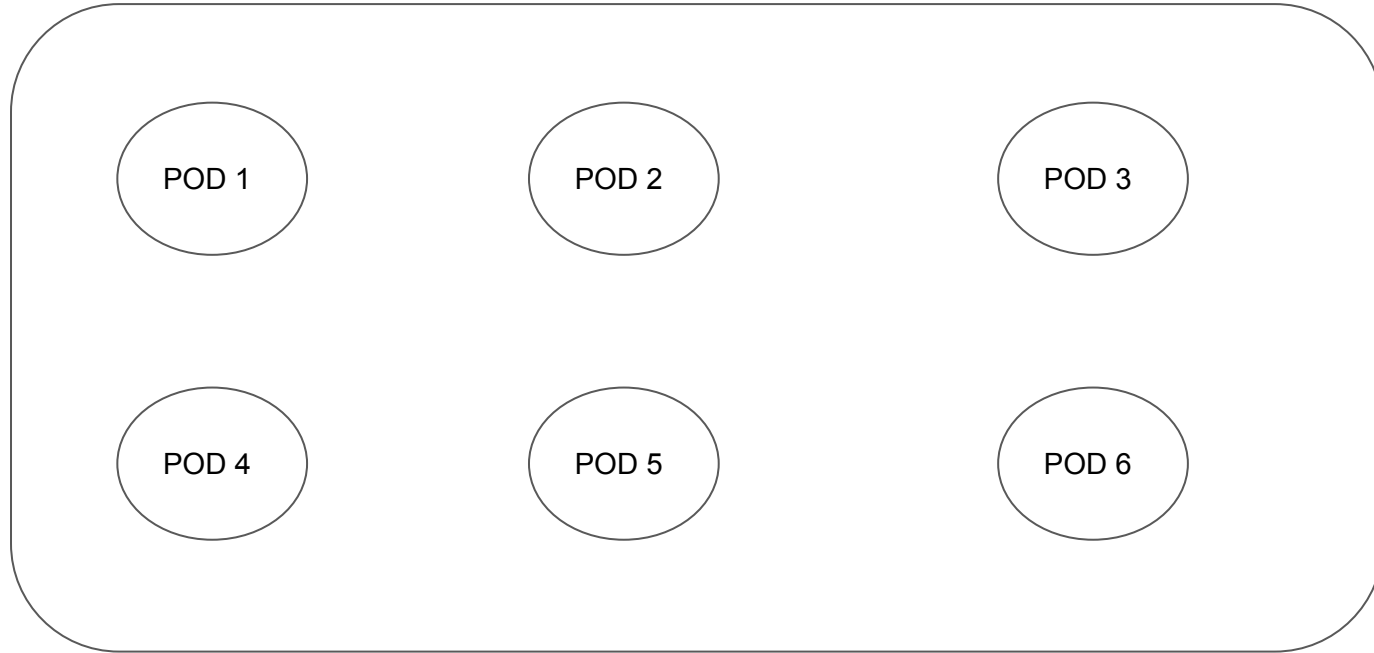
Kubernetes Perspective

There can be multiple objects within a Kubernetes cluster.

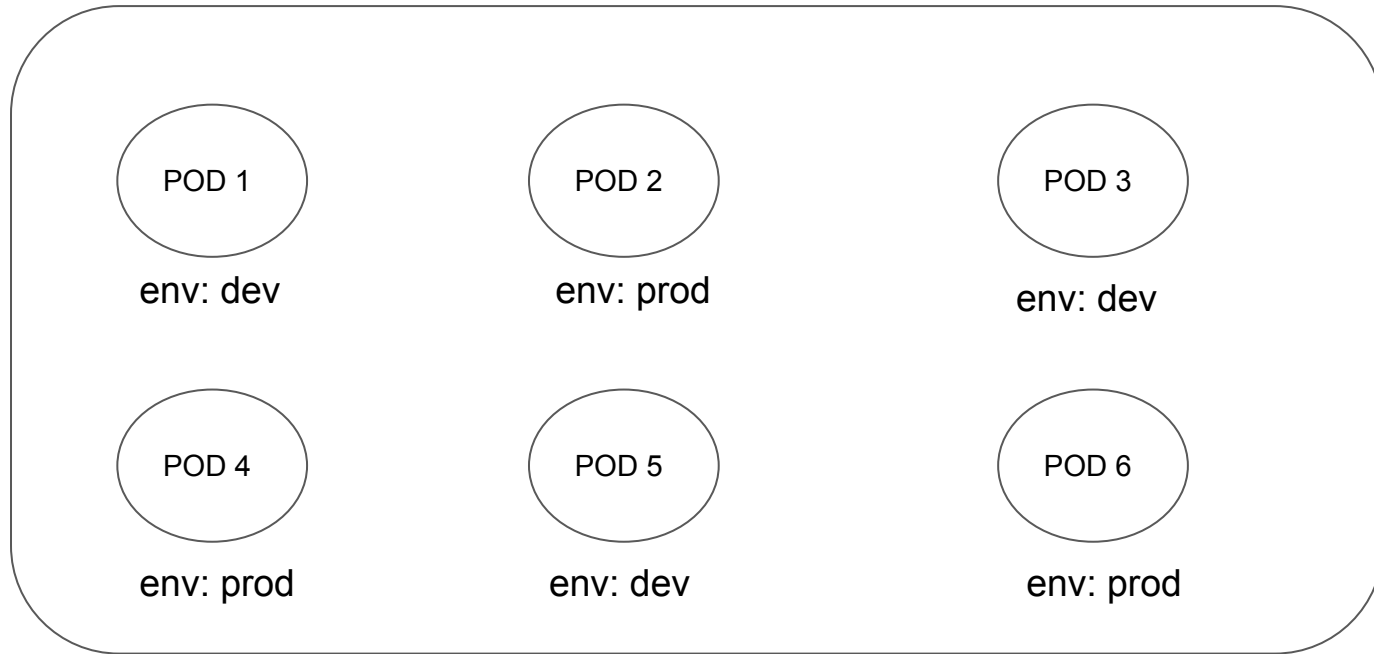
Some of the objects are as follows:

1. Pods
2. Services
3. Secrets
4. Namespaces
5. Deployments
6. Daemonsets

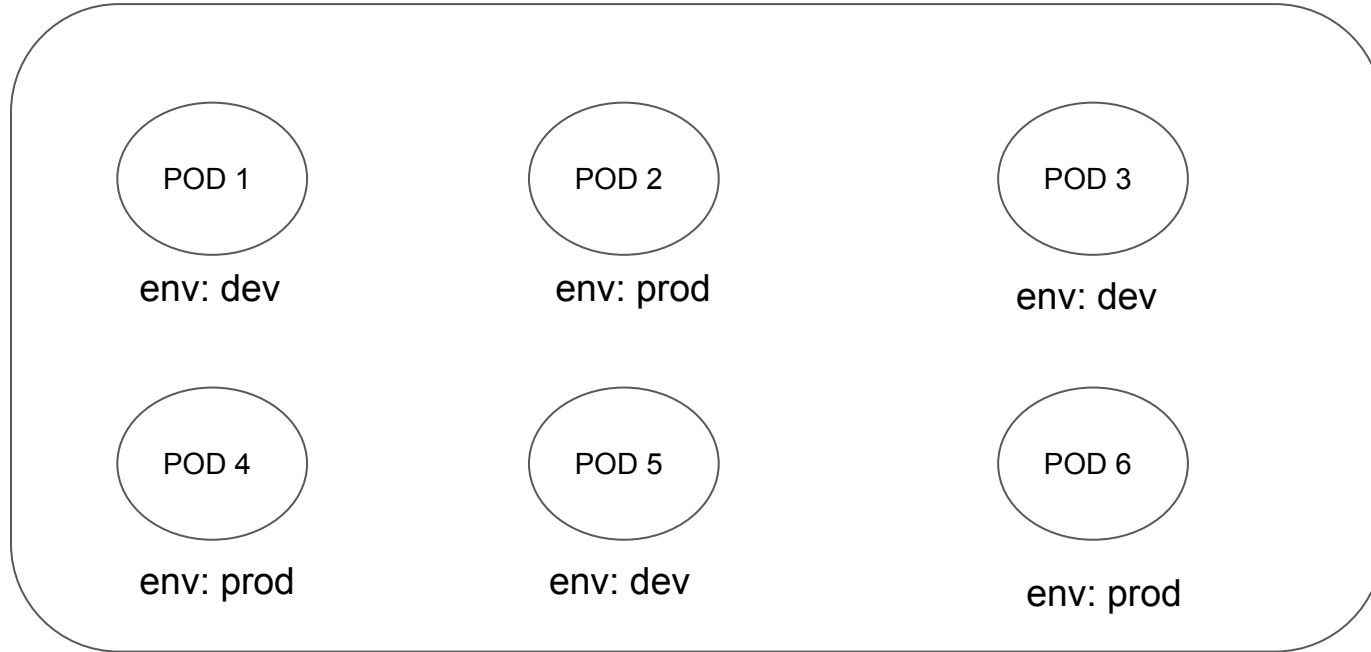
Kubernetes Perspective



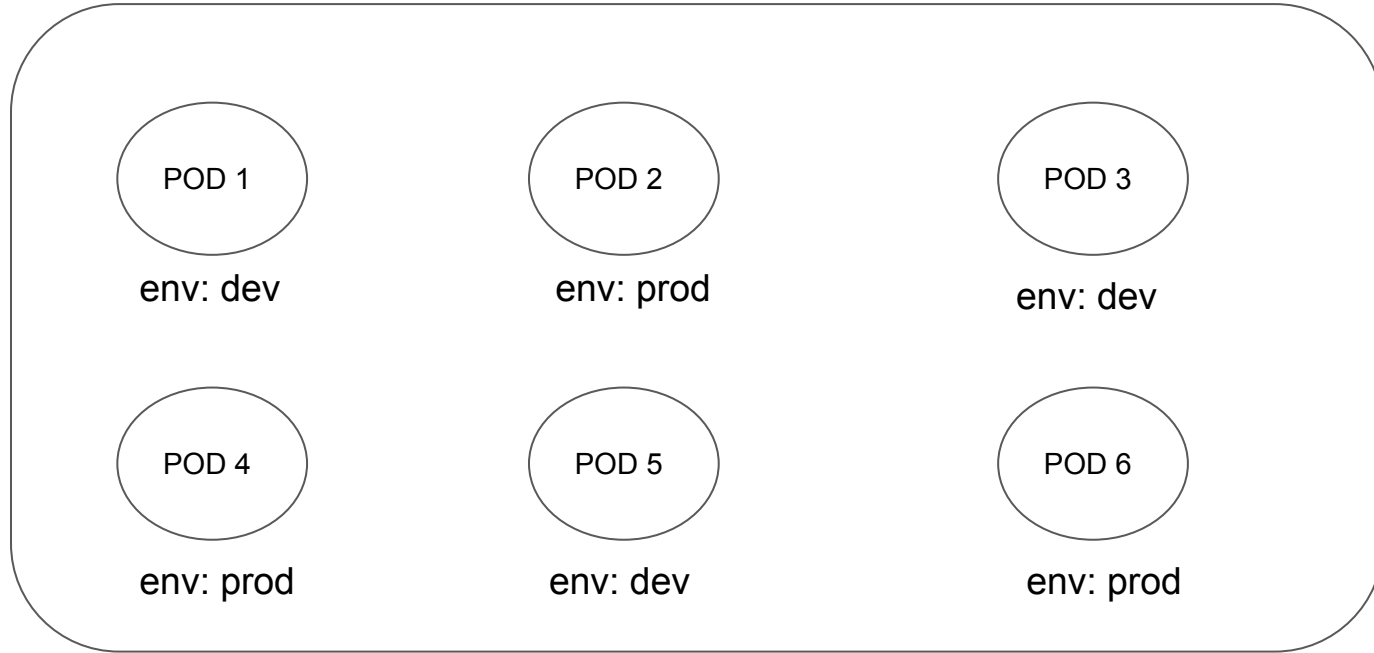
Assigning Labels to Kubernetes Objects



Selector: List All Pods from Dev Environment



Selector: List All Pods from Production Environment



ReplicaSet

Setting the Base

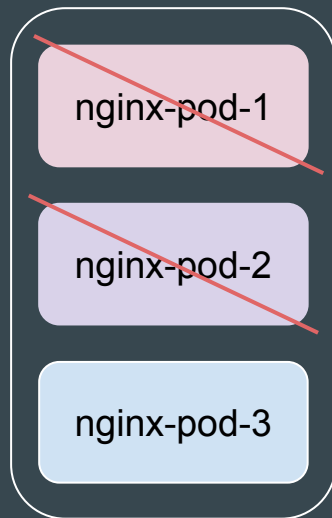
There is a requirement to run 3 Pods based on Nginx image all the time.



Issue with the Manual Step - Part 1

If a Pod fails (node failure, process crash, etc.), Kubernetes will not automatically recreate it.

You would have to manually detect the failure and recreate the Pod.



Issue with the Manual Step - Part 2

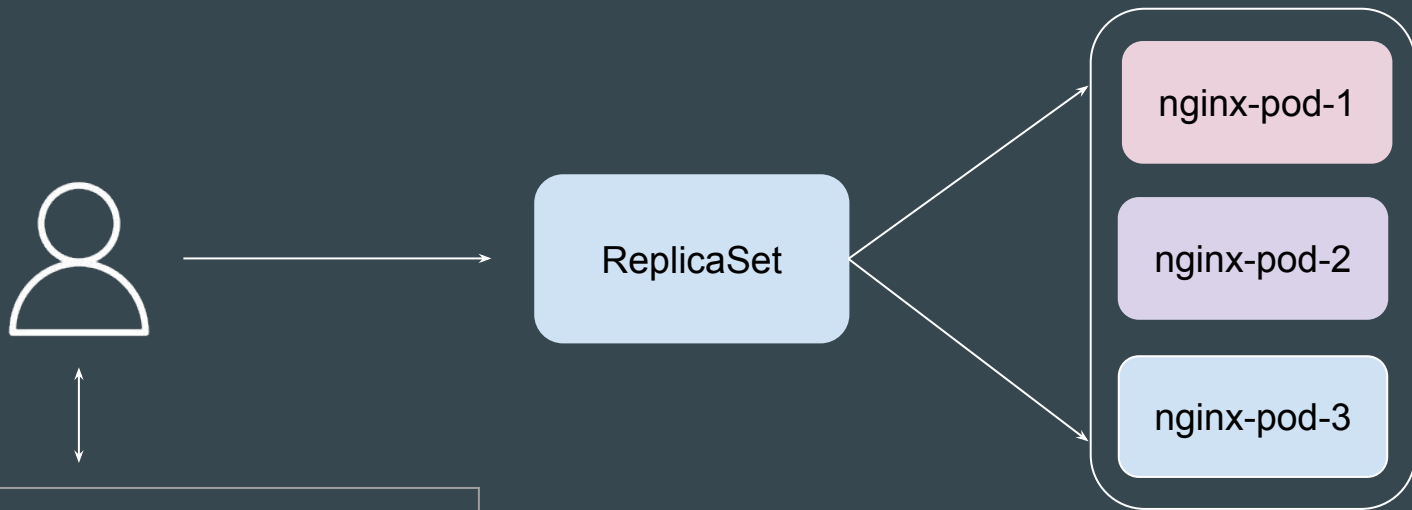
If you need to scale to more or fewer Pods, you would have to manually create or delete Pod definitions and apply the changes.

Example: Requirement is to run 20 Pods based on Nginx Image



Introducing Kubernetes ReplicaSet

A ReplicaSet's purpose is to maintain a stable set of replica Pods running at any given time.



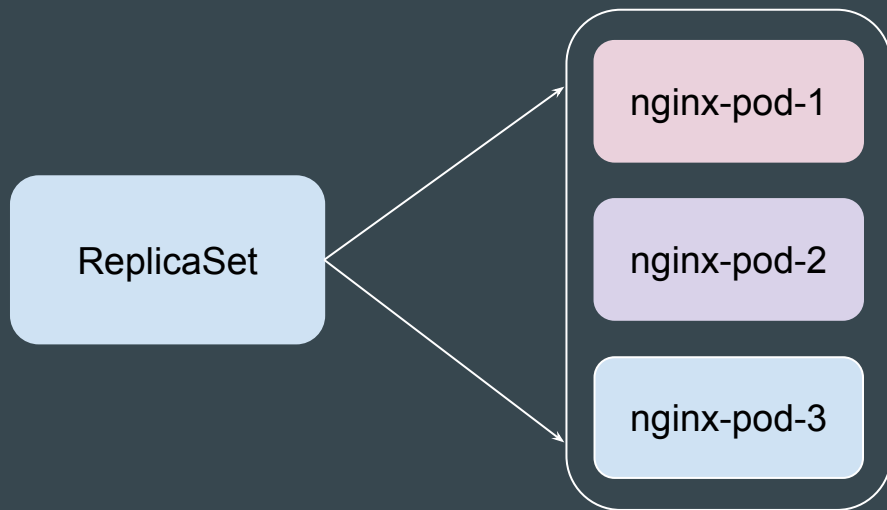
Requirement

I need 3 Pods of Nginx Run all the Time

Challenges with ReplicaSets

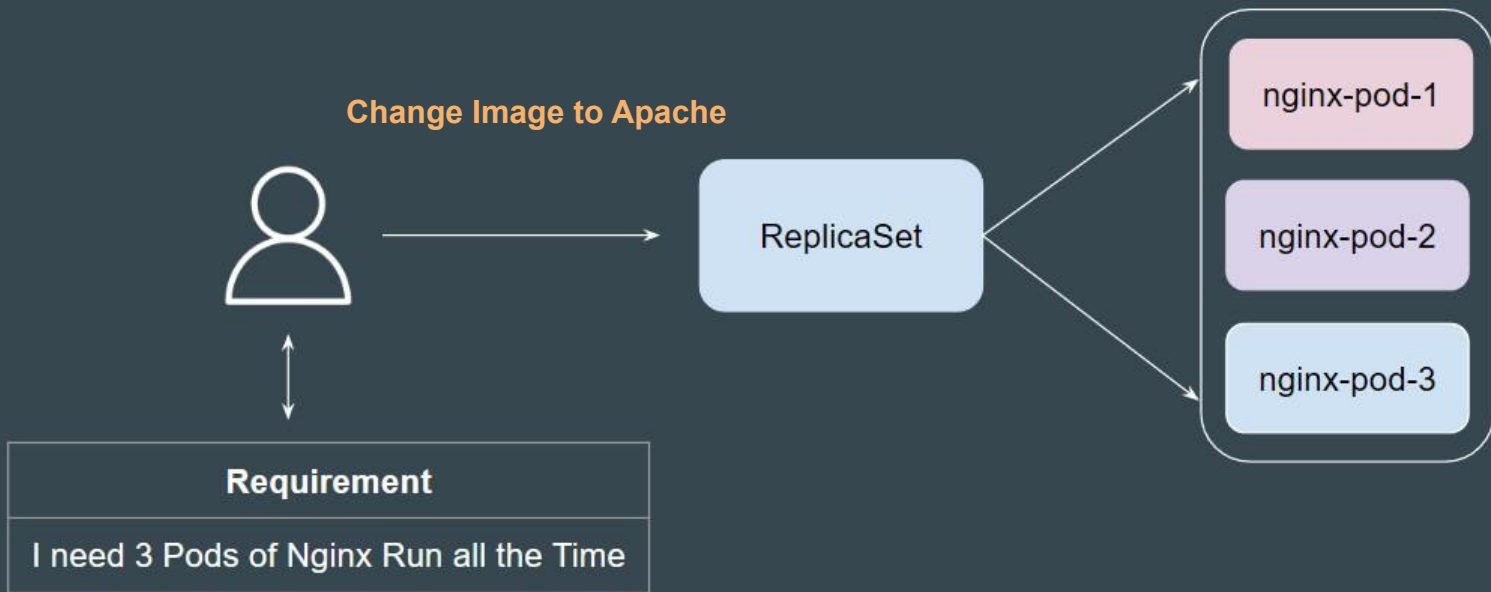
Setting the Base

ReplicaSets are **primarily designed to maintain a specific state of running Pods**, not to manage regular updates or changes to their configuration



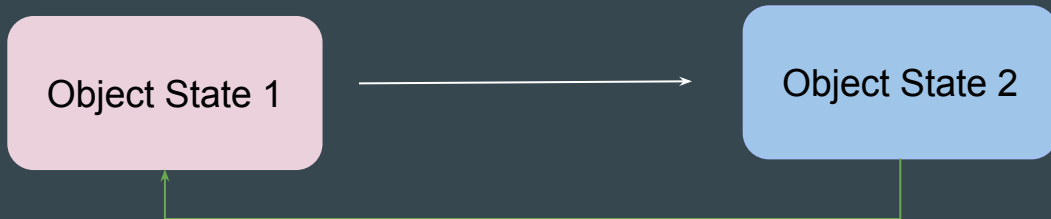
Challenge 1 - Updating Container Image

When you update the pod template (e.g., **change the container image**) in a ReplicaSet, the existing pods are not updated.



Challenge 2 - No Built-In Rollback Mechanism

ReplicaSets **lack a rollback mechanism** for reverting to a previous configuration in case of errors during an update.



Rollback to Previous State

Challenge 3 - Label Collision with ReplicaSet Selectors

When a ReplicaSet's selector matches labels of pods that it didn't create, it starts treating those pods as part of its managed set. This can cause unintended consequences.



Overview of Deployments

Simple Analogy - Camera and Lens

Think of the camera body as the ReplicaSet.

It is the core mechanism that controls and ensures that the right number of photos (or Pods) are being captured



Simple Analogy - Camera and Lens

Now, imagine attaching a big, powerful lens to the camera body. The lens is the Deployment.

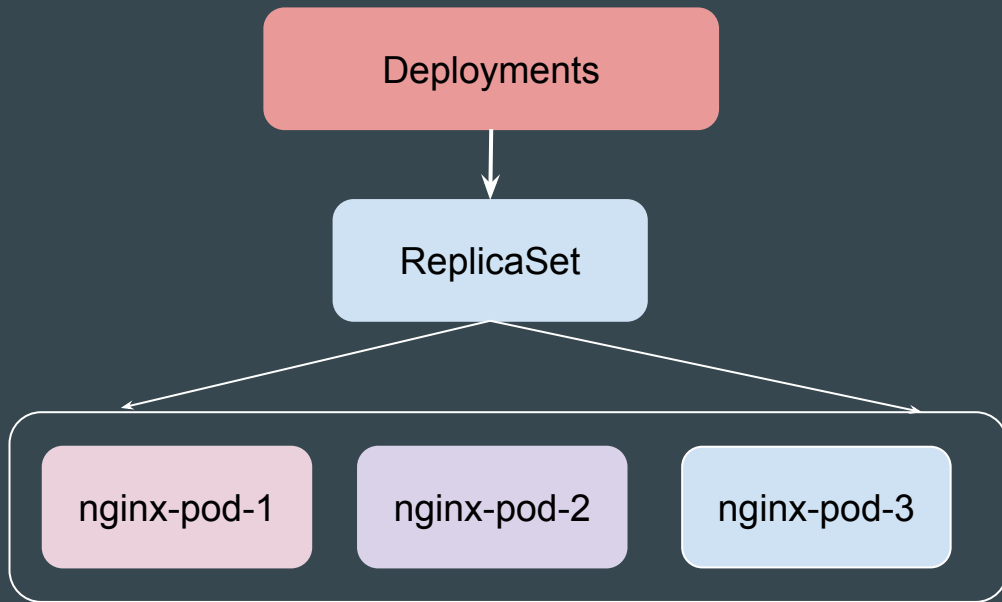
It doesn't just sit on top of the camera; it enhances and extends the camera's functionality, allowing for new perspectives.



Setting the Base

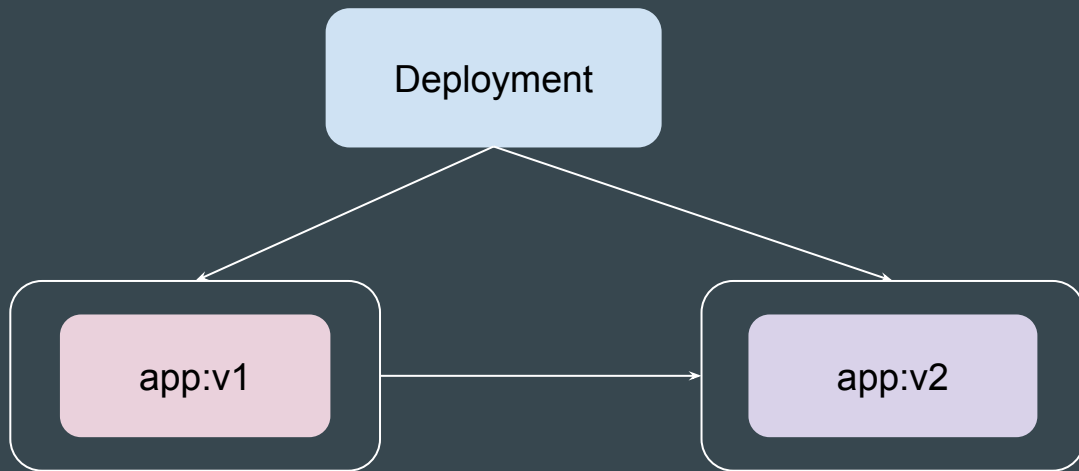
A Deployment is a higher-level abstraction built on top of ReplicaSets.

It not only manages ReplicaSets but also **provides advanced features** like rolling updates, rollbacks, and versioning.



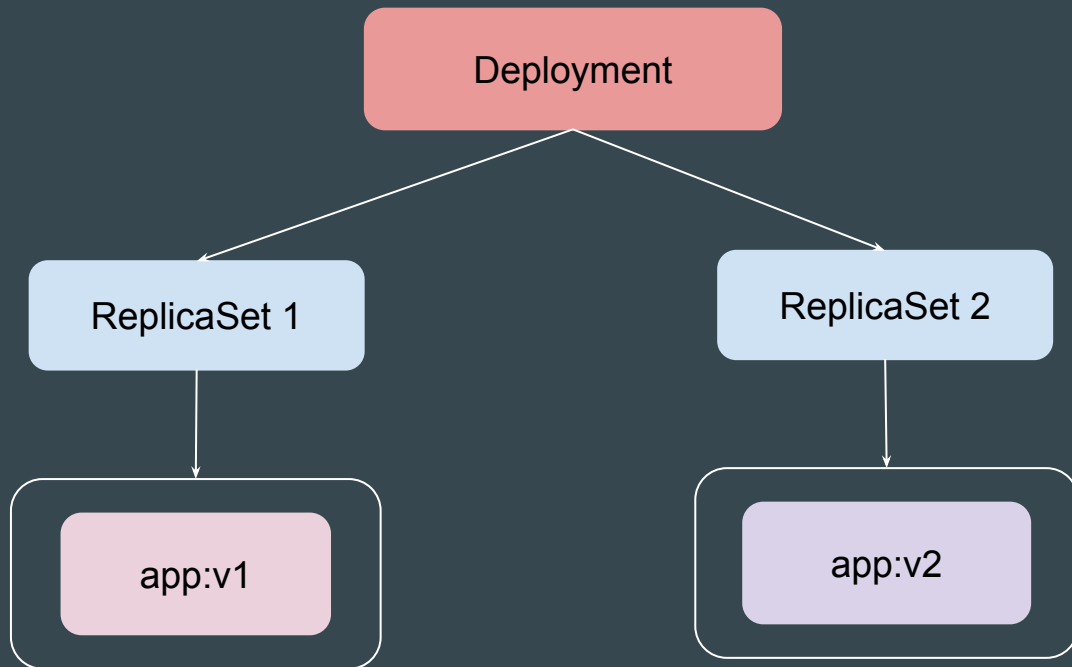
Use-Case: Update the Container Image

You can easily change the specifications like Container Image and other spec.



Use-Case: Rolling Update

Deployments will perform update in rollout manner to ensure that your app is not down.



Rollout History

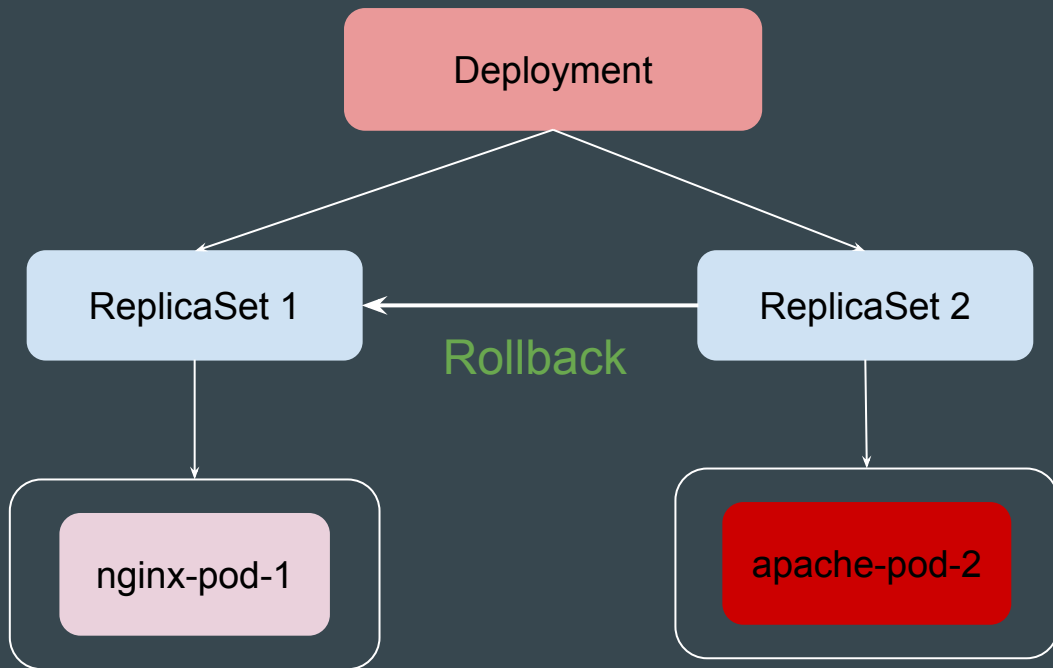
Deployment allows us to inspect the history of your deployments.

It provides a record of changes made to a deployment over time, enabling you to understand the evolution of your application and troubleshoot potential issues.

```
C:\>kubectl rollout history deployment/nginx-deployment
deployment.apps/nginx-deployment
REVISION  CHANGE-CAUSE
1          <none>
2          <none>
```

Rolling Back Changes

Sometimes, you may want to rollback a Deployment; for example, when the Deployment is not stable, such as crash looping



Key Differences - ReplicaSet and Deployments

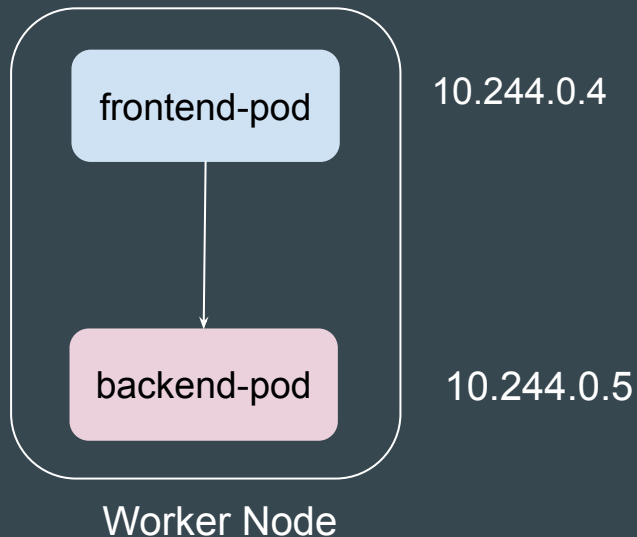
Feature	ReplicaSet	Deployment
Abstraction Level	Lower-level	Higher-level
Primary Function	Ensures a specific number of replicas are running.	Manages ReplicaSets and handles updates.
Rolling Updates	Not supported	Supported with configurable strategies.
Rollbacks	Not supported	Supported with version history.
Use Case	Simple, static workloads.	Dynamic workloads with frequent updates.

Overview of Service

Setting the Base

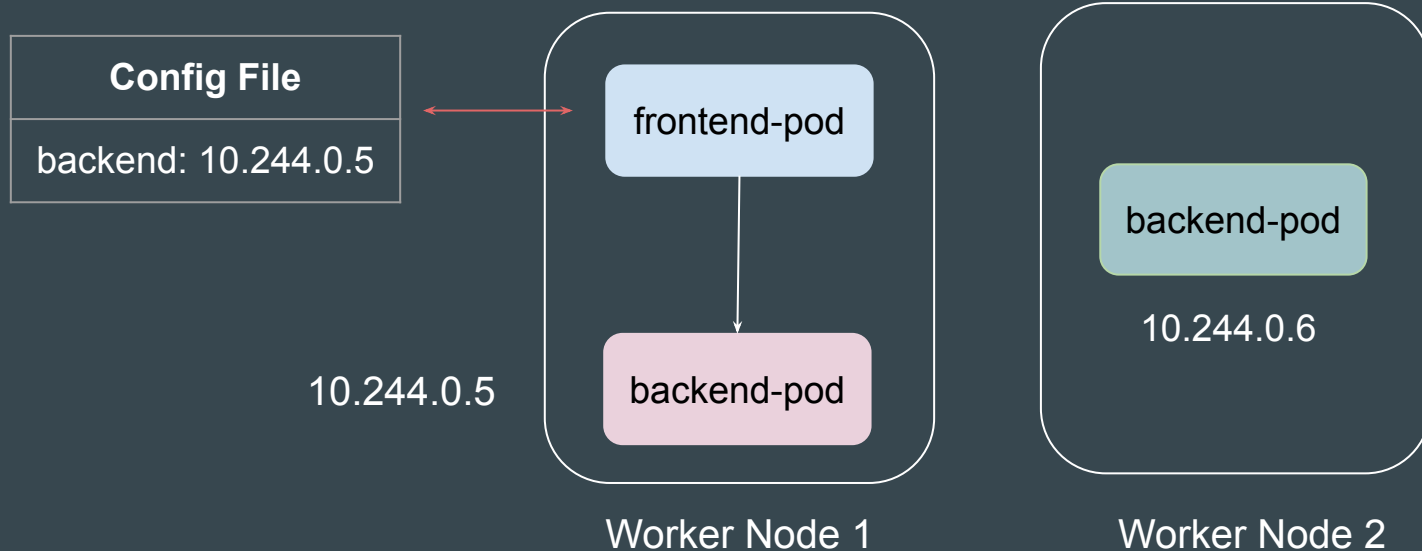
The Pods that get created in the Worker node have a private IP associated with them.

Pods in the same cluster can communicate with each other using Private IPs.



Use-Case: Frontend to Backend

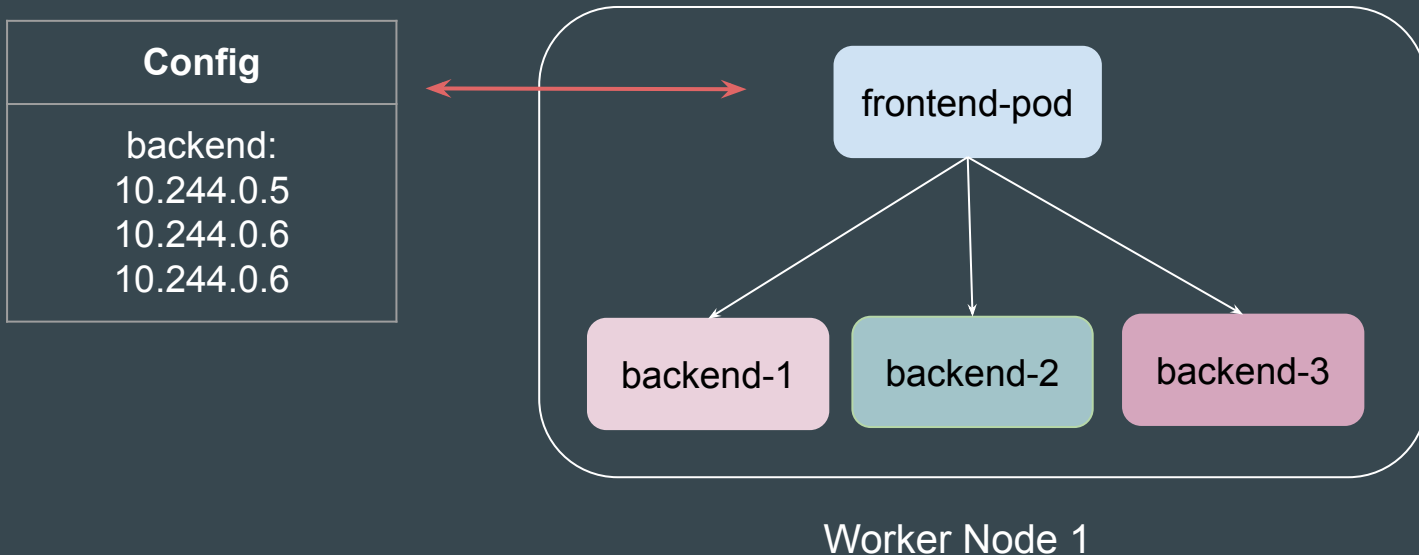
If the IP address of the backend pod is **hardcoded** in the front-end pod, it will create issues since Pods can be ephemeral.



Use-Case: Frontend to Backend

If backend pods are running as deployment, it is challenging to hardcode the IPs of each backend pod.

You would also like to distribute the traffic across all the backend pods.

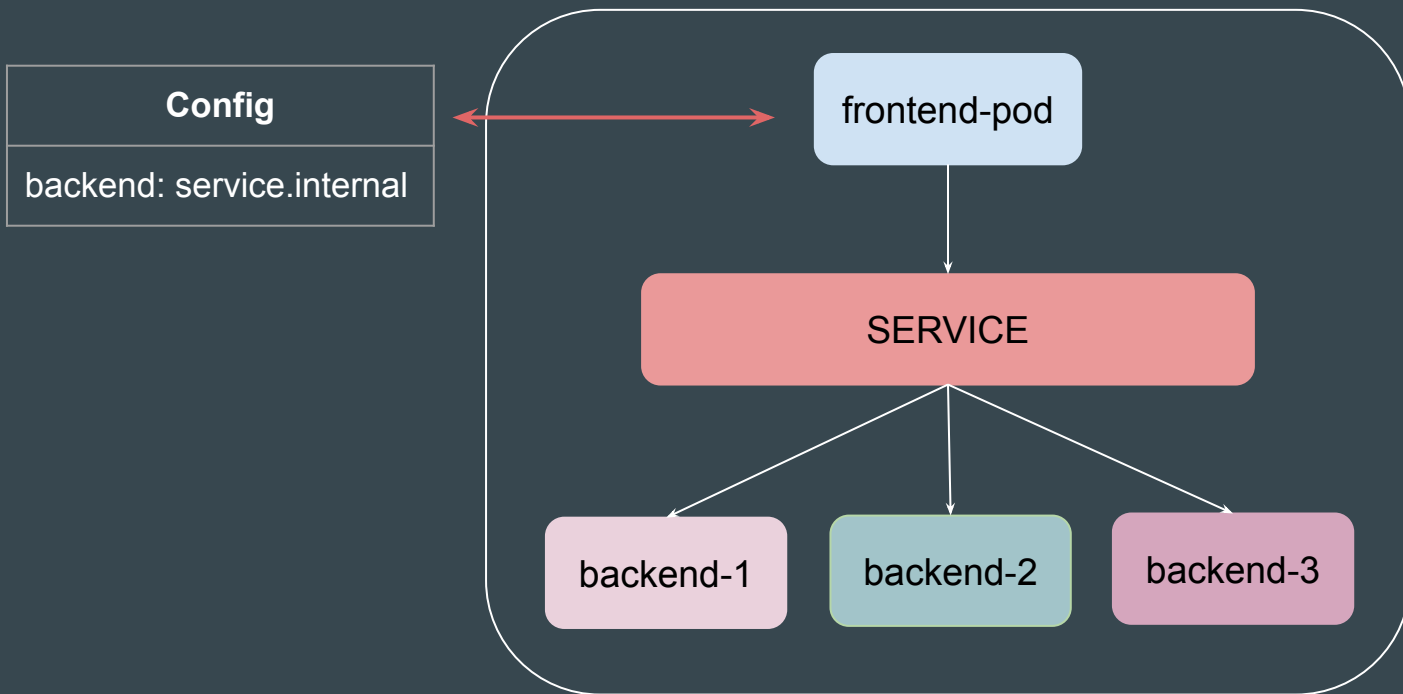


Reference Screenshot - Distributing Traffic

```
root@frontend-pod:/# curl service.internal
Backend Pod 2
root@frontend-pod:/# curl service.internal
Backend Pod 1
root@frontend-pod:/# curl service.internal
Backend Pod 2
root@frontend-pod:/# curl service.internal
Backend Pod 1
root@frontend-pod:/# curl service.internal
Backend Pod 2
root@frontend-pod:/# curl service.internal
Backend Pod 1
```

Introducing Service

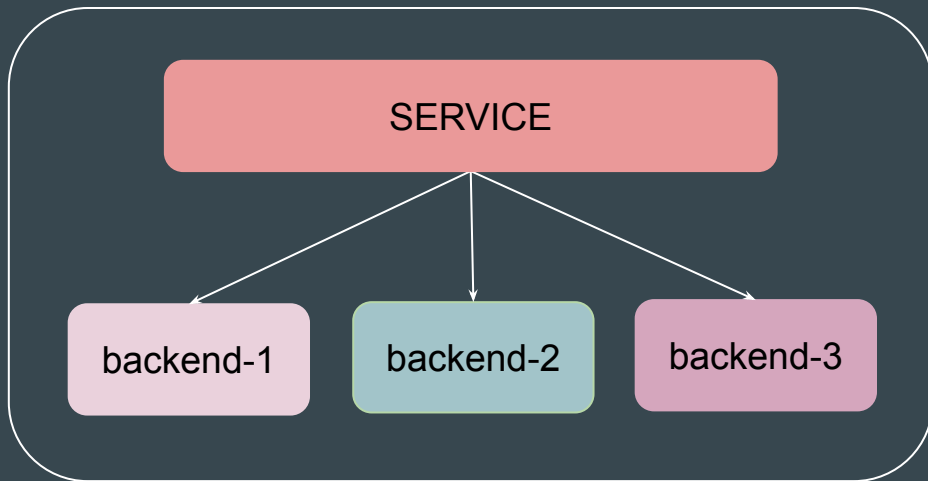
Service acts as a gateway that distributes incoming traffic between its endpoints.



Service and Deployments

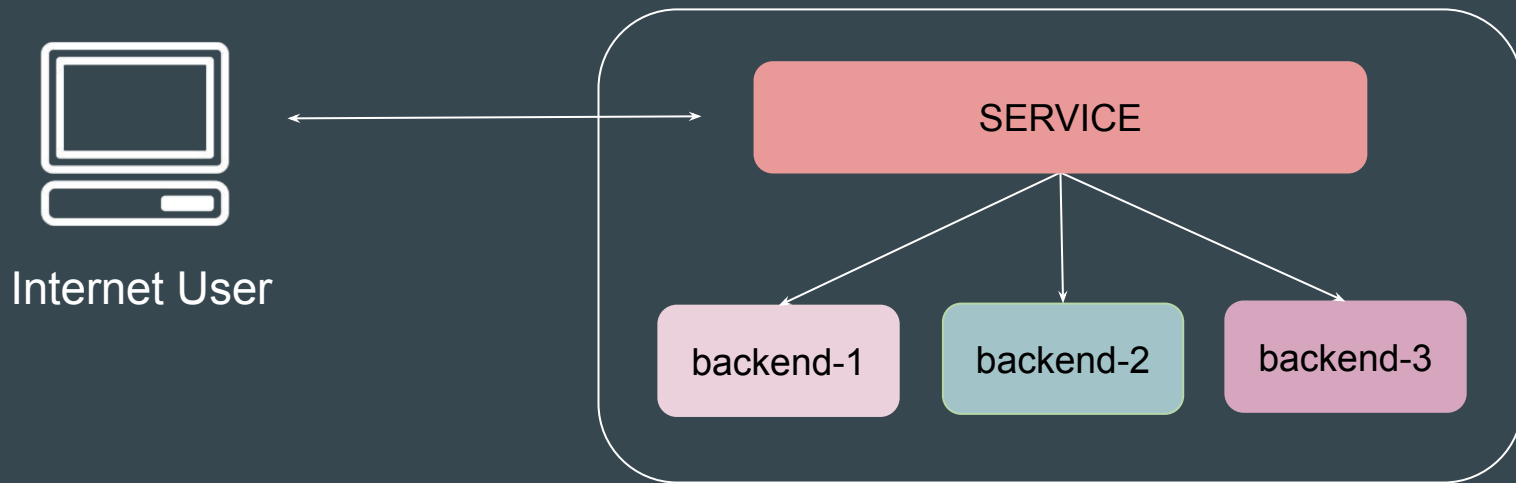
If you use a Deployment to run your app, that Deployment can create and destroy Pods dynamically.

Service can find the latest set of pods and route the traffic accordingly.



Points to Note

Using Service, users outside of Kubernetes cluster can also connect to the Pods internal to the cluster.



Benefits of Service

Pods are ephemeral, and their IPs change frequently. Services provide a stable endpoint.

Services distribute traffic across multiple Pod replicas.

Service enables exposing applications to external traffic like from the Internet.

Types of Services

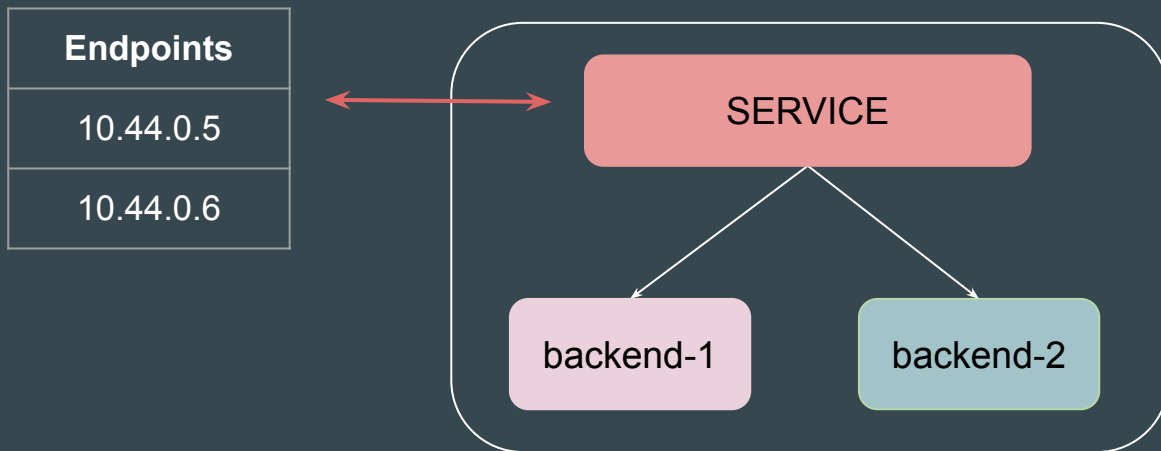
Service Type	Key Features	Use-Cases
ClusterIP	Default Service type. Accessible only within the cluster.	Internal microservices communication
NodePort	Exposes service on a static port (30000-32767) on each Node.	Development testing, demo applications
LoadBalancer	Exposes service externally using cloud provider's load balancer	Production applications requiring external access
ExternalName	Maps service to external DNS name	External service integration

Practical - Service and Endpoints

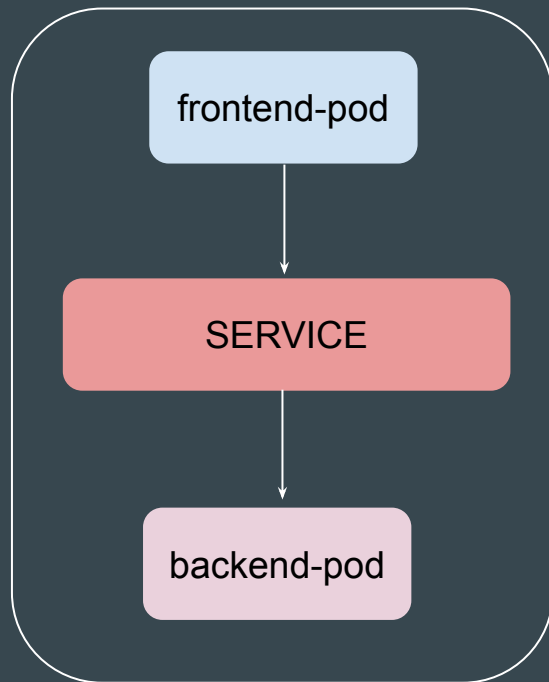
Setting the Base

Service is a gateway that distributes the incoming traffic between its endpoints

Endpoints contain the address of underlying Pods to which the service will route the traffic to.



Architecture of Today's Video



Step 1 - Create Simple Service

Create a Simple Service in Kubernetes. This service will have its own IP.

```
C:\>kubectl describe service kplabs-service
Name:                kplabs-service
Namespace:           default
Labels:              <none>
Annotations:         <none>
Selector:            <none>
Type:                ClusterIP
IP Family Policy:    SingleStack
IP Families:         IPv4
IP:                  10.109.24.231
IPs:                 10.109.24.231
Port:                <unset> 8080/TCP
TargetPort:          80/TCP
Endpoints:           <none>
Session Affinity:    None
Events:              <none>
```

Step 2 - Create Endpoint for Service

Add appropriate endpoint to the service manually for service to route traffic to.

```
C:\>kubectl describe service kplabs-service
```

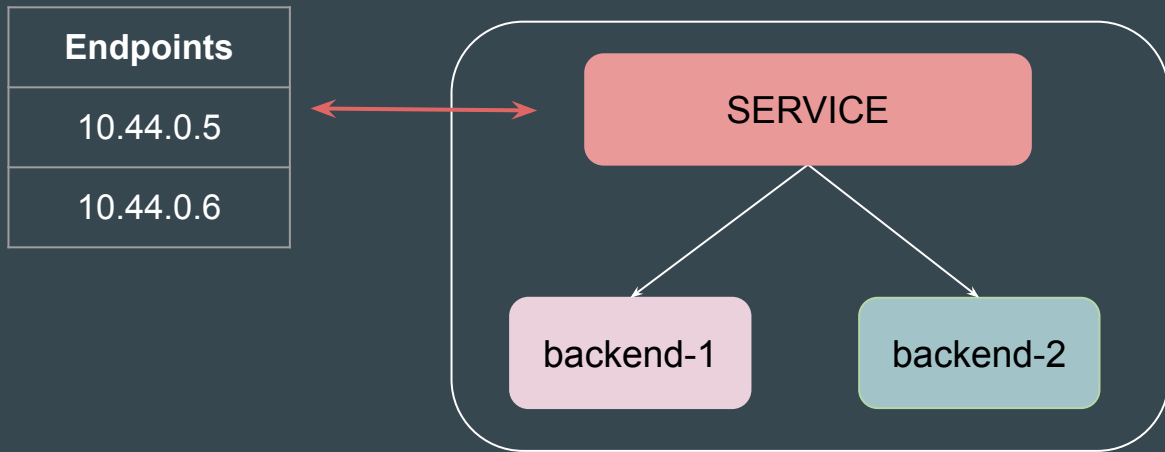
```
Name:                kplabs-service
Namespace:           default
Labels:              <none>
Annotations:         <none>
Selector:            <none>
Type:                ClusterIP
IP Family Policy:    SingleStack
IP Families:         IPv4
IP:                  10.109.12.34
IPs:                 10.109.12.34
Port:                <unset> 8080/TCP
TargetPort:          80/TCP
Endpoints:           10.108.0.1:80
Session Affinity:    None
Events:              <none>
```



Using Selector for Registering Service Endpoints

Setting the Base

In a simple manual workflow, you can create a service and manually add endpoints associated with that service.



Reference Manifest File

```
! service.yaml
  apiVersion: v1
  kind: Service
  metadata:
    name: simple-service
  spec:
    ports:
      - port: 80
        targetPort: 80
```

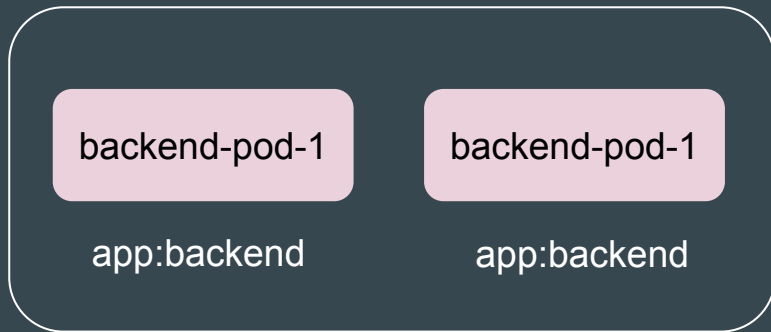


```
! endpoints.yaml
  apiVersion: v1
  kind: Endpoints
  metadata:
    name: simple-service
  subsets:
    - addresses:
        - ip: 10.108.0.67
      ports:
        - port: 80
```

Better Approach - Selectors

You can also configure the Service to use **Selectors** to automatically add appropriate Pod IPs within its endpoint.

```
! service-selector.yaml
apiVersion: v1
kind: Service
metadata:
  name: simple-service
spec:
  selector:
    app: backend
  ports:
    - port: 80
      targetPort: 80
```



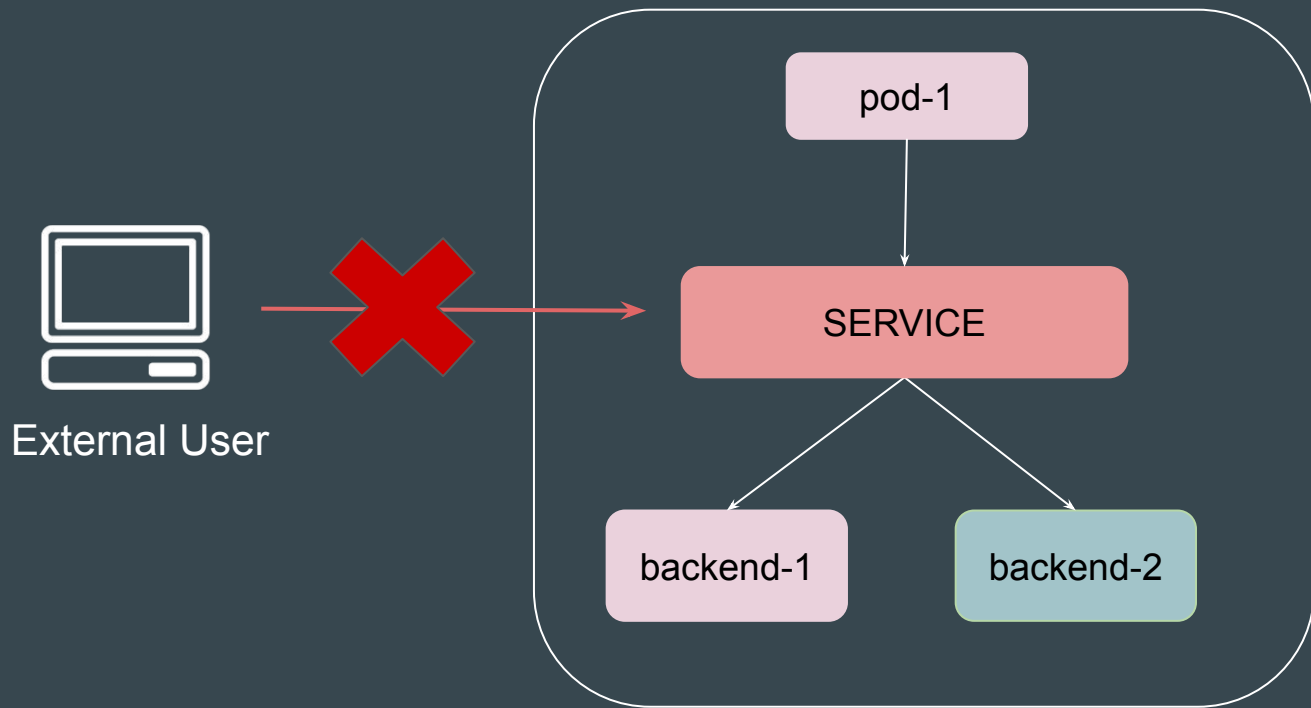
Service Type - ClusterIP

Types of Services

Service Type	Key Features	Use-Cases
ClusterIP	Default Service type. Accessible only within the cluster.	Internal microservices communication
NodePort	Exposes service on a static port (30000-32767) on each Node.	Development testing, demo applications
LoadBalancer	Exposes service externally using cloud provider's load balancer	Production applications requiring external access
ExternalName	Maps service to external DNS name	External service integration

Setting the Base

A Kubernetes Service of type **ClusterIP** provides an internal, stable IP address to expose your application **only within the Kubernetes cluster**.



Point to Note - Part 1

ClusterIP is a **default service type**. If you don't specify a type when creating a Service, Kubernetes defaults to ClusterIP.

```
! service.yaml
apiVersion: v1
kind: Service
metadata:
  name: kplabs-service
spec:
  ports:
    - port: 8080
      targetPort: 80
```



```
C:\>kubectl get service
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kplabs-service	ClusterIP	10.109.12.34	<none>	8080/TCP	6m47s
kubernetes	ClusterIP	10.109.0.1	<none>	443/TCP	21d

Point to Note - Part 2

The ClusterIP Service gets a virtual IP address (also called the cluster IP) that remains stable as long as the Service exists.

This IP can be used by other Pods inside the cluster to access the Service.

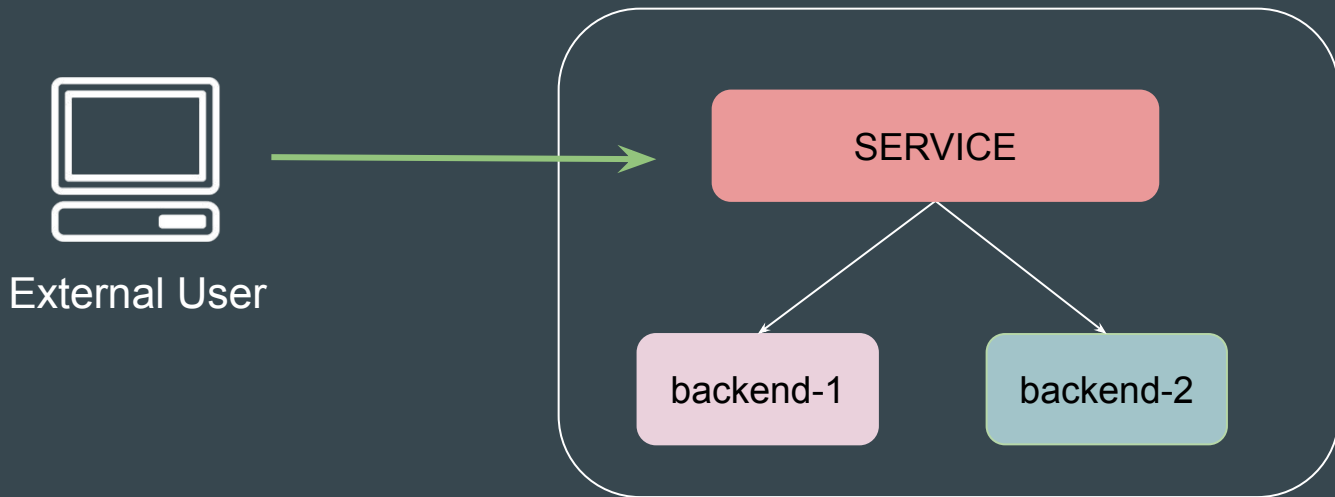
```
C:\>kubectl get service
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kplabs-service	ClusterIP	10.109.12.34	<none>	8080/TCP	6m47s
kubernetes	ClusterIP	10.109.0.1	<none>	443/TCP	21d

Service Type - NodePort

Setting the Base

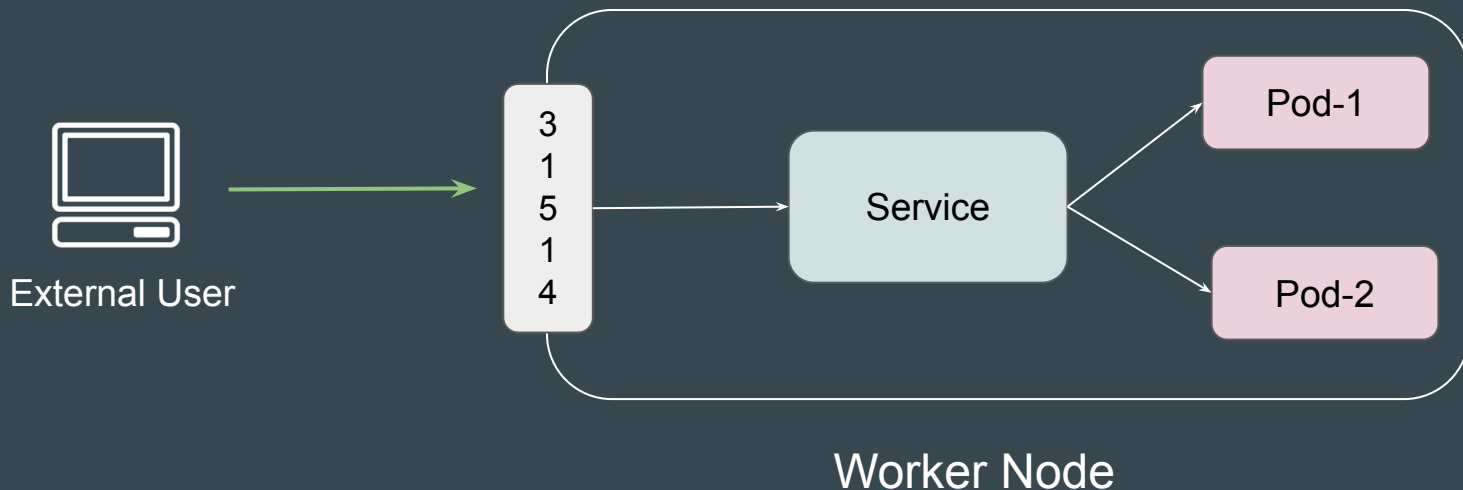
A **NodePort** is one of the service types used to **expose your application to the external world**.



How it Works

When you create a service of type NodePort, Kubernetes assigns a port from the NodePort range (default: 30000-32767) on all the nodes in the cluster..

Any traffic sent to `<NodeIP>:NodePort` is forwarded to the corresponding service



Reference Screenshot

```
C:\>kubectl get service
```

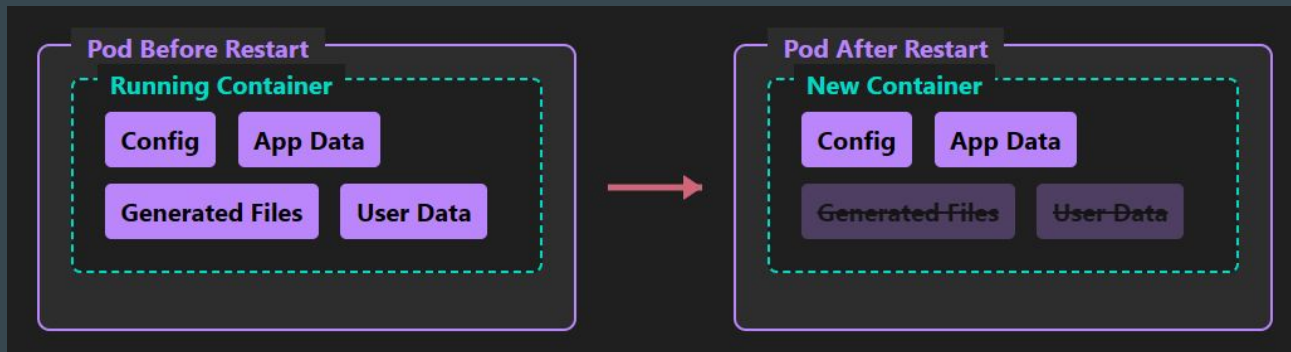
NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.109.0.1	<none>	443/TCP	22d
test-nodeport	NodePort	10.109.4.218	<none>	<u>80:30537/TCP</u>	10s

Volumes in Kubernetes

Understanding Challenge - State Persistence

When **a container crashes or restarted, the container state is not saved** so all of the files that were created during the lifetime of the container are lost.

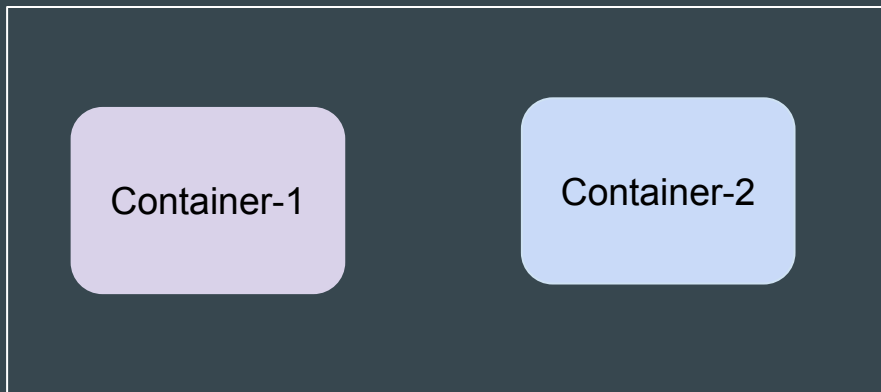
After a crash, kubelet restarts the container with a clean state.



Understanding Challenge - Shared Storage

Another problem occurs when **multiple containers are running in a Pod and need to share files.**

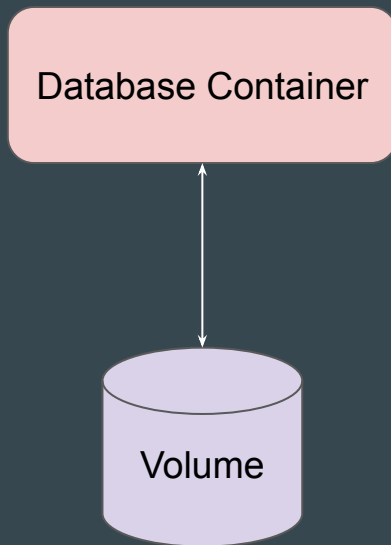
It can be challenging to set up and access a shared filesystem across all of the containers.



Multi-Container Pod

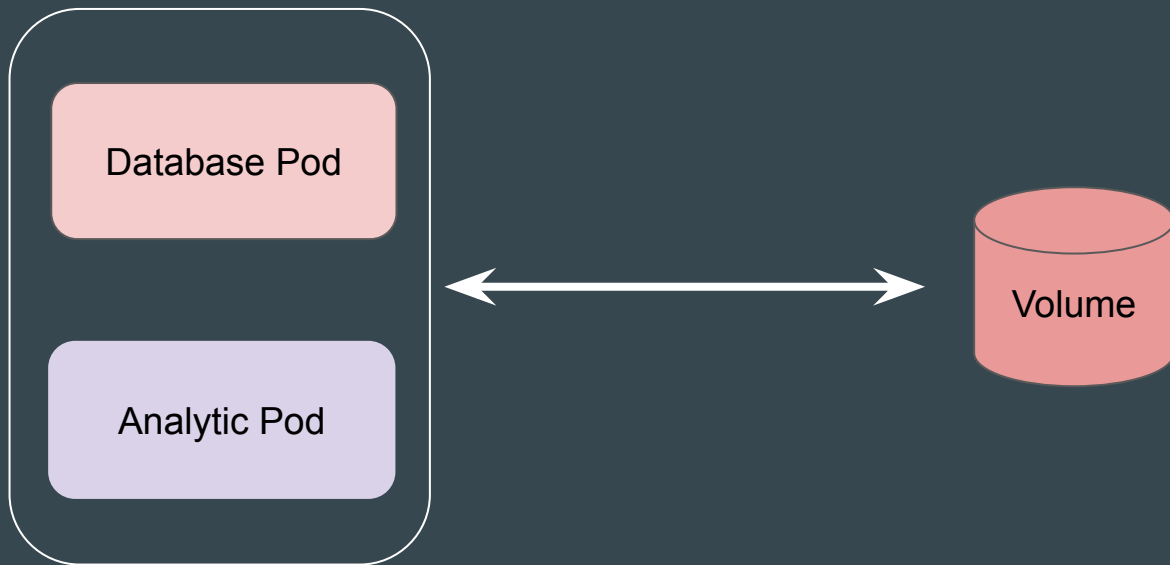
Introducing Volumes

Volumes can provide a way to **store data that persists beyond the lifecycle of a container.**



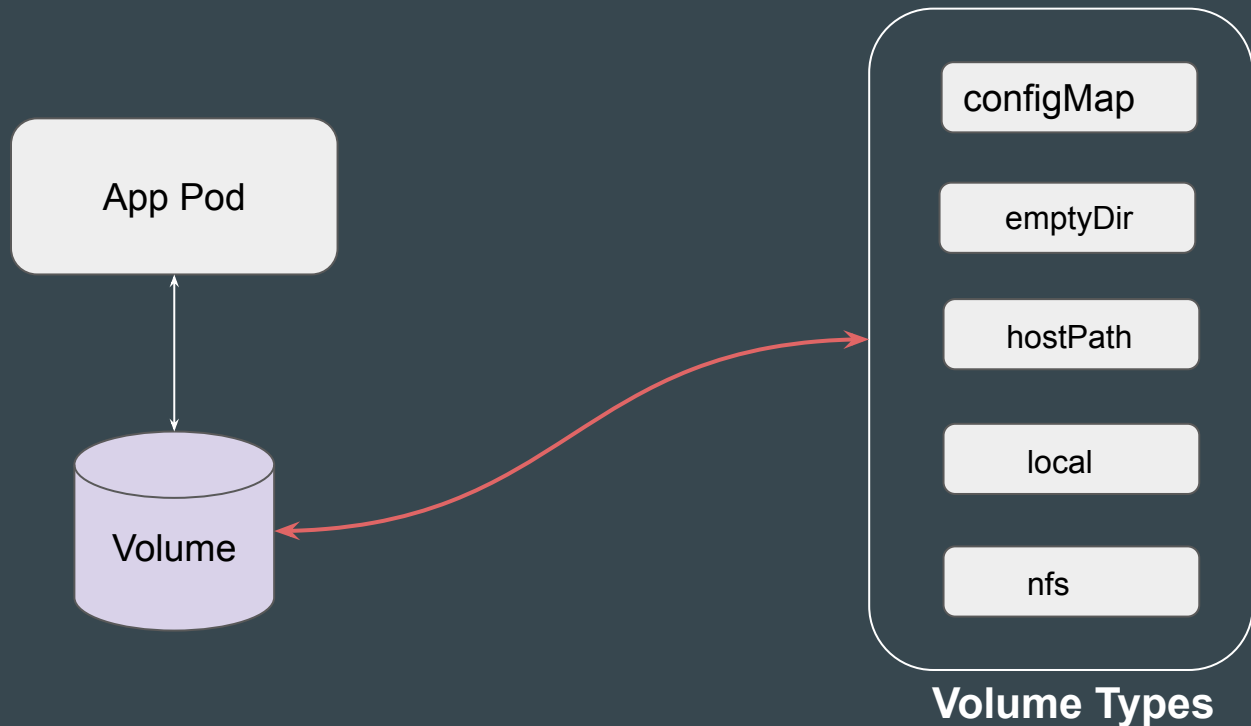
Benefit - Shared Storage

A single volume can be attached to multiple Pods.



Kubernetes Volumes

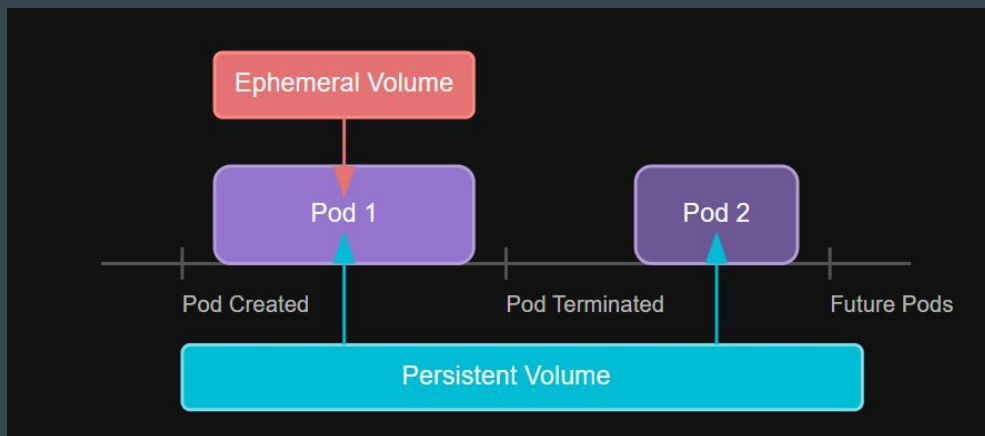
Kubernetes offers many volume types depending on the use-case.



Ephemeral and Persistent Volumes

Ephemeral volume types have a lifetime linked to a specific Pod, BUT persistent volumes exist beyond the lifetime of any individual pod.

When a pod ceases to exist, Kubernetes destroys ephemeral volumes; however, Kubernetes does not destroy persistent volumes.



Problem: Data Loss

Ephemeral Container

Container with Internal Storage

Application

Internal Data

↓ Container Restart ↓

After Restart

New Container

Application

Internal Data

Solution: Persistent Volume

Container with Persistent Storage

Container

Application

Persistent Volume

Important Data

User Files

↓ Container Restart ↓

After Restart

New Container

Application

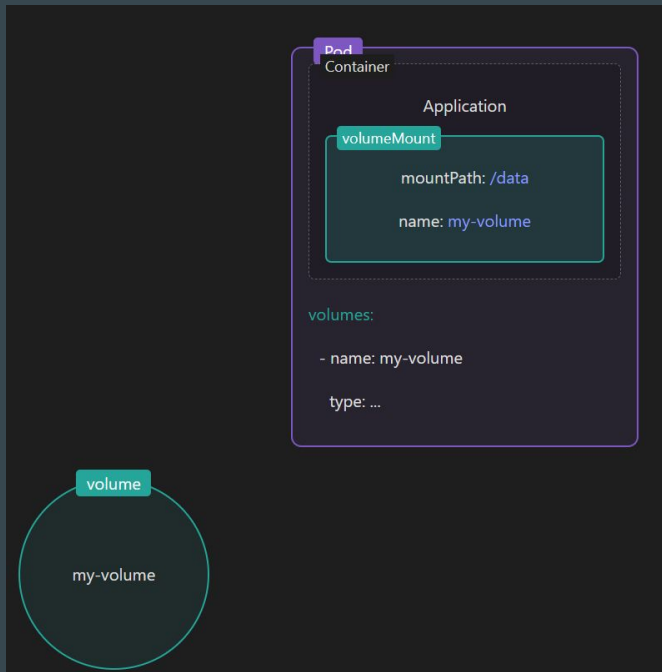
Persistent Volume (Preserved)

Important Data

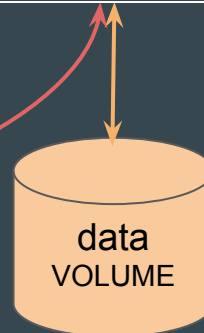
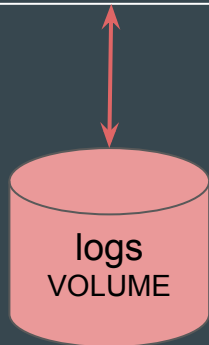
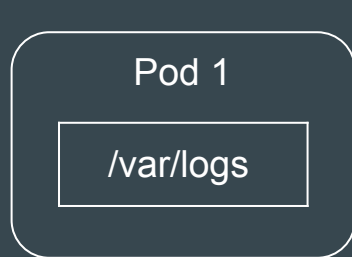
User Files

Volume and Volume Mounts

To use a volume, specify the volumes to provide for the Pod in `.spec.volumes` and declare where to mount those volumes into containers in `.spec.containers[*].volumeMounts`.



Volume Mount



Volume Mount

Sample Reference Code

```
apiVersion: v1
kind: Pod
metadata:
  name: test-pd
spec:
  containers:
  - image: nginx:latest
    name: test-container
    volumeMounts:
    - mountPath: /my-data
      name: data-volume
  volumes:
  - name: data-volume
    emptyDir:
      sizeLimit: 500Mi
```

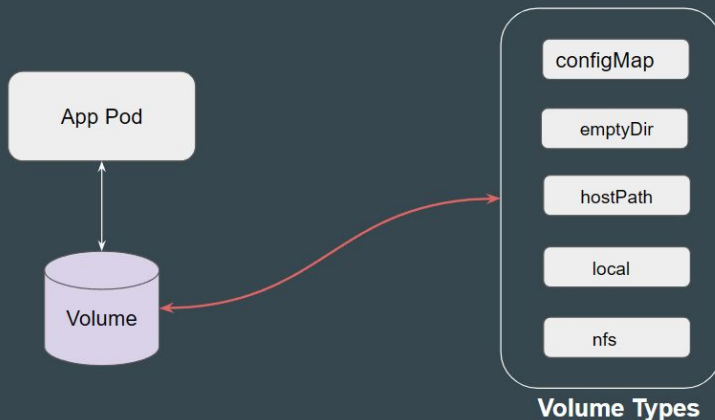
PersistentVolume and PersistentVolumeClaim

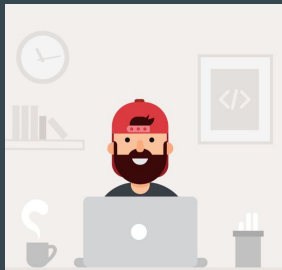
Setting the Base

Developers deploy application pods based on their their requirements.

Supplying storage specific configurations can introduce complexity due to the many different storage types and their settings

```
apiVersion: v1
kind: Pod
metadata:
  name: nfs-client-pod
spec:
  containers:
    - name: app-container
      image: nginx
      volumeMounts:
        - name: nfs-volume
          mountPath: /mnt/nfs
  volumes:
    - name: nfs-volume
      nfs:
        server: 192.168.1.100
        path: /exported/path
        readOnly: false
```





Developer



Being a developer, my goal is to create a straightforward Pod manifest that includes volume information. I'd prefer not to handle storage provisioning and its associated configurations.



Storage Administrator



As a Storage Administrator, I am responsible for provisioning storage and managing its configurations. Developers can then reference this storage within their Pod specifications.

Overview of Persistent Volume

Persistent Volume is a piece of storage in the cluster that has been provisioned by an administrator or dynamically provisioned using Storage Classes.

Every volume is created can be of different type and specifications.



EBS

Volume 1
Size: Small
Speed: Fast



NFS

Volume 2
Size: Medium
Speed: Fast



GlusterFS

Volume 3
Size: Big
Speed: Slow

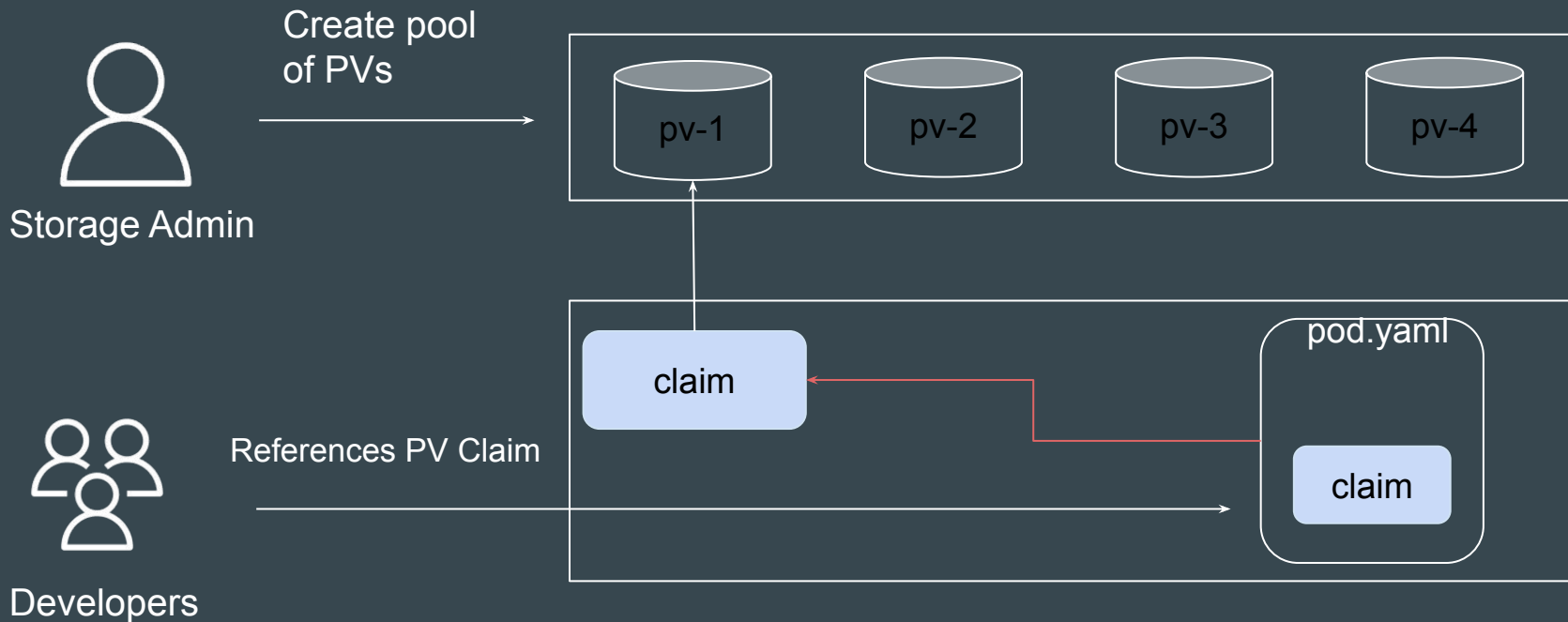


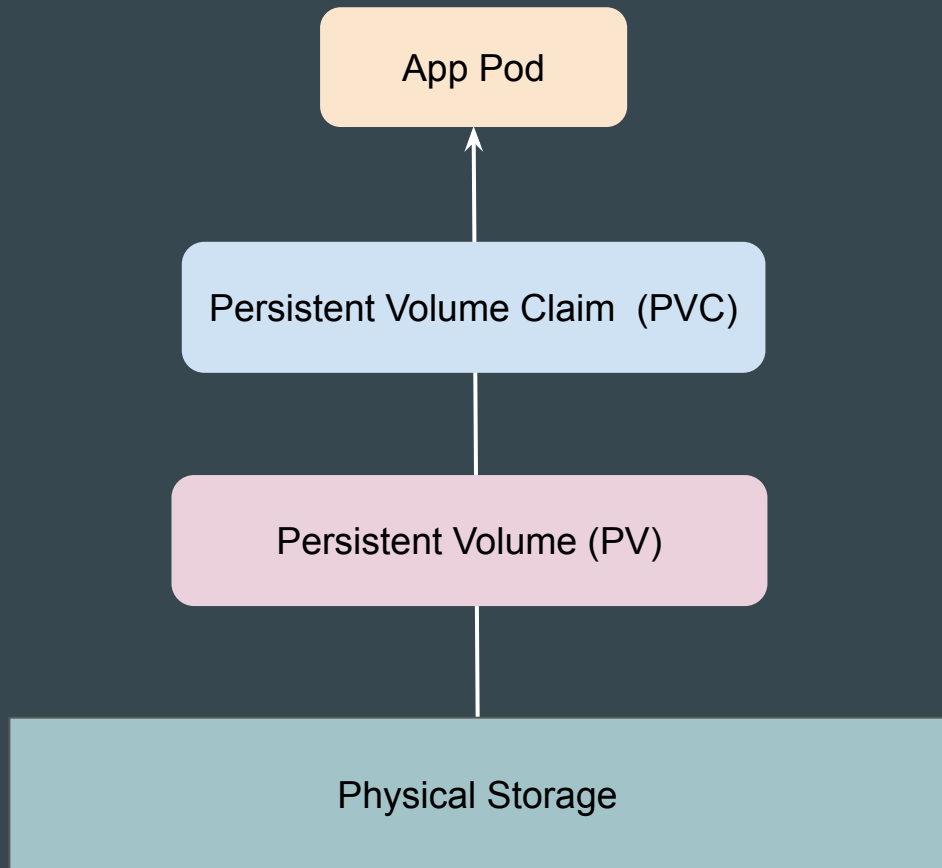
N

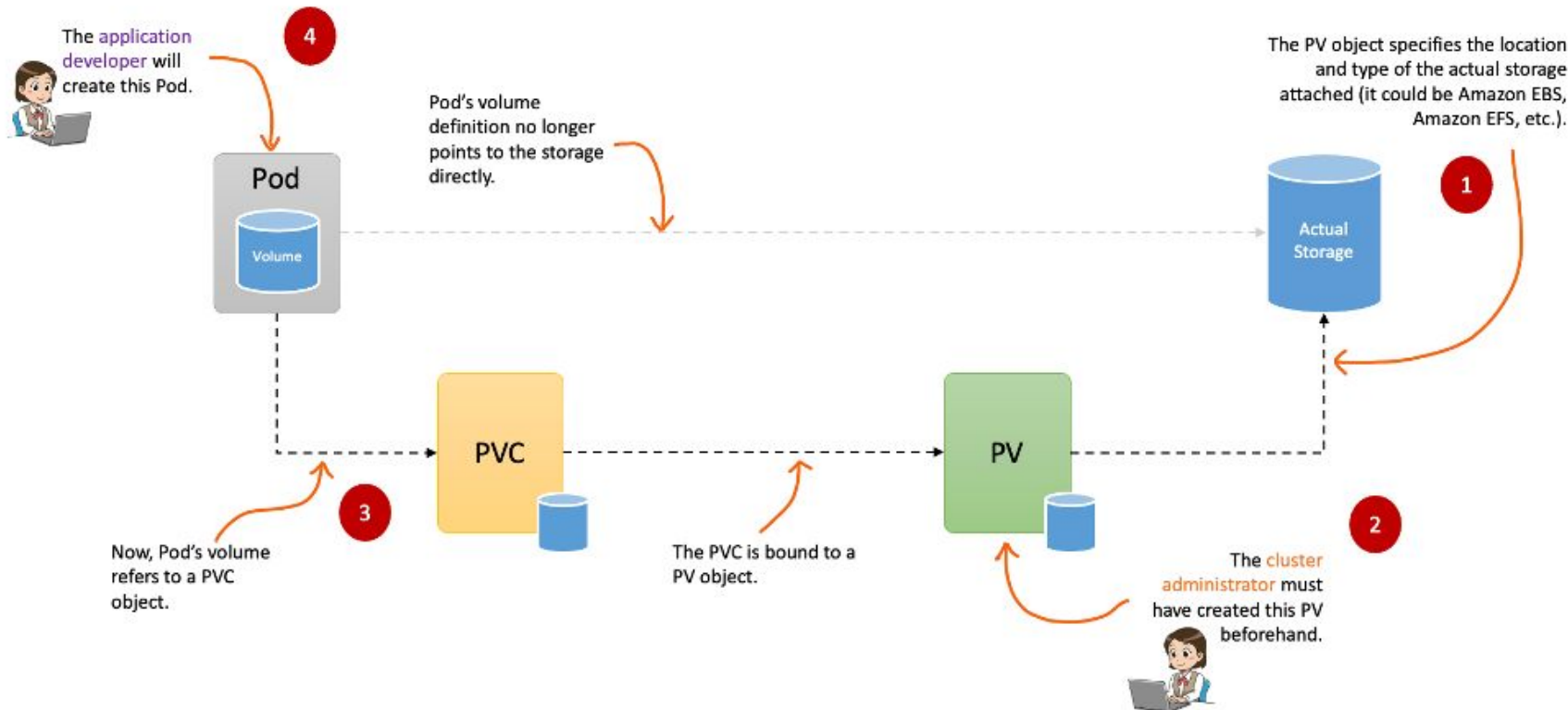
Volume 4
Size: VSmall
Speed: Ultra Fast

PersistentVolumeClaim

PersistentVolumeClaim (PVC) is a **request for storage by a user** (e.g., a developer). It specifies size, access modes, and other requirements.







Storage Class

Setting the Base

StorageClass specifies the plugin or driver that determines how persistent volumes (PVs) are dynamically provisioned.

They contain appropriate provisioner used to provision volumes.

```
C:\>kubectl get storageclass
```

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE	ALLOWVOLUMEEXPANSION
do-block-storage (default)	dobs.csi.digitalocean.com	Delete	Immediate	true
do-block-storage-retain	dobs.csi.digitalocean.com	Retain	Immediate	true
do-block-storage-xfs	dobs.csi.digitalocean.com	Delete	Immediate	true
do-block-storage-xfs-retain	dobs.csi.digitalocean.com	Retain	Immediate	true

Understanding the Structure

Each StorageClass contains the fields provisioner, parameters, and reclaimPolicy, which are used when a PersistentVolume belonging to the class needs to be dynamically provisioned to satisfy a PersistentVolumeClaim (PVC).

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: low-latency
provisioner: csi-driver.example-vendor.example
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
```

Default Storage Class

You can mark a StorageClass as the default for your cluster.

When a PVC does not specify a storageClassName, the default StorageClass is used.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: low-latency
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: csi-driver.example-vendor.example
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
```

Volume binding mode

VolumeBindingMode determines when volume binding and dynamic provisioning occurs.

Binding Mode	Bind Time	Description
Immediate	At PVC creation	Volumes are provisioned and bound as soon as the PVC is created.
WaitForFirstConsumer	Volume binding and provisioning is delayed until a Pod using the PVC is scheduled.	At Pod scheduling

ConfigMaps

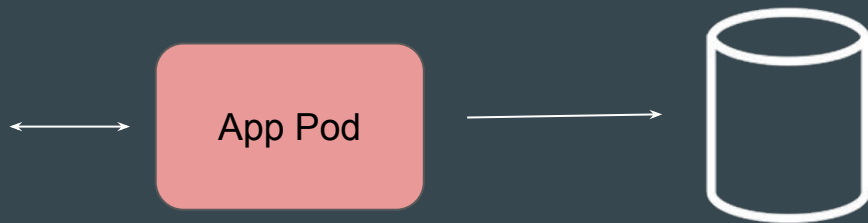
Understanding the Challenge - Part 1

Whenever an App pod gets deployed, it might need to connect to an external database to store data.

Issue: In many cases, these details are hard coded as part of the container image.

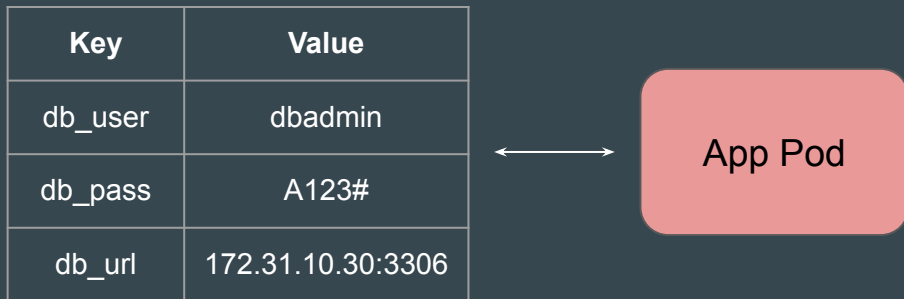
Key	Value
db_user	dbadmin
db_pass	A123#
db_url	172.31.10.30:3306

Hardcoded Data



Understanding the Challenge - Part 2

If a container has hardcoded data, any change to the data requires you to create a new set of Docker images and recreate the Pod



Hardcoded Data

Introduction to the ConfigMaps

A ConfigMap in Kubernetes is used to **store key-value pairs** of configuration data.

ConfigMap

Key	Value
db_user	dbadmin
db_pass	A123#
db_url	172.31.10.30:3306

Key	Value
db_user	devadmin
db_pass	A123#
db_url	10.77.0.5:3306

Fetch from Prod
ConfigMap



App Pod (Prod)

Fetch from Dev
ConfigMap



App Pod (Dev)

Point to Note

The primary purpose of ConfigMap is to store non-sensitive configuration data, such as configuration files, environment variables, etc.

It stores data in plain-text and is not intended for sensitive information.

Practical Approach for Entire Workflow

Part 1: Create ConfigMap

Part 2: Configure Pod to use that appropriate ConfigMap

ConfigMap Practical - Part 1 (Create ConfigMap)

Command to Create ConfigMap

The `kubectl create configmap` command allows us to create ConfigMap from a file, directory, or specified literal value

Examples:

```
# Create a new config map named my-config based on folder bar
kubectl create configmap my-config --from-file=path/to/bar
```

```
# Create a new config map named my-config with specified keys instead of file basenames on disk
kubectl create configmap my-config --from-file=key1=/path/to/bar/file1.txt --from-file=key2=/path/to/bar/file2.txt
```

```
# Create a new config map named my-config with key1=config1 and key2=config2
kubectl create configmap my-config --from-literal=key1=config1 --from-literal=key2=config2
```

```
# Create a new config map named my-config from the key=value pairs in the file
kubectl create configmap my-config --from-file=path/to/bar
```

```
# Create a new config map named my-config from an env file
kubectl create configmap my-config --from-env-file=path/to/foo.env --from-env-file=path/to/bar.env
```

Approach 1 - From a Literal

The `--from-literal` option allows you to directly specify key-value pairs as command-line arguments.

```
C:\>kubectl create configmap dev-config --from-literal=db_user=dbadmin --from-literal=db_host=172.31.0.5  
configmap/dev-config created
```

Approach 2 - From a File

The `--from-file` option allows you to create a ConfigMap from the contents of one or more files.



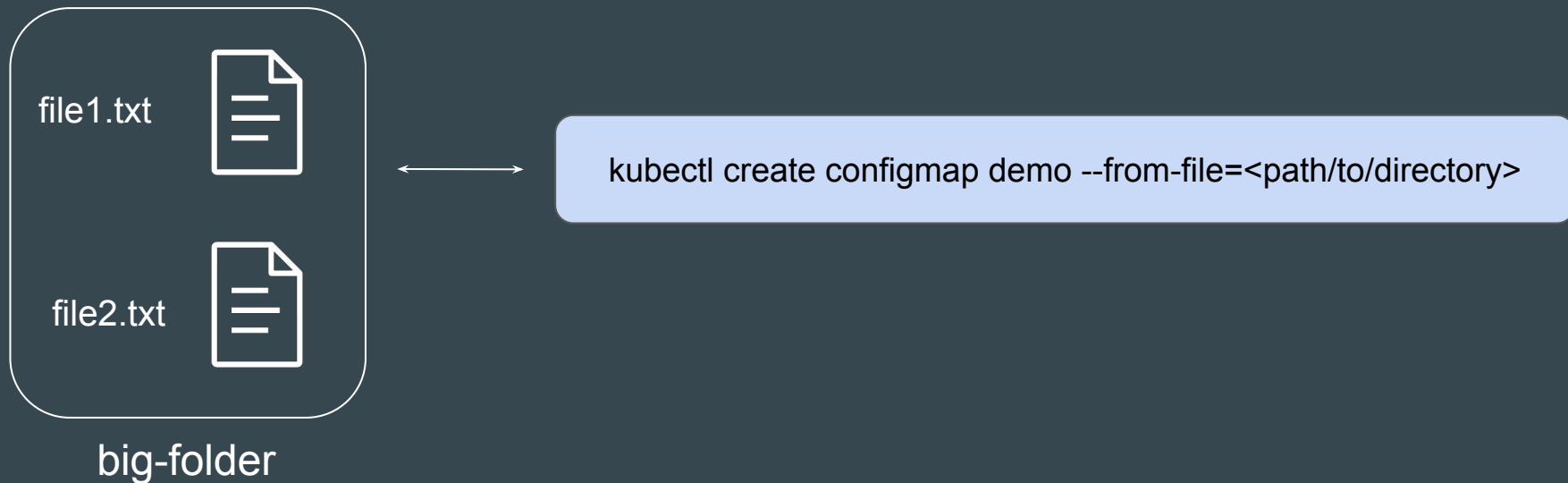
large-file.txt



```
kubectl create configmap --from-file=large-file.txt
```

Approach 3 - From a Directory

You can also use the `--from-file` option with a directory instead of a single file.



Comparison of All the Methods

Method	Use-Case	Advantage	Disadvantage
--from-literal	Simple, one-off key-value pairs	Quick and easy for simple configurations	Not suitable for complex configurations or large amounts of data
--from-file (file)	Individual configuration files	Good for managing separate configuration files	Can become cumbersome for many files
--from-file (dir)	Multiple configuration files organized in a directory	Convenient for grouping related configuration files	Less control over individual key names (filenames are used as keys)

Type of Data In ConfigMap

In the ConfigMap manifest file, you can represent data in multiple distinct formats.

Simple Key Value pair

```
! configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: demo-configmap
data:
  key1: "value1"
  key2: "value2"

  essay: |
    This is the first line of the essay1.
    It spans multiple lines and contains various information.
    This is Line 3

  json_data: |
    {
      "name": "Alice",
      "age": 26,
      "skills": ["Kubernetes", "Docker", "DevOps"]
    }
```

Multiline Block literal

Point to Note - Directory Approach

If you are referencing to an entire directory, Kubernetes will create a ConfigMap with each file in that directory becoming a key-value pair.

The filename (without extension) becomes the key, and the file content becomes the value

ConfigMap Practical - Part 2 (Mounting to Pods)

Setting the Base

Once ConfigMaps is created, we also need to reference it to appropriate Pod.

ConfigMap

Key	Value
db_user	dbadmin
db_pass	A123#
db_url	172.31.10.30:3306

Mount from Prod
ConfigMap

App Pod (Prod)

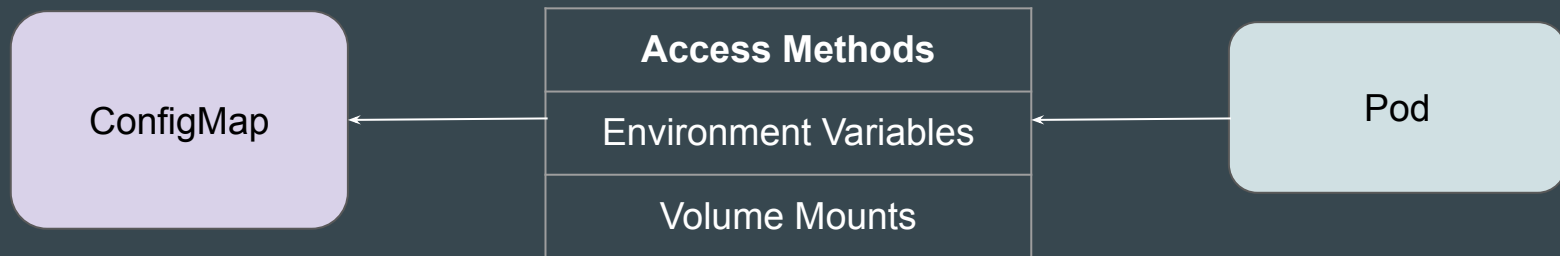
Key	Value
db_user	devadmin
db_pass	A123#
db_url	10.77.0.5:3306

Mount from Dev
ConfigMap

App Pod (Dev)

Different Approaches for Reference

There are multiple ways through which a Pod can fetch the data of a ConfigMap.



Approach 1 - Volume Mount

In this method, the ConfigMap is mounted directly as a volume in the Pod.

Each key-value pair in the ConfigMap appears as a file in the volume.



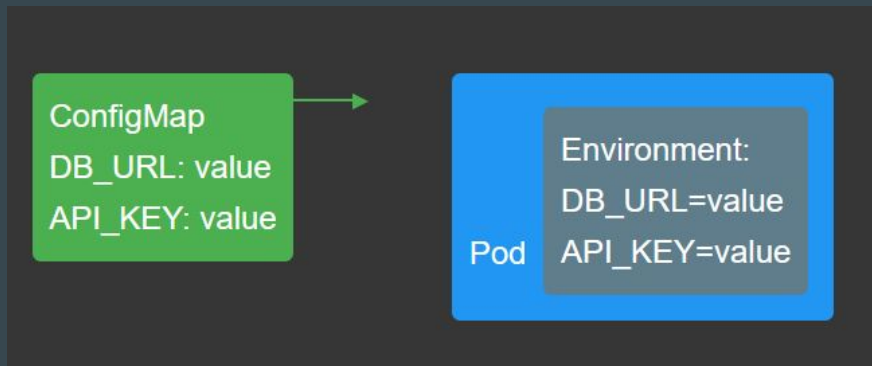
Reference Manifest File

```
! pod-volume.yaml
  apiVersion: v1
  kind: Pod
  metadata:
    name: app-pod
  spec:
    containers:
      - name: app-container
        image: nginx
        volumeMounts:
          - name: config-volume
            mountPath: /etc/config
    volumes:
      - name: config-volume
        configMap:
          name: app-config
```

Approach 2 - Environment Variables

In this method, the values in the ConfigMap are exposed as environment variables to the container.

The application can then access these values using standard environment variable lookup.



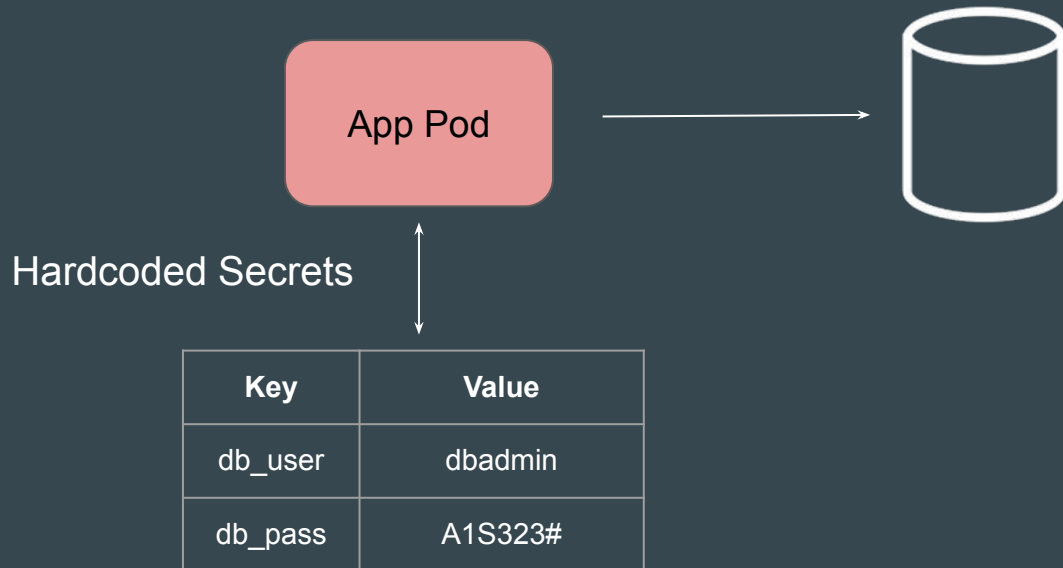
Reference Screenshot

```
! pod-env.yaml
  apiVersion: v1
  kind: Pod
  metadata:
    name: app-pod
  spec:
    containers:
      - name: app-container
        image: nginx
        env:
          - name: APP_MODE
            valueFrom:
              configMapKeyRef:
                name: app-config
                key: APP_MODE
          - name: APP_ENV
            valueFrom:
              configMapKeyRef:
                name: app-config
                key: APP_ENV
```


Kubernetes Secrets

HardCoding Secrets Should be Avoided

It is frequently observed that sensitive data like passwords, tokens, etc., are hard-coded as part of the container image.



Introducing Secret

Kubernetes Secrets is a feature that allows us to store these sensitive data.

Secrets

db_pass	db12#12
token	S2A2434

key	323@4dg
pass	admin@123

Mount from Secret

App Pod (Prod)

Reference Screenshot

```
C:\>kubectl get secret
```

NAME	TYPE	DATA	AGE
my-secret	Opaque	1	22m

```
C:\>kubectl get secret my-secret -o yaml
```

```
apiVersion: v1
```

```
data:
```

```
  db_pass: QTIjMTI1U0A=
```

```
kind: Secret
```

```
metadata:
```

```
  creationTimestamp: "2025-01-16T02:34:21Z"
```

```
  name: my-secret
```

```
  namespace: default
```

```
  resourceVersion: "5877928"
```

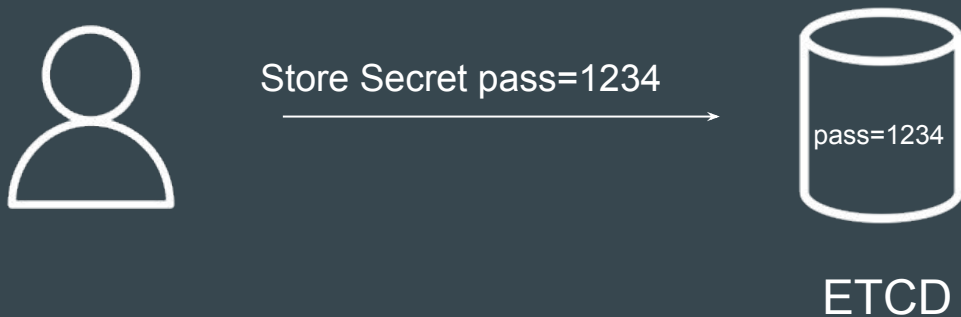
```
  uid: d3c383cb-c2e3-4f9b-95ef-d1f0ec6a04be
```

```
type: Opaque
```

Point to Note - Part 1

By default, Secrets are not very secure as they are not stored in encrypted format in the data store (ETCD). You can setup this configuration manually.

You can also additionally protect access to secrets using RBAC for access control.



Point to Note - Part 2

When you view a secret, Kubectl will print the Secret in **base64 encoded format**.

You'll have to use an external base64 decoder to decode the Secret fully

```
C:\>kubectl get secret my-secret -o yaml
apiVersion: v1
data:
  db_pass: QTIjMTI1U0A=
kind: Secret
metadata:
  creationTimestamp: "2025-01-16T02:34:21Z"
  name: my-secret
  namespace: default
  resourceVersion: "5877928"
  uid: d3c383cb-c2e3-4f9b-95ef-d1f0ec6a04be
type: Opaque
```

← base64 encoded

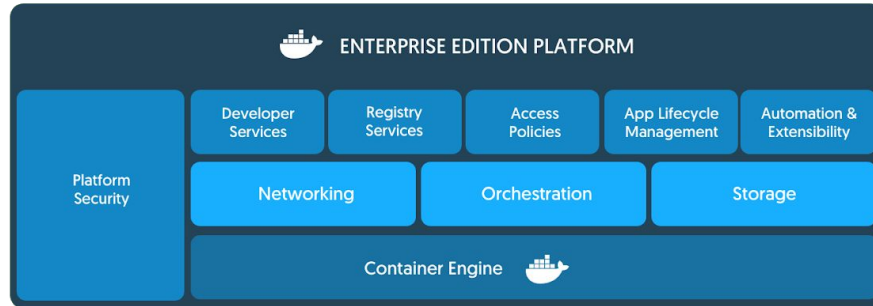
Docker Enterprise Edition (EE)

Build once, use anywhere

Overview of Docker EE

Docker Enterprise Edition (also referred as Docker EE) is designed for enterprise development as well as IT teams who build, ship, and run business-critical applications in production and at scale.

Docker EE is officially supported by the Docker Team.



Docker EE Tiers

There are multiple tiers available for Docker Enterprise edition. Referred as Basic, Standard and Advanced.

Capabilities	Docker Engine - Community	Docker Engine - Enterprise	Docker Enterprise
Container engine and built in orchestration, networking, security	✓	✓	✓
Certified infrastructure, plugins and ISV containers		✓	✓
Image management			✓
Container app management			✓
Image security scanning			✓

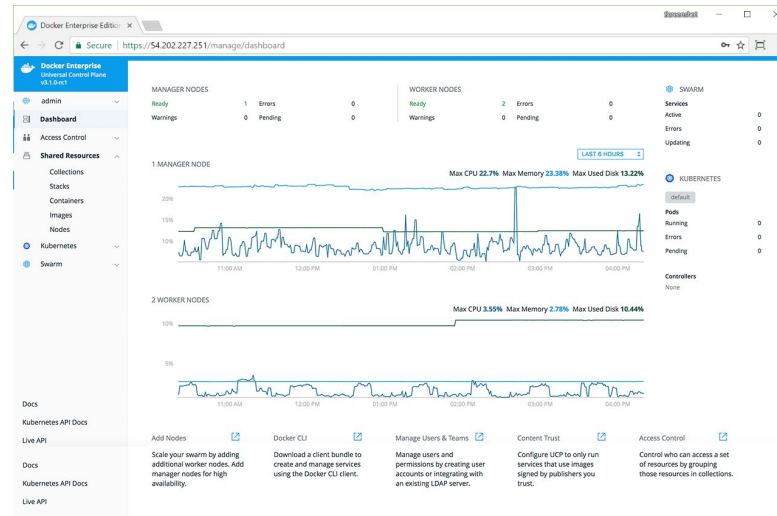
Universal Control Plane

Build once, use anywhere

Overview of Docker UCP

Docker Universal Control Plane (UCP) is the enterprise-grade cluster management solution from Docker.

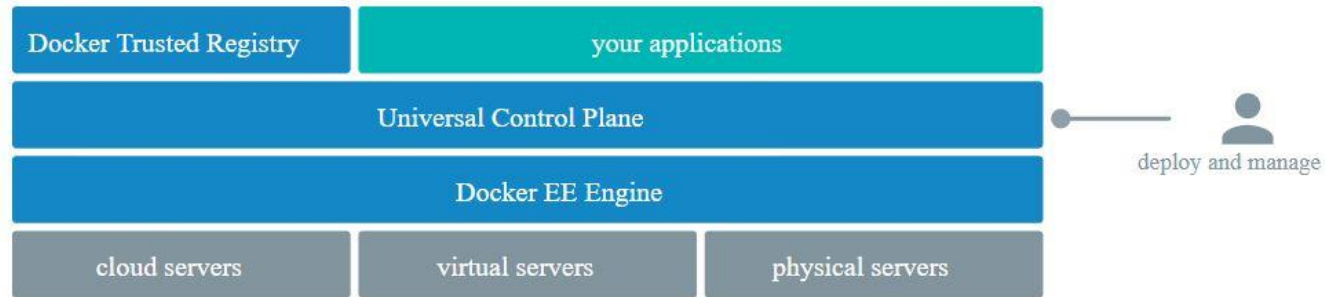
UCP helps you manage your Docker cluster and applications through a single interface.



UCP Architecture

Universal Control Plane is a containerized application that runs on Docker EE.

Once Universal Control Plane (UCP) instance is deployed, developers and IT operations no longer interact with Docker Engine directly, but interact with UCP instead.



System Requirements for UCP

Minimum Requirements for UCP:

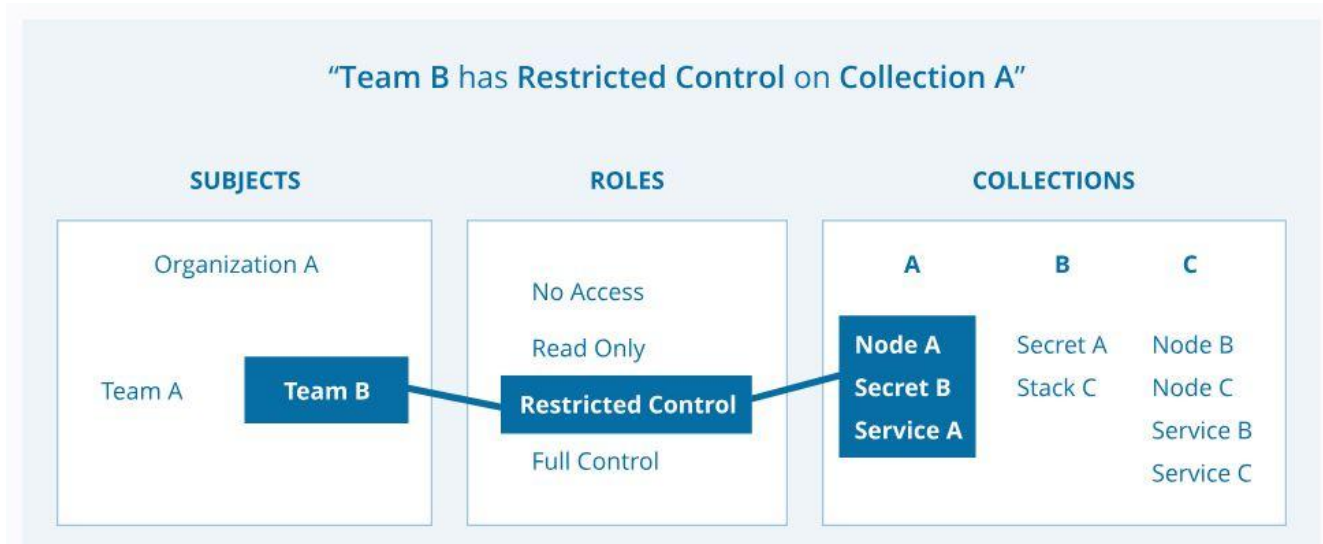
- Docker EE Engine
- 8GB of RAM for Manager Nodes
- 4GB of RAM for Worker nodes
- 5GB of free disk space for the /var partition for manager nodes
- 500MB of free disk space for the /var partition for worker nodes

UCP - Access Control Model

Build once, use anywhere

Getting Started

Docker UCP allows us provides **granular access control** feature which allows us to define various aspects like who can create and edit container resources in your swarm and others.



Understanding Subject

A subject represents a user, team, or organization.

A subject is in-turn granted an appropriate role.

User: Individual User.

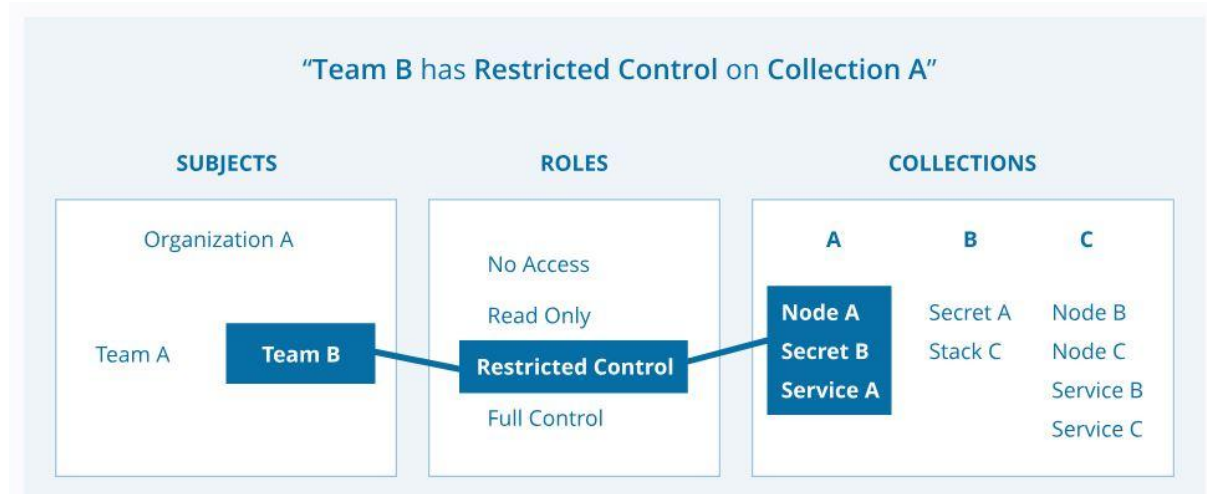
Organization: A group of users that share a specific set of permissions, defined by the roles of the organization.

Team: A group of users that share a set of permissions defined in the team itself.

A team exists only as part of an organization

Understanding Roles

A role is a set of permitted API operations that you can assign to a specific subject and collection



Understanding Collections

Docker EE enables controlling access to swarm resources by using collections.

Swarm resources that can be placed in to a collection include:

- Physical or virtual nodes
- Containers
- Services
- Networks
- Volumes
- Secrets
- Application configs

System Requirements for UCP

Minimum Requirements for UCP:

- Docker EE Engine
- 8GB of RAM for Manager Nodes
- 4GB of RAM for Worker nodes
- 5GB of free disk space for the /var partition for manager nodes
- 500MB of free disk space for the /var partition for worker nodes

DTR Backup

Build once, use anywhere

Overview of DTR Backup

Docker DTR has a backup command that allows us to take the backup.

The backup command doesn't cause downtime for DTR, so you can take frequent backups without affecting your users.

```
docker run --log-driver none -i --rm docker/dtr backup --ucp-url https://172.31.40.237  
-ucp-insecure-tls --ucp-username admin --ucp-password YOUR-PASSWORD-HERE >  
backup.tar
```

Important Pointer related to DTR Backup

This command only creates backups of configurations, and image metadata.

It does not back up users and organizations. Users and organizations can be backed up during a UCP backup.

It also does not back up Docker images stored in your registry.

Backup DTR Images

Build once, use anywhere

Backing up DTR Image

DTR supports multiple backends to store the images.

Depending on the backend that is being used, the backup procedure for images differs.

For local volumes based setup, you can backup the volume associated with DTR registry.

Routing Mesh

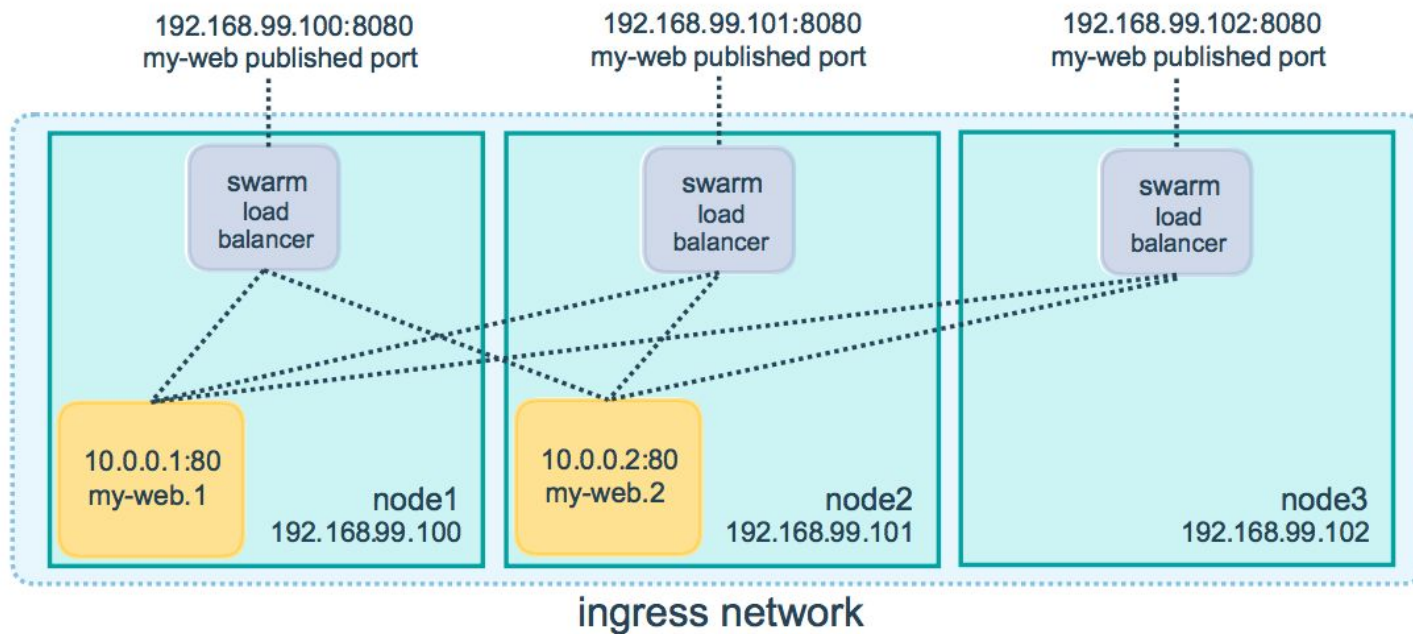
Build once, use anywhere

Overview of Routing Mesh

There can be multiple nodes within a swarm cluster.

When creating a service, the task can be assigned to nodes depending on various factors.

Routing mesh enables each node in the swarm to accept connections on published ports for any service running in the swarm, even if there's no task running on the node.



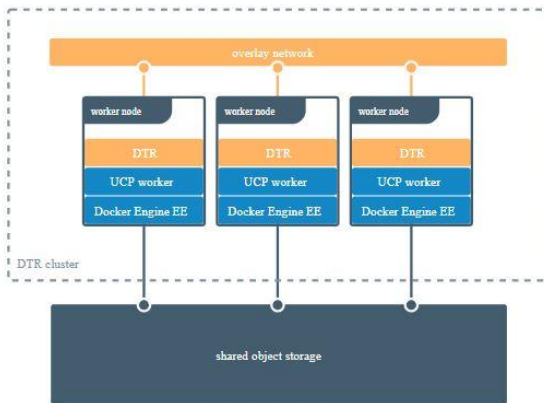
DTR Storage Driver

Build once, use anywhere

Overview of Storage driver

By default, Docker Trusted Registry stores images on the filesystem of the node where it is running, but you should configure it to use a centralized storage backend.

You can configure DTR to use an external storage backend, for improved performance or high availability.



Supported Storage Systems

Some of the supported storage systems in DTR are:

Local:

- NFS
- Bind Mount
- Volume

Cloud Storage Provider:

- AWS S3
- Azure
- Google Cloud

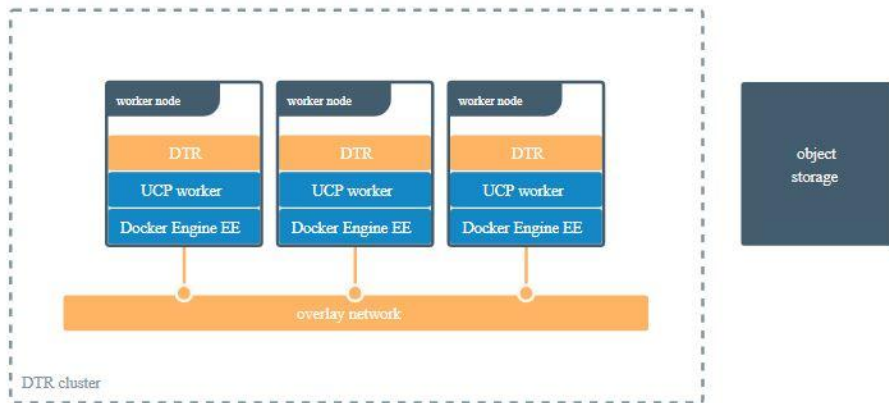
DTR High-Availability

Disaster Recovery

Overview of High Availability Setup

Docker Trusted Registry is designed to scale horizontally as your usage increases. You can add more replicas to make DTR scale to your demand and for high availability.

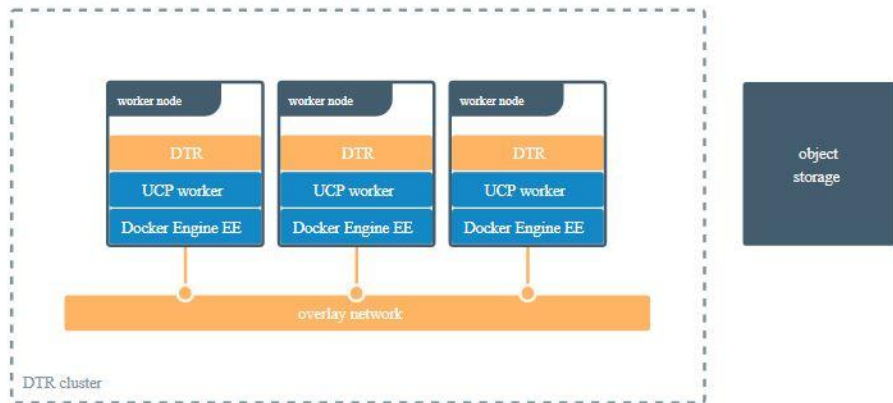
All DTR replicas run the same set of services and changes to their configuration are automatically propagated to other replicas.



Central Storage Backend

If your DTR deployment only has a single replica, you can continue using the local filesystem to store your Docker images.

If your DTR deployment has multiple replicas, for high availability, you need to ensure all replicas are using the same storage backend.



Fault Tolerance

Ensure sufficient amount of replicas are added to provide fault tolerance.

DTR Replicas	Fault Tolerated
1	0
3	1
5	2
7	3

Health Check Endpoints

DTR also exposes several endpoints you can use to assess if a DTR replica is healthy or not:

Endpoints	Description
/_ping:	Checks if the DTR replica is healthy, and returns a simple json response. This is useful for load balancing or other automated health check tasks
/api/v0/meta/cluster_status	Returns extensive information about all DTR replicas.

Important Pointers

To have high-availability on UCP and DTR, you need a minimum of:

- 3 dedicated nodes to install UCP with high availability,
- 3 dedicated nodes to install DTR with high availability,

When a replica fails, the number of failures tolerated by your cluster decreases. Don't leave that replica offline for long.

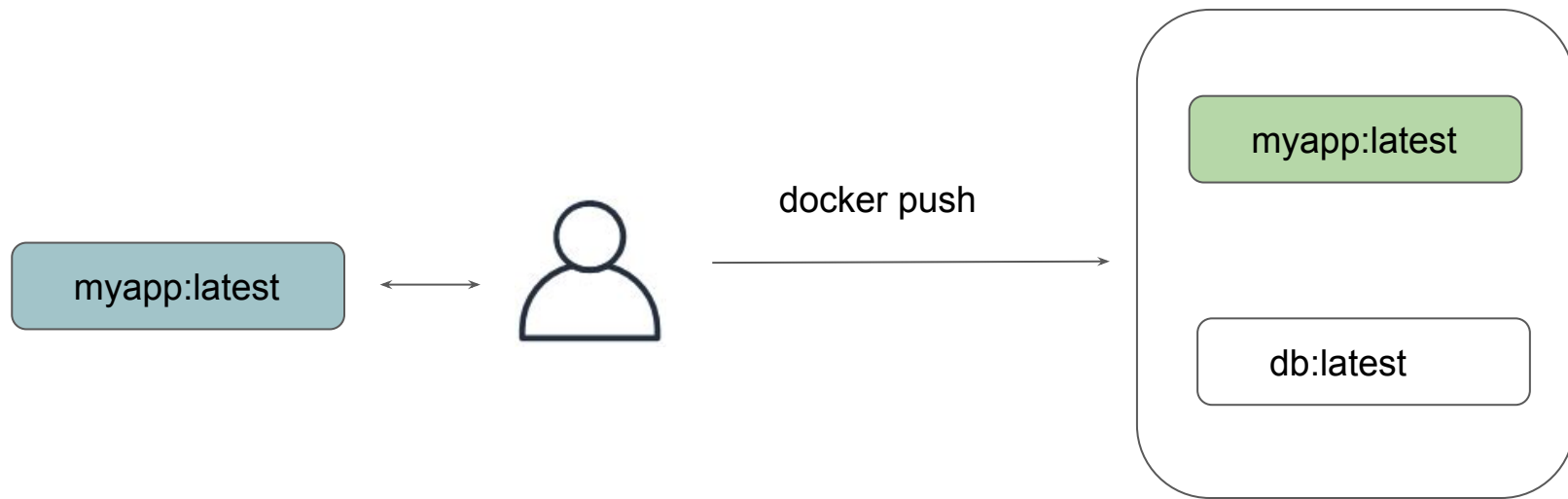
Adding too many replicas to the cluster might also lead to performance degradation, as data needs to be replicated across all replicas.

Immutable Tags in DTR

DTR In-Detail

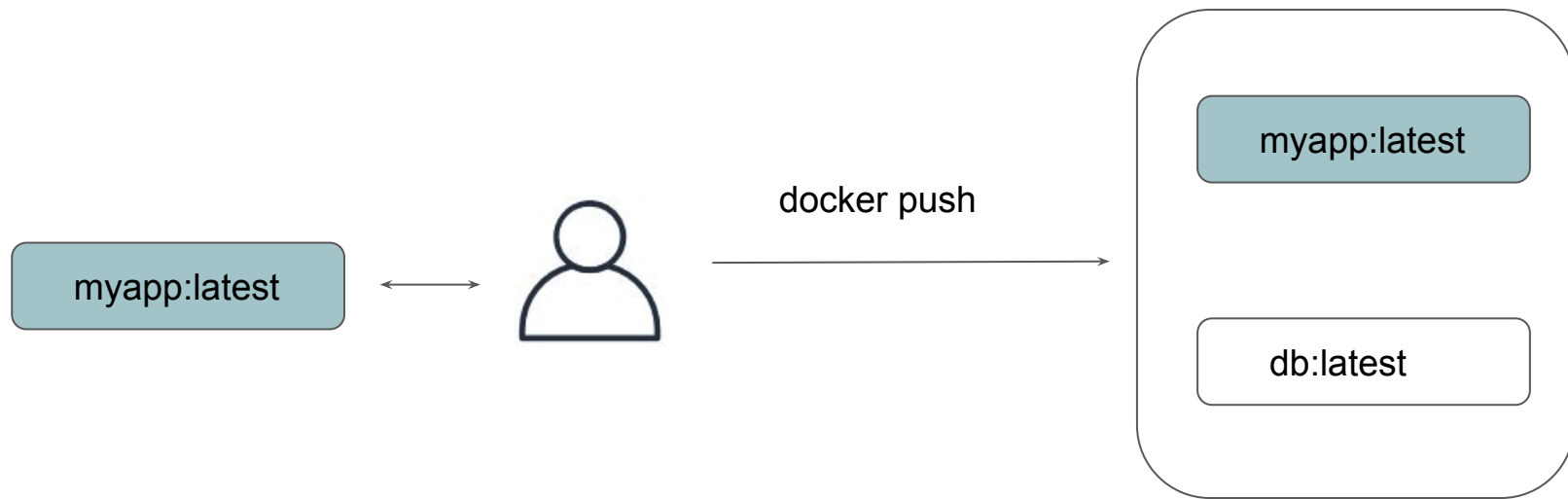
Overview of the Challenge

By default, users with read and write access to a repository can push the same tag multiple times to that repository.



Overview of the Challenge

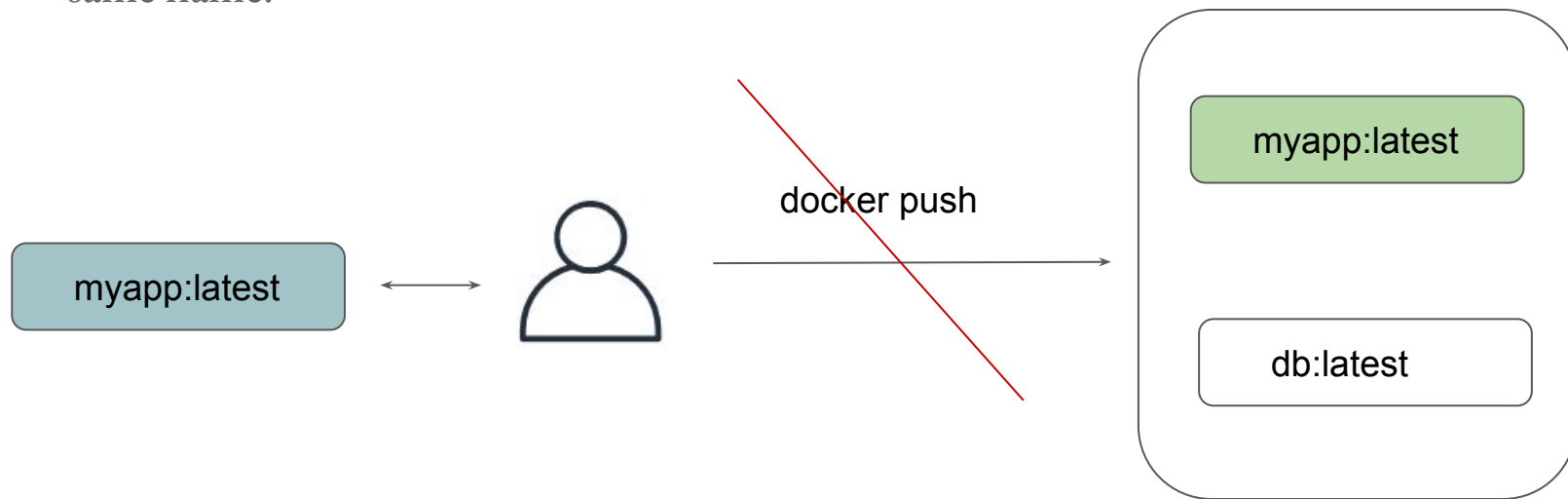
By default, users with read and write access to a repository can push the same tag multiple times to that repository.



Immutable Tags

To prevent tags from being overwritten, you can configure a repository to be immutable.

Once configured, DTR will not allow anyone else to push another image tag with the same name.



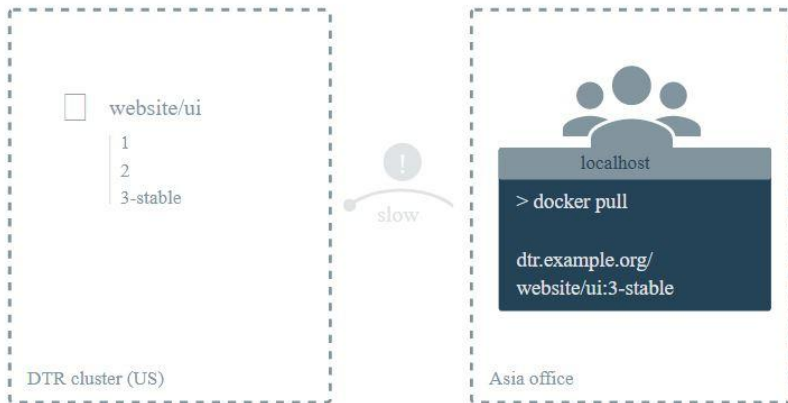
DTR Cache

DTR In Detail

Understanding the Challenge

The further away you are from the geographical location where DTR is deployed, the longer it will take to pull and push images.

This happens because the files being transferred from DTR to your machine need to travel a longer distance, across multiple networks.



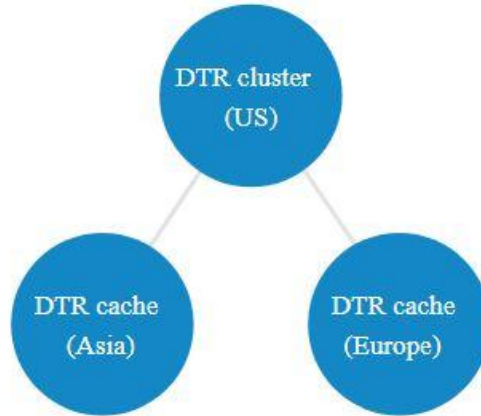
Implementing Cache at Location

To decrease the time to pull an image, you can deploy DTR caches geographically closer to users.

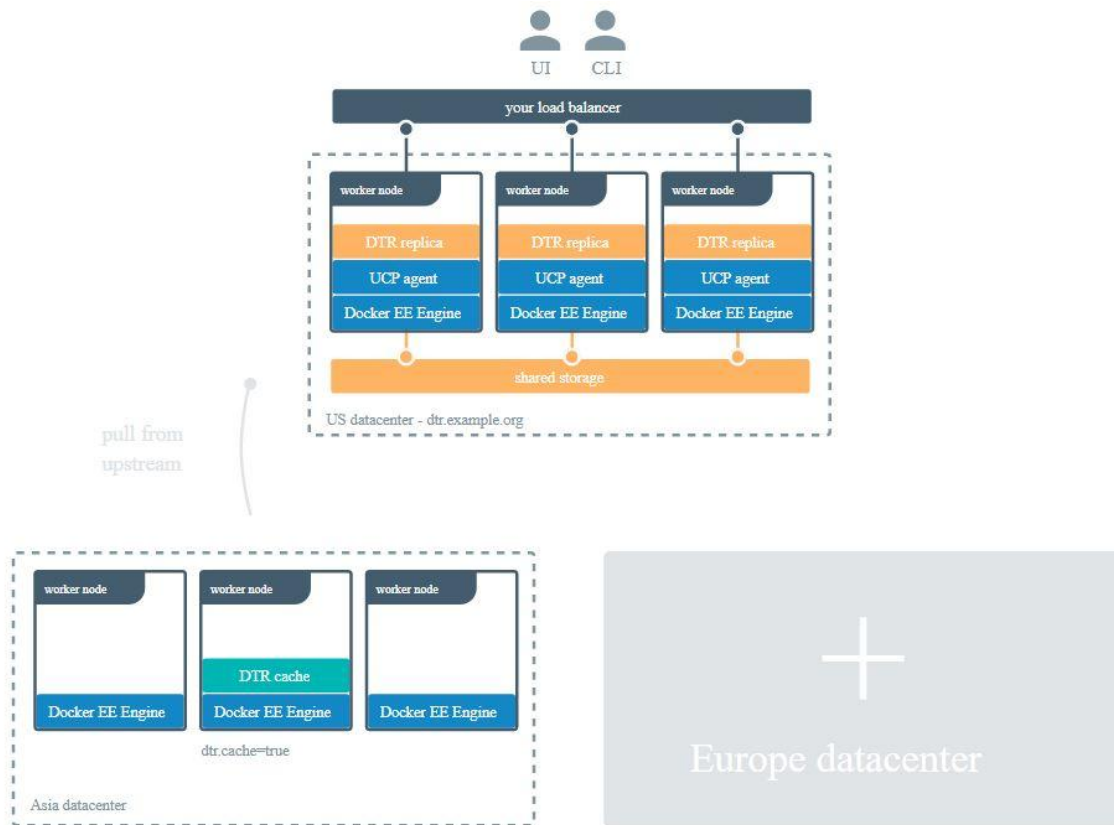
Caches are transparent to users, since users still log in and pull images using the DTR URL address. DTR checks if users are authorized to pull the image, and redirects the request to the cache.



Implementing Cache at Location



Architecture



Setting Orchestrator in UCP

UCP In-Detail

Orchestrator Types in UCP

Docker UCP supports both Swarm and Kubernetes.

When you add a node to the cluster, the node's workloads are managed by a default orchestrator, either Docker Swarm or Kubernetes.

When you install Docker Enterprise, new nodes are managed by Docker Swarm, but you can change the default orchestrator to Kubernetes in the administrator settings.

Scheduler

SET ORCHESTRATOR TYPE FOR NEW NODES

☒ Swarm

☐ Kubernetes

Orchestrator Types For Nodes

You can change the current orchestrator for any node that's joined to a Docker Enterprise cluster. The available orchestrator types are Kubernetes, Swarm, and Mixed.

The Mixed type enables workloads to be scheduled by Kubernetes and Swarm both on the same node.

When you change the orchestrator type for a node, existing workloads are evicted, and they're not migrated to the new orchestrator automatically

Change Orchestrator Type via CLI

To change the orchestrator type for a node from Swarm to Kubernetes:

```
docker node update --label-add com.docker.ucp.orchestrator.kubernetes=true <node-id>
```

```
docker node update --label-add com.docker.ucp.orchestrator.kubernetes=true <node-id>
```

Container Security Scanning

Container Security

Getting Started

Docker Containers can have security vulnerabilities.

If blindly pulled and if containers are running in production, it can result in breach.

latest

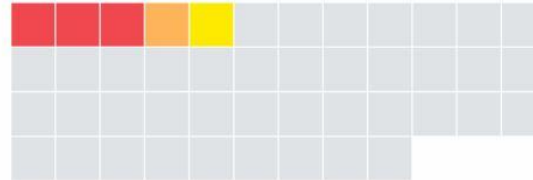
[View All Tags](#)

There are **20** vulnerable Core/components (Last scanned 4 days ago) [Provide Feedback](#)

1. ADD
file:1d214d2...e616f23870
in /

49.0MB

base
layer



Component	Vulnerability	Severity
bash 4.3-11+b1	CVE-2016-7543	Critical
GPLv3: Copyleft License	CVE-2016-9401	Minor

Overview of Security Scanning in DTR

DTR allows us to perform security scan for the containers.

These scan can perform “On Push” or even manually.

admin / webserver

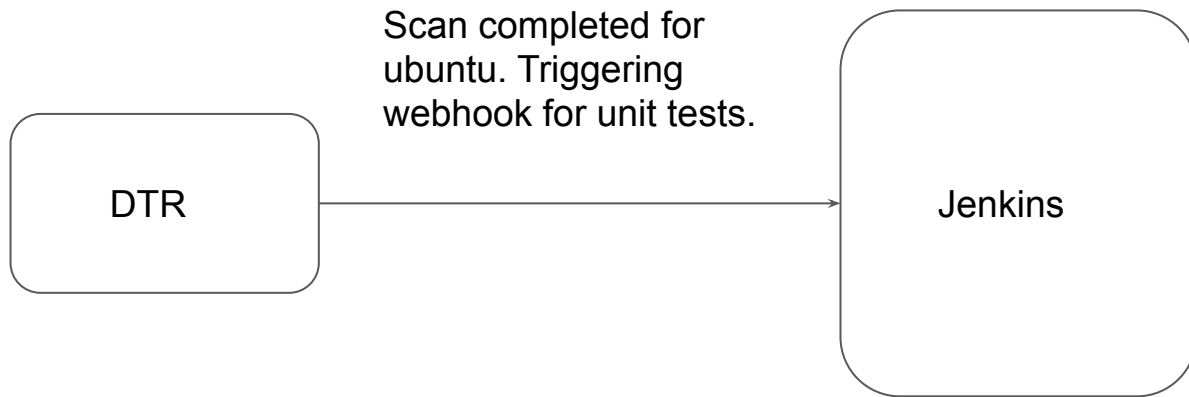
Info	Tags	Webhooks	Promotions	Pruning	Mirrors	Settings	Activity
☐	Image	Os/Arch	Image ID	Size (Compressed)	Signed	Last Pushed	Vulnerabilities
☐	ubuntu	linux / amd64	9361ce633ff1	43.56 MB		12 minutes ago by admin	5 Critical 31 Major 18 Minor View details
☐	v1	linux / amd64	d8233ab899d4	755.9 kB		29 minutes ago by admin	Clean View details

DTR Webhooks

Build once, use anywhere

Overview of the Webhooks

You can configure DTR to automatically post event notifications to a webhook URL of your choosing



Overview of the Webhooks

```
{
  "type": "TAG_PUSH",
  "createdAt": "2019-05-15T19:39:40.607337713Z",
  "contents": {
    "namespace": "foo",
    "repository": "bar",
    "tag": "latest",
    "digest": "sha256:b5bb9d8014a0f9b1d61e21e796d78dccdf1352f23cd32812f4850b878ae4944c",
    "imageName": "foo/bar:latest",
    "os": "linux",
    "architecture": "amd64",
    "author": "",
    "pushedAt": "2015-01-02T15:04:05Z"
  },
  "location": "/repositories/foo/bar/tags/latest"
}
```

UCP Client Bundles

Build once, use anywhere

Overview of UCP Client Bundles

A client bundle is a group of certificates downloadable directly from the Docker Universal Control Plane (UCP).

Depending on the permission associated with the user, you can now execute docker swarm commands from your remote machine that take effect on the remote cluster.

For example:

- You can create a new service in UCP from your laptop.
- Login to remote container from your laptop without SSH (via API)

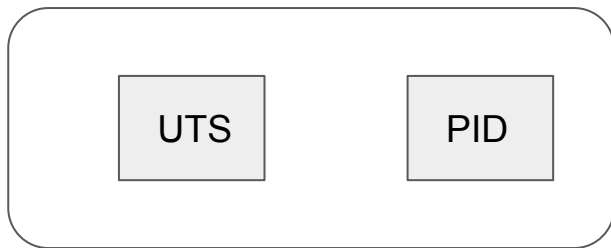
Linux Namespaces

Build once, use anywhere

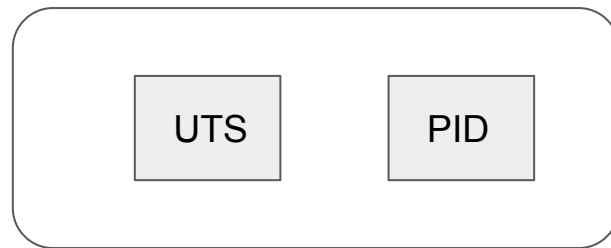
Overview of Namespaces

Docker uses a technology called namespaces to provide the isolated work space called the container

These namespaces provide a layer of isolation. Each aspect of a container runs in a separate namespace and its access is limited to that namespace.



Namespace A



Namespace B

Namespaces in Linux

Currently Linux provides six different types of namespaces as follows:

- Inter-Process Communication (IPC)
- Process (pid)
- Network (net)
- User Id (user)
- Mount (mnt)
- UTS

Control Groups

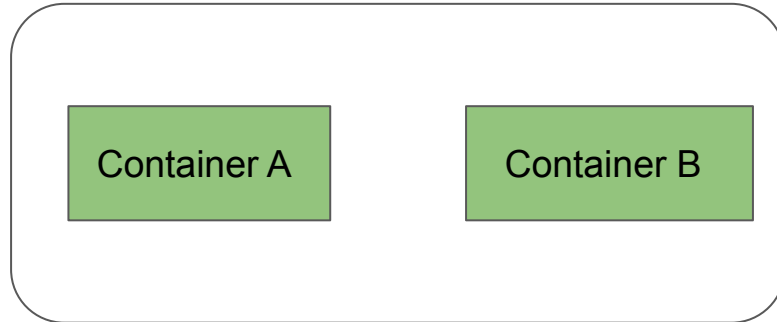
Build once, use anywhere

Overview of Control Groups

Control Groups (cgroups) is a Linux kernel feature that limits, accounts for, and isolates the resource usage (CPU, memory, disk I/O, network, etc.) of a collection of processes.

1 CPU core

1 GB RAM



Docker Host

Overview of Control Groups

Control Groups (cgroups) is a Linux kernel feature that limits, accounts for, and isolates the resource usage (CPU, memory, disk I/O, network, etc.) of a collection of processes.

1 CPU core

1 GB RAM



Docker Host

Limiting CPU for Containers

There are various ways in which you can limit the CPU for containers, these include:

Approaches	Description
<code>--cpus=<value></code>	If host has 2 CPUs and if you set <code>--cpus=1</code> , than container is guaranteed at most one CPU. You can even specify <code>--cpus=0.5</code>
<code>--cpuset-cpus</code>	Limit the specific CPUs or cores a container can use. A comma-separated list or hyphen-separated range of CPUs a container can use. 1st CPU is numbered 0 2nd CPU is numbered 1. Value of 0-3 means usage of first, second, third and fourth CPU. Value of 1,3 means second and fourth CPU.

Reservation vs Limit

Build once, use anywhere

Getting Started

By default, a container has no resource constraints and can use as much of a given resource as the host's kernel scheduler allows.

It is important not to allow a running container to consume too much of the host machine's memory.

On Linux hosts, if the kernel detects that there is not enough memory to perform important system functions, it throws an **Out Of Memory** Exception, and starts killing processes to free up memory.

Reservation vs Limit

Limit imposes an upper limit to the amount of memory that can potentially be used by a Docker container.

Reservations, on the other hand, is less rigid.

When the system is running low on memory and there is contention, reservation tries to bring the container's memory consumption at or below the reservation limit.

Swarm MTLs

Build once, use anywhere

Getting Started

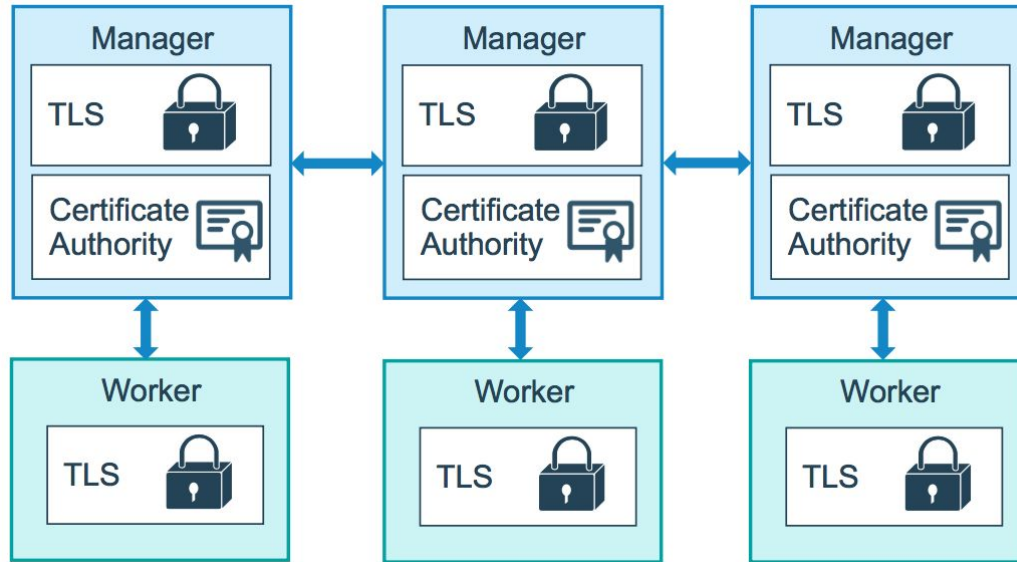
The nodes in a swarm use mutual Transport Layer Security (TLS) to authenticate, authorize, and encrypt the communications with other nodes in the swarm.

By default, the manager node generates a new root Certificate Authority (CA) along with a key pair, which are used to secure communications with other nodes that join the swarm.

You can specify your own externally-generated root CA, using the `--external-ca` flag of the `docker swarm init` command.

Overview of Generated Certificate

Each time a new node joins the swarm, the manager issues a certificate to the node.



Overview of Tokens

The manager node also generates two tokens to use when you join additional nodes to the swarm: one **worker** token and one **manager** token

Each token includes the digest of the root CA's certificate and a randomly generated secret.

When a node joins the swarm, the joining node uses the digest to validate the root CA certificate from the remote manager.

Certificate Details

By default, each node in the swarm renews its certificate every three months.

You can configure this interval by running the docker swarm update --cert-expiry <TIME PERIOD>

In the event that a cluster CA key or a manager node is compromised, you can rotate the swarm root CA so that none of the nodes trust certificates signed by the old root CA anymore.

Rotating the CA Certificate

Run `docker swarm ca --rotate` to generate a new CA certificate and key.

In the above command, docker generated a cross-signed certificate signed by previous old CA.

After every node in the swarm has a new TLS certificate signed by the new CA, Docker forgets about the old CA certificate and key material, and tells all the nodes to trust the new CA certificate only.

Join tokens also changes accordingly.

Managing Secrets in Docker Swarm

Build once, use anywhere

Overview of Docker Secrets

Docker secrets to centrally manage this data and securely transmit it to only those containers that need access to it.

Secrets are encrypted during transit and at rest in a Docker swarm.

A given secret is only accessible to those services which have been granted explicit access to it, and only while those service tasks are running.

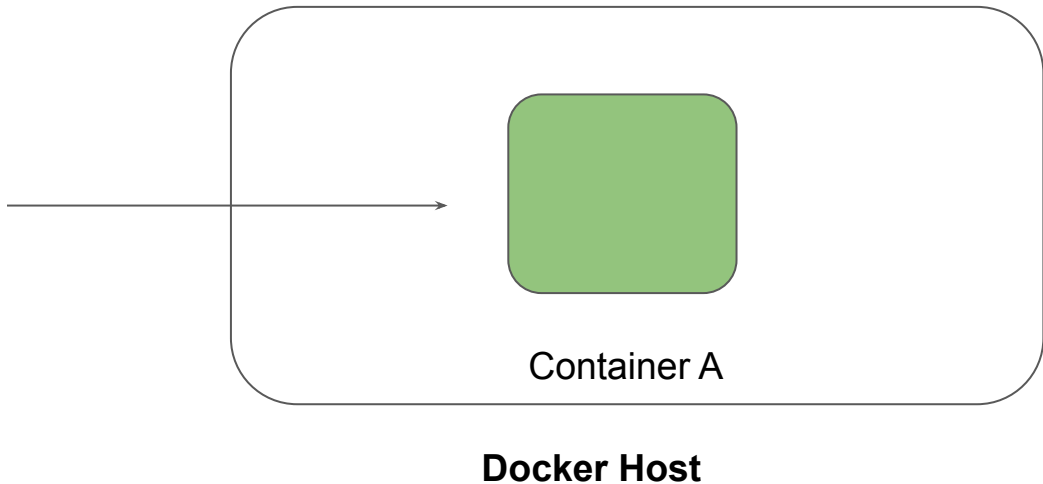
Linux Capabilities

Security Aspect

Understanding Capabilities

There are several types of capabilities which Linux provides to have granular access at the application level.

Capability	Action
SETPCAP	Yes
CHOWN	Yes
SYS_MODULE	No
LINUX_IMMUTABLE	No



Privileged container

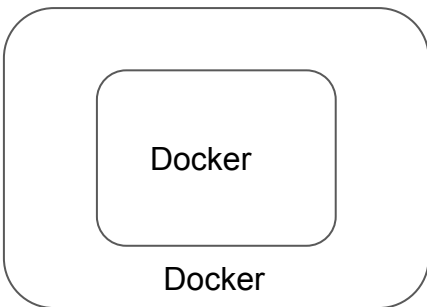
Security Aspect

Understanding the Challenge

By default, docker container does not have many capabilities assigned to it.

- Docker containers are also not allowed to access any devices.

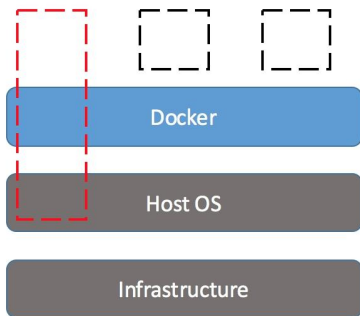
Hence, by default, docker container cannot perform various use-cases like run docker container inside a docker container



Privileged Containers

Privileged Container can access all the devices on the host as well as have configuration in AppArmor or SELinux to allow the container nearly all the same access to the host as processes running outside containers on the host.

- Limits that you set for privileged containers will not be followed.
- Running a privileged flag gives all the capabilities to the container



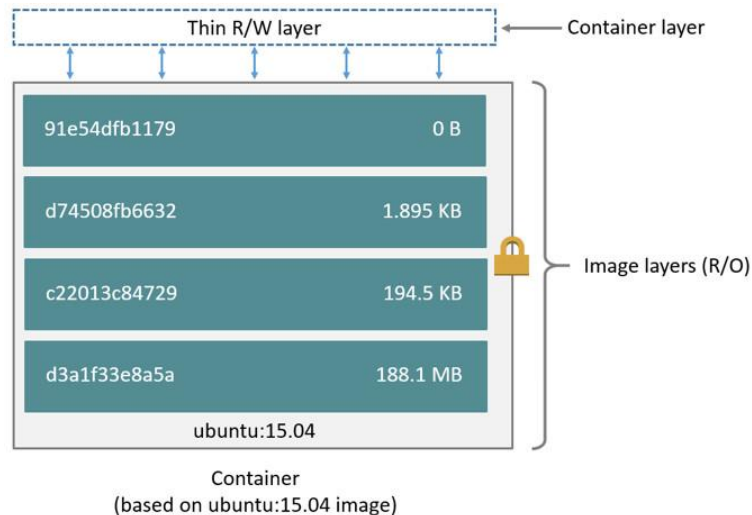
Storage Drivers

Build once, use anywhere

Overview of Storage Drivers

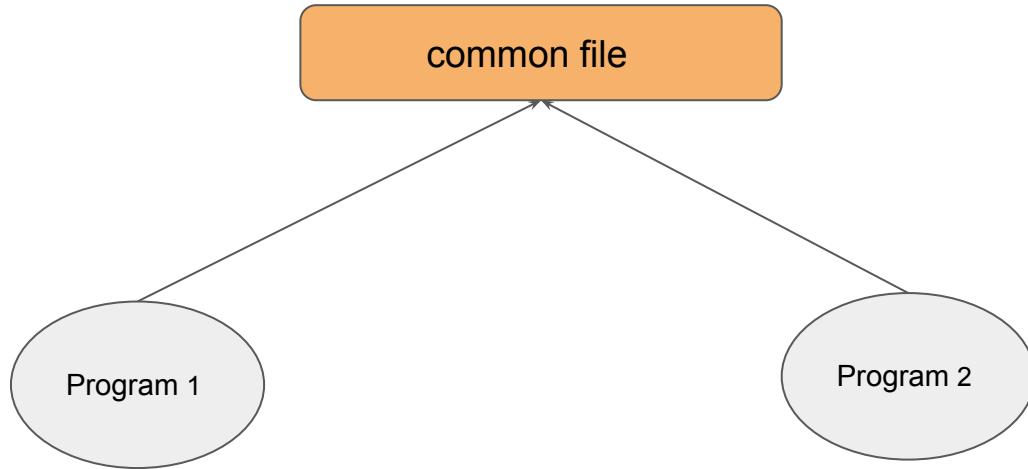
- A Docker image is built up from a series of layers.
- Storage Driver piece all these things together for you
- There are multiple storage drivers available with different trade-off..

```
FROM ubuntu:15.04  
COPY . /app  
RUN make /app  
CMD python /app/app.py
```

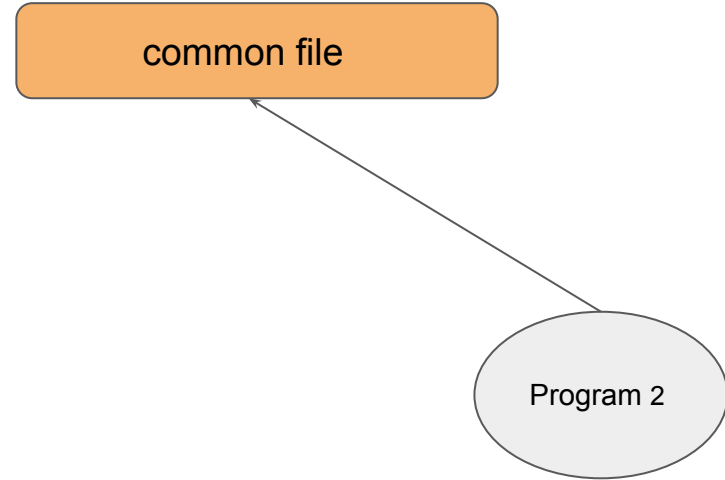
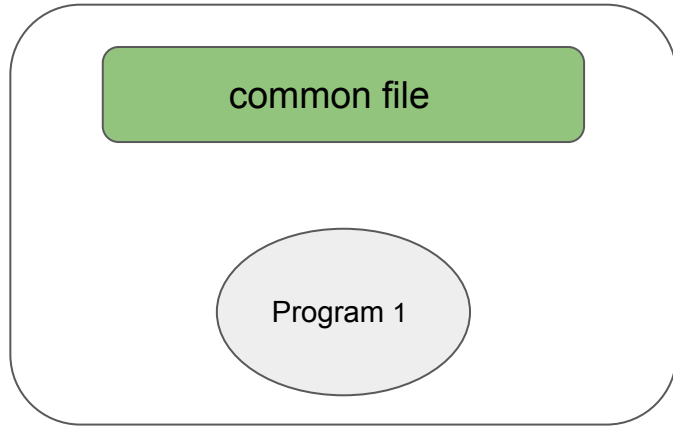


The copy-on-write (CoW) strategy

Copy on write" means: everyone has a single shared copy of the same data until it's written, and then a copy is made.



The copy-on-write (CoW) strategy



Various Storage Drivers Available

There are various storage drivers available in Docker, these includes:

- AUFS
- Device Mapper
- OverlayFS
- ZFS
- VFS
- Btrfs

Block vs Object Storage

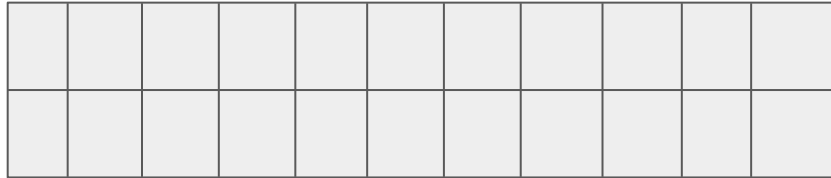
Everything can be lost

Block Storage

In block storage, the data is stored in terms of blocks

Data stored in blocks are normally read or written entirely a whole block at the same time

Most of the file systems are based on block devices.



Block Storage

Every block has an address and application can be called via SCSI call via it's address

There is no storage side meta-data associated with the block except the address.

Thus block has no description, no owner



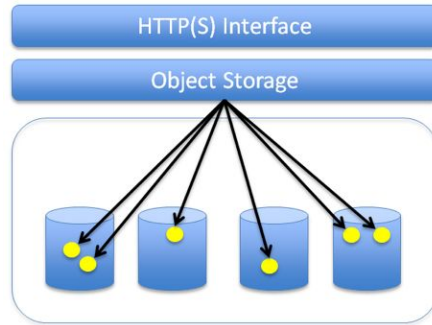
Object Storage

Object storage is a data storage architecture that manages data as objects as opposed to blocks of storage.

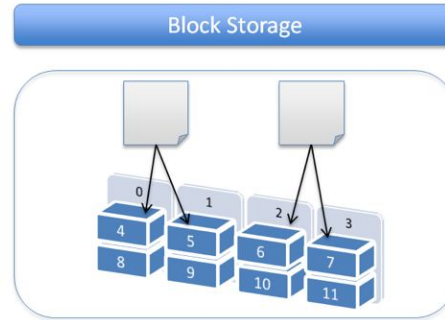
An object is defined as a data (file) along with all its meta-data which is combined together as an object.

This object is given an ID which is calculated from the content of the object (from the data and metadata). Application can then call object with the unique object ID.

Difference



- Store virtually unlimited files.
- Maintain file revisions.
- HTTP(S) based interface.
- Files are distributed in different physical nodes.



- File is split and stored in fixed sized blocks.
- Capacity can be increased by adding more nodes.
- Suitable for applications which require high IOPS, database, transactional data.

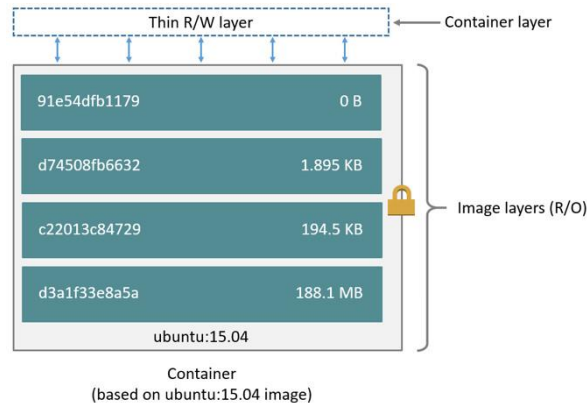
Docker Volumes

Build once, use anywhere

Challenges with files in Container Writable Layer

By default all files created inside a container are stored on a writable container layer. This means that:

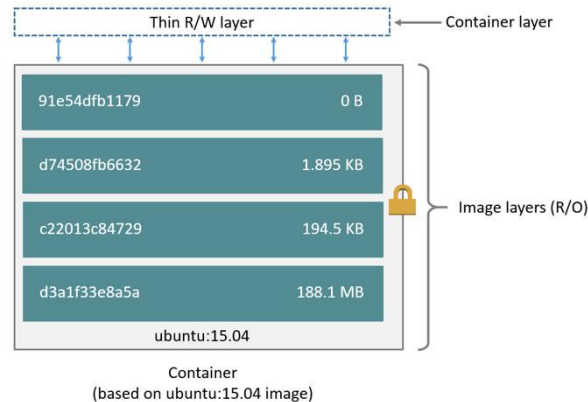
The data doesn't persist when that container no longer exists, and it can be difficult to get the data out of the container if another process needs it.



Challenges with files in Container Writable Layer - Part 2

Writing into a container's writable layer requires a storage driver to manage the filesystem. The storage driver provides a union filesystem, using the Linux kernel.

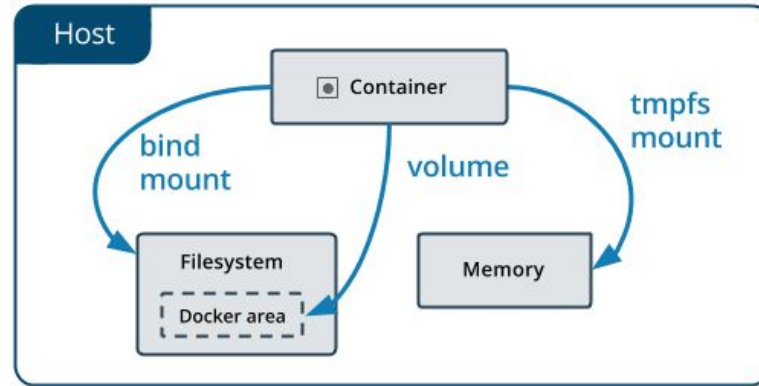
This extra abstraction reduces performance as compared to using data volumes, which write directly to the host filesystem.



Ideal Approach for Persistent Data

Docker has two options for containers to store files in the host machine, so that the files are persisted even after the container stops: volumes, and bind mounts.

If you're running Docker on Linux you can also use a tmpfs mount.



Important Pointers to Remember

A given volume can be mounted into multiple containers simultaneously.

When no running container is using a volume, the volume is still available to Docker and is not removed automatically.

When you mount a volume, it may be named or anonymous. Anonymous volumes are not given an explicit name when they are first mounted into a container, so Docker gives them a random name that is guaranteed to be unique within a given Docker host.

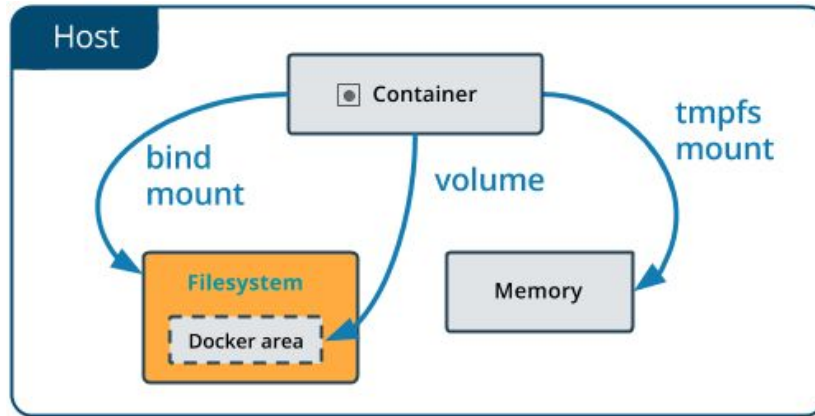
Bind Mounts

Build once, use anywhere

Understanding Bind Mounts

When you use a bind mount, a file or directory on the host machine is mounted into a container.

The file or directory is referenced by its full or relative path on the host machine.



Automatically Remove Volume on Container Removal

Build once, use anywhere

Getting Started

Whenever a container is created with `-dt` flag and if the main process completes/stops, then it goes into the Exited stage.

We can see list of all containers (Running/Exited) with `docker ps -a` command

Many a times, containers performs a job and we want container to automatically get deleted once it exits. This can be achieved with the `--rm` option.

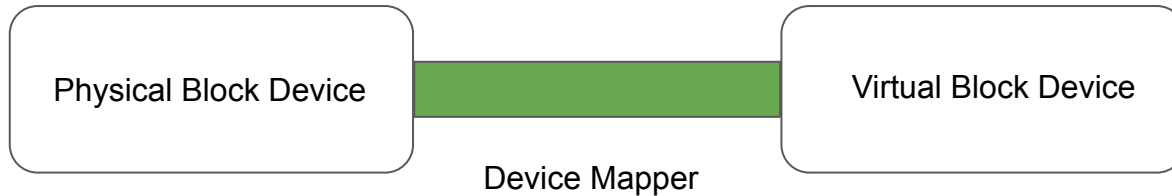
Device Mapper

Build once, use anywhere

Overview of Device Mapper

The device mapper is a framework provided by the Linux kernel for mapping physical block devices onto higher-level virtual block devices.

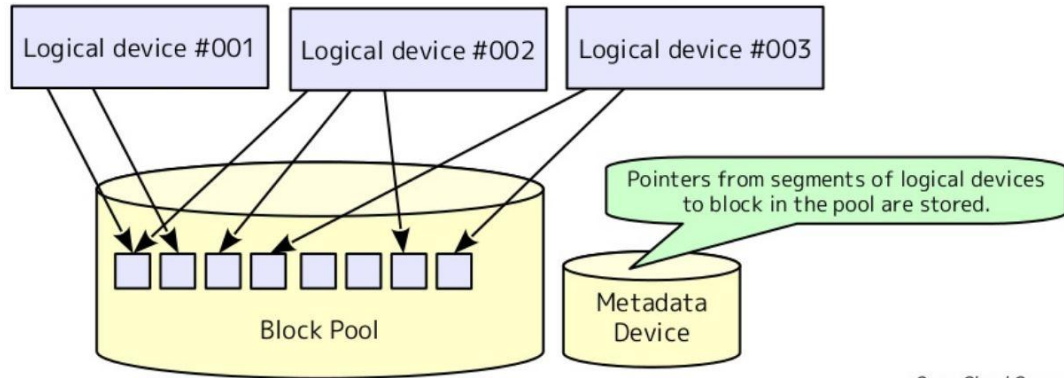
Device mapper works by passing data from a virtual block device, which is provided by the device mapper itself, to another block device.



Overview of Device Mapper

Device mapper creates logical devices on top of physical block device and provides feature likes :-

- RAID, Encryption, Cache and various others.



Open Cloud Campus

Few Important Pointers

There are two modes for devicemapper storage driver:

i) loop-lvm mode:

- Should only be used in testing environment.
- Makes use of loopback mechanism which is generally on the slower side.

ii) direct-lvm mode:

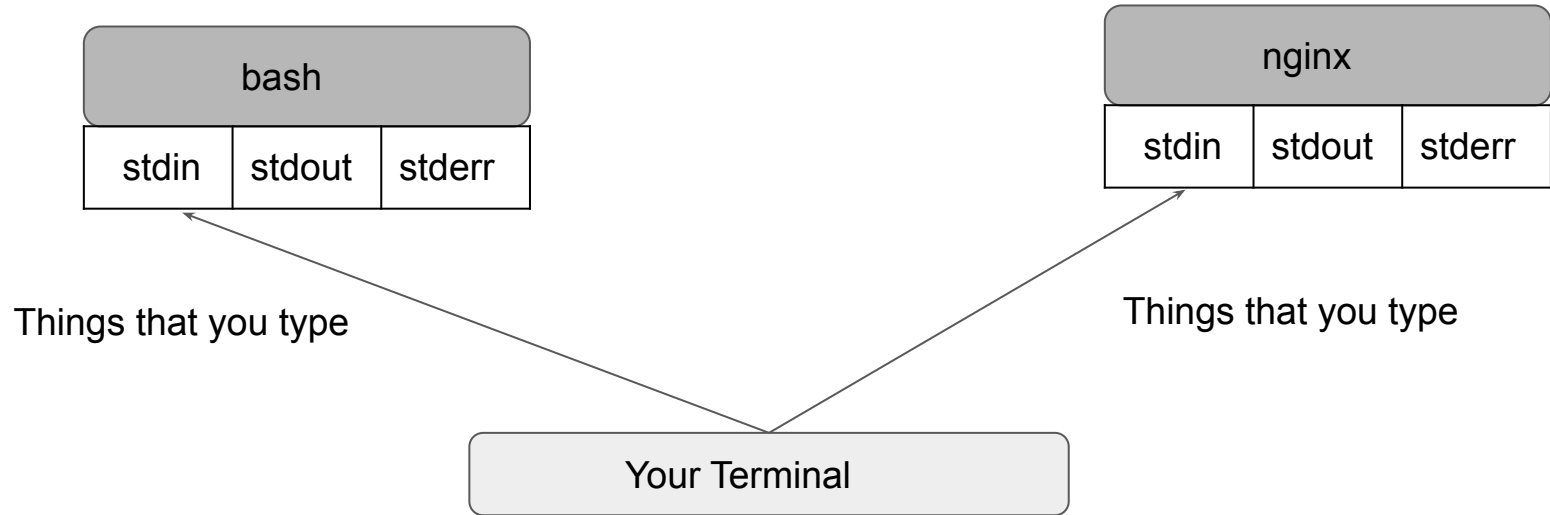
- Production hosts using the devicemapper storage driver must use direct-lvm mode.
- This is much more faster than the loop-lvm mode.

Logging Drivers

Build once, use anywhere

Getting Started

UNIX and Linux commands typically open three I/O streams when they run, called **STDIN**, **STDOUT**, and **STDERR**



Logging Drivers in Docker

There are a lot of logging driver options available in Docker, some of these include:

- json-file
- none
- syslog
- local
- journald
- splunk
- awslogs

The docker logs command is not available for drivers other than json-file and journald.

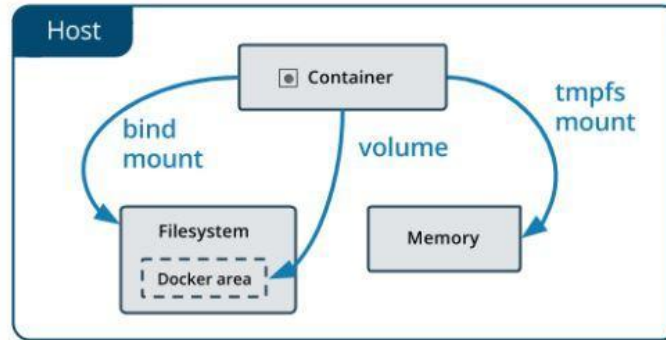
Volumes in Kubernetes

Security Aspect

Two Challenges

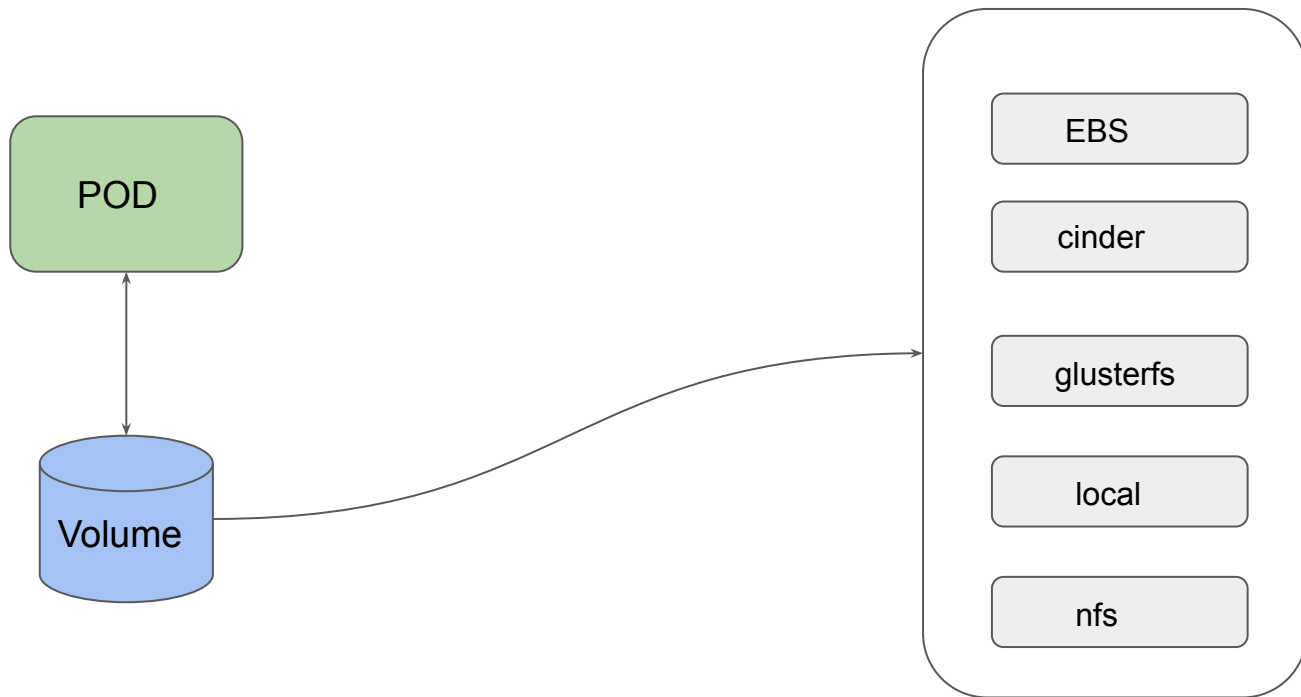
On-disk files in a Container are ephemeral.

When there are multiple containers who wants to share same data, it becomes a challenge.



Volumes in Kubernetes

One of the benefits of Kubernetes is that it supports multiple types of volumes.



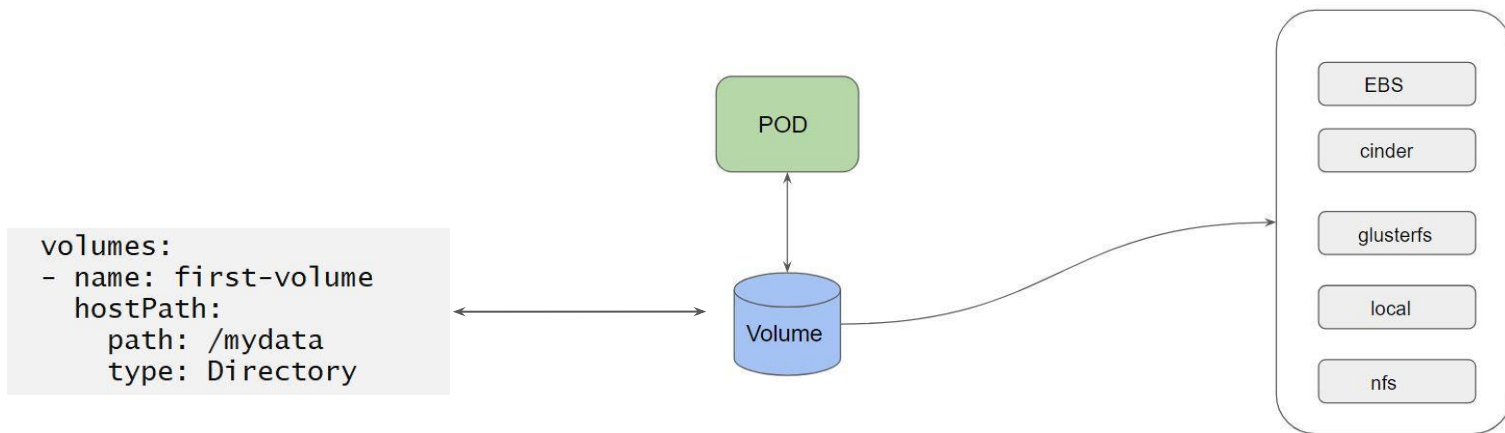
PersistentVolume and PersistentVolumeClaim

Storage Aspect

Volumes in Kubernetes

Generally developers create the YAML files associated with pods that they want to deploy.

In a scenario of directly referencing to volumes, developer needs to be aware of configurations



Understanding the Challenge



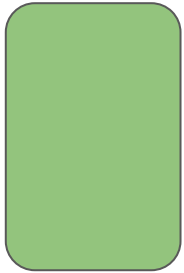
I am a Developer. I just want to write my code and pod.yaml file. I don't want to take care of storage provisioning, and its configurations.



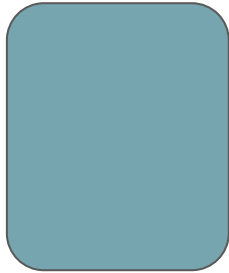
I am a Storage Administrator. I can take care of provisioning storage and its associated configurations. Developers can reference the storage within their podspec.

Persistent Volumes

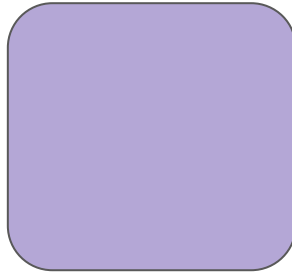
A Persistent Volume (PV) is a piece of storage in the cluster that has been provisioned by an administrator or dynamically provisioned using Storage Classes



Volume 1
Size: Small
Speed: Fast



Volume 2
Size: Medium
Speed: Fast



Volume 3
Size: Big
Speed: Slow



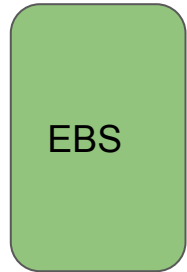
Volume 4
Size: VSmall
Speed: Ultra Fast



Volume 5
Size: Very Big
Speed: Very Slow

Different Types of Persistent Volumes

- Every Volume which is created can be of different type.
- This can be taken care by the Storage Administrator / Ops Team



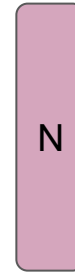
Volume 1
Size: Small
Speed: Fast



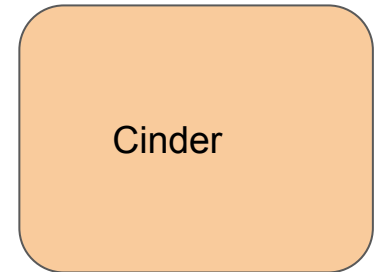
Volume 2
Size: Medium
Speed: Fast



Volume 3
Size: Big
Speed: Slow



Volume 4
Size: VSmall
Speed: Ultra Fast



Volume 5
Size: Very Big
Speed: Very Slow

PersistentVolumeClaim

A PersistentVolumeClaim is a request for the storage by a user.

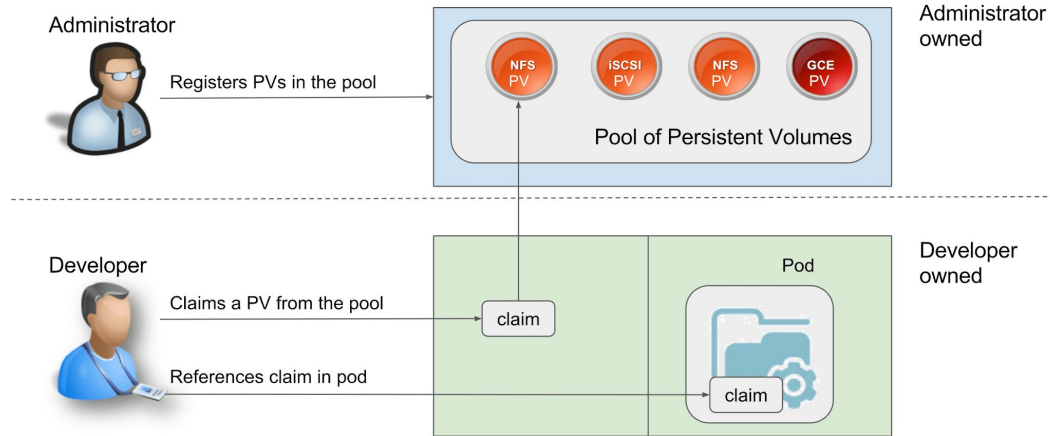
Within the claim, user need to specify the size of the volume along with access mode.

Developer:

I want a volume of size 10 GB which is has speed of Fast for my pod.

Overall Working Steps

1. Storage Administrator takes care of creating PV.
2. Developer can raise a “Claim” (I want a specific type of PV).
3. Reference that claim within the PodSpec file.

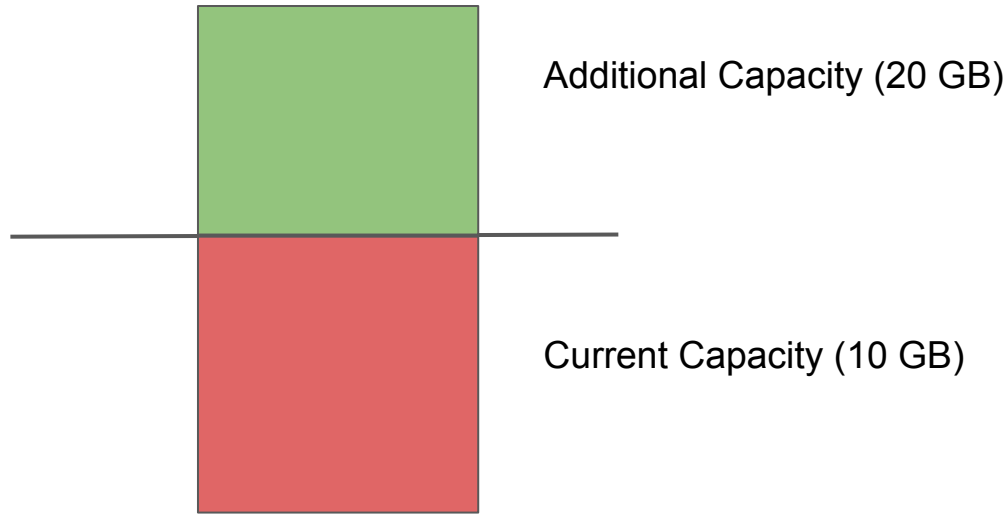


Volume Expansion in K8s

Mastering K8s

Understanding the Challenge

It can happen that your persistent volume has become full and you need to expand the storage for additional capacity.



Step 1 - Enable Volume Expansion

1. Enabling Volume Expansion in the Storage Class.

```
bash-4.2# kubectl describe storageclass do-block-storage
Name:          do-block-storage
IsDefaultClass: Yes
Annotations:    kubectl.kubernetes.io/last-applied-configuration={"allowVolumeExpansion":true,"apiVersion":"storage.k
8s.io/v1","kind":"StorageClass","metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"},"name
":"do-block-storage"},"provisioner":"dobs.csi.digitalocean.com","reclaimPolicy":"Delete"}
,storageclass.kubernetes.io/is-default-class=true
Provisioner:    dobs.csi.digitalocean.com
Parameters:     <none>
AllowVolumeExpansion: True
MountOptions:   <none>
ReclaimPolicy:  Delete
VolumeBindingMode: Immediate
Events:         <none>
```

Step 2 - Resizing the PVC

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"PersistentVolumeClaim",
      "status":{"phase":"Bound"},"spec":{"accessModes":["ReadWriteOnce"],"resource
      -storage"}}
    pv.kubernetes.io/bind-completed: "yes"
    pv.kubernetes.io/bound-by-controller: "yes"
    volume.beta.kubernetes.io/storage-provisioner: dobs.
creationTimestamp: "2020-08-30T07:08:20Z"
finalizers:
- kubernetes.io/pvc-protection
name: csi-pvc
namespace: default
resourceVersion: "2154"
selfLink: /api/v1/namespaces/default/persistentvolumeclaims/csi-pvc
uid: 2ec4833a-bdb7-49b1-8c32-6b73aaafea04
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 15Gi
```


Step 3 - Restart the POD

Once PVC object is modified, you will have to restart the POD.

```
kubectl delete pod [POD-NAME]
```

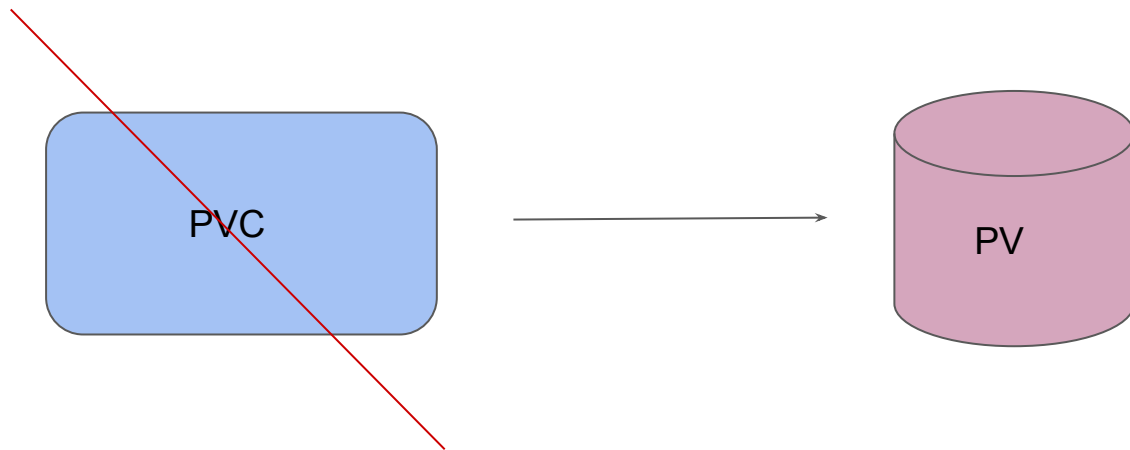
```
kubectl apply -f pod-manifest.yaml
```

Reclaim Policy

Mastering K8s Storage

Understanding the Challenge

The reclaim policy is responsible for what happens to the data in persistent volume when the kubernetes persistent volume claim has been deleted.



Types of Reclaim Policy

Reclaim Policy	Description
Retain	When the PersistentVolumeClaim is deleted, the PersistentVolume still exists and the volume is considered "released"
Delete	The persistent volume is deleted when the claim is deleted.
Recycle	If supported by the underlying volume plugin, the Recycle reclaim policy performs a basic scrub (<code>rm -rf /thevolume/*</code>) on the volume and makes it available again for a new claim.

Retain Policy

When PVC is deleted, the PersistentVolume still exists and the volume is considered "released".

It is not yet available for another claim because the previous claimant's data remains on the volume.

```
bash-4.2# kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM
pvc-41872ac3-55b7-412c-ac2c-a44e0a04fe33	5Gi	RWO	Retain	Released	default/kplabs-pvc

Steps for Reclamation

An administrator can manually reclaim the volume with the following steps.

Delete the PersistentVolume. The associated storage asset in external infrastructure (such as an AWS EBS, GCE PD, Azure Disk, or Cinder volume) still exists after the PV is deleted.

Manually clean up the data on the associated storage asset accordingly.

Manually delete the associated storage asset, or if you want to reuse the same storage asset, create a new PersistentVolume with the storage asset definition.

S3 Storage Classes

Cloud Storage is Saviour

Use-Case - Netflix

Netflix offers various different subscription plans for various category of requirements.

Main Aim: Watch the Entertainment Content



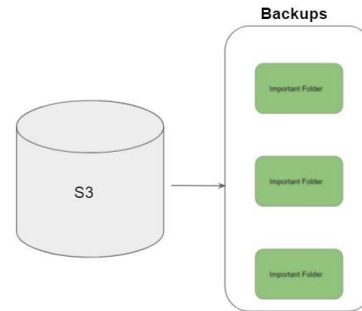
	Basic	Standard	Premium
Monthly cost* (Arab Emirates Dirham)	29 AED	39 AED	56 AED
Number of screens you can watch on at the same time	1	2	4
Number of phones or tablets you can have downloads on	1	2	4
Unlimited movies and TV shows	✓	✓	✓
Watch on your laptop, TV, phone or tablet	✓	✓	✓
HD available		✓	✓
Ultra HD available			✓

Initial Challenge - S3

AWS has millions of active customers.

Each customer might have different requirements for data storage.

Main Aim: Store Data.



S3 Storage Classes

Amazon S3 offers a range of storage classes designed for different use cases.

Storage Classes	Description
S3 Standard	Offers high durability, availability, and performance object storage for frequently accessed data
S3 Standard-Infrequent Access	For data that is accessed less frequently, but requires rapid access when needed.
Amazon S3 Glacier	Low-cost storage class for data archiving

AWS S3 Standard

S3 Standard offers high durability, availability, and performance object storage for frequently accessed data.

Designed for durability of 99.999999999% of objects (eleven nines)

Example :-

If we have 10,000 files stored in S3 (11 nines durability) then you can expect to lose one file every ten million years.

AWS S3 Standard IA

S3 Standard-IA is for data that is accessed less frequently, but requires rapid access when needed.

Comparing storage cost of 1TB data stored in S3 based on accessibility patterns.

Criteria	Amazon S3	Amazon S3 IA
Storage of 1TB Data	\$23.44	\$23.50
50% storage accessed in last 30 days	-	\$18.18
0% storage accessed in last 30 days	-	\$12.80

Amazon S3 Glacier

Glacier is meant to be for archiving and for storing long-term backups.

Ideally meant for data that needs to be archived for years without much requirement of access.

Criteria	Amazon S3	Glacier
Storage of 1TB Data	\$23.44	\$4.10

Multiple S3 Storage Classes

Performance across the S3 Storage Classes

	S3 Standard	S3 Intelligent-Tiering*	S3 Standard-IA	S3 One Zone-IA†	S3 Glacier	S3 Glacier Deep Archive
Designed for durability	99.999999999% (11 9's)	99.999999999% (11 9's)	99.999999999% (11 9's)	99.999999999% (11 9's)	99.999999999% (11 9's)	99.999999999% (11 9's)
Designed for availability	99.99%	99.9%	99.9%	99.5%	99.99%	99.99%
Availability SLA	99.9%	99%	99%	99%	99.9%	99.9%
Availability Zones	≥3	≥3	≥3	1	≥3	≥3
Minimum capacity charge per object	N/A	N/A	128KB	128KB	40KB	40KB
Minimum storage duration charge	N/A	30 days	30 days	30 days	90 days	180 days

Durability vs Availability

- Durability is percent (%) over one year period of time that the file which is stored in S3 will not be lost.
- Availability is percent (%) over one year period of time that the file stored in S3 will not be available.

Example :-

For Servers, Availability is one of the key metric and any minute of downtime is a loss. However what happens if component of server itself fails and server goes down ?

Overview of DCA Exam

Exam Experience & Important Tips

Overview of DCA Exam

DCA Exam is remotely proctored on your **Windows** or **Mac** computer

55 multiple choice questions in 90 minutes

USD \$195 or Euro €175 purchased online

Results are delivered immediately

Keep your ID card ready and be alone in the room.

DCA Initial Results



Docker Certified Associate

Examinee Details

First Name	Zeal
Last Name	Vora
Email	sunzealvora@gmail.com
Company	

Congratulations, you passed this exam

Docker Certified Associate

Overview of Imp Pointers for Exams

Exam Experience & Important Tips

Getting Started

The exam questions are generally small and you have more than sufficient time to complete the exam.

Certain questions can be tricky, so you should know how things work.

Why this section?

All the topics of this course are important for the exams.

The purpose of this section is to not skip the topics of course and directly jump to this section and sit for the exams.

The purpose is to make sure that your concepts are clear with 100% surity about the topics discussed henceforth.

Important Aspect to this section

1 topic can span across multiple domains.

In-case if that topic is not mentioned in Domain 1, it would be mentioned in other domain.

It is challenging to regularly update entire videos when a new kind of topic is asked in exam.

There is document called as “Updated Pointers” which has list of all such topics.

Make sure to complete the “Exam Preparation Quiz”

DCA Initial Results



Docker Certified Associate

Examinee Details

First Name	Zeal
Last Name	Vora
Email	sunzealvora@gmail.com
Company	

Congratulations, you passed this exam

Docker Certified Associate

Overview of DOMC Questions

Exam Experience & Important Tips

Overview of Exam Question Format

New exam question format is based on both DOMC and Multiple Choice

Type of Questions	Number of Questions
DOMC	42
Multiple Choice	13

DOMC Questions

MCQ

Overview of DOMC

DOMC stands for discrete option multiple choice.

Variations of DOMC

- Presented with only a single option, correct or incorrect, and then scoring the test item.
- Requiring more than one YES response to correct options.
- Presenting one or more non-scored options after the item has been scored.
- Increasing the number of correct and incorrect options

Approaching DOMC

If your concept of topic is clear, the DOMC questions would be easy to answer.

Make habit of calculation the answer of a question before reading any options from multiple-choice.

Example:

ADD Instruction provides which additional features over COPY?

Testing Against DOMC

Not so good news!

The DOMC item type is patented, and a license is needed to use the item in a quiz, test or exam.

Follow the tip that was previously suggested.

You cannot modify answers of DOMC questions (immutable)

Important Pointer - Part 1

Exam Experience & Important Tips

Important Pointers - Part 01

1. You should know how to create swarm Service

- `docker service create --name webserver --replicas 1 nginx`

2. Difference between replicated and global service

- Replicated service will have N number of containers defined with the `--replica` flag
- Global service will create 1 container in every node of the cluster.

Important Pointers - Part 02

3. Multiple approaches to scale swarm service

- `docker service scale mywebserver=5`
- `docker service update --replicas 5 mywebserver`

4. Difference between two approaches to scale swarm service

- `docker service scale` allows us to specify multiple services in same command.
- `docker service update` command only allows us to specify one service per command.

Important Pointers - Part 03

5. Draining Swarm Node

- `docker node update --availability drain swarm03`
- `docker node update --availability active swarm03`

6. Understanding the use-case of Docker Stack CLI

- `docker stack deploy --compose-file docker-compose.yml`

Important Pointers - Part 04

7. Placement Constraints

`docker service create --name webserver --constraint node.label.region==blr nginx`

`docker service create --name webserver --constraint node.label.region!=blr nginx`

8. Adding Labels to a node

`docker node update --label-add region=mumbai worker-node-id`

Important Pointers - Part 05

9. Overlay Networks & Security

Overlay network allows containers spreaded across different servers to communicate.

The communication between containers can be made secured with IPSec tunnels.

docker network create **--opt encrypted** --driver overlay my-overlay-secure-network

Important Pointers - Part 06

10. Revise and Double Revise the Quorum

Cluster Size	Majority	Fault Tolerance
1	0	0
2	2	0
3	2	1
4	3	1
5	3	2
6	4	2
7	4	3

Important Pointers - Part 07

11. Troubleshooting aspect

- docker system events provides you real-time even information from your server.
- Service Deployment would be in pending state if node is drained.
- Service deployments can also be in pending state due to placement constraints.
- You can further inspect the task to see more information.

Important Pointers 8 - Docker Stack Commands

Child Commands	Description
<code>docker stack deploy</code>	Deploy a new stack or update an existing stack
<code>docker stack ls</code>	List stacks
<code>docker stack ps</code>	List the tasks in the stack
<code>docker stack services</code>	List the services in the stack

Important Pointers 9 - Docker Service Commands

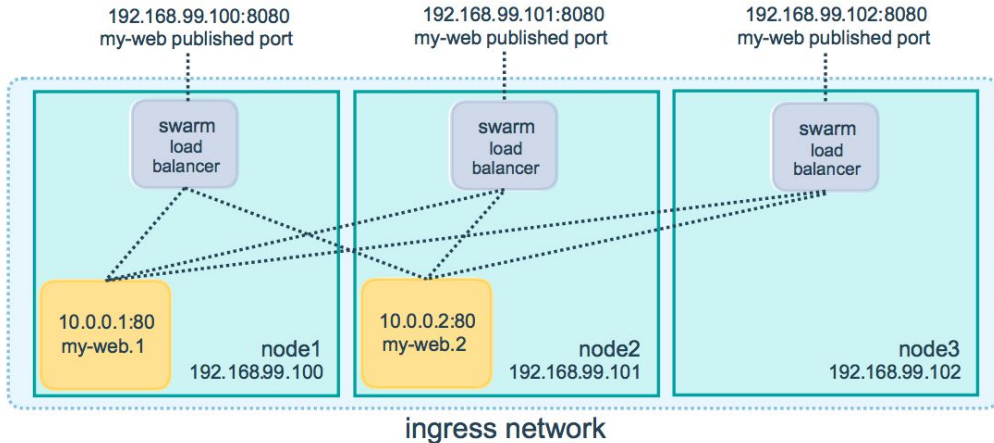
Child Commands	Description
docker service create	Create a new service
docker service inspect	Display detailed information on one or more services
docker service ps	List the tasks of one or more services
docker service update	Update a service
docker service scale	Scale one or multiple replicated services

Important Pointers 10 - Swarm Routing Mesh

Routing mesh enables each node in the swarm to accept connections on published ports for any service running in the swarm, even if there's no task running on the node.

`docker service create --name my_web --replicas 3 --publish published=8080,target=80 nginx`

8080 goes to 80



Important Pointers 11 - Updating Image of Swarm Service

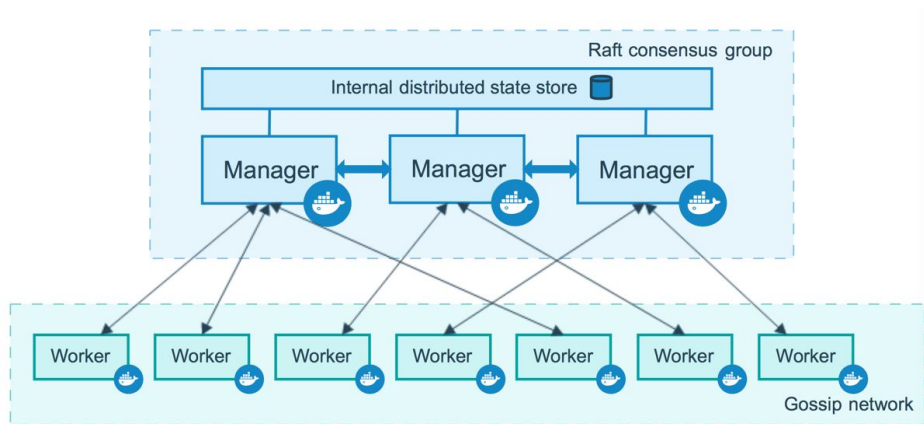
To update an image of a swarm service, following command can be used:

```
docker swarm update --image myimage:tag servicename
```


Important Pointers 12 - Manager and Worker Nodes

- Manager Nodes handles cluster management tasks like scheduling services.
- Raft implementation is used to maintain consistent internal state.
- Sole purpose of worker nodes is to execute containers.

To prevent the scheduler from placing tasks on a manager nodeset the availability for the manager node to Drain



Important Pointers 13 - Join Tokens in Swarm

Join tokens are secrets that allow a node to join the swarm.

There are two different join tokens available, one for the worker role and one for the manager role.

`docker swarm join-token worker`

Important Pointers 14 - Initializing Swarm

Initialize a swarm. The docker engine targeted by this command becomes a manager in the newly created single-node swarm.

```
docker swarm init
```

Important Pointers 15 - Miscellaneous Swarm

Leaving Docker Swarm Cluster:

If a worker intends to leave swarm, he can run the following command:

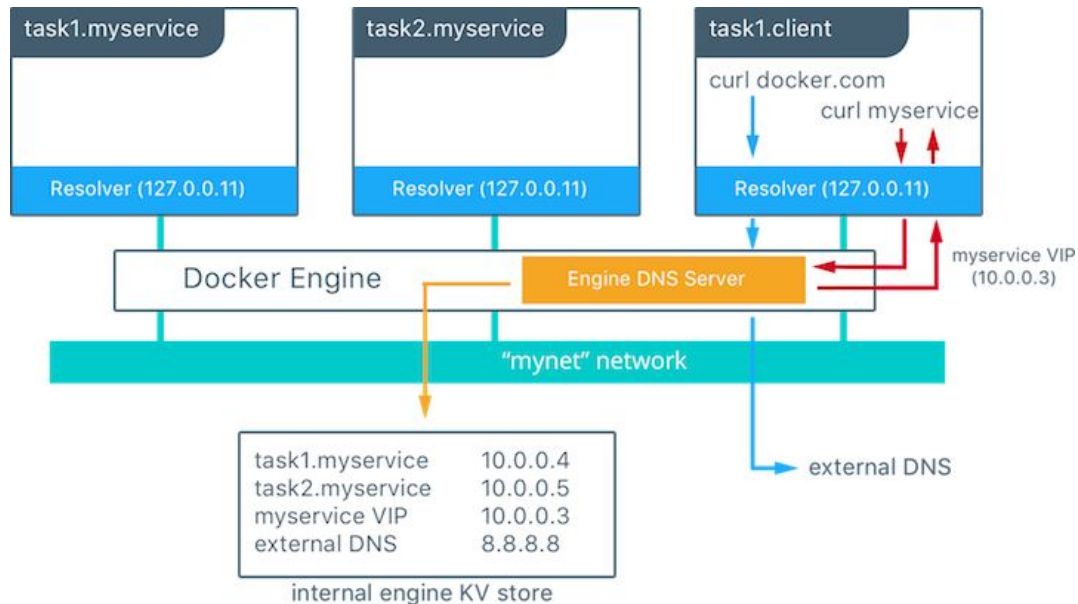
```
docker swarm leave
```

You can use the `--force` option on a manager to remove it from the swarm.

Only use `--force` in situations where the swarm will no longer be used after the manager leaves, such as in a single-node swarm.

Important Pointers 16 - Service Discovery in Swarm

Docker uses embedded DNS to provide service discovery for containers running on a single Docker engine and tasks running in a Docker swarm



Miscellaneous Pointer - 1

The docker system df command displays information regarding the amount of disk space used by the docker daemon.

```
C:\Users\Zeal Vora>docker system df
```

TYPE	TOTAL	ACTIVE	SIZE	RECLAIMABLE
Images	15	7	6.092GB	2.575GB (42%)
Containers	18	1	23.21GB	18.33GB (78%)
Local Volumes	88	3	32.09GB	30.85GB (96%)
Build Cache	0	0	0B	0B

Miscellaneous Pointer - 2

Use docker system events to get real-time events from the server. These events differ per Docker object type.

```
$ docker system events --filter 'event=stop'
```

```
2017-01-05T00:40:22.880175420+08:00 container stop 0fdb...ff37 (image=alpine:latest, name=test)
```

```
2017-01-05T00:41:17.888104182+08:00 container stop 2a8f...4e78 (image=alpine, name=kickass_brattain)
```

```
$ docker system events --filter 'image=alpine'
```

```
2017-01-05T00:41:55.784240236+08:00 container create d9cd...4d70 (image=alpine, name=happy_meitner)
```

```
2017-01-05T00:41:55.913156783+08:00 container start d9cd...4d70 (image=alpine, name=happy_meitner)
```

```
2017-01-05T00:42:01.106875249+08:00 container kill d9cd...4d70 (image=alpine, name=happy_meitner, signal=15)
```

```
2017-01-05T00:42:11.111934041+08:00 container kill d9cd...4d70 (image=alpine, name=happy_meitner, signal=9)
```

```
2017-01-05T00:42:11.119578204+08:00 container die d9cd...4d70 (exitCode=137, image=alpine, name=happy_meitner)
```

```
2017-01-05T00:42:11.173276611+08:00 container stop d9cd...4d70 (image=alpine, name=happy_meitner)
```

Miscellaneous Pointer - 3

dockerd is the persistent process that manages containers. Docker uses different binaries for the daemon and client. To run the daemon you type dockerd

For specifying the configuration options, you can make use of daemon.json file.

The default location of the configuration file on Linux is /etc/docker/daemon.json

The default location of the configuration file on Windows is
%programdata%\docker\config\daemon.json

Miscellaneous Pointer - 4

`docker container inspect` display detailed information on one or more containers.

It can also show you list of volumes that are attached to the container.

```
"Mounts": [  
  {  
    "Type": "bind",  
    "Source": "/host_mnt/d/containers/git",  
    "Destination": "/gitplace",  
    "Mode": "",  
    "RW": true,  
    "Propagation": "rprivate"  
  }  
],
```

Miscellaneous Pointer 5 - Inspecting Container

`docker container inspect` display detailed information on one or more containers.

It can also show you list of volumes that are attached to the container.

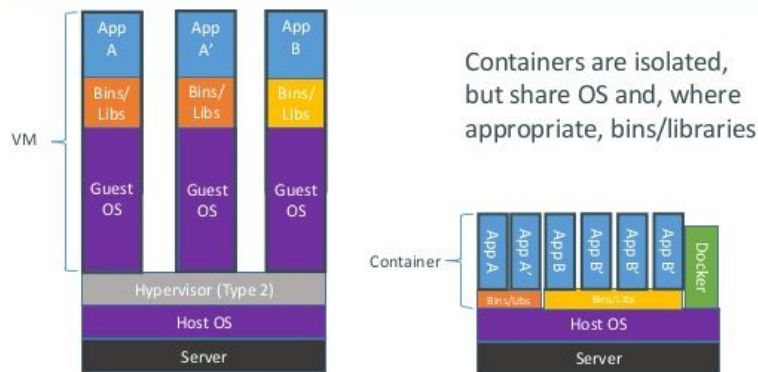
```
"Mounts": [  
  {  
    "Type": "bind",  
    "Source": "/host_mnt/d/containers/git",  
    "Destination": "/gitplace",  
    "Mode": "",  
    "RW": true,  
    "Propagation": "rprivate"  
  }  
],
```

Containers vs Virtual Machines

Virtual Machine contains entire Operating System.

Container uses the resource of the host operating system (primarily the kernel)

Containers vs. VMs



Important Pointer - Part 2

Image Creation, Management, and Registry

Important Pointers - Part 01

1. Have thorough understanding about various Dockerfile instructions.

- ADD
- COPY
- RUN
- ENTRYPOINT
- WORKDIR
- ENV
- VOLUMES
- CMD
- HEALTHCHECK

Important Pointers - Part 02

2. Understand difference between ADD and COPY

COPY allows us to copy files from local source to destination.

ADD allows the same in addition to using URL & extraction capabilities of TAR files.

3. Know difference between CMD and ENTRYPOINT

- CMD can be overridden.
- ENTRYPOINT cannot be overridden.

Important Pointers - Part 03

4. Use-cases of using ADD vs wget/curl

- Because image size matters, using ADD to fetch packages from remote URLs is strongly discouraged; you should use curl or wget instead.

```
ADD http://example.com/big.tar.xz /usr/src/things/
```

```
RUN tar -xJf /usr/src/things/big.tar.xz -C /usr/src/things
```

```
RUN make -C /usr/src/things all
```

```
RUN mkdir -p /usr/src/things \
```

```
&& curl -SL http://example.com/big.tar.xz \
```

```
| tar -xJC /usr/src/things \
```

```
&& make -C /usr/src/things all
```

Important Pointers 04 - WORKDIR

The **WORKDIR** instruction sets the working directory for any RUN, CMD, ENTRYPOINT, COPY and ADD instructions that follow it in the Dockerfile

The WORKDIR instruction can be used multiple times in a Dockerfile

Sample Snippet:

```
WORKDIR /a
```

```
WORKDIR b
```

```
WORKDIR c
```

```
RUN pwd
```

Output = /a/b/c

Important Pointers - Part 05

5. Understanding the format option

Format option allows us to format the output based on various criteria that we have defined with the command

```
[root@ip-172-31-45-184 ~]# docker images --format "{{.ID}}: {{.Repository}}"
87159635da2d: amd-ssh
d131e0fa2585: ubuntu
eaa8f22fecc5: registry.aquasec.com/enforcer
27a188018e18: nginx
af2f74c517aa: busybox
01da4f8f9748: amazonlinux
```

Important Pointers - Part 06

6. Understanding the filter option

Filter option allows us to filter output based on condition provided.

Sample Use-Case:

- Show all the dangling images
- Show all the images created after N image.
- Show all the containers which are in the exited stage.

Important Pointers - Part 07

7. Accessing in-secure docker registries

By default, docker will not allow you to perform operation with an insecure registry. You can override by adding following stanza within the `/etc/docker/daemon.json` file

```
{  
  "insecure-registries" : ["myregistrydomain.com:5000"]  
}
```

Important Pointers - Part 08

8. Pushing an image to a private repository

In order to push an image to private repository, you will have to tag it with the DNS name.

For example:

Registry Name: example.com

```
docker tag ubuntu:latest example.com/myrepo:ubuntu
```

Important Pointers - Part 09

9. Login to a private repository

`docker login example.com`

10. Searchability Aspect of Images

- `docker search nginx`
- `docker search nginx --filter "is-official=true"`

Important Pointer 10 - Moving Images Across Hosts

The docker save command will save one or more images to a tar archive

```
docker save busybox > busybox.tar
```

The docker load command will load an image from a tar archive

```
docker load < busybox.tar
```

Important Pointer 11 - Container Management

The docker commit can be used to to save the current state of a container to an image.

```
docker commit c3f279d17e0a container-image:v2
```

You can use docker export command to export a container to another system as an image tar file.

```
docker export my-container > container.tar
```

When we export a container, all the resultant imported image will be flattened.

Important Pointer 12 - Dockerfile ENV

The ENV instruction sets the environment variable <key> to the value <value>.

```
ENV NGINX 1.2
```

```
RUN curl -SL http://example.com/web-\$NGINX.tar.xz
```

```
RUN tar -xzf web-\$NGINX.tar.xz
```

You can use the -e, --env, and --env-file flags to set simple environment variables in the container you're running, or overwrite variables that are defined in the Dockerfile of the image you're running.

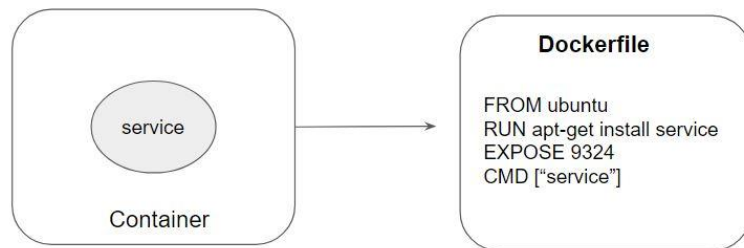
```
docker run --env VAR1=value1 --env VAR2=value2 ubuntu env | grep VAR
```


Important Pointer 13 - EXPOSE

The EXPOSE instruction informs Docker that the container listens on the specified network ports at runtime.

The EXPOSE instruction does not actually publish the port.

It functions as a type of documentation between the person who builds the image and the person who runs the container, about which ports are intended to be published.



Important Pointer 14 - HEALTHCHECK

HEALTHCHECK instruction in Docker allows us to tell the platform on how to test that our application is healthy.

HEALTHCHECK CMD curl --fail http://localhost || exit 1

That uses the curl command to make an HTTP request inside the container, which checks that the web app in the container does respond.

It exits with a 0 if the response is good, or a 1 if not - which tells Docker the container is unhealthy.

Important Pointer 15 - Tagging Docker Images

Create a tag TARGET_IMAGE that refers to SOURCE_IMAGE

Syntax:

```
docker tag SOURCE_IMAGE[:TAG] TARGET_IMAGE[:TAG]
```

Example Snippet:

```
docker tag httpd fedora/httpd:version1.0
```

Important Pointer 16 - Pruning Docker Images

`docker image prune` command allows us to clean up unused images.

By default, the above command will only clean up dangling images.

Dangling Images = Image without Tags and Image not referenced by any container

Downloading Images from Private Registry

If your image is available on a private registry which requires login, use the `--with-registry-auth` flag with `docker service create`, after logging in.

```
docker login registry.example.com
```

```
docker service create --with-registry-auth \--name my_service  
registry.example.com/acme/my_image:latest
```

This passes the login token from your local client to the swarm nodes where the service is deployed, using the encrypted WAL logs. With this information, the nodes are able to log into the registry and pull the image.

Important Pointer 18 - Build Cache

Docker creates container images using layers.

Each command that is found in a Dockerfile creates a new layer.

Docker uses a layer cache to optimize the process of building Docker images and make it faster.

```
[root@swarm02 build-cache]# docker build -t demo2
Sending build context to Docker daemon 3.072kB
Step 1/4 : FROM python:3.7-slim-buster
--> 87b1022604d5
Step 2/4 : COPY . .
--> Using cache
--> fe355c27a8ff
```

Important Pointer - Domain 3

Installation & Configuration

Important Pointers 01 - DTR Backup Process

When we backup DTR, there are certain things which are not backed.

User/Organization backup should be taken from UCP.

Data	Description	Backed up
Raft keys	Used to encrypt communication among Swarm nodes and to encrypt and decrypt Raft logs	yes
membership	List of the nodes in the cluster	yes
Services	Stacks and services stored in Swarm-mode	yes
(overlay) networks	The overlay networks created on the cluster	yes
configs	The configs created in the cluster	yes
secrets	Secrets saved in the cluster	yes
Swarm unlock key	Must be saved on a password manager !	no

Important Pointers 02 - Namespaces

2. Know what namespace are all about.

These namespaces provide a layer of isolation. Each aspect of a container runs in a separate namespace and its access is limited to that namespace.

User namespace is not enabled by default

Important Pointers 03 - Cgroups

Have clear understanding about cgroups.

Control Groups (cgroups) is a Linux kernel feature that limits, accounts for, and isolates the resource usage (CPU, memory, disk I/O, network, etc.) of a collection of processes.

Approaches	Description
--cpus=<value>	If host has 2 CPUs and if you set --cpus=1, than container is guaranteed at most one CPU. You can even specify --cpus=0.5
--cpuset-cpus	Limit the specific CPUs or cores a container can use. A comma-separated list or hyphen-separated range of CPUs a container can use. 1st CPU is numbered 0 2nd CPU is numbered 1. Value of 0-3 means usage of first, second, third and fourth CPU. Value of 1,3 means second and fourth CPU.

Important Pointers 04 - Reservation and Limits

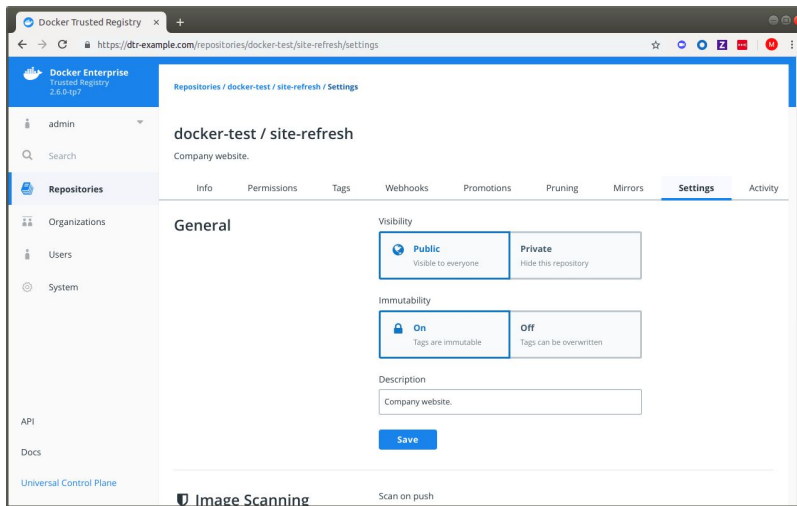
- Limit is a hard limit.
- Reservation is a soft limit.

- `docker container run -dt --name container01 --memory-reservation 250m ubuntu`
- `docker container run -dt --name container02 -m 500m ubuntu`

`--memory` and `--memory-reservation`

Important Pointers 05 - Immutable Tags in DTR

- By default, users with read and write access can overwrite tags.
- To prevent tags from being overwritten, we can configure repository to be immutable.



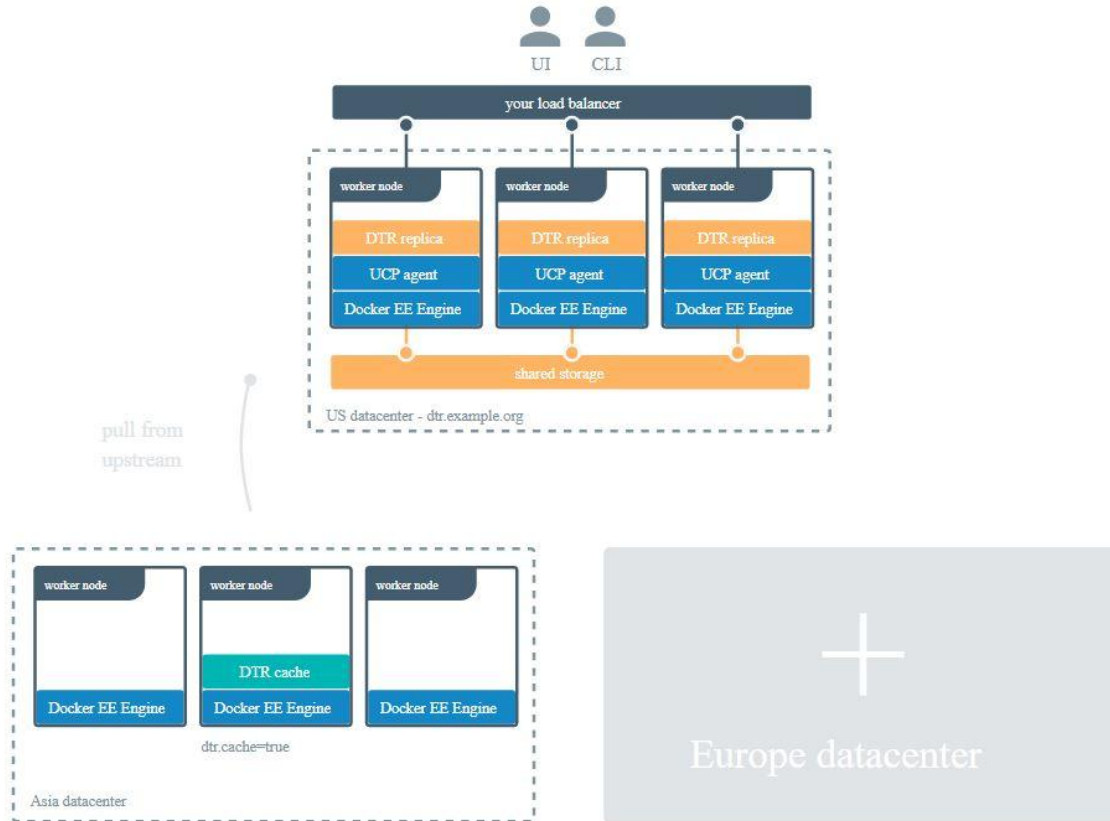
Important Pointers 06 - DTR Cache

To decrease the time to pull an image, you can deploy DTR caches geographically closer to users.

Caches are transparent to users, since users still log in and pull images using the DTR URL address. DTR checks if users are authorized to pull the image, and redirects the request to the cache.



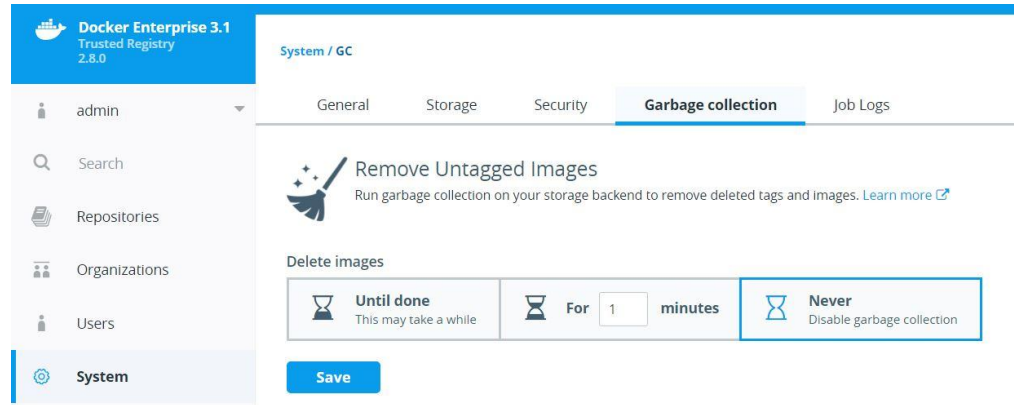
DTR Cache Architecture



DTR Garbage Collection

You can configure the Docker Trusted Registry (DTR) to automatically delete unused image layers, thus saving you disk space.

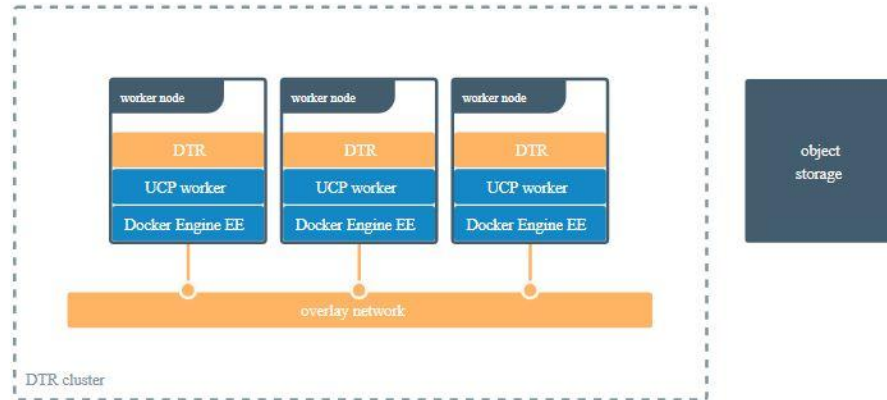
This process is also known as garbage collection.



Important Pointers 07 - DTR High Availability

Docker Trusted Registry is designed to scale horizontally as your usage increases. You can add more replicas to make DTR scale to your demand and for high availability.

If your DTR deployment has multiple replicas, for high availability, you need to ensure all replicas are using the same storage backend.



Important Pointers 08 - HA of UCP and DTR

To have high-availability on UCP and DTR, you need a minimum of:

- 3 dedicated nodes to install UCP with high availability,
- 3 dedicated nodes to install DTR with high availability,

You can monitor the status of UCP by using the web UI or the CLI. You can also use the [_ping](#) endpoint to build monitoring automation.

Important Pointers 09 - Orchestrator Types in UCP

Docker UCP supports both Swarm and Kubernetes.

When you install Docker Enterprise, new nodes are managed by Docker Swarm, but you can change the default orchestrator to Kubernetes in the administrator settings.

To change the orchestrator type for a node from Swarm to Kubernetes:

`docker node update --label-add com.docker.ucp.orchestrator.kubernetes=true <node-id>`

Scheduler

SET ORCHESTRATOR TYPE FOR NEW NODES

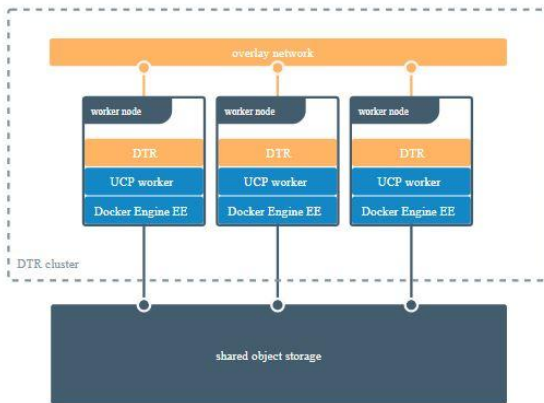
☒ Swarm

☐ Kubernetes

Important Pointers 10 - Storage Driver in DTR

By default, Docker Trusted Registry stores images on the filesystem of the node where it is running, but you should configure it to use a centralized storage backend.

You can configure DTR to use an external storage backend, for improved performance or high availability.



Important Pointers 11 - Supported Storage Systems

Some of the supported storage systems in DTR are:

Local:

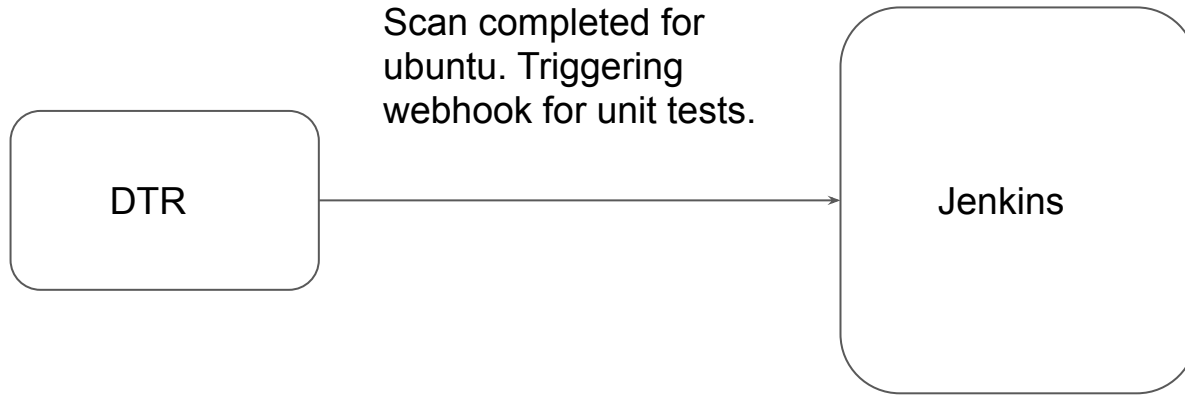
- NFS
- Bind Mount
- Volume

Cloud Storage Provider:

- AWS S3
- Azure
- Google Cloud

Important Pointers 12 - DTR Webhooks

You can configure DTR to automatically post event notifications to a webhook URL of your choosing



Important Pointer - Part 4

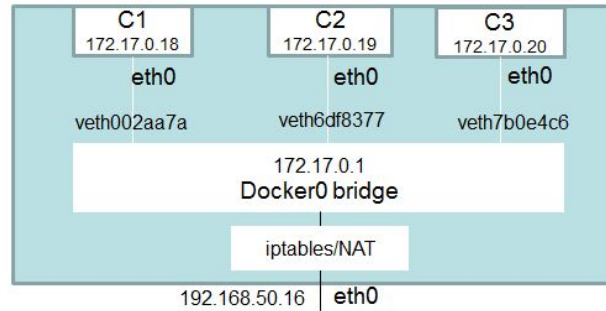
Networking

Important Pointers - Part 01

You should be aware of the primary networking drivers

1. Bridge Network Driver:

- Bridge is the default network driver for Docker.



Important Pointers - Part 02

2. Host Network

- This driver removes the network isolation between the docker host and the docker containers to use the host's networking directly.

4. User Defined Bridge and Legacy Link Approach

Remember that `--link` is a legacy approach and should not be used.

Important Pointers - Part 03

Overlay Network

- Default in Swarm.
- Allows containers across host to communicate with each other.
- Communication can be encrypted with `--opt encrypted` option.
- Do not confuse, `-o` is same as `--opt`

You don't need to create the overlay network on the other nodes, because it will be automatically created when one of those nodes starts running a service task which requires it.

Important Pointers - Part 04

6. Know difference between publish list (-p) and publish all (-P)

- Publish List (-p) will publish list of ports that you define. [-p 80:80]
- Publish All (-P) will assign random ports for all exposed ports of the container.
- -P will map the container port to random port above 32768

7. None Network

- If you want to completely disable the networking stack on a container, you can use the none network.
- This mode will not configure any IP for the container and doesn't have any access to the external network as well as for other containers.

Important Pointers - Part 05

8. Configuring docker for external DNS

- Docker Container's DNS configuration is taken from the host's `/etc/resolv.conf`
- DNS settings for containers be customized via `daemon.json` file.

```
{  
  "dns": ["8.8.8.8", "172.31.0.2"]  
}
```

Important Pointers 06 - Inspecting Network Details

To Fetch information about a specific network, you can run the following command:

```
docker network inspect <network-name>
```

Join us in our Adventure

Be Awesome



kplabs.in/twitter



kplabs.in/linkedin

instructors@kplabs.in

Important Pointer - Part 5

Security

Important Pointers 01 - UCP Client Bundles

- Client bundles are group of certificates and files downloaded from UCP.
- Depending on the permission associated with the user, you can now execute docker swarm commands from your remote machine that take effect on the remote cluster.

Important Pointers 02 - Docker Content Trust

Used for interacting with only trusted images (signed images)

Enabled with `export DOCKER_CONTENT_TRUST=1`

Example Dockerfile:

```
FROM myubuntu:latest
```

```
RUN apt-get install net-tools
```

```
CMD["bash"]
```


Important Pointers 02 - Docker Content Trust

Engine Signature Verification prevents the following:

- docker container run of an unsigned or altered image.
- docker pull of an unsigned or altered image.
- docker build where the FROM image is not signed or is not scratch.

Important Pointers 03 - Docker Secrets

To create a new secret, `docker secret create` command can be used.

This is a cluster management command, and must be executed on a swarm manager node.

Remember that you cannot update or rename a secret.

`--secret-add` and `--secret-rm` flags for docker service update

Important Pointers 04 - Secrets Mount Path

When you grant a newly-created or running service access to a secret, the decrypted secret is mounted into the container in following path:

`/run/secrets/<secret_name>`

We can also specify a custom location for the secret.

`docker service create --name redis --secret source=mysecret,target=/root/secretdata`

Important Pointers 05 - Swarm Auto-Lock

Know about the how you can lock the swarm cluster

If your Swarm is compromised and if data is stored in plain-text, an attack can get all the sensitive information.

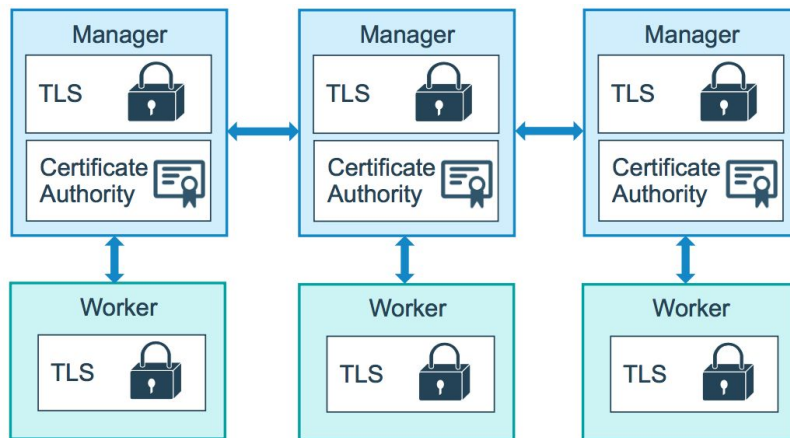
Docker Lock allows us to have control over the keys.

- `docker swarm update --autolock=true`

Important Pointers 06 - Mutual TLS Authentication

When you create a swarm by running `docker swarm init`, Docker designates itself as a manager node.

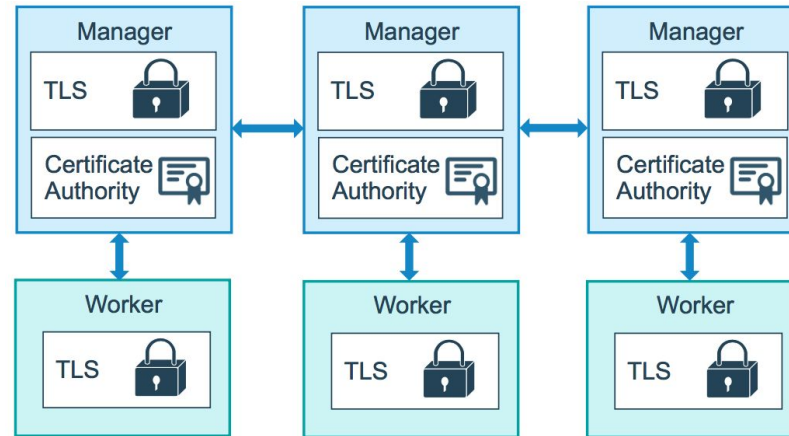
By default, the manager node generates a new root Certificate Authority (CA) along with a key pair, which are used to secure communications



Important Pointers 07 - Certificate for Each Node

Each time a new node joins the swarm, the manager issues a certificate to the node.

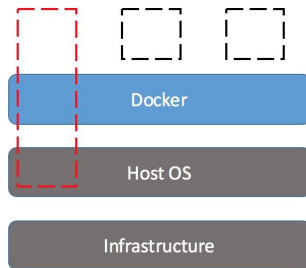
By default, each node in the swarm renews its certificate every three months.



Important Pointers 08 - Privileged container

By default, Docker containers are “unprivileged”

When the operator executes `docker run --privileged`, Docker will enable access to all devices on the host to allow the container nearly all the same access to the host as processes running outside containers on the host.



Important Pointers 09 - Roles in UCP

Built-in role	Description
None	Users have no access to Swarm or Kubernetes resources. Maps to No Access role in UCP 2.1.x.
View Only	Users can view resources but can't create them.
Restricted Control	Users can view and edit resources but can't run a service or container in a way that affects the node where it's running. Users cannot mount a node directory, exec into containers, or run containers in privileged mode or with additional kernel capabilities
Scheduler	Users can view nodes (worker and manager) and schedule (not view) workloads on these nodes. By default, all users are granted the Scheduler role against the /Shared collection
Full Control	Users can view and edit all granted resources. They can create containers without any restriction, but can't see the containers of other users.

Important Pointer - Part 6

Storage and Volumes

Important Pointers - Part 01

1. Mounting Volumes Inside Container

- `docker container run -dt --name webserver -v myvolume:/etc busybox sh`
- `docker container run -dt --name webserver --mount source=myvolume,target=/etc busybox sh`

You can even mount same volume to multiple containers.

Also know what bind mounts are all about

How you can map a directory from host to container.

Important Pointers - Part 02

2. Remove volume when container exits

If you have not named the volume, then when you specify `--rm` option, the volumes associated will also be deleted when container is deleted.

3. Device Mapper

- loop-lvm mode is for testing purpose only.
- direct-lvm mode can be used in production.

Join us in our Adventure

Be Awesome



kplabs.in/twitter



kplabs.in/linkedin

instructors@kplabs.in